



APPRENDIMENTO AUTOMATICO

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 1° semestre

Indice

1	Introduzione	7
1.1	Definizione di Machine Learning (ML)	7
1.2	Terminologia	8
1.3	Notazione	9
1.4	Altra terminologia	9
1.5	Tasks	9
1.6	Assunzione i.i.d	9
1.7	Spazio delle ipotesi	10
1.7.1	Deduzione Vs Induzione	10
1.7.2	Symbolic concept learning	10
1.7.3	Spazio delle ipotesi	11
1.7.4	Spazio delle versioni	11
1.7.5	Bias induttivo	12
1.7.6	No Free Lunch Theorem	13
2	Selezione e valutazione di modelli	15
2.1	Accuracy e Error rate	15
2.2	Overfitting	15
2.3	Metodi di valutazione	16
2.3.1	Validazione Hold-out	16
2.3.2	Cross validation	19
2.3.3	Leave-One-Out	20
2.3.4	Bootstrapping	20
2.4	Tuning dei parametri	21
2.5	Misurazione delle performance	23
2.5.1	Mean Square Error (MSE)	23
2.5.2	Error rate	24
2.5.3	Accuracy	24
2.5.4	Matrice di confusione	24
2.5.5	Precision, Recall	25
2.5.6	P-R Plots	25
2.5.7	ROC e AUC	29
2.6	Cost-Sensitive Error rates	31
2.7	Comparison Test	31
2.7.1	Test binomiale	31
2.7.2	Test di Student	33
2.7.3	Cross-Validated Student's t-test	36
2.7.4	5x2 Cross-Validated t-test	36

2.7.5	McNemar's Test	37
2.7.6	Friedman Test	38
2.7.7	Nemenyi Post-hoc Test	39
2.7.8	Critical difference diagram	39
2.8	Bias Variance decomposition	40
2.8.1	Bias-Variance Dilemma	42
3	Linear Models	43
3.1	Regressione lineare	43
3.2	Mean Squared Error and Best Fit Line	43
3.2.1	Rappresentazione matriciale	44
3.2.2	Coordinate omogenee	44
3.2.3	Risolvere il sistema	44
3.2.4	Migliore p	45
3.2.5	Soluzione dei minimi quadrati tramite calcolo	46
3.3	Regolarizzazione	48
3.4	Ridge regression	48
3.4.1	Informazioni sul miglioramento dell'invertibilità di una matrice	49
3.5	Lasso regression	49
3.6	Perché i metodi di regolarizzazione funzionano?	49
3.6.1	Minimizzazione della norma e scarsità	50
3.7	Modelli lineari generalizzati e regressione logistica	50
3.7.1	Generalized Linear Models (GLM)	50
3.8	Logistic Regression	51
3.9	Classificazione multiclasse	52
3.9.1	Idea principale	53
3.9.2	Matrice di codifica	53
3.9.3	One-vs-One (OvO) Classification	54
3.9.4	One-vs-Rest (OvR) Classification	55
3.9.5	Many-vs-Many (MvM) Classification	55
3.9.6	Error-Correcting Output Codes (ECOC)	55
3.9.7	OvR vs OvO	56
3.10	Problema di squilibrio di classe	56
3.10.1	Aggiornamento della Threshold	56
3.10.2	Oversampling	57
3.10.3	SMOTE	57
3.10.4	Undersampling	58
3.10.5	EasyEnsemble	58
4	Decision Trees	59
4.1	Processo di base	59
4.2	Split selection	62
4.2.1	Modi per misurare l'impurità	62
4.3	Decision trees	63
4.4	Entropia	63
4.4.1	Informazioni	64
4.4.2	Informazioni previste	64
4.4.3	Entropia per la misurazione della purezza	64
4.5	Information Gain	65
4.5.1	L'impurità attesa come valore atteso di una variabile casuale	65
4.5.2	Bias	67
4.6	Gain Ratio	67
4.6.1	Bias	68
4.7	Gini Index	68
4.8	Pruning	68
4.8.1	Pre-pruning	69

4.8.2	Post-pruning	71
4.9	Valori continui	73
4.9.1	Densità	73
4.10	Valori mancanti	75
4.11	Alberi decisionali multivariati	80
5	Reti neurali	82
5.1	Storia delle reti neurali	82
5.1.1	Prima fase	82
5.1.2	Seconda fase	82
5.1.3	Terza fase	82
5.2	Modello di neurone	83
5.2.1	Definizione di rete neurale	83
5.2.2	Modello di neurone M-P (McCulloch-Pitts, 1943)	83
5.2.3	Funzione di attivazione	84
5.3	Perceptron and Multi-layer Network	84
5.3.1	Perceptron	84
5.3.2	Perceptron Learning	85
5.3.3	Perceptron che risolve i problemi "AND", "OR" e "Not"	85
5.3.4	La capacità di apprendimento dei perceptron	88
5.3.5	Perceptron multistrato	88
5.3.6	Multi-layer feed-forward neural networks	89
5.4	Algoritmo di backpropagation degli errori	89
5.4.1	Algoritmo BP	90
6	Support Vector Machines	93
6.1	Margine funzionale	93
6.1.1	Support vectors	94
6.1.2	Relazione tra l'equazione e il vettore W	94
6.2	Margine geometrico	95
6.3	SVM: massimizzazione del margine	96
6.4	SVM problema di ottimizzazione	96
6.5	Dualità	96
6.5.1	Karush-Kuhn-Tucker conditions	97
6.5.2	SVM problema duale	97
6.6	Trovare soluzioni al problema SVM	98
6.6.1	Vantaggi della doppia formulazione	98
6.6.2	I moltiplicatori di Lagrange positivi sono associati ai support vectors	98
6.6.3	Consentire errori di margine	99
6.6.4	Doppia formulazione	99
6.6.5	Espansione delle feature	101
6.6.6	Non-separable case	101
6.6.7	Soft-margin SVM con kernel	102
6.7	Kernels	103
6.7.1	Funzione kernel	103
6.7.2	Prodotto interno	103
6.7.3	Motivazioni del kernel	103
6.7.4	Kernel polinomiale	103
6.7.5	Radial Basis Function kernel	104
6.7.6	Il trucco del kernel	105
6.7.7	Applicazioni del kernel	105
6.7.8	Progettazione di nuovi kernel	105
6.7.9	Proprietà di chiusura dei kernel	106

7	Classificatori Bayesiani	107
7.1	Teoria delle decisioni di Bayes	107
7.1.1	Teoria delle decisioni di Bayes nella classificazione	107
7.2	Stima della massima verosimiglianza	109
7.3	Naïve Bayes Classifier	110
7.3.1	Correzione laplaciana	111
7.4	Semi-Naïve Bayes Classifier	111
7.4.1	SPODE	112
7.4.2	TAN	112
7.4.3	AODE	113
7.5	Reti Bayesiane	113
7.5.1	Struttura di rete	113
7.5.2	Inferenza	114
7.5.3	Alcuni esempi noti di BN	114
7.5.4	Learning	118
7.5.5	Introduzione alla lunghezza minima della descrizione (MDL)	118
7.5.6	Modelli basati sulla compressione	119
7.5.7	Il principio della lunghezza minima della descrizione	119
7.5.8	Learning	120
7.5.9	Inferenza approssimativa	120
7.6	Algoritmo EM	121
8	Ensemble Learning	123
8.1	Cos'è Ensemble Learning	123
8.1.1	Come creare un ensemble efficace?	123
8.1.2	Come si integrano/combinano i classificatori di base	123
8.1.3	Come generare classificatori di base?	124
8.2	Bagging (Bootstrap Aggregating)	124
8.2.1	Bootstrap replicates	124
8.2.2	Stima dell'errore Out-of-bag	124
8.2.3	Random Forests (RFs)	126
8.3	AdaBoost (Adaptive Boosting)	127
8.3.1	Il problema dell'aumento delle ipotesi	127
8.3.2	Boosting	128
8.3.3	Derivazione dell'algoritmo AdaBoost	132
8.3.4	Imparare da set di dati ponderati	133
8.3.5	Ricampionamento del set di dati	134
8.3.6	Apprendimento da istanze ponderate	134
8.4	Perché l'ensemble learning funziona?	134
8.4.1	Scomposizione Bias/Varianza	134
8.4.2	Bagging è una macchina per la riduzione della varianza	135
8.4.3	Bagging e distribuzione binomiale	135
8.4.4	Benefit di ensemble learning	136
9	Clustering	137
9.1	Problema del Clustering	137
9.1.1	Formalizzazione	137
9.2	Misura delle prestazioni	137
9.2.1	Misura delle prestazioni per il clustering	137
9.2.2	Misura della performance - Indice esterno	138
9.2.3	Misura della performance - Indice interno	140
9.3	Calcolo della distanza	144
9.3.1	Attributi	144
9.3.2	Value Difference Metric, VDM	145
9.3.3	Misura della distanza con tipi di attributi misti	145
9.4	Prototype Clustering	145

9.4.1	K-means Clustering	145
9.4.2	Learning Vector Quantization (LVQ)	148
9.5	Density Clustering	149
9.5.1	Definizione di clustering di densità	149
9.5.2	DBSCAN	149
9.5.3	Come calcolare i parametri DBSCAN	153
9.6	Hierarchical Clustering	153
9.6.1	Punti di forza del clustering gerarchico	153
9.6.2	AGNES	154
9.6.3	Risultati del Clustering	156
9.7	Probabilistic Clustering	156
9.7.1	Prototype Clustering - Mixture of-Gaussian Clustering	156
9.7.2	Calcolo della matrice di covarianza	157
9.7.3	Mostrare la correlazione tra due features	157
9.7.4	Definizione della distribuzione mista di Gaussiane	157
9.7.5	Ottimizzazione dei parametri del modello	158
10	Riduzione della dimensionalità e Apprendimento delle metriche	161
10.1	k-Nearest Neighbor Learning	161
10.1.1	Introduzione a k-nearest neighbor learning	161
10.1.2	Lazy Learning e Eager Learning	161
10.1.3	Illustrazione di k-Nearest Neighbor Learning	161
10.1.4	Errore di classificazione binaria di 1NN	161
10.2	Low-dimensional Embedding	162
10.2.1	Maledizione della dimensionalità	162
10.2.2	Multiple Dimensional Scaling	163
10.2.3	Decomposizione degli autovalori di B	165
10.2.4	Riduzione delle dimensioni lineari	166
10.3	Analisi delle componenti principali (PCA)	168
10.3.1	Errore di ricostruzione minimo	168
10.3.2	Massima varianza	168
10.3.3	Risolvere PCA	169
10.4	Kernelized Linear Dimension Reduction	170
10.4.1	Kernelized PCA (KPCA)	170
10.5	Manifold learning	171
10.5.1	Isometric Mapping (Isomap)	171
10.5.2	Locally Linear Embedding (LLE)	172
10.6	Apprendimento delle metriche	173
10.6.1	Neighbourhood Component Analysis (NCA)	174
11	Selezione degli Attributi e Apprendimento Sparso	176
11.1	Feature	176
11.2	Selezione delle Feature	176
11.2.1	Metodo generale per la selezione delle features	176
11.2.2	Ricerca di sottoinsieme	177
11.2.3	Forward search	177
11.2.4	Valutazione del sottoinsieme	177
11.2.5	Valutare il sottoinsieme con entropia informativa	177
11.2.6	Metodi di selezione delle features comunemente utilizzati	178
11.3	Rappresentazione sparsa	181
11.4	Apprendimento del dizionario	182
11.4.1	Risolvere l'apprendimento del dizionario	182
11.4.2	Risolvere il dizionario di apprendimento di KSVD	182
11.5	Rilevamento compresso	183
11.5.1	Restricted Isometry Property	183
11.5.2	L'oggetto di ottimizzazione e la soluzione al rilevamento compresso	183

11.6 Completamento della matrice	184
11.6.1 L'oggetto di ottimizzazione e la soluzione per il completamento della matrice . .	184
12 Computational Learning Theory	185
12.1 Alcuni concetti e notazioni	185
12.2 Cos'è l'apprendimento	186
12.2.1 Concetti	186
12.2.2 Spazio delle ipotesi	186
12.2.3 Separabili e non separabili	186
12.2.4 Perché non possiamo apprendere esattamente il concetto target	186
12.3 Cos'è apprendibile	187
12.3.1 Probably Approximately Correct, PAC	187
12.3.2 L'importanza dell'apprendimento PAC	188
12.4 Spazio delle ipotesi Finito	188
12.4.1 Casi separabili	189
12.4.2 Casi Non separabili	190
12.5 Spazio delle ipotesi Infinito	192
12.6 Dimensione Vapnik-Chervonenkis (VC)	192
12.6.1 Dicotomia	193
12.6.2 Shattering	193
12.6.3 Limite superiore della funzione di crescita	196
12.7 Rademacher Complexity	197
12.8 Stabilità	199
12.8.1 Due cambi di set di allenamento	199
12.8.2 Loss function	199

1 Introduzione

1.1 Definizione di Machine Learning (ML)

Thomas Mitchell; 1996; Machine Learning

A computer program is said to learn from experience with respect to some class of tasks and performance measure, if its performance at tasks in, as measured by, improves with experience.

É la combinazione di tre oggetti l'**esperienza E** sotto forma di esempi del problema risolto al computer, un **task T** da risolvere (classificazione degli esempi in un certo numero di categorie, assegnazione di un numero associato agli esempi, riconoscimento di un'immagine), sotto una **misura di performance P** un valore per misurare quanto sta andando bene.

Il programma del computer fa apprendimento automatico basandosi sugli esempi se le sue performance migliorano man mano che vede gli esempi.

Esempio del melone Assumendo di avere dei dati sulla maturazione dei meloni vogliamo sapere se il frutto è maturo o no.

ID	Color	Root	Sound	Ripe
1	green	curly	muffled	true
2	dark	curly	muffled	true
3	green	straight	crisp	false
4	dark	slightly curly	dull	false

Figura 1: Un set di dati di angurie giocattolo

Ora troviamo un melone con le seguenti caratteristiche e vogliamo sapere se è maturo:

Color	Root	Sound
green	curly	dull

Figura 2: Esempio di anguria di prova

In questo caso abbiamo un **task**, sapere se è maturo, delle **performance**, quanto è accurata la nostra predizione e dell'**esperienza** i meloni comprati in precedenza.

Questo tipo di lavoro è chiamato **apprendimento supervisionato**, dove un essere umano andrà a segnare qual è la risposta giusta. Nell'**apprendimento non supervisionato** la colonna delle risposte non è presente, ma si andrà a raggruppare risposte simili.

1.2 Terminologia

Attributes/Features				Label/Class y
ID	Color	Root	Sound	Ripe
Training set	1 green	curly	muffled	true
	2 dark	curly	muffled	true
	3 green	straight	crisp	false
	4 dark	slightly curly	dull	false
Test set	1 green	curly	dull	?

Sample/Instance x_i
 Attribute value/Feature value x_{i1}

Figura 3: Terminologia sull'esempio delle angurie

Le colonne della tabella sono chiamate **attributi** o **features** dell'esempio, l'elemento che ci dice come risolvere il task è chiamato **etichetta** o **classe** dell'esempio, tipicamente la classe viene denotata con una y_i minuscola.

Una riga della tabella, senza considerare la colonna della classe, è chiamato **sample** o **istanza** dell'esempio, in genere denotata con una x_i per dire che è l'iesimo esempio. [Tutte le volte che parleremo di vettori saranno rappresentati da lettere maiuscole, invece i valori scalari saranno rappresentati in corsivo]

Il singolo valore di una colonna è chiamato **attribute value** o **feature value** ed è rappresentato da un valore in corsivo x_{i1} in questo caso la prima feature dell'iesimo esempio.

L'insieme degli esempi che usiamo per indurre la regola che poi utilizzeremo per classificare è chiamato **set di training** e spesso al fianco si trova un altro insieme di esempio, chiamato **test set**, che serve per riuscire a capire quanto bene stiamo performando.

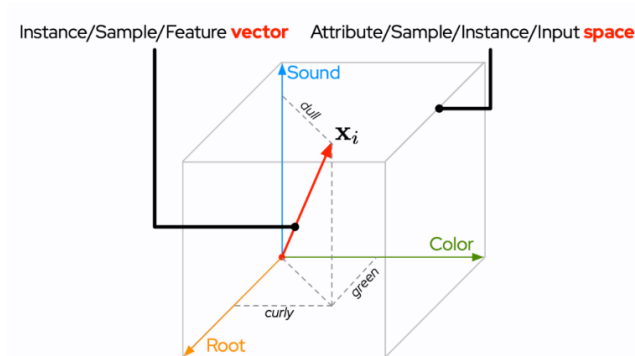


Figura 4: Terminologia in uno spazio euclideo

Quanto gli esempi vengono immaginati in uno spazio euclideo di qualche tipo, in questo caso a tre dimensioni, spesso ci si riferisce all'esempio come **instance vector** oppure **sample vector**, **feature vector**, l'intero insieme di tutti i possibili esempi è chiamato anche **attribute space**, **sample space**, **instance space**, **input space**.

1.3 Notazione

Symbol	Meaning
$\mathcal{X}$	instance space, in many cases $\mathcal{X} = \mathbb{R}^d$
$\mathcal{Y}$	label space, e.g., $\mathcal{Y} = \{0, 1\}$ for binary classification
$\mathbf{x}_i \in \mathcal{X}$	i -th instance/sample, $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^\top$
d	dimensionality of the instance space
$y_i \in \mathcal{Y}$	label of the i -th instance, e.g., $y_i = 1$ if the watermelon is ripe, $y_i = 0$ otherwise
D	Dataset, $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$
n	number of instances in the dataset
h	model/hypothesis, a function $h : \mathcal{X} \mapsto \mathcal{Y}$ that maps instances to labels
$\mathcal{L}$	Learning algorithm $\mathcal{L} : \mathcal{P}(\mathcal{X} \times \mathcal{Y}) \rightarrow \mathcal{H}$, where $\mathcal{P}$ is the power set and $\mathcal{H}$ is the hypothesis space
$\mathbb{I}$	Indicator function, $\mathbb{I}(P)$ (sometime denoted as $\mathbb{I}[P]$) is 1 if P is true, 0 otherwise

Figura 5: Notazione che utilizzeremo

1.4 Altra terminologia

Il **training** è un processo utilizzato da algoritmo di machine learning per costruire un modello dai dati. I dati utilizzati durante questo processo sono chiamati **training data**, dove ogni esempio è chiamato **training example** e l'insieme di tutti gli esempi è chiamato **training set**.

Obiettivo del Machine Learning

L'obiettivo dell'apprendimento automatico è trovare un **ipotesi** che è capace di approssimare la **ground-truth** su dati che non ha mai visto.

1.5 Tasks

I più importanti tasks che si possono fare con l'apprendimento automatico sono:

- **Predittivi**: predire un numero o una caratteristica, solitamente sono supervisionati
 - **Classificazione**, abbiamo un certo numero di etichette negli esempi, se le etichette sono solo due si parla di classificazione **binaria** invece se sono di più si parla di classificazione **multiclasse**.
 - **Regressione**, è un caso di predizione dove l'insieme delle etichette corrisponde con un intervallo continuo, cerchiamo di predire dei numeri.
- **Descrittivi**: descrivere qualcosa su come è strutturato il data set, solitamente non supervisionati
 - **Clustering**
 - **Regole di associazione**

Learning supervisionato l'etichetta è fornita dall'utente

Learning non supervisionato l'etichetta non esiste

1.6 Assunzione i.i.d

Nella maggioranza dei casi, in particolare in tutti gli algoritmi che vedremo nel corso, si assume che i dati siano **Indipendenti** e **Identicamente Distribuiti**.

Nel momento in cui ho estratto un istanza con la sua etichetta il prossimo lo estraggo dalla stessa distribuzione di probabilità, perché se cambia la distribuzione non ho speranza di apprendere nulla.

Indipendenti perché fissata la distribuzione, se io pesco due esempi, l'aver estratto un esempio non ha implicazione sull'estrazione di un altro esempio. [Ex. estraggo una pallina dall'urna, poi la rimetto dentro e riestraggo. La probabilità del primo e del secondo caso sarà la medesima]

- **Indipendenza:** $P((X_i, y_i) \mid (X_1, y_1), \dots, (X_{i-1}, y_{i-1})) = P((X_i, y_i)) \quad \forall i$
- **Identicamente distribuiti:** Tutte le istanze provengono dalla stessa distribuzione sottostante. Ciò significa anche che i dati di training e i dati di test devono provenire dalla stessa distribuzione.

Esempio

Immaginando di gestire una fabbrica di di cupcake. Ogni cupcake è fatto:

- usando la **stessa ricetta** (identicamente distribuita)
- e **e cotti in uno stampo separato** senza influenzare gli altri (indipendenti)

Quindi, se assaggi 10 cupcake a caso e hanno tutti un sapore dolce, puoi presumere che anche i cupcake futuri saranno dolci, perché il tuo campione è i.i.d.

Esempio - What if

Cosa succede se:

- All'improvviso usi **due ricette diverse** → non distribuite in modo identico.
- Cuoci i cupcake nello **stesso stampo e l'impasto si mescola** → non indipendente.

1.7 Spazio delle ipotesi

1.7.1 Deduzione Vs Induzione

Deduzione vuol dire che parto da degli assiomi, ho delle regole per combinare gli assiomi e ottengo una deduzione. La deduzione è un processo **corretto**, se assumo che gli assiomi siano corretti allora qualsiasi cosa io deduca da questi assiomi deve essere vera per forza.

L'Induzione è il processo inverso, si parte da oggetti specifici, gli esempi, e cerco di dedurre una regola generale.

Il processo di Apprendimento Automatico si basa interamente sul principio dell'induzione.

Esempio

Esempio di deduzione:

- Tutti gli umani sono mortali
- Socrate è un uomo
- Quindi, Socrate è mortale

Esempio di induzione:

- Dopo aver osservato che ogni essere umano conosciuto prima o poi muore, potremmo dedurre che tutti gli esseri umani sono mortali.

1.7.2 Symbolic concept learning

Con i metodi induttivi vogliamo provare a costruire delle regole logiche, e in questo caso ci troviamo nel campo del **symbolic concept learning** e una sua forma più estesa è chiamata **Inductive Logic Programming**, dei linguaggi che ci permettono di lavorare con la logica per costruire queste regole in modo induttivo.

La forma base del concept learning è chiamato **boolean concept learning** dove l'obiettivo è costruire una funzione $h : \chi \mapsto \{0, 1\}$.

Esempio

Supponendo di voler predire se un melone è maturo e la formula è:

$$ripe \iff (color = ?) \wedge (root = ?) \wedge (sound = ?)$$

Al posto dei punti interrogativi possiamo mettere un valore oppure un **asterisco *** per dire che **non interessa il valore specifico**.

In questo caso il melone sarà maturo se il suono è dull il resto non mi interessa.

$$ripe \iff (color = *) \wedge (root = *) \wedge (sound = dull)$$

1.7.3 Spazio delle ipotesi

Lo **Spazio delle ipotesi** è l'insieme di tutte le ipotesi (tutti i possibili modelli) che posso apprendere utilizzando uno specifico algoritmo di apprendimento. [Nell'esempio sopra citato è l'insieme di tutte le formule booleane che posso ottenere].

Possiamo pensare all'apprendimento automatico come un **processo di ricerca all'interno dello spazio delle ipotesi**, e devo trovare l'ipotesi migliore in base ai dati che ho.

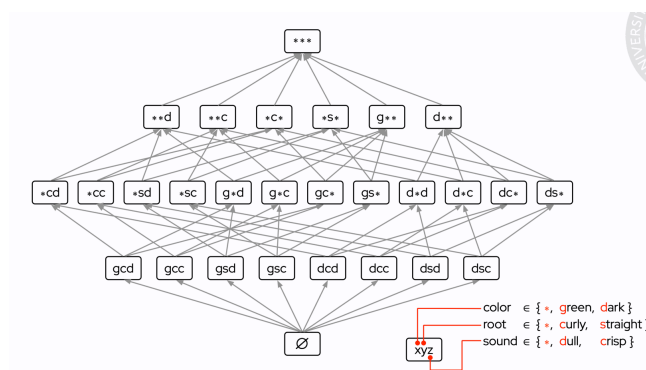


Figura 6: Spazio delle ipotesi di esempio, iniziale

1.7.4 Spazio delle versioni

Lo **spazio delle versioni** è un **sottoinsieme dello spazio delle ipotesi** che è consistente con tutti gli **esempi di training**. Sostanzialmente prendo solo gli esempi che danno una risposta giusta [nel caso dei meloni se trovo un esempio con suono *dull* ma non maturo lo andrò a scartare].

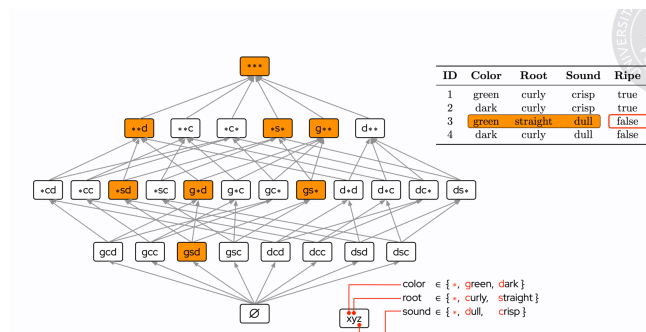


Figura 7: Spazio delle versioni di esempio, esempio ID 3

L'ipotesi in arancione non è consistente con l'esempio, perché questo avrebbe detto a qualsiasi cosa mature. ****d** dice che dovrebbe dare maturo e anche questo non va bene. Tutte le caselle evidenziate in arancione non sono consistenti con l'esempio in questione, quindi decido di eliminarle.

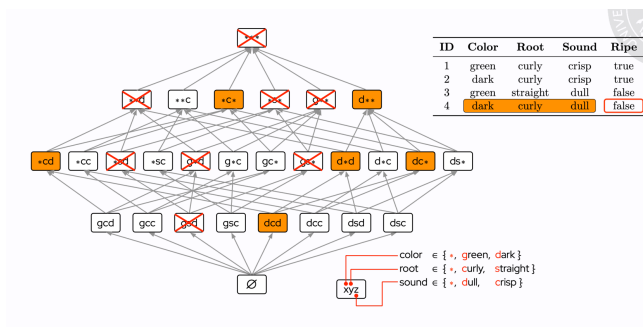


Figura 8: Spazio delle versioni di esempio, esempio ID 4

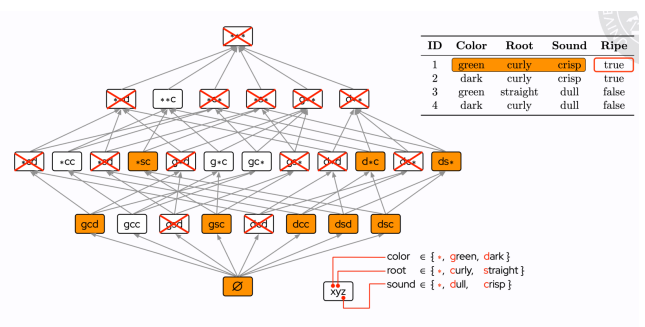


Figura 9: Spazio delle versioni di esempio, esempio ID 1

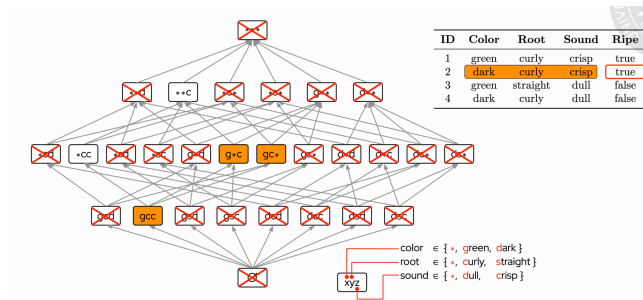


Figura 10: Spazio delle versioni di esempio, esempio ID 2

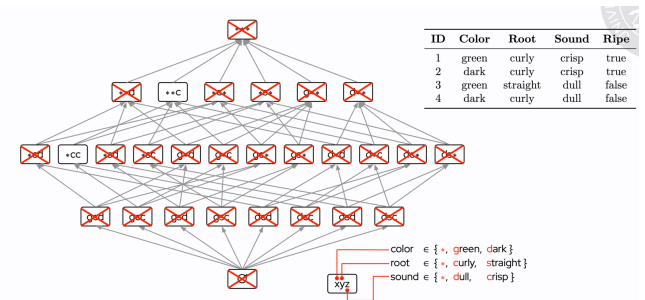


Figura 11: Spazio delle versioni di esempio, esempio finale

Vado a controllare tutti gli esempi e alla fine andrò a vedere che lo spazio delle versioni finale sarà:

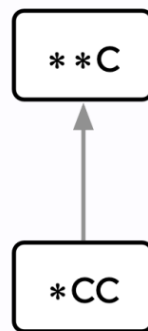


Figura 12: Spazio delle versioni di esempio, finale

Nell'esempio lo spazio delle versioni consiste in due ipotesi:

- $ripe \iff (color = *) \wedge (root = *) \wedge (sound = crisp)$
- $ripe \iff (color = *) \wedge (root = curly) \wedge (sound = crisp)$

Dato quello che sappiamo del dataset le due sono equivalenti e non c'è una giusta.

1.7.5 Bias induttivo

Nel caso dovesse arrivare un nuovo esempio del tipo $Color = green, Root = straight, Sound = crisp$, avremmo una regola che dice maturo e una che dice non maturo.

Senza altre informazioni non possiamo decidere quale delle due ipotesi è corretta, però un algoritmo non può restituire un insieme di ipotesi.

Tutti gli algoritmi di apprendimento hanno un **bias induttivo**, non basato sui dati.

Si può scegliere di preferire delle regole più generali o all'opposto potrebbe preferire una regola più specifica.

Ogni algoritmo di apprendimento DEVE avere un bias induttivo, altrimenti non potrebbe concludere l'apprendimento. Potrei anche scegliere una risposta a caso ma non sarebbe una soluzione furba da prendere.

Occam's Razor è un principio che a parità di ipotesi consistenti devo scegliere quella più semplice come bias induttivo.

Semplice può avere molti significati diversi, in genere la più utilizzata è quella che fa meno assunzioni sui dati.

No Free Lunch Theorem in modo informale dice che nessun algoritmo di apprendimento universalmente migliore di altri quando le sue performance sono misurate su tutti i possibili mondi.

1.7.6 No Free Lunch Theorem

Denotiamo con:

- $P(h \mid X, \mathcal{L}_a)$: **probabilità** che l'algoritmo $\mathcal{L}_a$ restituisca l'ipotesi h dati i dati X
 - h è l'ipotesi, quello che stiamo imparando
 - X è il training set
 - $\mathcal{L}_a$ è l'algoritmo di apprendimento
- f è la **ground truth** il modello vero sottostante che vogliamo apprendere
- l'errore **out-of-sample** dell'ipotesi h , denotato con E_{ote} , è la media dell'errore su tutti gli elementi che non sono presenti nel training set, che sono commessi dai modelli appresi tramite $\mathcal{L}_a$ sul training set X .

$$E_{ote}(\mathcal{L}_a \mid X, f) = \sum_h \sum_{x \in \mathcal{X} - X} P(X) \mathbb{I}(h(X) \neq f(X)) P(h \mid X, \mathcal{L}_a)$$

Faccio la somma su tutte le possibili ipotesi, dopo di che sommo su tutti gli esempi che non sono nel training set ottenuto da $\mathcal{X}$ (spazio di tutti i possibili esempi) togliendo gli elementi del training set. $P(X)$ è la probabilità di estrarre uno specifico esempio per la funzione indicatrice $\mathbb{I}(h(X) \neq f(X))$ (restituisce 1 se l'espressione è vera e il modello sta sbagliando a predire l'etichetta di X e 0 altrimenti) tutto per la probabilità di h su tutti i modelli.

Dimostrazione

Vogliamo dimostrare che l'errore non dipende da $\mathcal{L}_a$, quindi un algoritmo vale l'altro in media.

$$\begin{aligned}
 \sum_f E_{ote}(\mathcal{L}_a | X, f) &= \sum_f \sum_h \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \mathbb{I}(h(\mathbf{x}) \neq f(\mathbf{x})) P(h | X, \mathcal{L}_a) \\
 &= \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \sum_f \mathbb{I}(h(\mathbf{x}) \neq f(\mathbf{x})) \\
 &= \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \frac{1}{2} 2^{|\mathcal{X}|} \\
 &= \frac{1}{2} 2^{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \sum_h P(h | X, \mathcal{L}_a) \\
 &= \frac{1}{2} 2^{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}-X} P(\mathbf{x}) \cdot 1
 \end{aligned}$$

Figura 13: Svolgimento dimostrazione No Free Lunch

Nella diapositiva precedente, il termine $\frac{1}{2} 2^{|\mathcal{X}|}$ è il risultato della somma degli errori di tutte le possibili ipotesi binarie. Dato uno spazio campionario $\mathcal{X}$, esistono $2^{|\mathcal{X}|}$ modi per etichettare gli esempi e, quindi, $2^{|\mathcal{X}|}$ possibili ipotesi distinte. Se fissiamo un esempio X e un'ipotesi h , metà di queste ipotesi sarà in disaccordo con h su X , mentre l'altra metà sarà in accordo.

Esempio

Consideriamo $\mathcal{X} = \{X_1, X_2, X_3\}$, ci concentriamo su un esempio randomico (facciamo X_2), e assumiamo $h(X_2) = 0$, le funzioni in disaccordo con h su X_2 sono:

Function	$f(\mathbf{x}_1)$	$f(\mathbf{x}_2)$	$f(\mathbf{x}_3)$
f_1	0	0	0
f_2	0	0	1
f_3	0	1	0
f_4	0	1	1
f_5	1	0	0
f_6	1	0	1
f_7	1	1	0
f_8	1	1	1

Figura 14: Tabella di esempio, No Free Lunch

In generale, per qualsiasi X ed etichetta di h , metà delle $2^{|\mathcal{X}|}$ funzioni sono in disaccordo con h .

Quindi abbiamo dimostrato che:

$$\sum_f E_{ote}(\mathcal{L}_a | X, f) = \frac{1}{2} 2^{|\mathcal{X}|} \sum_{X \in \mathcal{X}-X} P(X)$$

Osservazione Il teorema assume che si possa mediare su tutte le possibili f ma nel mondo non tutte le ipotesi sono ugualmente importanti. Questo è un risultato interessante, perché dimostra che l'errore medio fuori campione di un algoritmo di apprendimento è **indipendente dall'algoritmo stesso**. Ciò significa che **nessun algoritmo di apprendimento è migliore di un altro** quando si calcola la media su tutti i possibili problemi.

2 Selezione e valutazione di modelli

2.1 Accuracy e Error rate

Il modo più semplice per misurare la bontà di un modello è quella di contare il numero di predizioni sbagliate che esso compie.

$$E = \frac{a}{m}$$

Dove a solo il numero di predizioni sbagliate ed m le predizioni totali E sarà il suo **error rate**.

L'**accuratezza** è calcolata:

$$accuracy = 1 - E = \frac{m - a}{m}$$

L'errore quando viene valutato sul training set è chiamato **empirical error** o **training error**. Quando calcolo l'errore su un data set indipendente si chiamerà **generalization error** o **test error**.

Nota

L'obiettivo principale di un algoritmo di apprendimento è quello di **minimizzare l'errore di generalizzazione**.

Durante l'addestramento non possiamo calcolare direttamente il **generalization error**, perché non abbiamo accesso ai nuovi esempi, quindi lo approssimiamo minimizzando il **training error**.

2.2 Overfitting

Quando un **modello si adatta troppo bene ai dati di training**, è probabile che alcune peculiarità degli esempi di training vengano assunte come proprietà generali dei dati. Questo porta a un modello che funziona bene sul set di training ma male sui nuovi campioni. Questo fenomeno è chiamato **overfitting**.

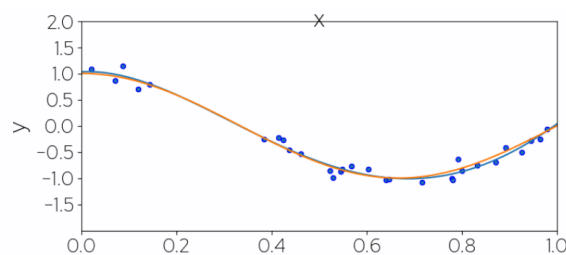


Figura 15: Esempio di Overfitting

Il **fenomeno opposto**, in cui il modello è troppo semplice per catturare i modelli sottostanti nei dati, è chiamato **underfitting**.

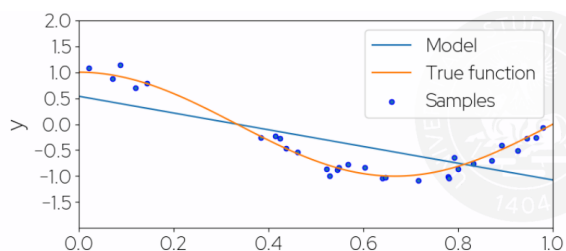


Figura 16: Esempio di Underfitting

L'**underfitting** è solitamente facilmente risolvibile utilizzando un modello più complesso. L'**overfitting** è più difficile da risolvere, poiché richiede un attento equilibrio tra complessità del modello e capacità di generalizzazione. Ciò nonostante, la maggior parte degli algoritmi di apprendimento dispone di meccanismi per prevenire l'overfitting, come **regularization techniques** o **early stopping**.

In genere si preferisce fare un leggero overfitting per avere una capacità espressiva sufficiente.

2.3 Metodi di valutazione

Possiamo valutare le prestazioni di un modello misurandone l'**errore di generalizzazione** su nuovi campioni. Tuttavia, non abbiamo accesso a nuovi campioni durante l'addestramento.

Utilizziamo invece diverse tecniche per stimare l'errore di generalizzazione sulla base dei dati di addestramento.

2.3.1 Validazione Hold-out

Divido l'insieme dei dati in due parti:

- **Training set:** dati che darò in pasto all'algoritmo per l'apprendimento.
- **Test set:** dati che utilizzerò per valutare.

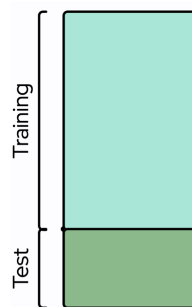


Figura 17: Validazione hold-out

```
1 from sklearn.model_selection import train_test_split
2 ...
3
4 #Assume X,y contains the dataset
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

Come effettuare lo split:

- **Simple split**
I primi n dati gli prendo come training e i restanti come test. Ma se ho dei dati non casuali rischio di avere un test set non veritiero.

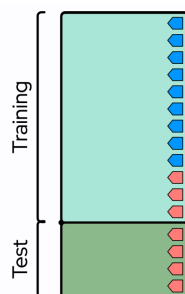


Figura 18: Simple split

- **Stratified split** Si mescolano i dati prima di estrarli poi procedo a prendere i primi n come training e il restante come test.

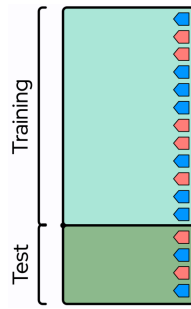


Figura 19: Stratified split

```

1  from model_selection import train_test_split
2  ...
3
4  #Assume X,y contains the dataset
5
6  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
7      =0.3, shuffle=True, #(default) sufficient for large datasets
8      stratify=y, #useful for small and/or imbalanced datasets
      random_state=42)

```

Se ripetiamo l'esperimento con il metodo di valutazione hold-out molte volte otterremo tanti risultati diversi, perché training e test set cambiano.

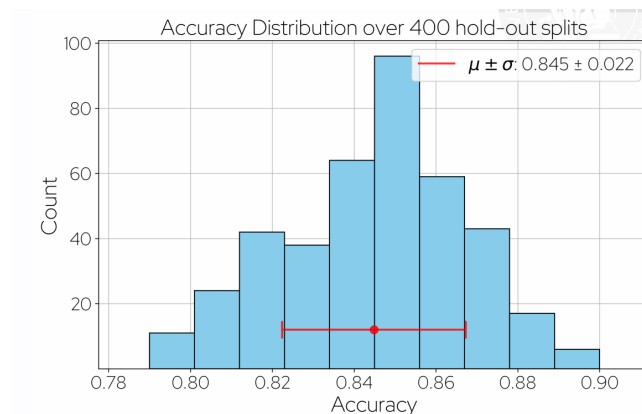


Figura 20: Distribuzione della precisione su 400 divisioni di hold-out

Per ottenere una stima dell'errore di generalizzazione più affidabile possiamo ripetere la validazione hold-out molteplici volte e facciamo la media dei risultati.

La media di n r.v. **indipendenti**, ognuno con media μ e varianza σ^2 è una nuova r.v. con:

Media

$$\frac{1}{n} \sum_{i=1}^n E_i = \frac{1}{n} \sum_{i=1}^n \mu = \mu$$

Varianza

$$\frac{1}{n^2} \sum_{i=1}^n \sigma^2 = \frac{\sigma^2}{n}$$

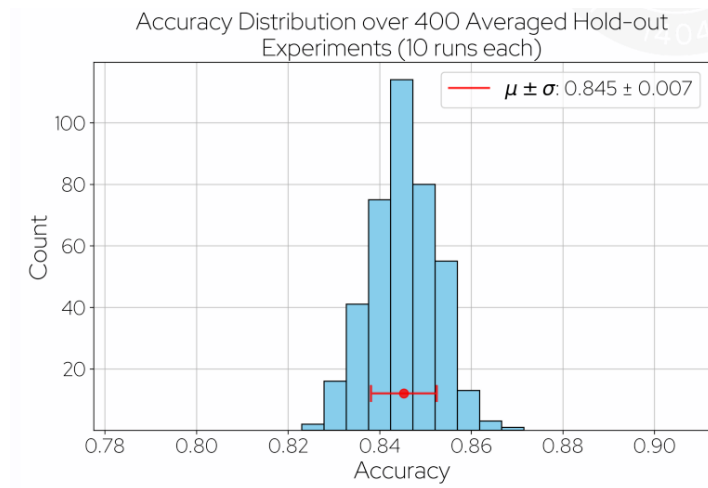


Figura 21: Distribuzione dell'accuratezza su 400 esperimenti di hold-out medi (10 esecuzioni ciascuno)

Modifica del rapporto di divisione Una cosa che influisce sull'ampiezza della campana è come stiamo separando il training set dal test set.

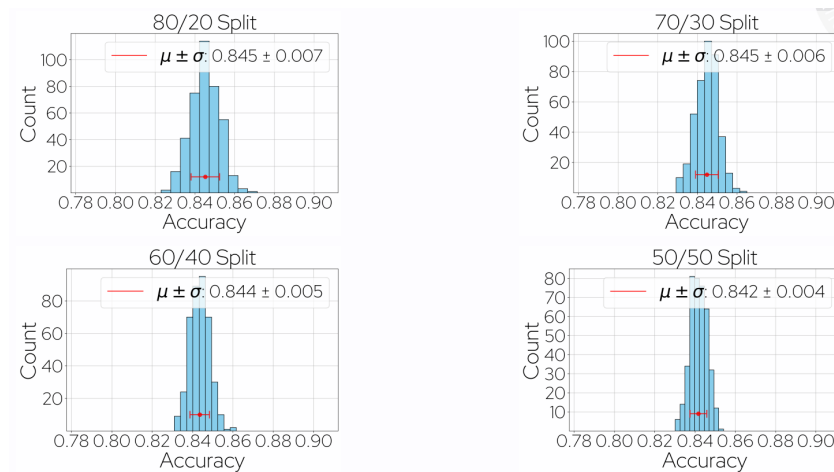


Figura 22: Modifica dello split ratio prima parte

Nel momento in cui mettiamo più esempi nel training set otterremo un classificatore che dovrebbe essere più bravo a predire.

All'aumentare della percentuale di esempi nel test set avremo un'accuratezza minore ma la varianza diminuisce.

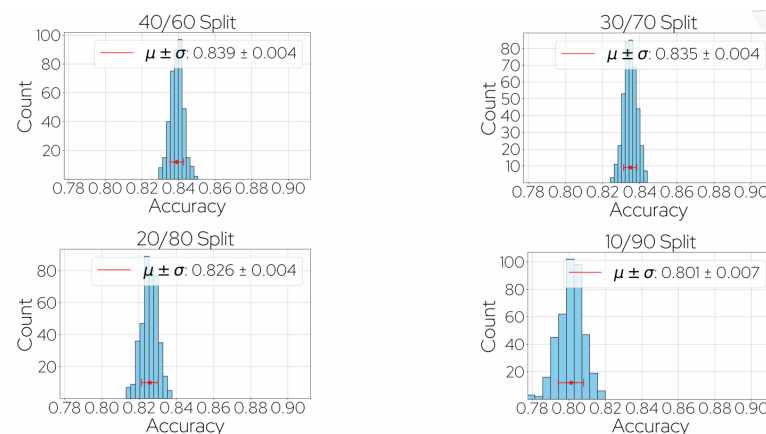


Figura 23: Modifica dello split ratio seconda parte

Continuando ad aumentare la percentuale di esempi per il test set la media andrà sempre peggiorando ma la varianza non migliorerà all'infinito anzi ricomincerà ad aumentare.

Se il dataset è grande abbastanza, la **validazione hold-out** è un **semplice** ed **efficace** metodo per stimare l'errore di generalizzazione.

Ma, se il **dataset** è **piccolo**, per tenere da parte una quantità sostanziale di dati per il testing potrebbe portare ad una stima non precisa dell'errore di generalizzazione. In questi casi si può usare la **cross-validation**.

2.3.2 Cross validation

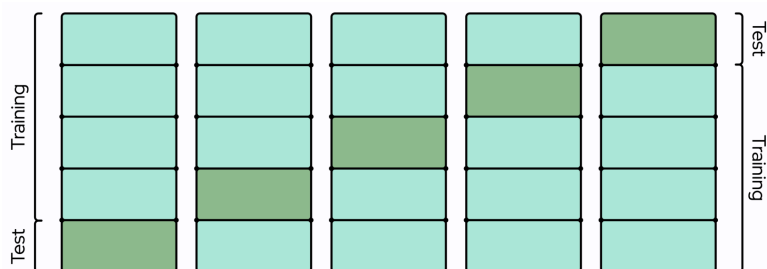


Figura 24: Cross validation

Dividiamo il dataset in k parti, **ripetiamo il processo di hold-out** k volte. La prima volta utilizziamo i primi $k - 1$ **fold** come training set e l'ultimo come test set, la volta successiva utilizziamo il penultimo come test set e tutti gli altri come training set e così via. Alla fine facciamo una **media** e otteniamo una **stima dell'errore di generalizzazione**.

Questa tecnica ha il vantaggio che l'algoritmo di apprendimento ha avuto l'opportunità di vedere, almeno una volta, tutti gli esempi.

Metodo più flessibile

```

1  from sklearn.model_selection import KFold
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.metrics import accuracy_score
4  import numpy as np
5
6  # Assume X,y contains the dataset
7
8  clf = LogisticRegression()
9
10 kf = KFold(n_splits=5, shuffle=True, random_state=42)
11 accuracies = []
12 for train_index, test_index in kf.split(X):
13     X_train, X_test = X[train_index], X[test_index]
14     y_train, y_test = y[train_index], y[test_index]
15
16     clf.fit(X_train, y_train)
17     score = accuracy_score(y_test, clf.predict(X_test))
18     accuracies.append(score)
19
20 print(f"x-val accuracy: {np.array(accuracies).mean()}")

```

Per sapere in modo veloce l'errore di generalizzazione.

```

1  from sklearn.model_selection import cross_val_score
2  from sklearn.linear_model import LogisticRegression
3
4  # Assume X,y contains the dataset
5
6  clf = LogisticRegression()
7  scores = cross_val_score(clf, X, y, cv=5)
8  print(f"x-val accuracy: {scores.mean()}")

```

Differenza tra hold-out e cross-validation: Nella **cross validation**, ogni campione viene utilizzato per il test **una sola volta**, mentre se **ripetiamo hold-out**, alcuni campioni possono essere utilizzati **più volte per il test** e altri potrebbero **non essere mai utilizzati per il test**. Ciò si traduce in una stima più affidabile dell'errore di generalizzazione (**varianza minore**).

Se il **dataset è sufficientemente grande**, la differenza è trascurabile

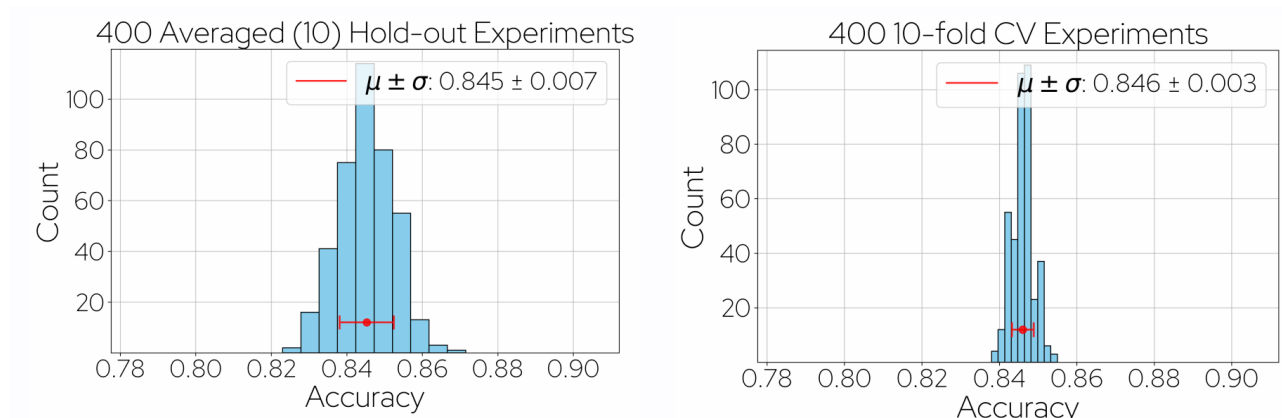


Figura 25: Cross-Validation vs Hold-out Validation

2.3.3 Leave-One-Out

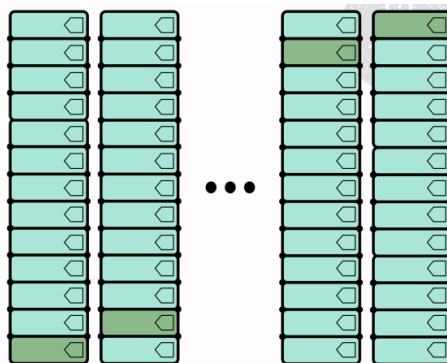


Figura 26: Leave-One-Out

Leave-One-Out è un caso estremo di cross-validation dove **ogni campione viene utilizzato per il test una sola volta**, mentre tutti gli altri campioni vengono utilizzati per l'addestramento. Di conseguenza, l'**errore di generalizzazione** viene stimato con una **varianza molto bassa**. Tuttavia, il **costo computazionale** è molto elevato, poiché è necessario addestrare il modello m volte, dove m è il numero di campioni nel set di dati.

2.3.4 Bootstrapping

Dato un set di dati D con campioni m , il nostro obiettivo è stimare l'**errore di generalizzazione** di un modello addestrato sull'**intero set di dati**. Tuttavia, la validazione hold-out e la cross-validation richiedono di addestrare il modello su un sottoinsieme di dati, il che può portare a una stima distorta dell'errore di generalizzazione. Per superare questo problema, possiamo utilizzare il **bootstrapping**. L'idea è quella di **creare più campioni bootstrap** dal set di dati originale D , ciascuno contenente m campioni, campionando con sostituzione.

Definizione

Un **campione bootstrap** è un campione ottenuto campionando con sostituzione dal set di dati originale un numero di esempi pari alla dimensione del set di dati originale.

Nota

I **campioni bootstrap non sono disgiunti**. Infatti, ogni campione è ottenuto campionando con sostituzione dal dataset originale, quindi **alcuni campioni potrebbero essere ripetuti** nello stesso campione bootstrap e **altri potrebbero non comparire in esso**.

In realtà è facile calcolare la probabilità che un campione non venga selezionato in un campione bootstrap:

$$P(\text{sample not selected}) = \left(1 - \frac{1}{m}\right)^m$$

Per valori grandi di m , questa probabilità è approssimativamente $e^{-1} \approx 0.3679$, il che significa che in media, circa il **37% dei campioni non viene selezionato** in un campione bootstrap.

Sia D' un campione bootstrap ottenuto da D . Possiamo **addestrare un modello** su D' e **valutarne le prestazioni** sui campioni in $D - D'$ che non sono stati selezionati nel campione bootstrap. In questo modo, possiamo ottenere una stima dell'errore di generalizzazione del modello addestrato su un campione che ha la stessa dimensione del dataset originale.

Nota

Il bootstrapping è **particolarmente utile** quando il **set di dati è piccolo** o quando non esiste un modo efficace per suddividerlo in set di training e test. Quando il set di dati è più grande, il bootstrapping di solito non è necessario e vengono utilizzate la validazione hold-out o la validazione incrociata.

2.4 Tuning dei parametri

Uno dei motivi per cui si utilizzano i metodi di valutazione sopra descritti è la **selezione del modello migliore** tra un insieme di modelli candidati. Questo può significare selezionare il **miglior algoritmo di apprendimento** o selezionare i **migliori iperparametri** per un dato algoritmo di apprendimento.

Definizione

I **parametri** sono gli "oggetti" che l'algoritmo di apprendimento manipola per adattare il modello ai dati, come i pesi in una rete neurale o i coefficienti in un modello di regressione lineare. Gli **iperparametri** sono parametri dell'algoritmo di apprendimento stesso, come l'altezza di un albero decisionale o il parametro di regolarizzazione in un modello di regressione lineare.

La stima dell'errore di generalizzazione utilizzando le tecniche descritte può essere utilizzata per due scopi:

- **Selezione del modello o ottimizzazione dei parametri:** ovvero, selezionare il modello migliore tra un insieme di modelli candidati o selezionare i migliori iperparametri per un dato algoritmo di apprendimento.
- **Stima delle prestazioni:** ovvero stimare le prestazioni del modello finale sulla base di dati non visibili.

Per capire quale è l'algoritmo migliore facciamo un esperimento, prendiamo il dataset, facciamo l'addestramento prima con un set di iperparametri poi con un altro set, valutiamo l'errore di generalizzazione in entrambi i casi e scegliamo quello con l'errore più basso.

Così facendo stiamo valutando basandoci sul test set per fare una scelta quindi bisogna fare **attenzione a non sovraadattare il modello** al set di test. Ciò significa che non dovremmo utilizzare il **set di test per ottimizzare gli iperparametri** del modello, poiché ciò porterebbe a una stima distorta dell'errore di generalizzazione del modello finale.

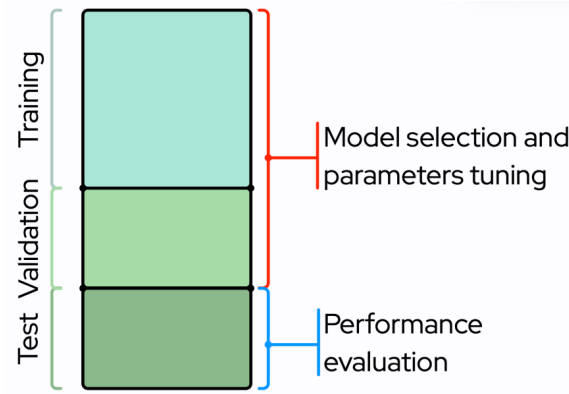


Figura 27: Divisione Dataset in 3 parti

Per non utilizzare il test set per decidere quali iperparametri utilizzare si divide il dataset in **3 parti**:

- **Training set**
- **Validation set**: una parte di esempi messa da parte per fare la stima degli **iperparametri**
- **Test set**: utilizzato alla fine di tutto quando il modello è stato già scelto per sapere quanto vale l'**errore di generalizzazione** del modello

Esempio

Supponiamo di voler selezionare i migliori iperparametri per un algoritmo di apprendimento automatico. Si dispone di un set di 1000 iperparametri tra cui scegliere e si desidera misurare l'errore utilizzando il set di test. Poiché abbiamo visto che questo potrebbe rappresentare un problema, teniamo da parte anche un set aggiuntivo di esempi, il set di validazione, per misurare l'errore sul classificatore scelto.

Poiché sia i set di validazione che quelli di test sono **campioni finiti**, la misurazione dell'errore su di essi introdurrà un po' di rumore nella stima dell'errore di generalizzazione, ovvero:

$$\hat{E}_{Te} = E + \epsilon_{Te}, \quad \hat{E}_{Val} = E + \epsilon_{Val}$$

Si scelgono quindi gli iperparametri che riducono al minimo l'errore sul set di convalida.

Domanda: alla fine del processo, i due errori $\hat{E}_{Te}$ e $\hat{E}_{Val}$ saranno ancora stimatori senza bias dell'errore di generalizzazione? In altri termini, $\mathbb{E}[\hat{E}_{Te}] = \mathbb{E}[\hat{E}_{Val}] = E$?

La media dello stimatore basata sul Validation set avrà un bias, invece quella basata sul Test set sarà senza bias.

Esempio

```
1 # 1000 hyper-parameters to set, each yielding an error in (0, 100)
2 means = np.random.uniform(0, 100, 1000)
3
4 valid_error_dist = []
5 test_error_dist = []
6
7 N = 5000
8 for i in range(N):
9     samples = np.array(
10         [
11             [
12                 m,                                #E
13                 m + np.random.uniform(-10, 10)    #E_val
14                 m + np.random.uniform(-10, 10)    #E_test
15             ]
16             for m in means
17         ]
18     )
19
20 #selects the best hp according to R_val
21 amin = np.argmin(samples[:, 1])
22
23 valid_error_dist.append(samples[amin, 1] - samples[amin, 0])
24 test_error_dist.append(samples[amin, 2] - samples[amin, 0])
```

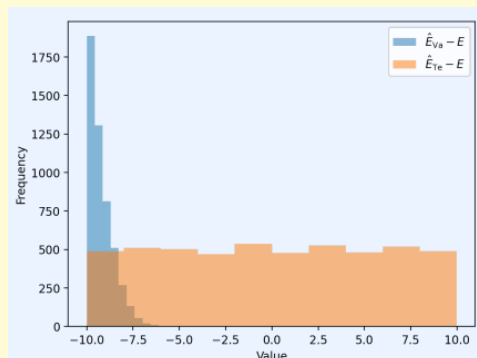


Figura 28: Valutazione esperimento

2.5 Misurazione delle performance

Diversi indicatori di performance riflettono le diverse esigenze dei compiti e producono risultati di valutazione diversi. La scelta dell'indicatore di performance corretto è fondamentale per una corretta valutazione del modello.

2.5.1 Mean Square Error (MSE)

L'**errore quadratico medio** è una misura di prestazione comune per le attività di regressione. In generale, se si conosce la **distribuzione dei dati** $\mathcal{D}$, l'errore quadratico medio di una funzione è definito come:

$$E(f; \mathcal{D}) = \mathbb{E}_{(X,y) \sim \mathcal{D}}[(f(X) - y)^2] = \int_{(X,y) \sim \mathcal{D}} (f(X) - y)^2 p(X, y) dX dy$$

Nella maggior parte dei casi, non conosciamo la distribuzione dei dati $\mathcal{D}$, ma disponiamo di un set di dati $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$. In questo caso, possiamo stimare l'errore quadratico medio come:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(X_i) - y_i)^2$$

2.5.2 Error rate

Analogamente all'errore quadratico medio, possiamo definire il **tasso di errore** e l'**accuratezza** per le attività di classificazione. La forma generale, supponendo di conoscere la distribuzione dei dati, è:

$$E(f; \mathcal{D}) = \mathbb{E}_{(X,y) \sim \mathcal{D}}[\mathbb{I}(f(X) \neq y)] = \int_{(X,y) \sim \mathcal{D}} \mathbb{I}(f(X) \neq y) p(X, y) dX dy$$

Dove $\mathbb{I}$ è la **funzione indicatrice** che restituisce 1 se la condizione è vera e 0 altrimenti.

Se disponiamo di un set di dati $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$, possiamo stimare il tasso di errore come:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(X_i) \neq y_i)$$

2.5.3 Accuracy

L'accuratezza è il **complementare dell'error rate**:

$$acc(f; \mathcal{D}) = 1 - E(f; \mathcal{D}) = 1 - \mathbb{E}_{(X,y) \sim \mathcal{D}}[\mathbb{I}(f(X) \neq y)]$$

o, considerando un dataset $D = \{(X_i, y_i)\}_{i=1}^m$ di m campioni estratti da $\mathcal{D}$:

$$acc(f; D) = 1 - E(f; D) = 1 - \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(X_i) \neq y_i)$$

2.5.4 Matrice di confusione

È possibile definire molte altre misure di performance, a seconda del compito e dei requisiti specifici. Per le attività di classificazione, molte di queste misure possono essere derivate dalla **matrice di confusione**.

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Figura 29: Matrice di confusione

con:

- **TP: True Positive** (numero degli esempi che sono stati predetti positivi ed erano veramente positivi)
- **FP: False Positive** (predetti positivi ma in realtà negativi)
- **TN: True Negative** (predetti negativi e veramente negativi)
- **FN: False Negative** (predetti negativi ma in realtà positivi)

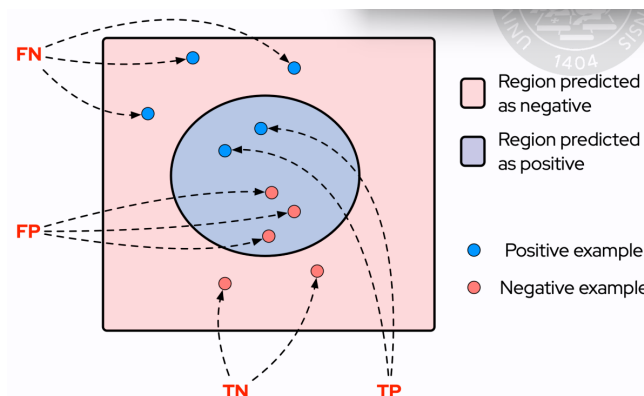


Figura 30: Esempi positivi e negativi per la matrice di confusione

2.5.5 Precision, Recall

A volte, gli **errori** non sono **ugualmente importanti**. Ad esempio, in un'attività di diagnosi di cancro, un falso negativo (mancanza di una malattia) può essere più critico di un falso positivo (diagnosi errata di una malattia).

In questo esempio, potremmo voler dare **priorità al richiamo** (recall) (la capacità di identificare tutti i casi di cancro) rispetto alla **precisione** (la capacità di evitare false diagnosi di cancro).

In un'attività di rilevamento dello spam, d'altra parte, potremmo voler dare **priorità alla precisione** (per evitare di etichettare erroneamente email legittime come spam) rispetto al **richiamo** (recall) (per identificare tutti i messaggi di spam).

Più formalmente, la **precisione** è definita come:

$$precision = \frac{TP}{TP + FP} = \frac{TP}{\#predicted\ positives}$$

e il **richiamo** (recall) è definito come:

$$recall = \frac{TP}{TP + FN} = \frac{TP}{\#actual\ positives}$$

Precision e **recall** vengono spesso utilizzati insieme, poiché forniscono informazioni complementari sulle prestazioni del modello. **Di solito sono contraddittorie**, il che significa che l'aumento di una spesso porta a una diminuzione dell'altra.

Ad esempio, se iniziamo a classificare **più campioni come positivi**, aumenteremo il numero di veri positivi, ma anche il numero di falsi positivi, **diminuendo** così la **precisione**, ma **aumentando** il **recall**.

D'altra parte, se iniziamo a classificare **meno campioni come positivi**, diminuiranno il numero di falsi positivi, **aumentando** così la **precisione**, ma anche il numero di falsi negativi, **diminuendo** così il **recall**.

2.5.6 P-R Plots

Supponiamo di avere un classificatore che fornisca un **punteggio di confidenza** per ciascun campione, indicando quanto è sicuro che il campione appartenga alla classe positiva. È quindi possibile variare una **soglia** su questo punteggio per ottenere diversi valori di precision e recall.

Esempio

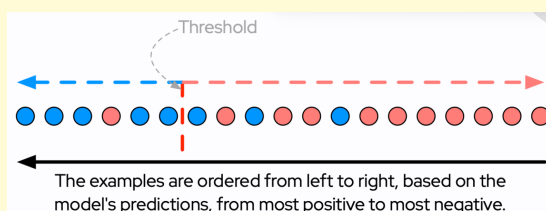


Figura 31: Soglia di esempio
questo esempio ho $FP = 1$, $FN = 3$, $TP = 5$, $TN = 10$

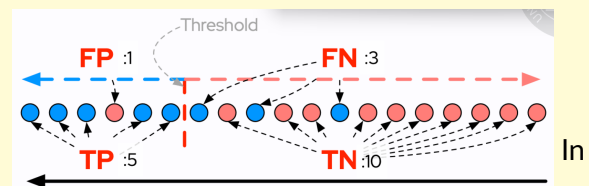


Figura 32: Soglia di esempio con valori della
matrice di confusione

$$precision = \frac{TN}{TP + FP} = \frac{5}{6}$$

$$recall = \frac{TP}{TP + FN} = \frac{5}{8}$$

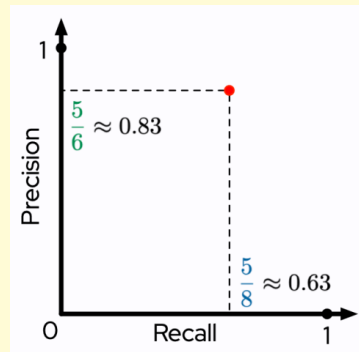


Figura 33: Precisione e recall dell'esempio

Cambiando la posizione della soglia otterrò:

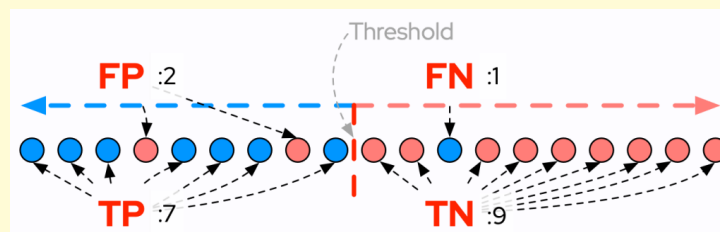


Figura 34: Soglia spostata

$$precision = \frac{TN}{TP + FP} = \frac{7}{9}$$

$$recall = \frac{TP}{TP + FN} = \frac{7}{8}$$

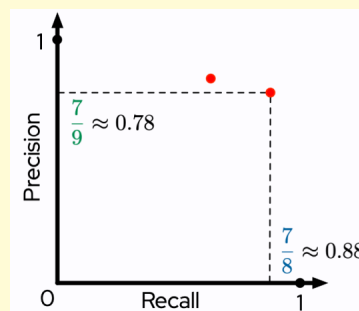


Figura 35: Precisione e recall dell'esempio con soglia spostata

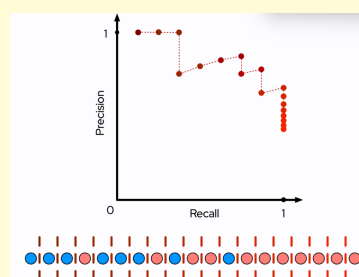


Figura 36: Precisione e recall per tutte le soglie

Dovendo confrontare più classificatori che restituiscono una soglia di confidenza ottengo un grafico del tipo:

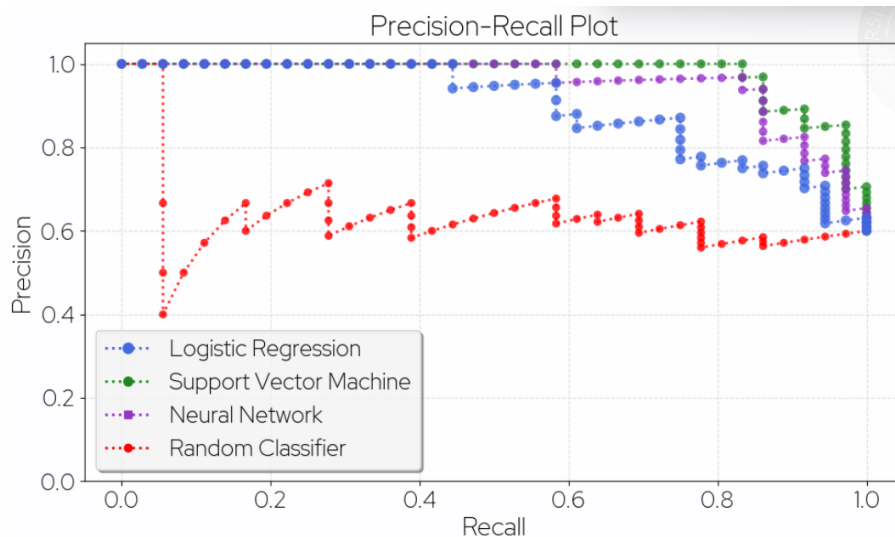


Figura 37: Precision-Recall Plot

In particolare si può vedere che le **Support Vector Machine** (in verde) dominano tutti gli altri classificatori, se dovessi scegliere un classificatore sceglierei questo perché a parità di threshold vince su tutti gli altri.

Come selezioniamo gli studenti migliori date alcune curve precisione-richiamo?

- **Fissare un dato punto di precision** o recall e selezionare il modello che raggiunge il miglior richiamo o precisione in quel punto.
- Selezionare la curva con l'**area sotto la curva (AUC)** più alta (n.b. difficile da calcolare).
- Selezionare il **miglior punto di break-even**, ovvero il punto in cui precision e recall sono uguali.

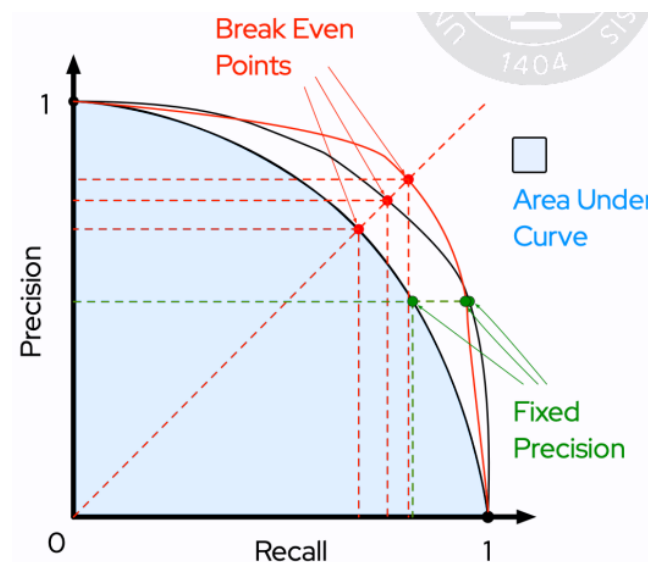


Figura 38: Curva P-R e break-even point

F1-score Una misura importante che combina precision e recall è il **punteggio F1**, definito come la **media armonica** di precision e recall:

$$F1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = \frac{2 \times precision \times recall}{precision + recall} = \frac{2 \times TP}{TOT + FP + FN}$$

o in forma più generale (F_β -score):

$$F_\beta = \frac{1 + \beta^2}{\frac{1}{precision} + \frac{\beta^2}{recall}} = \frac{(1 + \beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$$

Media armonica ponderata

La media armonica ponderata di due valori a e b è definita come:

$$\frac{(w_a + w_b)}{\frac{w_a}{a} + \frac{w_b}{b}}$$

Nota

La F-misura definita da F_β misura quanto sono bravo a recuperare gli esempi rispetto ad un utente che pensa che la recall sia β volte più importante della precision.

Supponiamo di voler **calcolare la media** dei contributi di **diverse matrici di confusione** (ad esempio, perché ci troviamo in un problema multi-classe, o vogliamo combinare i risultati di diversi round di convalida incrociata). Indichiamo con P e R le statistiche di **precision** e **recall**.

Questo può essere fatto in due modi:

1. **Macro-averaging:** calcolo la Precision e la Recall per ogni matrice di confusione, denotate come $(P_1, R_1), (P_2, R_2), \dots, (P_n, R_n)$. Prendendo la loro media abbiamo la precision, recall e F_1 macro-averaged:

$$macro-P = \frac{1}{n} \sum_{i=1}^n P_i$$

$$macro-R = \frac{1}{n} \sum_{i=1}^n R_i$$

$$macro-F_1 = \frac{2 \times macro-P \times macro-R}{macro-P + macro-R}$$

2. **Micro-averaging:** calcolare le medie elemento per elemento nelle matrici di confusione, per ottenere $\overline{TP}$, $\overline{FP}$, $\overline{TN}$, $\overline{FN}$, e quindi calcola la micro precision, recall e F_1 come:

$$micro-P = \frac{\overline{TP}}{\overline{TP} + \overline{FP}}$$

$$micro-R = \frac{\overline{TP}}{\overline{TP} + \overline{FN}}$$

$$micro-F_1 = \frac{2 \times micro-P \times micro-R}{micro-P + micro-R}$$

- **Macro-averaging** tratta tutte le matrici di confusione allo stesso modo, indipendentemente dalle loro dimensioni. Questo significa che matrici di confusione di piccole dimensioni possono avere un impatto significativo sul risultato finale.
- **Micro-averaging** attribuisce maggiore peso alle matrici di confusione più grandi, poiché contribuiscono maggiormente al conteggio complessivo di TP, FP, TN e FN. Ciò significa che il risultato finale è più rappresentativo della performance complessiva di tutte le matrici di confusione.

Ciò è importante, ad esempio, nei problemi di classificazione multiclasse in cui le classi possono essere sbilanciate.

Esempio $Macro-F_1$

Assumendo di avere tre matrici di confusione:

$$CM_1 = \begin{bmatrix} 10 & 2 \\ 1 & 7 \end{bmatrix} \quad CM_2 = \begin{bmatrix} 8 & 3 \\ 2 & 6 \end{bmatrix} \quad CM_3 = \begin{bmatrix} 9 & 1 \\ 1 & 8 \end{bmatrix}$$

CM	precision	recall	F1-score
CM ₁	0.9091	0.8333	0.8695
CM ₂	0.8000	0.7273	0.7619
CM ₃	0.9000	0.9000	0.9000
Avg	0.8697	0.8202	0.8438

Figura 39: Esempio precision, recall e F1-score

$$macro-F_1 = \frac{2 \times macro-P \times macro-R}{macro-P + macro-R} = \frac{2 \times 0.8697 \times 0.8202}{0.8697 + 0.8202} \approx 0.8442$$

Esempio $Micro-F_1$

Con le stesse matrici di confusione di prima:

$$CM_1 = \begin{bmatrix} 10 & 2 \\ 1 & 7 \end{bmatrix} \quad CM_2 = \begin{bmatrix} 8 & 3 \\ 2 & 6 \end{bmatrix} \quad CM_3 = \begin{bmatrix} 9 & 1 \\ 1 & 8 \end{bmatrix} \quad \Rightarrow \quad \overline{CM} = \begin{bmatrix} 9 & 2 \\ 1.3333 & 7 \end{bmatrix}$$

La precision e recall micro-averaged sono:

$$micro-P = \frac{9}{9 + 1.3333} = 0.8710 \quad micro-R = \frac{9}{9 + 2} = 0.8182$$

La F_1 micro-averaged è:

$$micro-F_1 = \frac{2 \times micro-P \times micro-R}{micro-P + micro-R} = \frac{2 \times 0.8710 \times 0.8182}{0.8710 + 0.8182} = 0.8438$$

2.5.7 ROC e AUC

Se, invece di utilizzare precision e recall, tracciamo il **false positive rate (FPR)** rispetto al **true positive rate (TPR)**, otteniamo un grafico **ROC** (Receiver Operating Characteristic curve).

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{\#actual\ negative}$$

$$TPR = \frac{TP}{TP + FN} = \frac{TP}{\#actual\ positives}$$

Come nei grafici P-R, diciamo che un learner A è migliore di un learner B se la curva ROC di A racchiude interamente la curva ROC di B . Tuttavia, quando esistono intersezioni, nessun learner è generalmente migliore dell'altro.

Un modo per confrontare le curve ROC intersecate è calcolare le **aree sotto le curve ROC**.

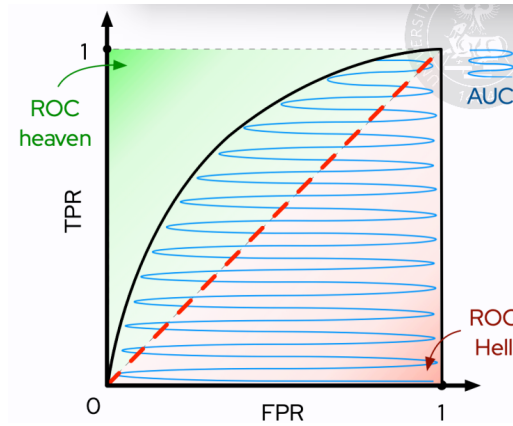


Figura 40: Illustrazione delle curve ROC e AUC

In alto a sinistra (**ROC heaven**) troviamo classificatori che **non sbagliano mai**, in basso a destra (**ROC Hell**) troviamo classificatori che **sbagliano sempre** e al centro sono presenti i peggiori classificatori perché rispondono sempre a caso. Noi **vogliamo stare il più lontano possibile dalla diagonale**.

Area Under the Curve (AUC) Nel caso delle curve ROC, l'area sotto la curva (AUC) è facile da calcolare (anche per le curve teoriche, poiché è sempre non decrescente).

Si può anche dimostrare che è correlata all'errore di ranking del classificatore.

$$1 - AUC = \ell_{rank} \triangleq \frac{1}{m^+ m^-} \sum_{X^+ \in D^+} \sum_{X^- \in D^-} \mathbb{I}[f(X^+) < f(X^-)] + \frac{1}{2} \mathbb{I}[f(X^+) = f(X^-)]$$

Somma su tutti gli esempi positivi, somma su tutti gli esempi negativi calcoliamo un punto di errore se il ranking dell'esempio positivo è minore del ranking dell'esempio negativo, abbiamo anche un altro termine perché possiamo avere dei casi in cui c'è un pareggio in questo caso assegnano mezzo punto di errore. Facciamo la somma di tutto e per normalizzarlo dividiamo tutto per il numero di esempi positivi moltiplicata per il numero di esempi negativi.

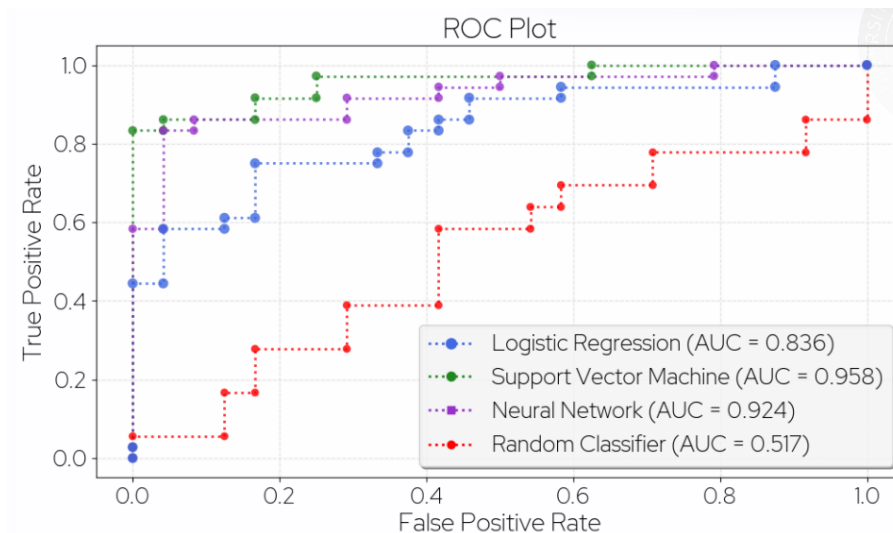


Figura 41: ROC plot

```

1 # ...
2 # Assume X_train, X_test, y_train, y_test are defined
3
4 clf = LogisticRegression()
5 clf.fit(X_train, y_train)
6
7 y_scores = clf.predict_proba(X_test)[:, 1]

```

```

8
9 # Compute ROC curve and AUC
10 fpr, tpr, _ = roc_curve(y_test, y_scores)
11 roc_auc = auc(fpr, tpr)
12
13 # You can easily plot the ROC curve using matplotlib
14 plt.plot(fpr, tpr)
15
16 # Alternatively, you can directly use roc_auc_score
17 roc_auc2 = roc_auc_score(y_test, y_scores)
18
19 # For Precision-Recall, we don't usually compute
20 # AUC, but we can compute the average precision score
21 avg_precision = average_precision_score(y_test, y_scores)

```

2.6 Cost-Sensitive Error rates

A volte si potrebbe voler adattare l'accuratezza (o l'errore) in modo che siano **sensibili** ai diversi costi di errore per FP e FN . Si consideri $cost_{xy}$ il costo di una classificazione errata x come y .

Per un compito di classificazione binaria potremmo avere la matrice $cost$:

	Predicted as 0	Predicted as 1
Actual 0	0	$cost_{01}$
Actual 1	$cost_{10}$	0

Figura 42: Matrice dei costi della classificazione binaria

Quindi, supponendo che la classe sia positiva, si può definire il **tasso di errore sensibile ai costi** come:

$$E(f; \mathcal{D}; cost) = \frac{1}{m} \left(\sum_{(X,y) \in \mathcal{D}^+} \mathbb{I}[f(X) \neq y] \times cost_{01} + \sum_{(X,y) \in \mathcal{D}^-} \mathbb{I}[f(X) \neq y] \times cost_{10} \right)$$

Allo stesso modo, possiamo definire il tasso di errore e l'accuratezza basati sulla distribuzione.

2.7 Comparison Test

Confrontare i risultati di diversi learner **non** è semplice per diversi motivi:

- Vogliamo valutare le **prestazioni di generalizzazione**, ma abbiamo accesso solo a un **set di test finito**.
- Le prestazioni del modello possono variare tra **diversi set di test**, anche se hanno le stesse dimensioni.
- La **casualità** intrinseca di molti algoritmi di apprendimento significa che possono produrre modelli diversi se addestrati più volte sugli stessi dati.

L'**Hypothesis testing** consente un rigoroso confronto statistico tra diversi modelli.

2.7.1 Test binomiale

Dato un modello f con un **tasso di errore reale sconosciuto** ϵ , vogliamo verificare se il modello ha prestazioni significativamente migliori rispetto a una determinata soglia di tasso di errore $\epsilon_0 \in [0, 1]$ e ipotizziamo di osservare un **tasso di errore empirico** $\hat{\epsilon}$ su un set di test di dimensioni m tali da consentire un modello f .

Formalizziamo questo come un test di ipotesi:

- **Null hypothesis** (il modello non è migliore di ϵ_0):

$$H_0 : \epsilon \geq \epsilon_0$$

- **Alternative hypothesis** (il modello è migliore di ϵ_0):

$$H_1 : \epsilon < \epsilon_0$$

La difficoltà di tutto questo sta nel fatto che ϵ è sconosciuto, quindi non possiamo calcolare direttamente la probabilità di osservare un tasso di errore empirico $\hat{\epsilon}$ dato un tasso di errore reale ϵ . Poter assumere l'**ipotesi nulla** ci permette di fare dei ragionamenti probabilistici. L'**ipotesi alternativa** è quella che vogliamo dimostrare.

Sia E la variabile casuale che conta il numero di errori fatti dal modello sul test set. Sotto l'ipotesi nulla H_0 , E segue una distribuzione binomiale con parametri m e ϵ_0 : $E \sim \text{Bin}(m, \epsilon_0)$

Nota

Di seguito, indicheremo con $\hat{\epsilon}$ il tasso di errore osservato e con $\hat{e} = \hat{\epsilon} \cdot m$ il numero di errori.

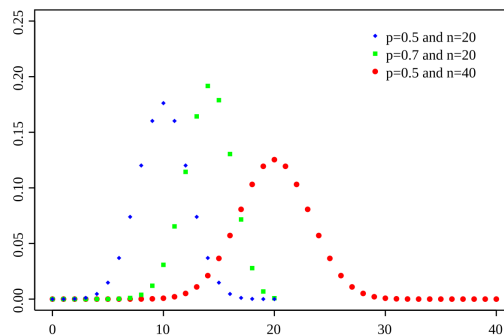


Figura 43: Distribuzione binomiale

Decision Rule

Rifiutare l'ipotesi nulla H_0 a **livello di significatività** α (o con **confidenza** $1 - \alpha$) se:

$$P(E \leq \hat{e} \mid \epsilon = \epsilon_0) < \alpha$$

In altre parole, se è **improbabile** (sotto l'ipotesi nulla H_0) osservare un numero di errori pari o inferiore a $\hat{e}$, allora concludiamo che il modello **funziona meglio** di ϵ_0 .

In caso contrario, **non rifiutiamo** H_0 .

P-value

Il **p-value** di un test statistico è la probabilità di **osservare** una statistica del test **altrettanto estrema o più estrema di quella effettivamente osservata**, supponendo che l'ipotesi nulla sia vera.

Il **p-value**, nel nostro caso, è definito come:

$$p\text{-value} = P(E \leq \hat{e} \mid \epsilon = \epsilon_0) = \sum_{k=0}^{\hat{e}} \binom{m}{k} \epsilon_0^k (1 - \epsilon_0)^{m-k}$$

La probabilità di osservare esattamente k successi su m esperimenti.

Se il p-value è **minore** di α , il risultato è considerato statisticamente significativo e si rifiuta l'ipotesi nulla H_0 .

Esempio

Assumiamo di avere un modello che mostra un tasso di errore del 20%: $\hat{\epsilon} = 0.2$ su un set di test di dimensione $m = 100$, cioè compie $\hat{\epsilon} \times 100 = 20$ errori sul set di test. Vogliamo verificare l'ipotesi che il vero tasso di errore è significativamente minore di 0.3.

L'ipotesi nulla è:

$$H_0 : \epsilon \geq 0.3$$

L'ipotesi alternativa è:

$$H_1 : \epsilon < 0.3$$

Sotto l'ipotesi nulla, la probabilità di osservare $\hat{\epsilon}$ errori è data da:

$$P(\hat{\epsilon}; \epsilon_0) = \binom{m}{\hat{\epsilon}} \epsilon_0^{\hat{\epsilon}} (1 - \epsilon_0)^{m - \hat{\epsilon}}$$

Il p-value del test è data da:

$$P(e \leq \hat{e} \mid \epsilon = 0.3) = \sum_{k=0}^{20} \binom{100}{k} (0.3)^k (1 - 0.3)^{100-k} = 0.0165$$

Se **impostiamo il livello di significatività** a $\alpha = 0.05$, possiamo rifiutare l'ipotesi nulla H_0 fino a quando $0.0165 < 0.05$. Questo significa che abbiamo abbastanza prove per concludere che il modello performa significativamente meglio di un tasso di errore del 30%.

Fissato un **livello di confidenza** α , in rosso nella figura è mostrato la barra corrispondente alla coda sinistra della distribuzione binomiale tale che la probabilità cumulativa è minore o uguale a α .

Poiché $\hat{\epsilon}$ cade nella regione, **possiamo rifiutare l'ipotesi nulla**.

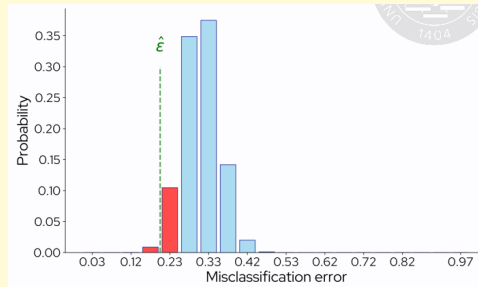


Figura 44: Esempio a livello di confidenza $\alpha = 0.05$

Nel caso in cui volessimo ripetere il test molteplici volte, vorremmo calcolare il tasso di errore massimo $\bar{\epsilon}$ che possiamo osservare per rifiutare ancora l'ipotesi nulla.

Questo valore è chiamato **valore critico** e, per il test binomiale, è definito come il più grande valore di $\hat{\epsilon}$ tale che il p-value è minore o uguale a α :

$$\bar{\epsilon} = \max_{\epsilon} \epsilon \quad \text{subject to} \quad \sum_{i=0}^{\lfloor \epsilon \times m \rfloor} \binom{m}{i} \epsilon^i (1 - \epsilon_0)^{m-i} \leq \alpha$$

2.7.2 Test di Student

Spesso otteniamo più tassi di errore nei test da cross-validation o da hold-out validation ripetuti. Sia $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_n$ i tassi di errore osservati in k ripetizioni indipendenti del test. Possiamo facilmente calcolare la **media** e la **deviazione standard** dei tassi di errore osservati:

$$\mu = \frac{1}{k} \sum_{i=1}^k \hat{\epsilon}_i$$

$$\sigma^2 = \frac{1}{k-1} \sum_{i=1}^k (\hat{\epsilon}_i - \mu)^2$$

Nota

Per essere rigorosi, l'hold-out ripetuto e (soprattutto) la cross-validation non sono indipendenti, poiché i campioni sono estratti dallo stesso set di dati. Tuttavia, se il set di dati è sufficientemente ampio, la dipendenza è trascurabile e la conclusione rimane valida.

Possiamo considerare questi valori come k campioni indipendenti e identicamente distribuiti (i.i.d.) da una distribuzione normale con media μ e varianza σ^2 e quindi la variabile:

$$\tau = \sqrt{k} \frac{\mu - \epsilon_0}{\sigma}$$

è distribuita come una distribuzione di Student con $k - 1$ gradi di libertà. La distribuzione t di Student è una famiglia di distribuzioni parametrizzata dal valore k che è chiamato **gradi di libertà** (degrees of freedom, df) della distribuzione. Un parametro da fissare per dire quale distribuzione voglio usare.

Il **test di Student** è un test statistico utilizzato per determinare se le medie di due gruppi sono significativamente diverse tra loro. Nel nostro caso, possiamo utilizzarlo per verificare se il tasso di errore medio μ è significativamente diverso da una determinata soglia di tasso di errore ϵ_0 .

L'ipotesi nulla è:

$$H_0 : \mu = \epsilon_0$$

L'ipotesi alternativa è:

$$H_1 : \mu \neq \epsilon_0$$

La statistica del test è data da:

$$\tau = \sqrt{k} \frac{\mu - \epsilon_0}{\sigma}$$

Possiamo **rifutare l'ipotesi nulla** a un **livello di significatività** α se τ è **al di fuori** dell'intervallo $[-t_{\alpha/2}, t_{\alpha/2}]$, dove t_z è il valore critico della distribuzione di Student con $k - 1$ gradi di libertà a livello di significatività z . t_z lo ottengo da delle tabelle precompilate e controllo che il mio τ sia fuori dall'intervallo.

Nelle formule, **rifiuta l'ipotesi nulla se**:

$$\tau \notin [-t_{\alpha/2}, t_{\alpha/2}]$$

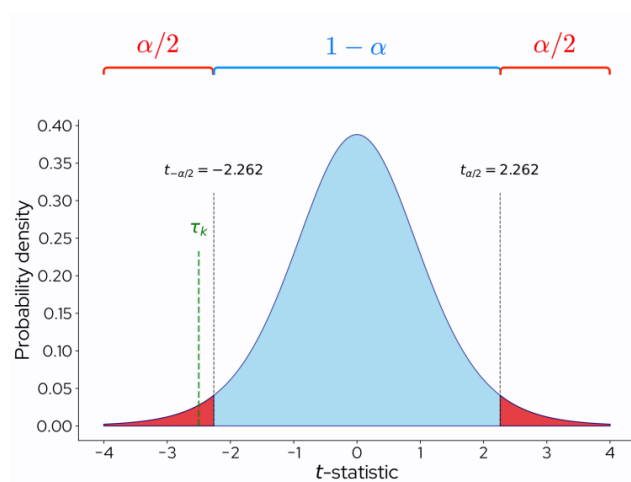


Figura 45: Distribuzione t-student

Valori critici comunemente utilizzati

df (k-1)	$\alpha = 0.10$	$\alpha = 0.05$	$\alpha = 0.025$	$\alpha = 0.01$
1	3.078	6.314	12.706	31.821
2	1.886	2.920	4.303	6.965
3	1.638	2.353	3.182	4.541
4	1.533	2.132	2.776	3.747
5	1.476	2.015	2.571	3.365
6	1.440	1.943	2.447	3.143
7	1.415	1.895	2.365	2.998
8	1.397	1.860	2.306	2.896
9	1.383	1.833	2.262	2.821
10	1.372	1.812	2.228	2.764
11	1.363	1.796	2.201	2.718
12	1.356	1.782	2.179	2.681
13	1.350	1.771	2.160	2.650
14	1.345	1.761	2.145	2.624
15	1.341	1.753	2.131	2.602
16	1.337	1.746	2.120	2.583
17	1.333	1.740	2.110	2.567
18	1.330	1.734	2.101	2.552
19	1.328	1.729	2.093	2.539
20	1.325	1.725	2.086	2.528
25	1.316	1.708	2.060	2.485
30	1.310	1.697	2.042	2.457
∞	1.282	1.645	1.960	2.326

Figura 46: Valori critici comuni per la distribuzione t di Student

Il libro riporta i valori critici usand k come numero di campioni (qui sono riportati i gradi di libertà), e α è già diviso per 2. Quindi, la voce per $k = 5$ e $\alpha = 0.05$ sul libro corrisponde alla voce per $df = 4$ e $\alpha = 0.025$ nella tabella.

```

1 from scipy.stats import t
2 import numpy as np
3
4 # Assume we have k error rates in a list
5 error_rates = [0.2, 0.18, 0.22, 0.19, 0.21]
6 k = len(error_rates)
7 mu = np.mean(error_rates)
8 sigma = np.std(error_rates, ddof=1) # ddof=1 for sample standard deviation
9
10 alpha = 0.05
11
12 # ppf == percent point function, i.e., the inverse of the cumulative
13   distribution function (CDF)
14 t_crit_0 = t.ppf(alpha/2, k-1) # negative crit for left tail
15 t_crit_1 = t.ppf(1 - alpha/2, k-1) # positive critical value for right tail
16
17 tau_k = no.sqrt(k) * (mu - 0.3) / sigma
18
19 if(tau_k < t_crit_0 or tau_k > t_crit_1):
20     print("Reject null hypothesis H0")
21 else:
22     print("Do not reject null hypothesis H0")
23
24 # Alternatively, you can use the ttest_1samp function from scipy.stats
25 from scipy.stats import ttest_1samp
26 t_stat, p_value = ttest_1samp(error_rates, 0.3)
27 if p_value < alpha:
28     print("Reject null hypothesis H0")
29 else:
30     print("Do not reject null hypothesis H0")

```

2.7.3 Cross-Validated Student's t-test

Assumiamo di avere due algoritmi di apprendimento A e B , e denotiamo con $\epsilon_1^A, \dots, \epsilon_k^A$ e $\epsilon_1^B, \dots, \epsilon_k^B$ i tassi di errore ottenuti da **k-fold cross validation**.

Se vogliamo confrontare i due algoritmi, possiamo usare il **k-fold cross-validation Student's t-test**.

L'idea è che, se le performance dei due algoritmi sono le stesse, allora i tassi di errore nei test dovrebbero essere simili nelle stesse suddivisioni training/test: $\epsilon_i^A \approx \epsilon_i^B$ per tutte le $i = 1, \dots, k$.

Il test richiede di calcolare $\Delta_i = \epsilon_i^A - \epsilon_i^B$ per $i = 1, \dots, k$, e quindi calcolare la media μ e la deviazione standard σ della differenza Δ_i .

Note

Nota 1: si presume che i tassi di errore siano indipendenti, il che non è vero nella pratica, ma l'ipotesi è ragionevole se il set di dati è sufficientemente ampio.

Nota 2: la tecnica può essere utilizzata anche con validation hold-out accoppiate (vale a dire, le stesse suddivisioni di formazione/test vengono utilizzate per entrambi gli algoritmi).

Ipotesi nulla: i due algoritmi hanno le stesse prestazioni, cioè, $\mu = 0$.

Ipotesi alternativa: i due algoritmi hanno prestazioni diverse, cioè, $\mu \neq 0$.

La statistica del test è data da:

$$\tau = \frac{\sqrt{k} \cdot \mu}{\sigma}$$

Regola di decisione: rifiutare l'ipotesi nulla H_0 a un livello di significatività α se:

$$\tau \notin [-t_{\alpha/2}, t_{\alpha/2}]$$

dove $t_{\frac{\alpha}{2}}$ è il valore critico della distribuzione di Student con $k - 1$ gradi di libertà a livello di significatività $\frac{\alpha}{2}$.

2.7.4 5x2 Cross-Validated t-test

Come notato, l'ipotesi di indipendenza è violata nel caso di CV k-fold. Per risolvere questo problema, è possibile utilizzare metodi più robusti come il **5x2 cross-validated t-test**.

La procedura è la seguente:

1. Eseguire 2-fold cross-validation. Siano $\Delta_{1,1}$ e $\Delta_{1,2}$ le differenze di errore tra i due algoritmi nei due fold.
2. Ripetere questo processo 4 altre volte, ogni volta con una diversa suddivisione dei dati, ottenendo così un totale di 5 paia di differenze in totale. Denotiamo le differenze di errore come $\Delta_{i,1}$ e $\Delta_{i,2}$ per $i = 1, \dots, 5$.
3. Per ognuna delle 5 ripetizioni, calcoliamo la varianza delle due differenze come:

$$\sigma_i^2 = (\Delta_{i,1} - \bar{\Delta}_i)^2 + (\Delta_{i,2} - \bar{\Delta}_i)^2$$

$$\bar{\Delta}_i = \frac{\Delta_{i,1} + \Delta_{i,2}}{2}$$

4. Il test statistico è calcolato usando la differenza tra i due fold della prima ripetizione e la media delle varianze calcolate nelle 5 ripetizioni:

$$\tau = \frac{\Delta_{1,1}}{\sqrt{\frac{1}{5} \sum_{i=1}^5 \sigma_i^2}}$$

5. Sotto l'ipotesi nulla, questa statistica è distribuita secondo una **distribuzione di Student con 5 gradi di libertà**. La regola di decisione è la stessa di prima, ma usando il valore critico $t_{\alpha/2}$ con 5 gradi di libertà.

Nota

Il libro di testo suggerisce di usare la media $0.5(\Delta_{i,1} + \Delta_{i,2})$ al posto di $\Delta_{i,1}$ nella formula per τ andando in contrasto con *original paper by Dietterich*.

```

1 from scipy.stats import t
2
3 # ...
4
5 # Assume errors_A and errors_B are lists of error rates for learners A and B
  # obtained from k-fold cross-validation
6
7 k = len(errors_A)
8 deltas = np.array(errors_A) - np.array(errors_B)
9 mu = np.mean(deltas)
10 sigma = np.std(deltas, ddof=1) # Sample standard deviation
11
12 tau = np.sqrt(k) * mu / sigma
13 t_crit = t.ppf(1 - alpha/2, k-1) # critical value for two-tailed test
14
15 # Decision rule
16 if abs(tau) > t_crit:
17     print("Reject null hypothesis H0: learners A and B have different
18           performance")
19 else:
20     print("Do not reject null hypothesis H0: learners A and B have similar
21           performance")

```

2.7.5 McNemar's Test

Assumiamo di avere due classificatori, A e B , e vogliamo confrontare le loro prestazioni usando il metodo hold-out. Addestriamo entrambi i classificatori sullo stesso set di addestramento e li valutiamo sullo stesso set di test, così facendo possiamo ottenere la seguente tabella di contingenza:

	Classifier A Correct	Classifier A Incorrect
Classifier B Correct	e_{11}	e_{10}
Classifier B Incorrect	e_{01}	e_{00}

Figura 47: La tabella di contingenza di due learners

L'idea è che, sotto l'**ipotesi nulla** che i due classificatori abbiano le stesse prestazioni, ci aspettiamo che il **numero di errori fatti da entrambi i classificatori sia simile**, cioè:

$$e_{01} \approx e_{10}$$

Il McNemar's test considera la variabile:

$$\tau_{\chi^2} = \frac{(|e_{10} - e_{01}| - 1)^2}{e_{10} + e_{01}}$$

che segue una distribuzione χ^2 con 1 grado di libertà sotto l'ipotesi nulla.

Regola di decisione: rifiutare l'ipotesi nulla H_0 a un livello di significatività α se:

$$\tau_{\chi^2} > \chi_{\alpha,1}^2$$

dove $\chi^2_{\alpha,1}$ è il valore critico della distribuzione χ^2 con 1 grado di libertà a livello di significatività α .
 Tipicamente i valori critici più comuni sono:

$\alpha = 0.10$	$\alpha = 0.05$	$\alpha = 0.01$
2.706	3.841	6.635

Figura 48: Valori critici comuni per la distribuzione χ^2 con 1 grado di libertà

2.7.6 Friedman Test

Entrambi i test di Student e McNemar sono progettati per confrontare due algoritmi di apprendimento su un singolo dataset. Tuttavia, in pratica, spesso vogliamo **confrontare più di due algoritmi su più dataset**.

In questo caso, possiamo confrontare ciascuna coppia di algoritmi utilizzando il test di Student o il test di McNemar, ma questo può essere **computazionalmente costoso** e può comportare un aumento del rischio di errori di tipo I (falsi positivi).

Il **test di Friedman** è un test di ranking che consente di confrontare più algoritmi su più set di dati utilizzando un singolo test. Se l'ipotesi nulla viene rifiutata, è necessario utilizzare un **test post-hoc** (come il test di Nemenyi) per identificare le coppie specifiche di algoritmi che presentano prestazioni **significativamente diverse**.

Assumendo di comparare gli algoritmi A , B , e C sui dataset D_1 , D_2 , D_3 e D_4 . Iniziamo usando la **validazione hold-out o la cross-validation** per **ottenere i tassi di errore** per ogni algoritmo su ogni dataset, poi **ordiniamo gli algoritmi** per ogni dataset in base al loro tasso di errore (il più basso ottiene il rango 1, il secondo più basso ottiene il rango 2, e così via), per ogni set di dati in cui agli algoritmi con lo stesso tasso di errore viene assegnato il rango medio.

Esempio

Supponiamo di avere i seguenti tassi di errore per i tre algoritmi sui quattro set di dati:

Error rates:

Dataset	Algorithm A	Algorithm B	Algorithm C
D_1	0.2	0.3	0.4
D_2	0.1	0.2	0.2
D_3	0.05	0.1	0.2
D_4	0.3	0.4	0.5

Figura 49: Tassi di errore per i tre algoritmi sui quattro set di dati

Ranks:

Dataset	Algorithm A	Algorithm B	Algorithm C
D_1	1	2	3
D_2	1	2.5	2.5
D_3	1	2	3
D_4	1	2	3
Averages	1	2.125	2.875

Figura 50: Ranghi assegnati agli algoritmi per ogni set di dati

Sia k il **numero di algoritmi**, N il numero di dataset, e r_i il rango medio dell'algoritmo i su tutti i dataset. Per calcolare la statistica del test di Friedman, partiamo a calcolare la formula:

$$\tau_{\chi^2} = \frac{12N}{k(k+1)} \left(\sum_{i=1}^k r_i^2 - \frac{k(k+1)^2}{4} \right)$$

Il test statistico segue una distribuzione χ^2 con $k - 1$ gradi di libertà sotto l'ipotesi nulla che tutti gli algoritmi abbiano le stesse prestazioni.

Tuttavia, questa statistica **può essere utilizzata solo quando è di k grandi dimensioni**. Normalmente, la statistica viene corretta utilizzando la **statistica corretta di Iman e Davenport**:

$$\tau_F = \frac{(N - 1) \cdot \tau_{\chi^2}}{N(k - 1) - \tau_{\chi^2}}$$

Questa statistica corretta segue una distribuzione F con $k - 1$ e $(k - 1)(N - 1)$ gradi di libertà sotto l'ipotesi nulla.

Critical values for the F-test at $\alpha = 0.05$									
N	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = 9$	$k = 10$
4	10.128	5.143	3.863	3.259	2.901	2.661	2.488	2.355	2.250
5	7.709	4.459	3.490	3.007	2.711	2.508	2.359	2.244	2.153
8	5.591	3.739	3.072	2.714	2.485	2.324	2.203	2.109	2.032
10	5.117	3.555	2.960	2.634	2.422	2.272	2.159	2.070	1.998
15	4.600	3.340	2.827	2.537	2.346	2.209	2.104	2.022	1.955
20	4.381	3.245	2.766	2.492	2.310	2.179	2.079	2.000	1.935

Critical values for the F-test at $\alpha = 0.1$									
N	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = 9$	$k = 10$
4	5.538	3.463	2.813	2.480	2.273	2.130	2.023	1.940	1.874
5	4.545	3.113	2.606	2.333	2.158	2.035	1.943	1.870	1.811
8	3.589	2.726	2.365	2.157	2.019	1.919	1.843	1.782	1.733
10	3.360	2.624	2.299	2.108	1.980	1.886	1.814	1.757	1.710
15	3.102	2.503	2.219	2.048	1.931	1.845	1.779	1.726	1.682
20	2.990	2.448	2.182	2.020	1.909	1.826	1.762	1.711	1.668

Figura 51: Valori critici per F-test ad $\alpha = 0.05$ e $\alpha = 0.1$

2.7.7 Nemenyi Post-hoc Test

Se l'ipotesi nulla del test di Friedman viene rifiutata, possiamo procedere con un test post-hoc per identificare quali coppie di algoritmi hanno prestazioni significativamente diverse. Il **test di Nemenyi** è una scelta comune a questo scopo.

Il test di Nemenyi confronta le prestazioni di due classificatori osservando la differenza nei loro ranghi medi. Le prestazioni di due classificatori sono considerate significativamente diverse se i ranghi medi corrispondenti differiscono almeno della differenza critica (CD), che si calcola come:

$$CD = q_{\alpha} \sqrt{\frac{k(k+1)}{6N}}$$

dove k è il numero di algoritmi, N il numero di dataset, e q_{α} è il valore critico della distribuzione di Studentizzata al livello di significatività α .

Si ritiene che due algoritmi abbiano prestazioni significativamente diverse se la differenza assoluta dei loro ranghi medi è maggiore del CD:

$$|r_i - r_j| > CD$$

2.7.8 Critical difference diagram

La critical difference viene spesso visualizzata utilizzando un **diagramma della differenza critica**, che mostra con CD la distanza della critical difference tra i ranghi medi degli algoritmi confrontati.

A e B hanno una barra che li collega, quindi non sono significativamente diversi, invece A e C non hanno una barra che li collega, quindi sono significativamente diversi.

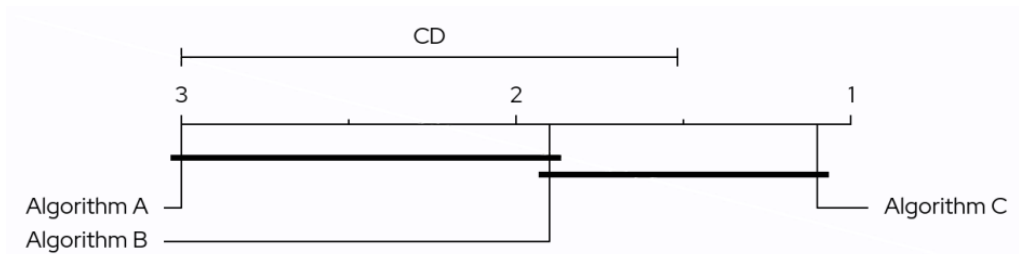


Figura 52: Il grafico del test di Friedman

2.8 Bias Variance decomposition

La **decomposizione Bias-Varianza** è un modo per comprendere le fonti di errore in un modello di apprendimento automatico.

Per diversi set di apprendimento, il risultato è spesso diverso, sebbene i campioni siano estratti dalla stessa distribuzione. La decomposizione Bias-Varianza ci aiuta a capire perché questo accade scomponendo l'errore in tre componenti:

1. **Bias**: l'errore dovuto a ipotesi errate nel modello di apprendimento, che può portare a dell'underfitting.
2. **Variance**: l'errore dovuto alla sensibilità del modello a piccole fluttuazioni nel set di addestramento, che può portare a un overfitting.
3. **Irreducible error**: l'errore che non può essere ridotto da nessun modello, spesso dovuto al rumore nei dati.

Consideriamo un problema di **regressione** e sia X un **campione di prova**, y_D è l'etichetta associata al dataset D^1 , y l'etichetta della ground truth per X , e $f(X; D)$ la precisione del modello f addestrato su D .

Expected prediction: la previsione prevista per il modello su X è data da:

$$\bar{f} = \mathbb{E}_D[f(X; D)]$$

Varianza:

$$var = \mathbb{E}_D[(f(X; D) - \bar{f})^2]$$

Rumore:

$$\epsilon^2 = \mathbb{E}_D[(y_D - y)^2]$$

Bias: cioè, la differenza tra la previsione prevista e l'etichetta reale:

$$bias^2 = (\bar{f} - y)^2$$

$\mathbb{E}_D$ è la media su tutti i possibili set di addestramento D .

Dimostrazione

Per facilitare la discussione, supponiamo che l'aspettativa di rumore sia zero, ovvero, $\mathbb{E}_D[y_D - y] = 0$.

L'MSE previsto del modello può essere scritto come:

$$\begin{aligned}
 E(f; D) &= \mathbb{E}_D[(f(\mathbf{x}; D) - y_D)^2] \\
 &= \mathbb{E}_D[\underbrace{(f(\mathbf{x}; D) - \bar{f})}_a + \underbrace{(\bar{f} - y_D)}_b]^2 = \mathbb{E}_D[(a + b)^2] \\
 &= \mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})^2] + \mathbb{E}_D[(\bar{f} - y_D)^2] + \cancel{2\mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})(\bar{f} - y_D)]} \quad \text{since } y_D \perp f_D \\
 &= \mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f})^2] + \mathbb{E}_D[(\bar{f} - y_D)^2] \\
 &= \text{var} + \mathbb{E}_D[(\bar{f} - y + y - y_D)^2] \\
 &= \text{var} + \cancel{\mathbb{E}_D[(\bar{f} - y)^2]} + \mathbb{E}_D[(y - y_D)^2] + \cancel{2\mathbb{E}_D[(\bar{f} - y)(y - y_D)]} \quad \text{since } \mathbb{E}_D[y_D - y] = 0 \Rightarrow \mathbb{E}_D[y_D] = y \\
 &= \text{var} + \text{bias}^2 + \varepsilon^2
 \end{aligned}$$

Figura 53: Dimostrazione della decomposizione bias-varianza

Calcoliamo l'errore f su tutto il dataset D : vado a prendere la media su tutti i possibili dataset D , della predizione di f meno l'etichetta reale al quadrato (sto calcolando l'errore empirico di f sul test set, sull'esempio specifico X).

Sottraggo e aggiungo $\bar{f}$ all'interno della parentesi quadra e sviluppo il quadrato della somma.

Il terzo termine va a 0 nel momento in cui y_D è indipendente da f_D , vero perché l'etichetta dell'esempio nel dataset è indipendente dalla funzione f perché addestrata sulla parte di training del dataset D , quindi non ha mai visto quell'etichetta rendendola indipendente. (Svolgendo la moltiplicazione ottengo: $\mathbb{E}[f_d \bar{f}] - \mathbb{E}[f_D y_d] - \mathbb{E}[\bar{f}^2] + \mathbb{E}[\bar{f} y_D] \rightarrow \bar{f}^2 - \mathbb{E}[f_D y_D] - \bar{f}^2 + \bar{f} \mathbb{E}_D[y_D] \rightarrow -\mathbb{E}[f_d] \mathbb{E}_D[y_D] + \bar{f} \mathbb{E}_D[y_D] = 0$).

Aggiungo e sottraggo nuovamente il valore vero dell'etichetta e ottengo la media su tutti i dataset.

Bias misura la differenza tra la previsione attesa dell'algoritmo di apprendimento e l'etichetta di ground-truth, che riflette la **capacità di adattamento dell'algoritmo di apprendimento**.

Varianza misura quanto le prestazioni di apprendimento cambiano a causa delle variazioni nel set di addestramento, riflettendo l'**impatto delle fluttuazioni dei dati sul risultato dell'apprendimento**.

Noise rappresenta il limite inferiore dell'errore di generalizzazione atteso che può essere raggiunto da **qualsiasi algoritmo di apprendimento** per il compito dato.

Un altro modo di vedere la cosa è che la decomposizione bias-varianza ci dice che la **performance della generalizzazione** è determinata congiuntamente da:

- la capacità dell'algoritmo di apprendimento
- sufficienza dei dati
- e l'errore intrinseco del problema di apprendimento

Per ottenere eccellenti prestazioni di generalizzazione, un modello necessita di un **piccolo bias** (per adattarsi adeguatamente ai dati) e di una **piccola varianza** (per ridurre al minimo l'impatto delle fluttuazioni dei dati).

2.8.1 Bias-Variance Dilemma

Il **dilemma bias-varianza** si riferisce al compromesso tra bias e varianza in un algoritmo di apprendimento: in generale gli algoritmi con un basso bias tendono ad avere un'alta varianza, e viceversa.

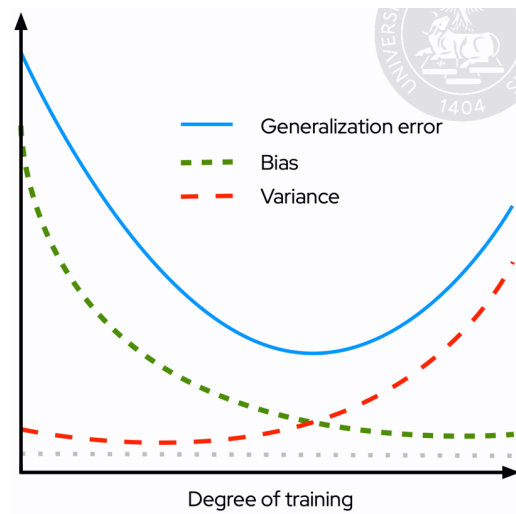


Figura 54: Relazioni tra errore di generalizzazione, distorsione e varianza

3 Linear Models

Sia $\mathbf{x} = (x_1, x_2, \dots, x_d)^\top$ un'esempio descritto da d variabili.

Un **modello lineare** mira a imparare una funzione che fa predizioni da una **combinazione lineare** delle variabili di input, che è:

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_dx_d + w_{d+1} = \mathbf{w}^\top \mathbf{x} + w_{d+1}$$

dove $\mathbf{w} = (w_1, w_2, \dots, w_d)^\top$ è il vettore dei pesi e w_{d+1} è il bias.

Il **modello** è **determinato** una volta che $\mathbf{w}$ e w_{d+1} sono stati **appresi dai dati**.

3.1 Regressione lineare

Dato un dataset $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$, dove $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{id})$ e $y_i \in \mathbb{R}$.

La **regressione lineare** mira ad apprendere un **modello lineare** in grado di prevedere con precisione il valore reale delle etichette di output.

L'idea chiave è misurare e minimizzare la differenza tra i valori previsti e i valori effettivi nel dataset:

$$MSE(\mathbf{w}, w_{d+1}) = \frac{1}{n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (\mathbf{w}^\top \mathbf{x}_i + w_{d+1}))^2$$

3.2 Mean Squared Error and Best Fit Line

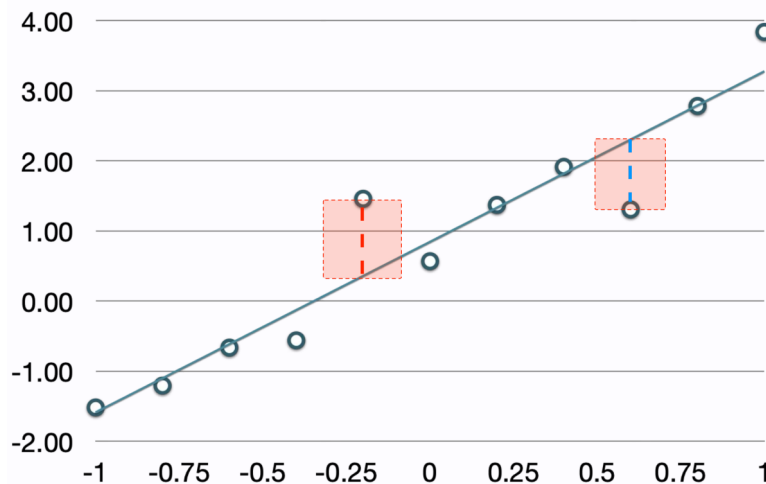


Figura 55: Errore quadratico medio e retta di miglior adattamento

x	y
-1	-1.52
-0.8	-1.21
-0.6	-0.67
-0.4	-0.56
-0.2	1.46
0	0.57
0.2	1.37
0.4	1.91
0.6	1.31
0.8	2.78
1	3.84

Nel caso univariato nell'esempio, vogliamo trovare i parametri w_1 e w_2 del modello lineare $f(x) = w_1x + w_2$ che meglio si adatta ai dati.

Idealmente l'equazione lineare varrebbe per ogni riga della tabella a sinistra. Cioè, il nostro obiettivo è trovare w_1 e w_2 tali che:

$$\begin{cases} w_1 \cdot (-1) + w_2 = -1.52 \\ w_1 \cdot (-0.8) + w_2 = -1.21 \\ w_1 \cdot (-0.6) + w_2 = -0.67 \\ \dots \end{cases}$$

Figura 56: Esempio univariato

3.2.1 Rappresentazione matriciale

Il sistema:

$$\begin{cases} w_1 \cdot (-1) + w_2 = -1.52 \\ w_1 \cdot (-0.8) + w_2 = -1.21 \\ w_1 \cdot (-0.6) + w_2 = -0.67 \\ \dots \end{cases}$$

può essere scritto in forma matriciale come:

$$\begin{bmatrix} \overset{x}{-1} \\ -0.8 \\ -0.6 \\ \vdots \end{bmatrix} \begin{bmatrix} \overset{w}{w_1} \end{bmatrix} + w_2 = \begin{bmatrix} \overset{y}{-1.52} \\ -1.21 \\ -0.67 \\ \vdots \end{bmatrix} = \mathbf{xw} + w_2$$

3.2.2 Coordinate omogenee

Incorporeremo il bias b nel vettore dei pesi, che diventa $\mathbf{w}' = (w_1, w_2, \dots, w_d, w_{d+1})^\top$, e aggiungiamo una colonna di uno alla matrice di input $\mathbf{x}$. Questa rappresentazione è chiamata **homogeneous coordinates**:

$$\begin{bmatrix} \overset{x}{-1} & 1 \\ -0.8 & 1 \\ -0.6 & 1 \\ \dots & 1 \end{bmatrix} \begin{bmatrix} \overset{w}{w_1} \\ w_2 \end{bmatrix} = \begin{bmatrix} \overset{y}{-1.52} \\ -1.21 \\ -0.67 \\ \dots \end{bmatrix} \equiv \mathbf{xw} = y$$

Domanda

Dato l'esempio:

$$\begin{aligned} x_1 &= (1, 2), & y_1 &= 0.8 \\ x_2 &= (3, 4), & y_2 &= 0.4 \\ x_3 &= (1, 4), & y_3 &= 0.7 \\ x_4 &= (2, 2), & y_4 &= 0.9 \end{aligned}$$

Quali sono le matrici $\mathbf{x}$ e il vettore $\mathbf{y}$?

3.2.3 Risolvere il sistema

Nel caso ideale $\mathbf{x}$ è al quadrato e di rango pieno, e l'equazione può essere risolta semplicemente moltiplicando entrambi i membri per l'inverso di $\mathbf{x}$:

$$\begin{aligned} \mathbf{xw} &= \mathbf{y} \\ \Downarrow \\ \mathbf{x}^{-1}\mathbf{xw} &= \mathbf{x}^{-1}\mathbf{y} \\ \Downarrow \\ \mathbf{w} &= \mathbf{x}^{-1}\mathbf{y} \end{aligned}$$

Sfortunatamente, questo è vero solo nei casi più banali. $\mathbf{x}$ non è sempre invertibile ed è **necessario un approccio più generale**.

Iniziamo notando che variando $\mathbf{w}$ in realtà stiamo coprendo lo spazio colonna di $\mathbf{x}$.

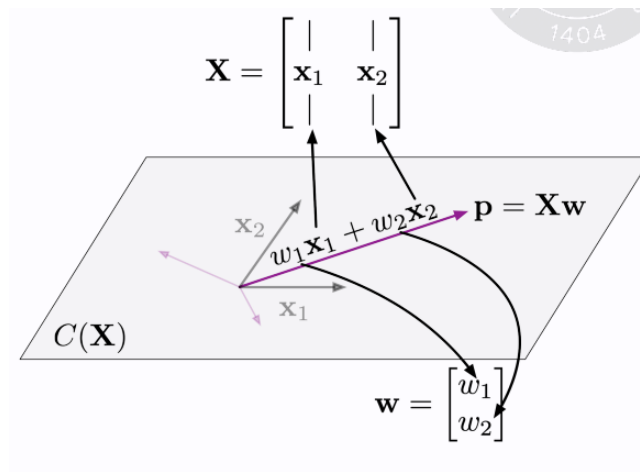


Figura 57: Variazione di w nello spazio colonna di x

Poiché il sistema di equazioni non è risolvibile, ne consegue che non esiste alcuna combinazione di w_i che ci permetta di ottenere y , implicando che y non appartiene a $C(x)$.

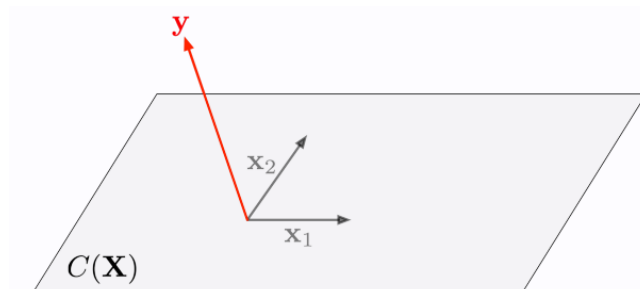


Figura 58: Nessuna combinazione di w può raggiungere y

Il meglio che possiamo fare è trovare il vettore w che minimizza la lunghezza dell'errore degli errori $e = y - xw$.

In altre parole, vogliamo trovare:

$$\arg \min_{\mathbf{w}} \|\mathbf{e}\| = \arg \min_{\mathbf{w}} \|\mathbf{e}\|^2 = \arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{xw}\|^2$$

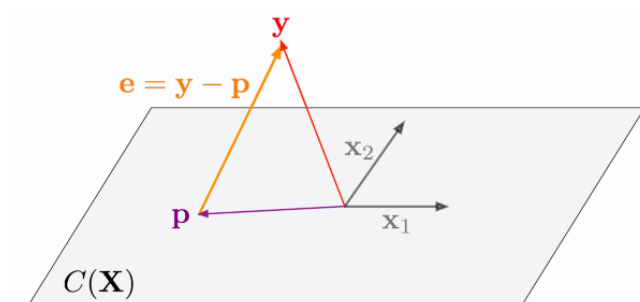


Figura 59: Trovare il punto sul piano $C(x)$ più vicino a y

3.2.4 Migliore p

Domanda Come scegliamo il punto migliore sul piano $C(x)$ che si avvicina a y ?

Il migliore punto sul piano $C(x)$ che approssima y è la **proiezione ortogonale** di y su $C(x)$.

In altre parole, è **sufficiente trovare il punto sul piano $C(x)$ tale che il vettore errore e sia ortogonale al piano.**

Questo punto corrisponde a $p = xw$, dove w è il vettore dei pesi ottimali.

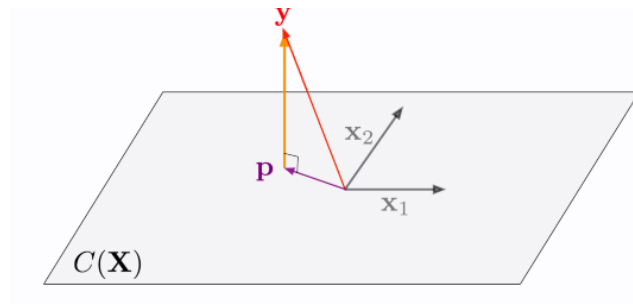


Figura 60: Trovare il punto sul piano $C(\mathbf{x})$ più vicino a $\mathbf{y}$

Per imporre che un vettore sia ortogonale a un piano, è sufficiente imporre che sia ortogonale a tutti i vettori che attraversano il piano:

$$\mathbf{x}_1 \perp \mathbf{e} \Rightarrow \mathbf{x}_1^T \mathbf{e} = 0$$

$$\mathbf{x}_2 \perp \mathbf{e} \Rightarrow \mathbf{x}_2^T \mathbf{e} = 0$$

...

Che può essere scritto in forma matriciale come:

$$\mathbf{x}^T \mathbf{e} = 0$$

La soluzione può essere trovata risolvendo la seguente equazione:

$$\mathbf{x}^T (\mathbf{y} - \mathbf{x}\mathbf{w}) = 0$$

$$\mathbf{x}^T \mathbf{y} - \mathbf{x}^T \mathbf{x}\mathbf{w} = 0$$

$$\mathbf{x}^T \mathbf{x}\mathbf{w} = \mathbf{x}^T \mathbf{y}$$

$$\mathbf{w} = (\mathbf{x}^T \mathbf{x})^{-1} \mathbf{x}^T \mathbf{y}$$

Regolarizzazione

Se i dati sono scarsi o rumorosi, la matrice $\mathbf{x}^T \mathbf{x}$ potrebbe essere **mal condizionata** o **singolare**, rendendo problematica l'inversione. In questi casi, è possibile impiegare tecniche di **regolarizzazione** (ad esempio, la regressione di Ridge) per stabilizzare la soluzione.

3.2.5 Soluzione dei minimi quadrati tramite calcolo

Un altro modo per trovare la soluzione dei minimi quadrati è utilizzare il calcolo.

Possiamo riscrivere il problema come:

$$\arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{x}\mathbf{w}\|^2 = \arg \min_{\mathbf{w}} (\mathbf{y} - \mathbf{x}\mathbf{w})^T (\mathbf{y} - \mathbf{x}\mathbf{w}) = \arg \min_{\mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{x}\mathbf{w} + \mathbf{w}^T \mathbf{x}^T \mathbf{x}\mathbf{w})$$

Derivando rispetto a $\mathbf{w}$ e ponendo la derivata a zero, otteniamo:

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{x}\mathbf{w} + \mathbf{w}^T \mathbf{x}^T \mathbf{x}\mathbf{w}) = 0$$

$$-2\mathbf{x}^T \mathbf{y} + 2\mathbf{x}^T \mathbf{x}\mathbf{w} = 0$$

$$\mathbf{x}^T \mathbf{x}\mathbf{w} = \mathbf{x}^T \mathbf{y}$$

$$\mathbf{w} = (\mathbf{x}^T \mathbf{x})^{-1} \mathbf{x}^T \mathbf{y}$$

Dove usiamo:

$$\frac{\partial}{\partial \mathbf{w}} [\mathbf{w}^T \mathbf{x}^T \mathbf{x}\mathbf{w}] = 2\mathbf{x}^T \mathbf{x}\mathbf{w}$$

e

$$\frac{\partial}{\partial \mathbf{w}} [\mathbf{y}^\top \mathbf{x} \mathbf{w}] = \mathbf{x}^\top \mathbf{y}$$

Esempio usando tre punti

$$\mathbf{x} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$

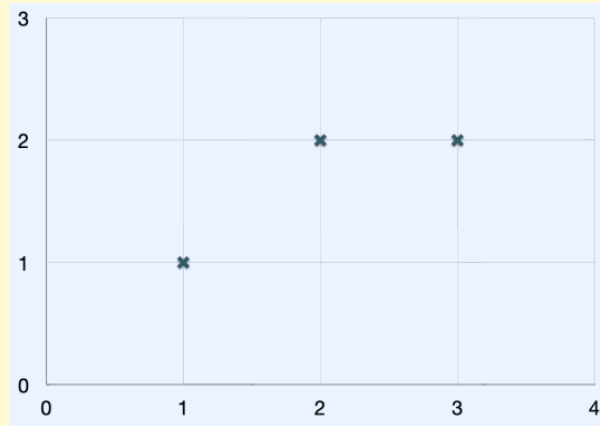


Figura 61: Esempio con tre punti

Obiettivo: calcolare $\hat{\mathbf{w}} = (\mathbf{x}^\top \mathbf{x})^{-1} \mathbf{x}^\top \mathbf{y}$

$$\mathbf{x}^\top \mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

Possiamo calcolare l'inverso di $\mathbf{x}^\top \mathbf{x}$ usando il metodo di Gauss-Jordan:

$$\begin{bmatrix} 14 & 6 & | & 1 & 0 \\ 6 & 3 & | & 0 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 0 & | & 1 & -2 \\ 6 & 3 & | & 0 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & | & \frac{1}{2} & -1 \\ 6 & 3 & | & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & | & \frac{1}{2} & -1 \\ 0 & 3 & | & -3 & 7 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & | & \frac{1}{2} & -1 \\ 0 & 1 & | & -1 & \frac{7}{3} \end{bmatrix}$$

Implicando:

$$(\mathbf{x}^\top \mathbf{x})^{-1} = \begin{bmatrix} \frac{1}{2} & -1 \\ -1 & \frac{7}{3} \end{bmatrix}$$

$$\mathbf{x}^\top \mathbf{y} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 11 \\ 5 \end{bmatrix}$$

La soluzione è quindi:

$$\hat{\mathbf{w}} = (\mathbf{x}^\top \mathbf{x})^{-1} \mathbf{x}^\top \mathbf{y} = \begin{bmatrix} \frac{1}{2} & -1 \\ -1 & \frac{7}{3} \end{bmatrix} \begin{bmatrix} 11 \\ 5 \end{bmatrix} =$$

$$\begin{bmatrix} \frac{1}{2} \cdot 11 - 1 \cdot 5 \\ -1 \cdot 11 + \frac{7}{3} \cdot 5 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ \frac{2}{3} \end{bmatrix}$$

Esempio usando tre punti

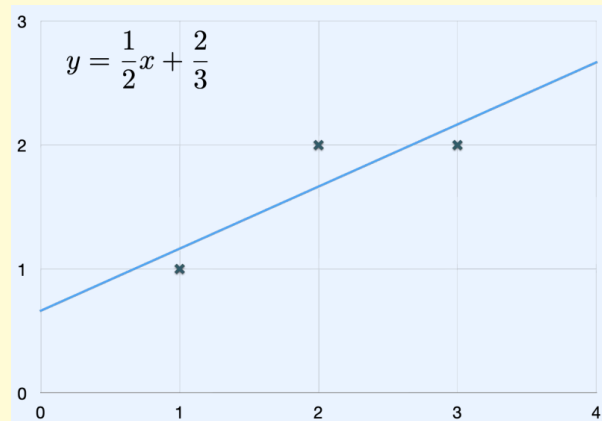


Figura 62: Risultato dell'esempio con tre punti

3.3 Regolarizzazione

Il metodo dei minimi quadrati è uno strumento potente per trovare la retta di approssimazione migliore per un insieme di punti. Tuttavia, è importante notare che **il metodo non è immune all'overfitting**.

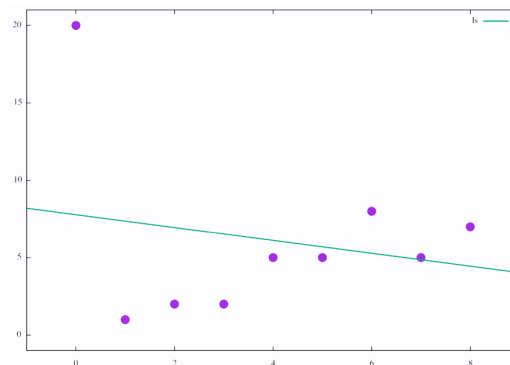


Figura 63: Esempio di partenza per la regolarizzazione

La **regolarizzazione** è un metodo generale per evitare l'overfitting applicando vincoli aggiuntivi al vettore dei pesi. Un approccio comune consiste nell'assicurarsi che i pesi siano, in media, di piccola entità: questo fenomeno è definito "**restringimento**" (**shrinkage**).

La versione regolarizzata dell'ottimizzazione dei minimi quadrati è:

$$\arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{xw}\|^2 + \lambda \|\mathbf{w}\|_p^p$$

dove λ è un iperparametro che controlla la quantità di regolarizzazione, e p è la norma utilizzata per misurare la dimensione dei pesi.

3.4 Ridge regression

Nel caso della norma L_2 (cioè, $p = 2$), la versione regolarizzata dell'ottimizzazione dei minimi quadrati è chiamata **regressione di Ridge**.

La regressione di Ridge ha una soluzione in forma chiusa:

$$\hat{\mathbf{w}} = (\mathbf{x}^\top \mathbf{x} + \lambda \mathbf{I})^{-1} \mathbf{x}^\top \mathbf{y}$$

dove $\mathbf{I}$ è la matrice identità.

La regolarizzazione equivale ad aggiungere λ alla diagonale di $\mathbf{x}^\top \mathbf{x}$, un trucco ben noto per migliorare la stabilità numerica dell'inversione di matrice.

3.4.1 Informazioni sul miglioramento dell'invertibilità di una matrice

Ci si potrebbe chiedere perché aggiungere $\lambda \mathbf{I}$ a una matrice ne migliori l'invertibilità. Il motivo risiede nel fatto che **una matrice è invertibile se e solo se tutti i suoi autovalori sono diversi da zero**.

Autovalori e invertibilità

Numericamente, quando alcuni autovalori sono prossimi allo zero, l'invertibilità diventa difficile, e diventa matematicamente impossibile quando uno o più autovalori raggiungono esattamente lo zero.

Aggiungendo $\lambda \mathbf{I}$ a una matrice aumenta tutti gli autovalori di λ . Poiché $\mathbf{x}^\top \mathbf{x}$ è reale e simmetrica, **tutti i suoi autovalori sono reali e non negativi**. Pertanto, anche un valore piccolo di λ è sufficiente a garantire l'invertibilità e, in generale, migliora la stabilità numerica del processo di inversione.

3.5 Lasso regression

Nel caso della norma L_1 (cioè, $p = 1$), la versione regolarizzata dell'ottimizzazione dei minimi quadrati è chiamata **regressione Lasso** (operatore di selezione e restringimento minimo assoluto).

In pratica, la regressione Lasso sostituisce il termine di regolarizzazione di Ridge $\|\mathbf{w}\|_2^2$ con la somma dei pesi assoluti:

$$\|\mathbf{w}\|_1 = \sum_i |w_i|$$

Il risultato è che alcuni pesi vengono ridotti, ma altri vengono impostati a 0, quindi **la regressione Lasso favorisce le soluzioni sparse**.

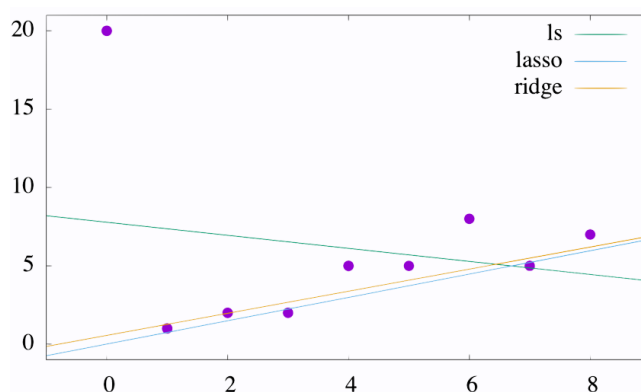


Figura 64: Confronto tra Lasso, Ridge e ls

3.6 Perché i metodi di regolarizzazione funzionano?

- Perché minimizzare la norma del vettore peso migliora i risultati?
- Perché l'approccio lasso favorisce soluzioni sparse?

Diverse ragioni giustificano la minimizzazione della norma del vettore dei pesi:

- se si presume che $\mathbf{x}$ sia affetto da un errore $\mathbf{D}$ allora:

$$(\mathbf{x} + \mathbf{D})\mathbf{w} = \mathbf{x}\mathbf{w} + \mathbf{D}\mathbf{w}$$

e minimizzando la norma del vettore dei pesi si minimizza l'effetto dell'errore su $\mathbf{x}$.

- Limitando i pesi, **introduciamo una tendenza verso modelli più semplici**, che a sua volta riduce la varianza del modello.
 - Questo compromesso è al centro della regolarizzazione: **una notevole riduzione della varianza** può compensare un **piccolo aumento della distorsione**, portando a un errore totale inferiore sui dati non osservati.

- Operiamo partendo dal presupposto che i modelli più semplici siano più generalizzabili, un principio che si allinea con il **rasoio di Occam** menzionato in precedenza.

3.6.1 Minimizzazione della norma e scarsità

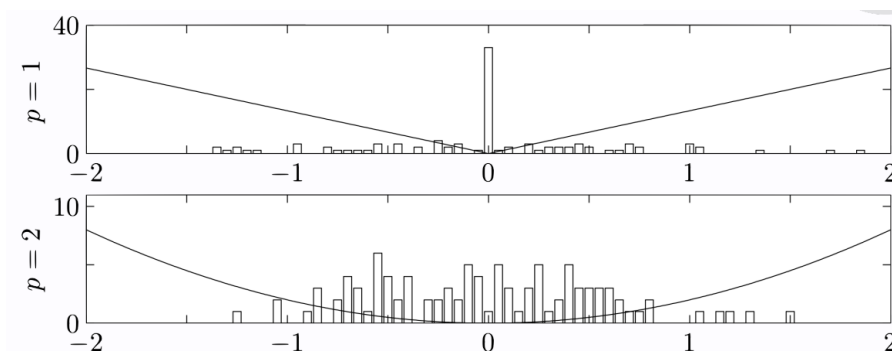


Figura 65: Esempio di minimizzazione con $p = 1$ e $p = 2$

3.7 Modelli lineari generalizzati e regressione logistica

3.7.1 Generalized Linear Models (GLM)

Considerando il modello lineare:

$$y = \mathbf{w}^\top \mathbf{x} + b$$

è possibile far sì che il valore previsto **approssimi una variabile derivata da y** ?

Esempio

$y \approx e^{0.5x+0.8}$ può essere modellato usando un modello lineare se usassimo la trasformazione $y' = \log(y)$, portando a:

$$y' = 0.5x + 0.8$$

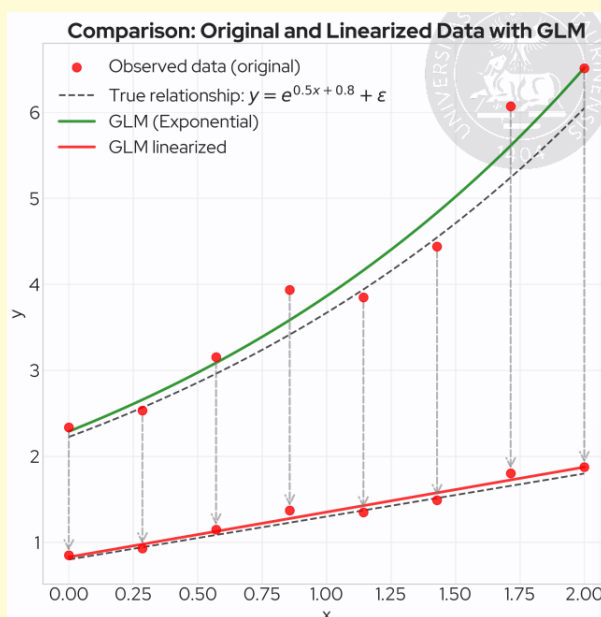


Figura 66: Confronto: originale e linearizzato con GLM

I **Modelli Lineari Generalizzati (GLM)** estendono il modello lineare per consentire diversi tipi di variabili di risposta. L'idea chiave è quella di utilizzare una **funzione di collegamento** per correlare il predittore lineare al valore atteso della variabile di risposta, ovvero, per qualsiasi funzione monotona

differenziabile, possiamo scrivere:

$$y = g^{-1}(\mathbf{w}^\top \mathbf{x} + b)$$

dove g è la **funzione di collegamento** e g^{-1} è la sua inversa.

Nota

C'è un po' di teoria sui GLM, qui adotteremo l'approccio del libro e ci concentreremo sugli aspetti pratici dei GLM, e in particolare sulla loro applicazione per derivare la regressione logistica.

3.8 Logistic Regression

Come possiamo risolvere i **problemi di classificazione** utilizzando **modelli lineari**?

Possiamo cercare funzioni differenziabili monotone che colleghino l'output della regressione lineare alle etichette di verità di base del problema di classificazione.

In particolare, vogliamo **convertire l'output** del modello lineare in etichette 0/1. Idealmente, si desidera la funzione a passo unitario:

$$y = f(z) = \begin{cases} 0, & \text{se } z < 0 \\ 0.5, & \text{se } z = 0 \\ 1, & \text{se } z > 0 \end{cases}$$

Tuttavia, questa funzione non è differenziabile e quindi non può essere utilizzata come g^{-1} .

Un surrogato differenziabile monotono della funzione passo unitario è la **funzione logistica**:

$$y = f(z) = \frac{1}{1 + e^{-z}}$$

La sua inversa è:

$$z = f^{-1}(y) = \ln \frac{y}{1 - y}$$

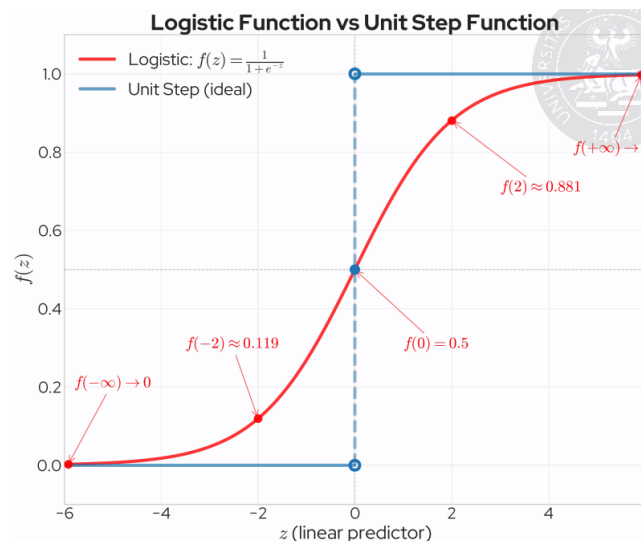


Figura 67: Funzione unitaria e funzione logistica

Utilizzando la **funzione logistica** come **inversa della funzione di collegamento**, possiamo modellare la probabilità che un dato input appartenga a una particolare classe. L'output della funzione logistica può essere interpretato come la **probabilità della classe positiva**:

$$y' = p(y = 1 | \mathbf{x}) = g^{-1}(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Applicando la **funzione collegamento** ad entrambi i lati, otteniamo:

$$g(y') = \ln \frac{y'}{1 - y'} = \ln \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} = g(g^{-1}(\mathbf{w}^\top \mathbf{x} + b)) = \mathbf{w}^\top \mathbf{x} + b$$

cedendo:

$$\mathbf{w}^\top \mathbf{x} + b = \ln \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})}$$

cioè, il **modello lineare** prevede le **probabilità logaritmiche** della classe positiva.

Avremmo anche potuto **partire dall'equazione del logaritmo di probabilità** di cui sopra e derivare la **funzione logistica** come segue. Da:

$$\ln \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} = z \wedge p(y = 0 | \mathbf{x}) + p(y = 1 | \mathbf{x}) = 1$$

abbiamo:

$$p(y = 1 | \mathbf{x}) = e^z p(y = 0 | \mathbf{x}) = e^z (1 - p(y = 1 | \mathbf{x}))$$

cedendo:

$$p(y = 1 | \mathbf{x}) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$$

e:

$$p(y = 0 | \mathbf{x}) = \frac{1}{1 + e^z}$$

Per stimare i parametri del modello di regressione logistica, possiamo utilizzare l'**approccio di stima della massima verosimiglianza (MLE)**. L'idea è quella di trovare i parametri che massimizzano la verosimiglianza dei dati osservati.

La verosimiglianza per un singolo punto dati $(\mathbf{x}_i, y_i)$, dove $y_i \in \{0, 1\}$, è data dalla distribuzione di Bernoulli:

$$p(y_i | \mathbf{x}_i) = p(y_i = 1 | \mathbf{x}_i)^{y_i} \cdot p(y_i = 0 | \mathbf{x}_i)^{1-y_i}$$

Per trovare i parametri ottimali, possiamo lavorare con la logaritmo-verosimiglianza, che è più facile da differenziare. Sia $z_i = \mathbf{w}^\top \mathbf{x}_i + b$

$$\begin{aligned} \ln p(y_i | \mathbf{x}_i) &= y_i \ln p(y_i = 1 | \mathbf{x}_i) + (1 - y_i) \ln p(y_i = 0 | \mathbf{x}_i) \\ &= y_i \ln \frac{e^{z_i}}{1 + e^{z_i}} + (1 - y_i) \ln \frac{1}{1 + e^{z_i}} \\ &= y_i(z_i - \ln(1 + e^{z_i})) - (1 - y_i) \ln(1 + e^{z_i}) \\ &= y_i z_i - y_i \ln(1 + e^{z_i}) - \ln(1 + e^{z_i}) + y_i \ln(1 + e^{z_i}) \\ &= y_i z_i - \ln(1 + e^{z_i}) \end{aligned}$$

Massimizzare la verosimiglianza logaritmica equivale a minimizzare la sua negativa. In coordinate omogenee, con $\beta = (\mathbf{w}, b)$ e $\mathbf{x}'_i = (\mathbf{x}_i, 1)$, la funzione di costo (verosimiglianza logaritmica negativa per un singolo punto) è:

$$\ell(\beta; \mathbf{x}'_i, y_i) = -\ln p(y_i | \mathbf{x}'_i) = \ln(1 + e^{\beta^\top \mathbf{x}'_i}) - y_i(\beta^\top \mathbf{x}'_i)$$

Per un set di dati con n osservazioni indipendenti, la verosimiglianza complessiva è il prodotto delle verosimiglianze individuali:

$$\ell(\beta) = \sum_{i=1}^n \ell(\beta; \mathbf{x}'_i, y_i) = \sum_{i=1}^n \left(\ln(1 + e^{\beta^\top \mathbf{x}'_i}) - y_i(\beta^\top \mathbf{x}'_i) \right)$$

La **loss function** sopra è convessa in β e differenziabile. Il suo minimo può essere trovato mediante la discesa del gradiente o altri algoritmi di ottimizzazione che risolvono:

$$\beta^* = \arg \min_{\beta} \ell(\beta)$$

3.9 Classificazione multiclasse

Sebbene esistano numerose tecniche per la classificazione multiclasse, esse possono generalmente essere raggruppate in due categorie: **metodi di trasformazione dei problemi** e **metodi di adattamento degli algoritmi**.

In questa sezione, ci concentreremo sui metodi di trasformazione dei problemi.

Metodi di trasformazione dei problemi

I metodi di trasformazione dei problemi mirano a **ridurre il problema di classificazione multiclasse in problemi di classificazione binaria multipla**.

3.9.1 Idea principale

Affronteremo il problema multiclasse suddividendolo in diverse **attività di classificazione binaria più piccole**. Per ogni attività, creeremo un **nuovo set di dati**, addestreremo un **semplice classificatore binario** e poi **combineremo i risultati** per ottenere la previsione multiclasse finale.

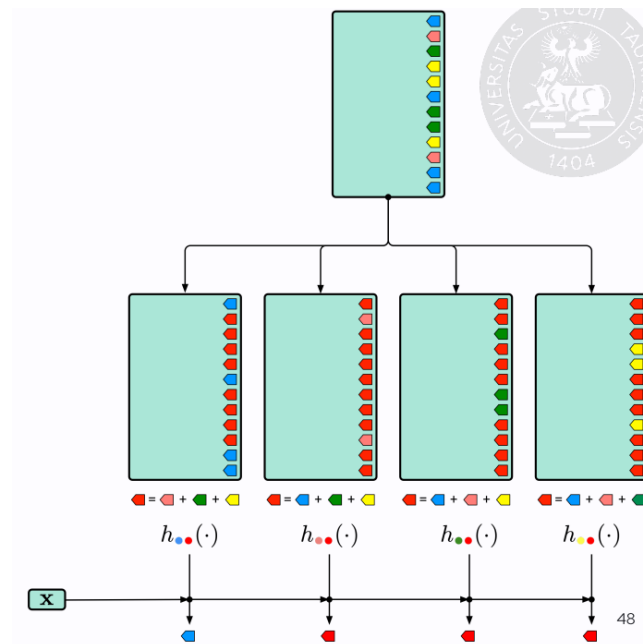


Figura 68: Idea principale della classificazione multiclasse

Esistono diverse alternative all'approccio di base descritto sopra. Queste alternative differiscono nel modo in cui costruiscono le attività di classificazione binaria e nel modo in cui combinano i risultati:

- One-vs-One (OvO)
- One-vs-Rest (OvR)
- Many-vs-Many (MvM)
- Error-Correcting Output Codes (ECOC)

Tutti possono essere descritti in termini di una **matrice di codifica** che definisce la relazione tra le classi originali e i classificatori binari.

3.9.2 Matrice di codifica

Una **matrice di codifica** è una matrice che definisce la relazione tra le classi originali e i classificatori binari.

Esempio

	h_1	h_2	h_3	h_4
C_1	1	1	-1	-1
C_2	1	-1	1	-1
C_3	-1	1	-1	1
C_4	-1	-1	1	1

Figura 69: Esempio di matrice di codifica

Ogni colonna corrisponde a un classificatore binario che distingue due gruppi di classi. Le voci nella matrice indicano quali classi devono essere considerate positive o negative per ciascun classificatore.

Ad esempio, per costruire il primo classificatore, rietichetteremo il set di dati assegnando la **classe positiva** a C_1 e C_2 e la classe negativa a C_3 e C_4 .

Al **momento del test**, applicheremo tutti i classificatori a un nuovo campione e combineremo le loro previsioni in un vettore riga, ad esempio, $[1, 1, -1, -1]$.

La riga nella matrice di codifica più simile al vettore di previsione indicherà la classe prevista.

					h_1	h_2	h_3	h_4				
$\mathbf{x}$	$\rightarrow$	(h_1, h_2, h_3, h_4)	$\rightarrow$	$[1, 1, -1, -1]$	$\rightarrow$	C_1	1	1	-1	-1	$\rightarrow$	C_1
						C_2	1	-1	1	-1		
						C_3	-1	1	-1	1		
						C_4	-1	-1	1	1		

Figura 70: Esempio di utilizzo della matrice di codifica

3.9.3 One-vs-One (OvO) Classification

Supponiamo di avere k classi. Nell'approccio **uno contro uno**, **addestriamo un classificatore binario per ogni coppia di classi**. Questo si traduce in un totale di $\frac{k(k-1)}{2}$ classificatori.

In questo contesto, ogni classificatore viene addestrato a distinguere tra due classi specifiche, astenendosi dal fare previsioni sulle altre classi.

Esempio di una matrice di codifica OvO per un problema di 4 classi

	h_{12}	h_{13}	h_{14}	h_{23}	h_{24}	h_{34}
C_1	1	1	1	0	0	0
C_2	-1	0	0	1	1	0
C_3	0	-1	0	-1	0	1
C_4	0	0	-1	0	-1	-1

Figura 71: Esempio di matrice di codifica OvO per un problema di 4 classi

3.9.4 One-vs-Rest (OvR) Classification

Nel metodo **One-vs-Rest** (OvR) (noto anche come **One-vs-All** (OvA)), addestriamo k classificatori binari. Ogni classificatore viene addestrato a **distinguere una classe da tutte le altre**.

Esempio di una matrice di codifica OvR per un problema di 4 classi

	h_1	h_2	h_3	h_4
C_1	1	-1	-1	-1
C_2	-1	1	-1	-1
C_3	-1	-1	1	-1
C_4	-1	-1	-1	1

Figura 72: Esempio di matrice di codifica OvR per un problema di 4 classi

3.9.5 Many-vs-Many (MvM) Classification

I metodi **molti contro molti** (MvM) rappresentano un approccio più generale e flessibile. In questo contesto, ogni classificatore binario viene addestrato su un sottoinsieme di classi, dove alcune classi vengono trattate come positive e altre come negative.

Sia **OvO** che **OvR** possono essere considerati **casi speciali di MvM**.

Esempio di una matrice di codifica MvM per un problema di 4 classi

	h_1	h_2	h_3	h_4	h_5	h_6
C_1	1	1	-1	-1	-1	-1
C_2	1	-1	-1	1	1	1
C_3	1	1	1	-1	1	1
C_4	-1	-1	1	-1	1	-1

Figura 73: Esempio di matrice di codifica MvM per un problema di 4 classi

3.9.6 Error-Correcting Output Codes (ECOC)

La matrice di codifica è **progettata** in modo che **le righe** (codici di classe) **siano ben separate in base alla distanza di Hamming**. Questa separazione consente al sistema di correggere gli errori commessi dai singoli classificatori binari durante la previsione.

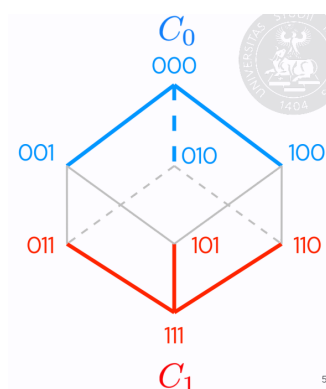


Figura 74: Esempio di matrice di codifica ECOC per un problema di 2 classi

Trovare l'**ECOC ottimale** per un dato numero di classi è **NP-hard**. Tuttavia, il semplice utilizzo di una **matrice di codifica casuale** può essere una buona euristica nella pratica se il codice è sufficientemente lungo.

Tuttavia, **quando k è piccolo**, potrebbero essere possibili solo poche combinazioni univoche di parole di codice, il che può limitare l'efficacia dell'approccio ECOC.

3.9.7 OvR vs OvO

Le classificazioni One-vs-Rest (OvR) e One-vs-One (OvO) presentano ciascuna vantaggi e svantaggi distinti, e la scelta tra le due dipende dalle caratteristiche del compito di classificazione.

OvR

- Utilizza l'intero set di dati per ogni classificatore binario, il che può migliorare l'efficienza statistica.
- Tuttavia, produce sottoproblemi fortemente sbilanciati, anche se il set di dati originale è bilanciato.

OvO

- Genera molti problemi binari più piccoli, $\frac{k(k-1)}{2}$ classificatori per k classi, che possono essere computazionalmente costosi quando k è grande.
- Ogni classificatore viene addestrato solo su un sottoinsieme di dati, il che può rappresentare una limitazione quando i dati sono scarsi.

3.10 Problema di squilibrio di classe

Supponiamo di avere un **set di dati con un significativo squilibrio di classi**, in cui la classe di maggioranza ha 998 istanze e la classe di minoranza ne ha solo 2.

Un learner può facilmente raggiungere una precisione del 99,8% semplicemente prevedendo la classe di maggioranza per tutte le istanze. Tuttavia, **questo approccio non riesce a identificare la classe di minoranza**, che spesso è la **classe di interesse**.

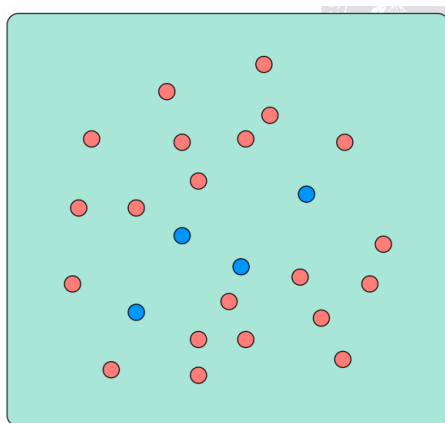


Figura 75: Esempio di problema di squilibrio di classe

3.10.1 Aggiornamento della Threshold

Supponiamo di utilizzare un classificatore lineare:

$$y = \mathbf{w}^\top \mathbf{x} + b$$

e classifichiamo le istanze come positive se $y > 0.5$, che equivale a usare $\frac{y}{1-y} > 1$. Ciò equivale a supporre che le classi siano ugualmente probabili.

Se abbiamo m^+ istanze positive e m^- istanze negative, possiamo riscalare il confine decisionale utilizzando come rapporto di verosimiglianza $\frac{m^+}{m^-}$:

$$\frac{y}{1-y} > \frac{m^+}{m^-} \Leftrightarrow y > \frac{m^+}{m^+ + m^-}$$

3.10.2 Oversampling

I metodi di sovracampionamento mirano a bilanciare la distribuzione delle classi aumentando il numero di istanze nella classe di minoranza. L'approccio più semplice è il **sovracampionamento casuale**, che prevede la duplicazione casuale dei campioni della classe di minoranza.

Sebbene semplice, il **sovracampionamento casuale può portare a un overfitting**, poiché il modello potrebbe imparare a riconoscere istanze duplicate specifiche anziché caratteristiche generali della classe di minoranza.

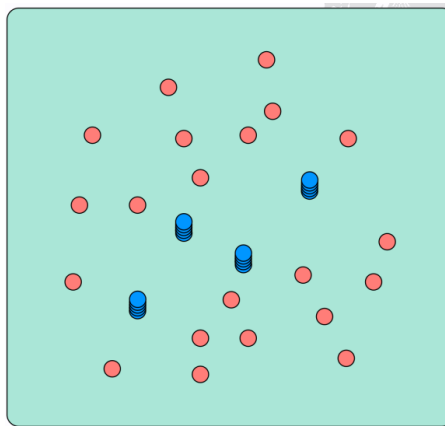


Figura 76: Esempio di sovracampionamento casuale

3.10.3 SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) è una delle più utilizzate **tecniche di oversampling** per creare dati sintetici della classe di minoranza.

Nota

SMOTE seleziona a caso uno dei k vicini più prossimi (solitamente $k = 5$) e interpola con l'originale per creare un esempio sintetico

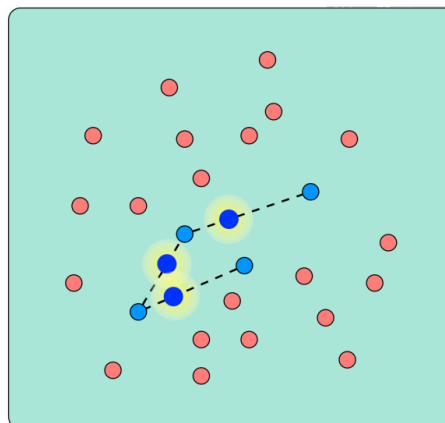


Figura 77: Esempio di SMOTE

3.10.4 Undersampling

I metodi di sottocampionamento mirano a bilanciare la distribuzione delle classi **riducendo il numero di istanze nella classe di maggioranza**. Il metodo più semplice è il **sottocampionamento casuale**, che consiste nel rimuovere casualmente campioni dalla classe di maggioranza fino a quando il set di dati non è bilanciato.

Il principale svantaggio del sottocampionamento è la **potenziale perdita di informazioni preziose**. Questo può essere particolarmente **problematico** se il **set di dati** è di **piccole dimensioni**.

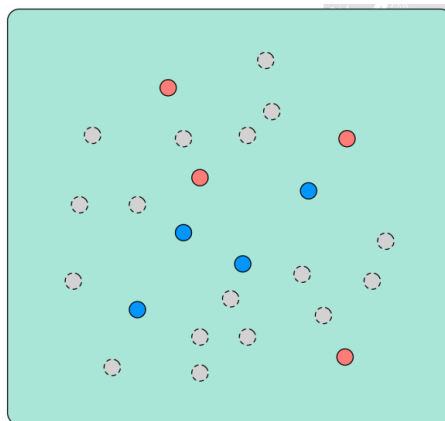


Figura 78: Esempio di sottocampionamento casuale

3.10.5 EasyEnsemble

Per mitigare la perdita di informazioni associata al sottocampionamento casuale, **EasyEnsemble** combina il sottocampionamento con un approccio di apprendimento d'insieme.

Approccio

Invece di creare un singolo set di dati sottocampionato, EasyEnsemble genera più sottoinsiemi diversi della classe di maggioranza. Per ogni sottoinsieme, addestra un classificatore separato su quel sottoinsieme combinato con tutte le istanze della classe di minoranza. La previsione finale è quindi un'aggregazione (ad esempio, tramite voting) delle previsioni di tutti i classificatori nell'ensemble.

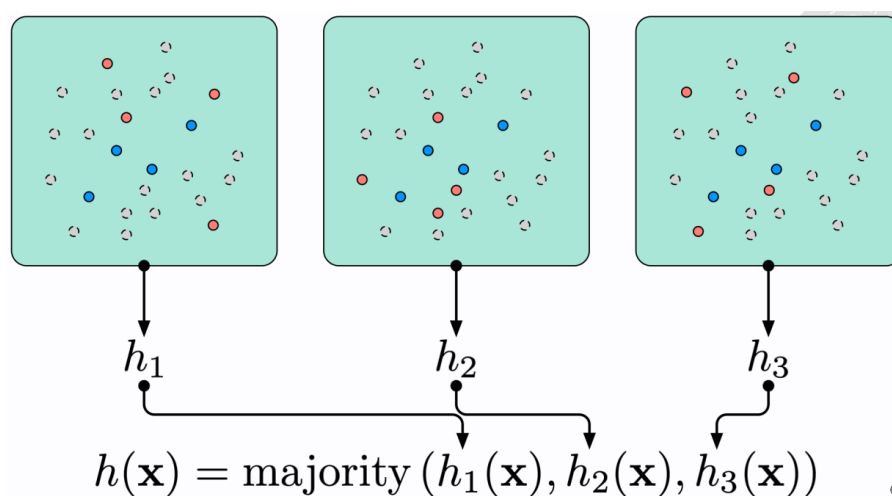


Figura 79: Esempio di EasyEnsemble

4 Decision Trees

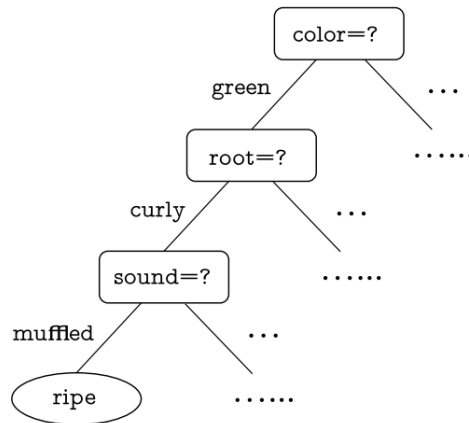


Figura 80: Esempio di albero decisionale per classificare angurie

4.1 Processo di base

- Ogni domanda posta nel processo decisionale è un "test" su una feature.
- Le conclusioni al termine del processo decisionale corrispondono alle possibili classificazioni.
- Ogni test porta alla conclusione o a un test aggiuntivo condizionato alla risposta corrente.
- Ogni percorso dal nodo radice al nodo foglia è una sequenza decisionale.

Obiettivo

L'obiettivo è produrre un albero che possa **generalizzare per prevedere campioni inediti**.

Algorithm 1 Decision Tree Learning

Require: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$; Feature set $A = \{a_1, a_2, \dots, a_d\}$.

```

1: Process: Function TreeGenerate( $D, A$ )
2:   Generate node  $i$ ;
3:   if All samples in  $D$  belong to the same class  $C$  then
4:     Mark node  $i$  as a class  $C$  leaf node;
5:     return ;
6:   end if
7:   if  $A = \emptyset$  OR all samples in  $D$  take the same value on  $A$  then
8:     Mark node  $i$  as a leaf node, and its class label is the majority class in  $D$ ;
9:     return ;
10:  end if
11:  Select the optimal splitting feature  $a_*$  from  $A$ ;
12:  for each value  $a_*^v$  in  $a_*$  do
13:    Generate a branch for node  $i$ ; Let  $D_v$  be the subset of samples taking value  $a_*^v$  on feature  $a_*$ ;
14:    if  $D_v = \emptyset$  then
15:      Mark this child node as a leaf node, and label it with the majority class in  $D$ ;
16:      return ;
17:    else
18:      Use TreeGenerate( $D_v, A \setminus \{a_*\}$ ) as the child node;
19:    end if
20:  end for

```

Ensure: A decision tree with root node i .

Caso 1: Tutti i campioni nel nodo corrente appartengono alla stessa classe

Algorithm 4.1 Decision Tree Learning.

Input: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;

Feature set $A = \{a_1, a_2, \dots, a_d\}$: ID, color, root, sound, texture, umbilicus, surface)

Process: Function TreeGenerate(D, A)

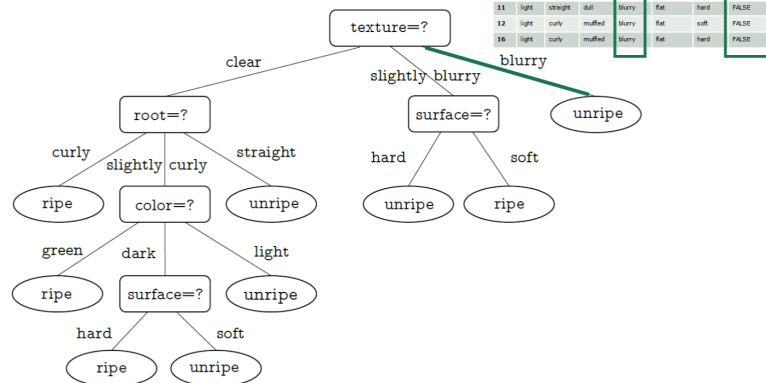
```

1: Generate node  $i$ ;
2: if All samples in  $D$  belong to the same class  $C$  then
3:   Mark node  $i$  as a class  $C$  leaf node; return
4: end if
5: if  $A = \emptyset$  OR all samples in  $D$  take the same value on  $A$  then
6:   Mark node  $i$  as a leaf node, and its class label is the majority class in  $D$ ; return
7: end if
8: Select the optimal splitting feature  $a_s$  from  $A$ ;
9: for each value  $a_s^*$  in  $a_s$  do
10:  Generate a branch for node  $i$ ; Let  $D_s$  be the subset of samples taking value  $a_s^*$  on  $a_s$ ;
11:  if  $D_s$  is empty then
12:    Mark this child node as a leaf node, and label it with the majority class in  $D$ ; return
13:  else
14:    Use TreeGenerate( $D_s, A \setminus \{a_s\}$ ) as the child node.
15:  end if
16: end for
Output: A decision tree with root node  $i$ .

```

(1) All samples in the current node belong to the same class.

same class



Caso 2: L'attuale set di feature è vuoto o tutti i campioni hanno gli stessi valori di feature

Algorithm 4.1 Decision Tree Learning.

Input: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;

Feature set $A = \{a_1, a_2, \dots, a_d\}$: ID, color, root, sound, texture, umbilicus, surface)

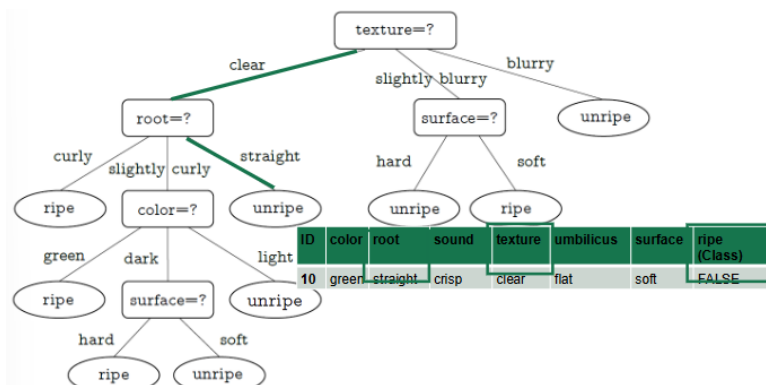
Process: Function TreeGenerate(D, A)

```

1: Generate node  $i$ ;
2: if All samples in  $D$  belong to the same class  $C$  then
3:   Mark node  $i$  as a class  $C$  leaf node; return
4: end if
5: if  $A = \emptyset$  OR all samples in  $D$  take the same value on  $A$  then
6:   Mark node  $i$  as a leaf node, and its class label is the majority class in  $D$ ; return
7: end if
8: Select the optimal splitting feature  $a_s$  from  $A$ ;
9: for each value  $a_s^*$  in  $a_s$  do
10:  Generate a branch for node  $i$ ; Let  $D_s$  be the subset of samples taking value  $a_s^*$  on  $a_s$ ;
11:  if  $D_s$  is empty then
12:    Mark this child node as a leaf node, and label it with the majority class in  $D$ ; return
13:  else
14:    Use TreeGenerate( $D_s, A \setminus \{a_s\}$ ) as the child node.
15:  end if
16: end for
Output: A decision tree with root node  $i$ .

```

(2) The current feature set is empty, or all samples have the same feature values.



Operazione di divisione su a^*

Algorithm 4.1 Decision Tree Learning.

Input: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
 Feature set $A = \{a_1, a_2, \dots, a_d\}$. (ID, color, root, sound, texture, umbilicus, surface)
Process: Function TreeGenerate(D, A)

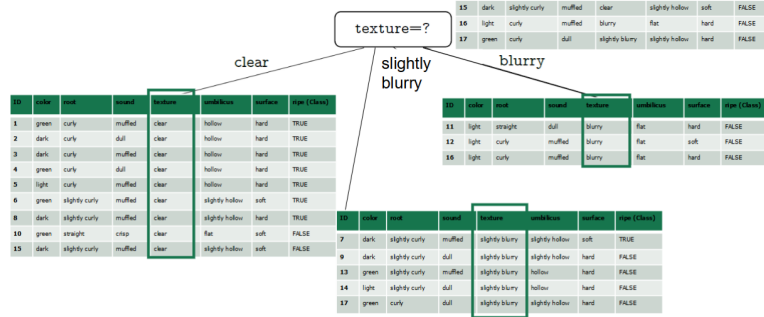
- 1: Generate node i ;
- 2: if All samples in D belong to the same class C then
- 3: Mark node i as a class C leaf node; **return**
- 4: **end if**
- 5: if $A = \emptyset$ OR all samples in D take the same value on A then
- 6: Mark node i as a leaf node, and its class label is the majority class in D ; **return**
- 7: **end if**
- 8: Select the optimal splitting feature a_* from A ;
- 9: **for** each value a_i^* in a_* **do**
- 10: Generate a branch for node i ; Let $D_{a_i^*}$ be the subset of samples taking value a_i^* on a_* ;
- 11: if $D_{a_i^*}$ is empty then
- 12: Mark this child node as a leaf node, and label it with the majority class in D ; **return**
- 13: else
- 14: Use TreeGenerate($D_{a_i^*}, A \setminus \{a_*\}$) as the child node.
- 15: **end if**
- 16: **end for**

Output: A decision tree with root node i .

Splitting operation on a^*

Splitting operation on a^* :

□ Assuming a^* is "texture":



Caso 3: Non c'è alcun campione nel nodo corrente

Algorithm 4.1 Decision Tree Learning.

Input: Training set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
 Feature set $A = \{a_1, a_2, \dots, a_d\}$. (ID, color, root, sound, texture, umbilicus, surface)
Process: Function TreeGenerate(D, A)

- 1: Generate node i ;
- 2: if All samples in D belong to the same class C then
- 3: Mark node i as a class C leaf node; **return**
- 4: **end if**
- 5: if $A = \emptyset$ OR all samples in D take the same value on A then
- 6: Mark node i as a leaf node, and its class label is the majority class in D ; **return**
- 7: **end if**
- 8: Select the optimal splitting feature a_* from A ;
- 9: **for** each value a_i^* in a_* **do**
- 10: Generate a branch for node i ; Let $D_{a_i^*}$ be the subset of samples taking value a_i^* on a_* ;
- 11: if $D_{a_i^*}$ is empty then
- 12: Mark this child node as a leaf node, and label it with the majority class in D ; **return**
- 13: else
- 14: Use TreeGenerate($D_{a_i^*}, A \setminus \{a_*\}$) as the child node.
- 15: **end if**
- 16: **end for**

Output: A decision tree with root node i .

(3) There is no sample in the current node.



4.2 Split selection

Il fulcro dell'algoritmo di apprendimento dell'albero decisionale è la selezione della caratteristica di suddivisione ottimale. In generale, man mano che il processo di suddivisione procede, vorremmo che più campioni all'interno di ciascun nodo appartenessero a una singola classe, ovvero **aumentando la purezza di ciascun nodo**.

Linee guida classiche per la selezione delle feature di divisione:

- Minority class;
- Gini index;
- Information gain;
- Gain ratio;

4.2.1 Modi per misurare l'impurità

Minority class:

$$\min\{p^+, p^-\} \quad 1 - \max\{p_k\}, k > 2 \text{ classes}$$

O il tasso di errore come verrebbe misurato con la proporzione di esempi classificati erroneamente se tutti gli esempi nel nodo foglia fossero etichettati con la classe di maggioranza.

Gini index:

$$2p^+p^- \quad \sum_{k=1}^{n \text{ classes}} p_k(1 - p_k)$$

È l'errore atteso se gli esempi nel nodo foglia fossero etichettati in modo casuale: positivo con probabilità p^+ , negativo con probabilità p^- . La probabilità di un falso positivo è p^+p^- e la probabilità di un falso negativo è p^-p^+ .

Entropia:

$$-p^+ \cdot \log_2(p^+) - p^- \cdot \log_2(p^-) \quad \sum_{k=1}^{n \text{ classes}} p_k \cdot \log_2(p_k)$$

Si tratta dell'informazione attesa contenuta in un messaggio che ci informa sulla classe di un esempio scelto casualmente nel nodo foglia.

Notazione

p^+ e p^- sono le probabilità rispettivamente della classe positiva e negativa.

Ossevazione: perché l'indice di Gini è l'errore previsto

Assumiamo che gli esempi nella popolazione sono positivi con probabilità p e negativi con probabilità $1 - p$.

Sono etichettati in modo casuale e **indipendentemente** dalla loro vera etichetta. L'etichettatura sarà positiva con probabilità p e negativa con probabilità $1 - p$.

Quando estraiamo un esempio, è positivo con probabilità p . Lo etichettiamo come negativo con probabilità $(1 - p)$ e commettiamo un errore di falso negativo (FN). Probabilità di questo errore: $p(1 - p)$

C'è una probabilità $(1 - p)$ di estrarre un esempio negativo. Lo etichettiamo come positivo con probabilità p e commettiamo un errore di falso positivo (FP). Probabilità di questo errore: $(1 - p)p$
Probabilità totale di errore:

$$(FN + FP) = p(1 - p) + (1 - p)p = 2p(1 - p)$$

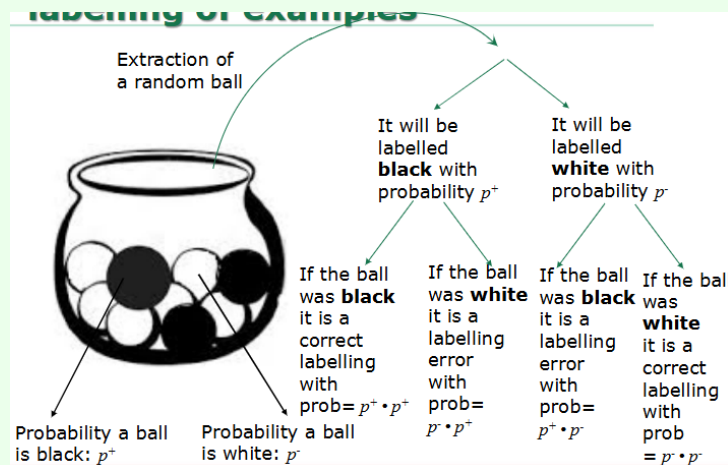


Figura 81: Visualizzazione dell'etichettatura casuale degli esempi

4.3 Decision trees

- Separare il set di dati in partizioni disgiunte guidate dalla funzione obiettivo:
 - Obiettivo: ogni partizione è omogenea nell'attributo di destinazione (la classe).
- La funzione obiettivo misura l'impurità delle partizioni ottenute dopo una suddivisione. Una di queste misure è l'entropia.
- L'**entropia** è una misura della **quantità di confusione**.
 - Obiettivo: ridurre l'entropia dell'attributo di destinazione in ogni partizione.

4.4 Entropia

L'Entropia è misura in **bits**. Può essere usata per:

- quantificare il numero di unità (bit) necessarie per rappresentare l'informazione (ed eventualmente comunicarla tramite un messaggio)
- misurare la quantità di informazioni associate a un evento (il cui esito potrebbe essere comunicato in un messaggio)

Se un esperimento ha n possibili esiti (equiprobabili), il numero di bit necessari per rappresentare uno qualsiasi di questi esiti è b :

$$b = \lceil \log_2(n) \rceil$$

Esempio 1

Supponiamo di lanciare un dado. Ha 6 risultati.

Se vogliamo rappresentare e comunicare un risultato abbiamo bisogno di b bits:

$$b = \lceil \log_2(6) \rceil = \lceil 2.58 \rceil = 3$$

Se comunichiamo prima solo la parità del risultato, abbiamo bisogno di b_1 bits:

$$b_1 = \lceil \log_2(2) \rceil = 1$$

Dopo il primo messaggio sulla parità dell'esito, rimane una certa confusione sull'esito, ma questa confusione è ridotta dalla quantità di informazioni già inviate.

Esempio 2

Ci sono 3 possibili risultati per ogni risultato pari o dispari di un lancio di dado. Abbiamo bisogno di b_2 bit per rappresentarlo:

$$b_2 = \lceil \log_2(3) \rceil = \lceil 1.58 \rceil = 2$$

Questo misura la quantità di confusione residua sul risultato ed è la quantità di informazioni che dobbiamo ancora comunicare, date le informazioni già diffuse (parità).

$$b_2 = b - b_1 = \log_2 6 - \log_2 2 = 2.58 - 1 = 1.58$$

Le informazioni acquisite dal primo messaggio sono: Informazioni acquisite = quantità iniziale di confusione – quantità residua di confusione

$$b_1 = b - b_2 = \log_2 6 - \log_2 3 = 2.58 - 1.58 = 1$$

4.4.1 Informazioni

Se un evento E ha probabilità p di verificarsi, le informazioni ottenute quando lo osserviamo sono:

$$I(E) = \log_2 \left(\frac{1}{p} \right) = -\log_2 p$$

In un evento E **altamente probabile** ($p \sim 1$), la quantità di informazione che ricaviamo dall'osservazione di E è **bassa**.

Se un evento E **non è probabile** ($p \sim 0$), la quantità di informazione che ricaviamo da E è **alta**.

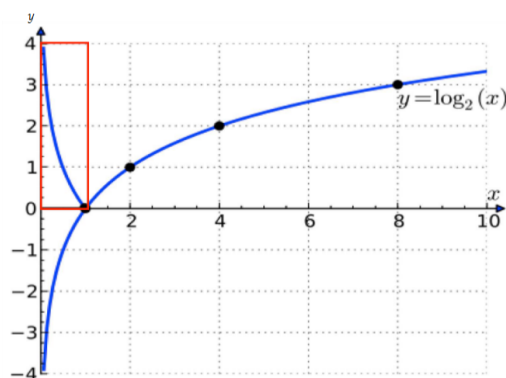


Figura 82: La curva della misura della quantità di informazioni

4.4.2 Informazioni previste

Se un esperimento ha n risultati ($E_1, \dots, E_n$), ciascuno con probabilità $p_1, p_2, \dots, p_n$, allora la quantità media di informazioni che otteniamo quando osserviamo un risultato generico dell'esperimento è l'informazione attesa.

$$H(E) = \sum_{i=1}^n p_i \cdot \log_2 \left(\frac{1}{p_i} \right) = - \sum_{i=1}^n p_i \cdot \log_2(p_i)$$

È il valore atteso di una variabile casuale pari a $\log_2 \frac{1}{p_i}$ che rappresenta la quantità di informazione di E_i che si verifica con probabilità p_i .

L'entropia è l'informazione attesa.

4.4.3 Entropia per la misurazione della purezza

L'entropia è una delle misure più comunemente utilizzate per l'impurità. Indichiamo con p_k la proporzione della classe k -esima nel set di dati D , dove $K = 1, 2, \dots, |\mathcal{Y}|$. (*)

Quindi, l'entropia è definita come:

$$Ent(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

Più bassa è $Ent(D)$, più alta sarà la purezza di D .

- Nel calcolo dell'entropia, $p \log_2 p = 0$ quando $p = 0$.
- Il minimo di $Ent(D)$ è 0 e il massimo è $\log_2 |Y|$.

(*): p_k è l'interpretazione frequentista della probabilità della classe k in D .

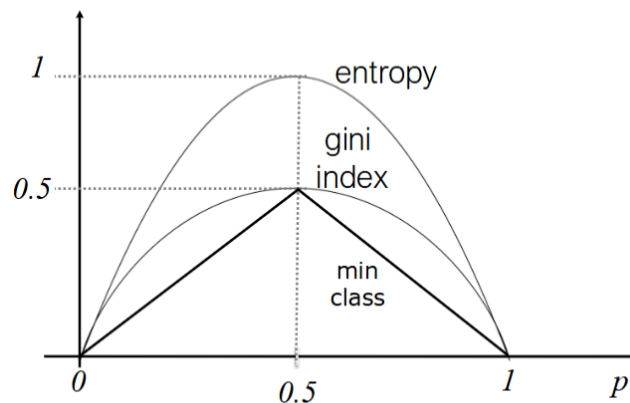


Figura 83: Funzione di impurità, entropia e indice di Gini

4.5 Information Gain

Supponiamo che la feature discreta a abbia V valori possibili $\{a^1, a^2, \dots, a^V\}$. Quindi, suddividendo il set di dati D per la feature a si ottengono V nodi figli, dove il nodo figlio D^v include tutti i campioni in D che assumono il valore a^v per la feature a . Quindi, il guadagno informativo derivante dalla suddivisione del set di dati D con la feature a viene calcolato come:

$$Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

$\frac{|D^v|}{|D|}$ è l'importanza di ciascun nodo. Maggiore è il numero di campioni, maggiore è l'impatto del nodo di diramazione.

In generale, maggiore è il guadagno di informazioni, maggiore è il miglioramento della purezza che possiamo aspettarci dalla suddivisione con le feature.

L'algoritmo dell'albero decisionale ID3 [Quinlan, 1986] utilizza il guadagno di informazioni come linea guida per la selezione delle feature di suddivisione.

4.5.1 L'impurità attesa come valore atteso di una variabile casuale

Il valore atteso di una variabile casuale V con valori del dominio $v_1, v_2, \dots, v_d$ che hanno probabilità di essere osservati in una realizzazione di V pari a $p_1, p_2, \dots, p_d$ rispettivamente, è dato da:

$$Expected\ value\ of\ (V) = \sum_{i=1}^{i=d} (p_i \cdot v_i)$$

Nel caso della diapositiva precedente, trattiamo Ent di un sottoinsieme D^v come un valore di una variabile casuale che descrive v e $\frac{|D^v|}{|D|}$ è la sua probabilità (probabilità che D^v si verifichi, supponendo che sia data dalla frequenza degli esempi nei set di dati).

Esempio concreto

ID	color	root	sound	texture	umbilicus	surface	ripe
1	green	curly	muffled	clear	hollow	hard	true
2	dark	curly	dull	clear	hollow	hard	true
3	dark	curly	muffled	clear	hollow	hard	true
4	green	curly	dull	clear	hollow	hard	true
5	light	curly	muffled	clear	hollow	hard	true
6	green	slightly curly	muffled	clear	slightly hollow	soft	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
8	dark	slightly curly	muffled	clear	slightly hollow	hard	true
9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	false
10	green	straight	crisp	clear	flat	soft	false
11	light	straight	crisp	blurry	flat	hard	false
12	light	curly	muffled	blurry	flat	soft	false
13	green	slightly curly	muffled	slightly blurry	hollow	hard	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	false
15	dark	slightly curly	muffled	clear	slightly hollow	soft	false
16	light	curly	muffled	blurry	flat	hard	false
17	green	curly	dull	slightly blurry	slightly hollow	hard	false

Figura 84: Dataset Watermelon 2.0

Questo dataset include 17 esempi di training. $p_1 = \frac{8}{17}$ di loro sono positivi e $p_2 = \frac{9}{17}$ sono negativi.

L'entropia del nodo radice è:

$$Ent(D) = - \sum_{k=1}^2 p_k \log_2 p_k = - \left(\frac{8}{17} \log_2 \frac{8}{17} + \frac{9}{17} \log_2 \frac{9}{17} \right) = 0.998$$

Se D viene diviso in base al colore, allora otteniamo tre sottoinsiemi:

$$D^2(\text{color} = \text{green}), D^2(\text{color} = \text{dark}), D^3(\text{color} = \text{light})$$

Il sottoinsieme D^1 include 6 esempi $\{1, 4, 6, 10, 13, 17\}$, nei quali $p_1 = \frac{3}{6}$ sono positivi e $p_2 = \frac{3}{6}$ sono negativi.

D^2 e D^3 possono essere discussi in modo simile: $|D^2| = 6$ con $p_1 = \frac{4}{6}$ e $p_2 = \frac{2}{6}$; $|D^3| = 5$ con $p_1 = \frac{1}{5}$ e $p_2 = \frac{4}{5}$.

L'entropia dei 3 nodi figli è:

$$Ent(D^1) = - \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) = 1.000$$

$$Ent(D^2) = - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.918$$

$$Ent(D^3) = - \left(\frac{1}{5} \log_2 \frac{1}{5} + \frac{4}{5} \log_2 \frac{4}{5} \right) = 0.722$$

Il guadagno informativo della suddivisione per colore è:

$$\begin{aligned}
 Gain(D, \text{color}) &= Ent(D) - \sum_{v=1}^3 \frac{|D^v|}{|D|} Ent(D^v) \\
 &= 0.998 - \left(\frac{6}{17} \cdot 1.000 + \frac{6}{17} \cdot 0.918 + \frac{5}{17} \cdot 0.722 \right) \\
 &= 0.109
 \end{aligned}$$

Esempio concreto

Similmente, possiamo calcolare il guadagno informativo sulle altre feature:

$$Gain(D, root) = 0.143$$

$$Gain(D, texture) = 0.381$$

$$Gain(D, surface) = 0.006$$

$$Gain(D, sound) = 0.141$$

$$Gain(D, umbilicus) = 0.289$$

Poiché la suddivisione per texture fornisce il massimo guadagno di informazioni, viene scelta come funzione di suddivisione.

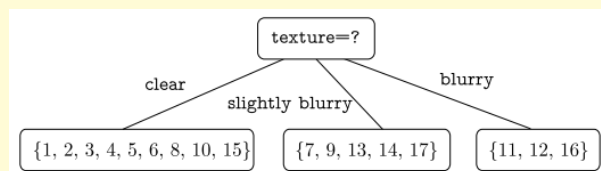


Figura 85: Divisione del nodo radice in base alla texture

Quindi, ogni nodo figlio viene ulteriormente suddiviso dall'algoritmo dell'albero decisionale. Possiamo ottenere l'albero decisionale finale:

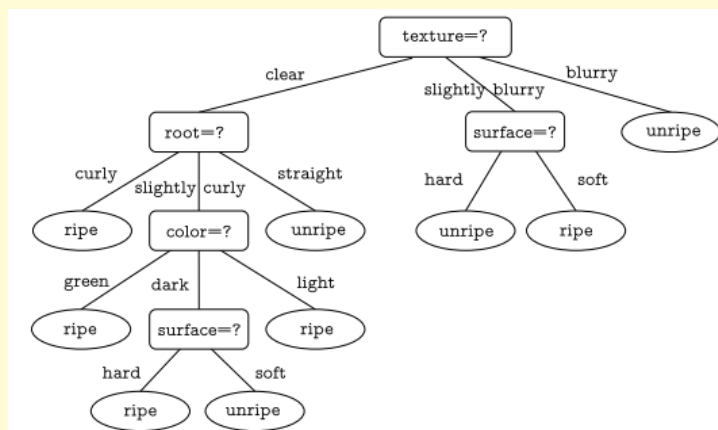


Figura 86: Albero decisionale finale

4.5.2 Bias

Se consideriamo l'ID della feature come una feature di suddivisione candidata, il suo guadagno informativo sarà molto più elevato di quello di qualsiasi altra feature in generale. Tuttavia, un tale albero decisionale non ha capacità di generalizzazione e non può predire efficacemente nuovi campioni.

Si scopre che il criterio di guadagno informativo è **sbilanciato verso caratteristiche con più valori possibili**.

4.6 Gain Ratio

Il rapporto di guadagno della feature a è definito come:

$$Gain_{ratio}(D, a) = \frac{Gain(D, a)}{IV(a)}$$

dove:

$$IV(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

è chiamato **valore intrinseco** della feature a . $IV(a)$ è grande quando la feature a ha molti valori possibili.

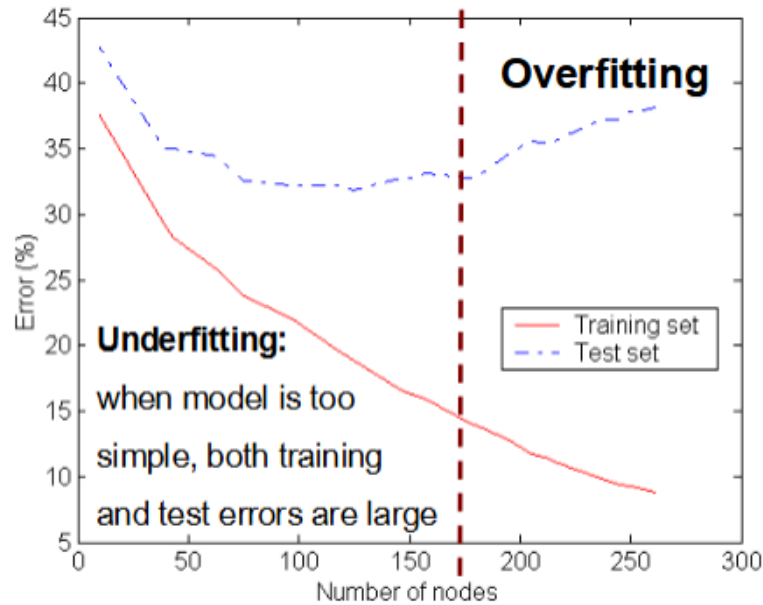


Figura 87: Esempio di overfitting e underfitting

4.6.1 Bias

Il rapporto di guadagno è sbilanciato verso le features con meno valori possibili.

L'algoritmo C4.5 [Quinlan, 1993] utilizza un metodo euristico: seleziona la feature con il rapporto di guadagno più elevato dall'insieme delle feature candidate con un **guadagno di informazione superiore alla media**.

4.7 Gini Index

Il valore di Gini del set di dati D è definito come:

$$Gini(D) = \sum_{k=1}^{|Y|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|Y|} p_k^2$$

rappresenta la probabilità di un'etichettatura errata se etichettiamo campioni in modo casuale con l'etichetta di classe k per probabilità p_k

Più basso è $Gini(D)$, più alta è la purezza di D .

L'indice di Gini della feature a è definito come:

$$Gini_{index}(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} Gini(D^v)$$

Selezioniamo la feature con l'indice di Gini più basso come feature di divisione:

$$a_* = \arg \min_{a \in A} Gini_{index}(D, a)$$

CART [Breiman et al., 1984] utilizza l'indice di Gini per selezionare la feature di divisione.

4.8 Pruning

La **potatura** è la strategia principale degli algoritmi di apprendimento degli alberi decisionali per **gestire l'overfitting**.

Se ci sono troppi rami, il learner potrebbe essere ingannato dalle peculiarità dei campioni di addestramento e considerarli erroneamente come la verità di fondo. Strategie generali di potatura:

- **Pre-pruning** (con una propensione verso modelli più compatti)

- **Post-pruning** (con una propensione verso modelli più estesi)

Come valutare la capacità di generalizzazione dopo il pruning?

Possiamo utilizzare il metodo hold-out per riservare una parte dei dati come set di convalida per la valutazione delle prestazioni.

	ID	color	root	sound	texture	umbilicus	surface	ripe
Training Set	1	green	curly	muffled	clear	hollow	hard	true
	2	dark	curly	dull	clear	hollow	hard	true
	3	dark	curly	muffled	clear	hollow	hard	true
	6	green	slightly curly	muffled	clear	slightly hollow	soft	true
	7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
	10	green	straight	crisp	clear	flat	soft	false
	14	light	slightly curly	dull	slightly blurry	hollow	hard	false
	15	dark	slightly curly	muffled	clear	slightly hollow	soft	false
	16	light	curly	muffled	blurry	flat	hard	false
	17	green	curly	dull	slightly blurry	slightly hollow	hard	false
Validation Set	4	green	curly	dull	clear	hollow	hard	true
	5	light	curly	muffled	clear	hollow	hard	true
	8	dark	slightly curly	muffled	clear	slightly hollow	hard	true
	9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	false
	11	light	straight	crisp	blurry	flat	hard	false
	12	light	curly	muffled	blurry	flat	soft	false
	13	green	slightly curly	muffled	slightly blurry	hollow	hard	false

Figura 88: Suddivisione del dataset per il pruning

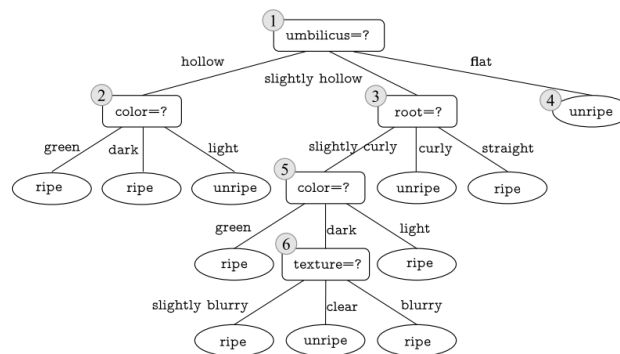


Figura 89: Albero decisionale non potato

4.8.1 Pre-pruning

Il pre-pruning decide se potare i nodi figlio immediatamente dopo la loro generazione, suddividendo il loro nodo antenato.

Decide se suddividere confrontando le capacità di generalizzazione prima e dopo la suddivisione.

- Se l'accuratezza della convalida diminuisce dopo il pruning (dei nodi figlio), i nodi figlio vengono mantenuti e la suddivisione viene confermata.
- In caso contrario, il pruning elimina i nodi figlio e la suddivisione viene interrotta.

Se non viene eseguita alcuna suddivisione, il **nodo antenato** viene contrassegnato come **nodo foglia** e la sua etichetta viene impostata sulla **classe di maggioranza**.

Funziona dall'alto verso il basso dell'albero, man mano che l'albero cresce.

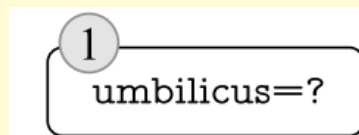
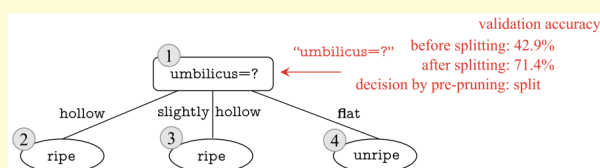


Figura 90: Prima dello splitting: 57.14%

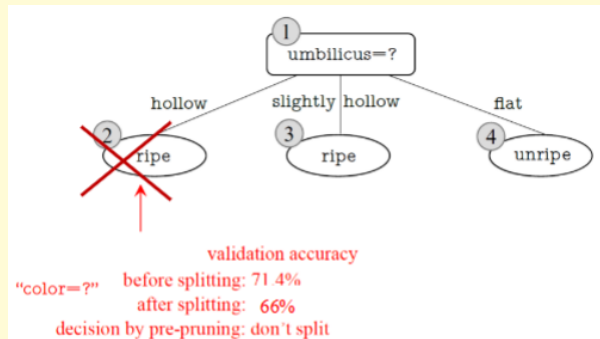
Se non viene eseguita alcuna suddivisione, questo nodo viene contrassegnato come nodo foglia e la sua etichetta viene impostata sulla classe di maggioranza (ovvero, **immaturo**(unripe)). Nel set di convalida, {9, 11, 12, 13} sono classificati correttamente. Quindi l'accuratezza della convalida è:

$$\frac{4}{7}100\% = 57.14\%$$

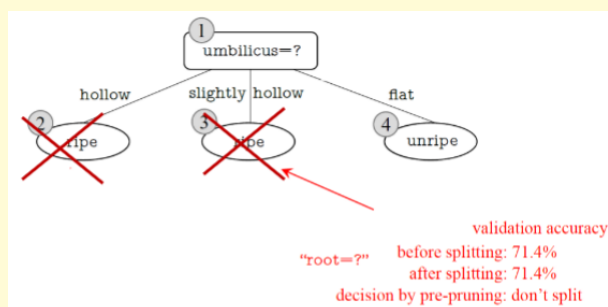


Dopo la suddivisione, i campioni vengono inseriti in 4 nodi figlio. Contrassegniamo questi 3 nodi come nodi foglia e impostiamo le etichette sulle classi di maggioranza. L'accuratezza della convalida è quindi:

$$\frac{5}{7}100\% = 71.4\%$$

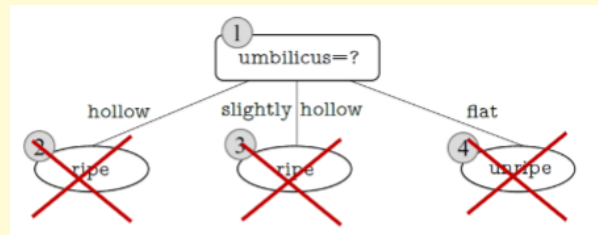


La strategia di pre-potatura impedisce la divisione del nodo 2.



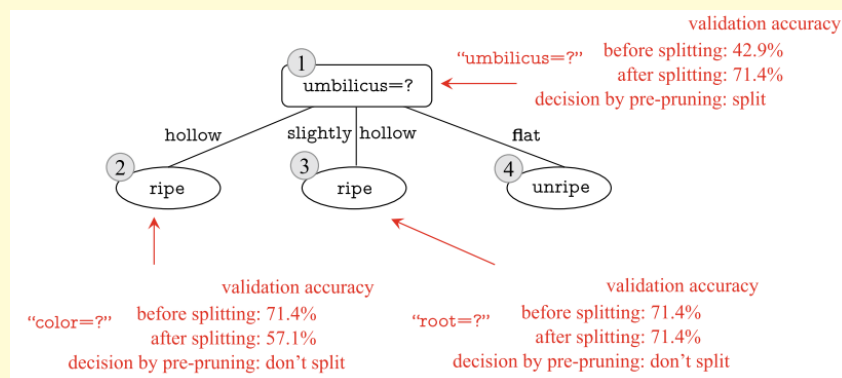
La strategia di pre-potatura impedisce la divisione del nodo 3.
 Ha una propensione verso la decisione di non dividersi

Esempio



Per il nodo 4, non è necessaria alcuna suddivisione poiché tutti i campioni appartengono alla stessa classe.

Infine, otteniamo un albero decisionale con una sola suddivisione. Tale albero decisionale è chiamato **decision stump**.



Vantaggi del pre-pruning:

- Ridurre il rischio di **overfitting**
- Ridurre i costi computazionali di addestramento e test

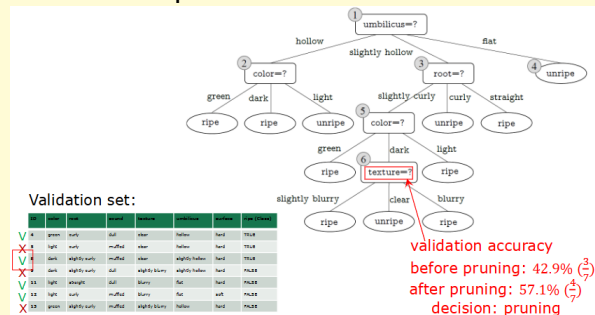
Svantaggi del pre-pruning:

- **Rischio di underfitting:** sebbene alcuni rami siano ostacolati dalla pre-potatura a causa di un miglioramento scarso o addirittura negativo della capacità di generalizzazione, è comunque possibile che i loro **successivi splits possano portare a un miglioramento significativo**. Questi rami vengono potati a causa della natura greedy della pre-potatura e ciò può comportare il rischio di underfitting.

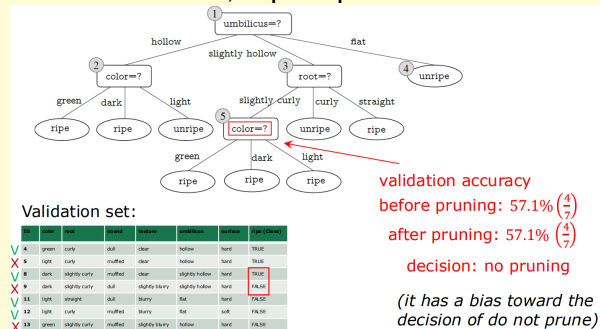
4.8.2 Post-pruning

La post-potatura consente a un albero decisionale di crescere fino a diventare un albero completo. Quindi, è necessaria una strategia bottom-up per esaminare ogni nodo non foglia nell'albero decisionale completamente cresciuto.

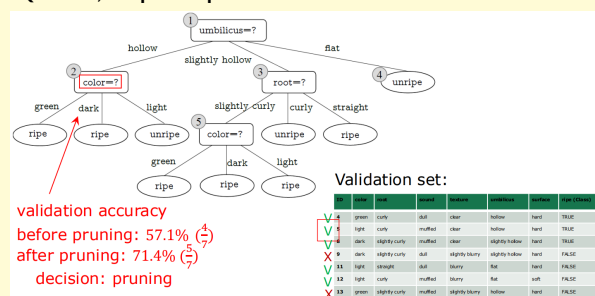
Il nodo 6 è il primo ad essere esaminato tramite post-pruning.



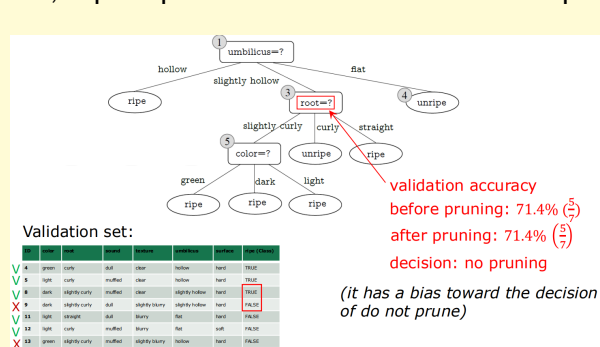
Successivamente, la post-potatura esamina il nodo 5.



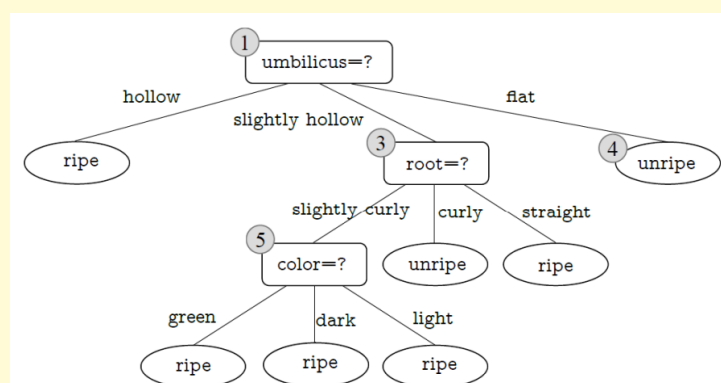
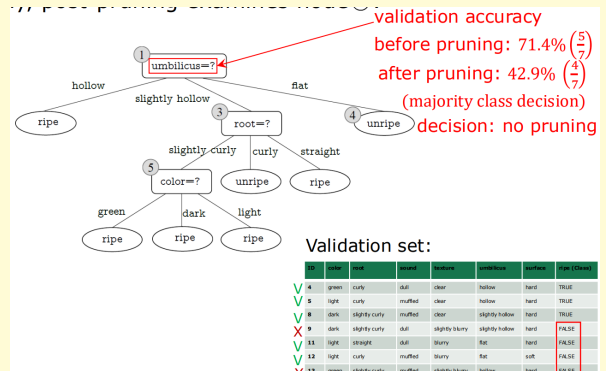
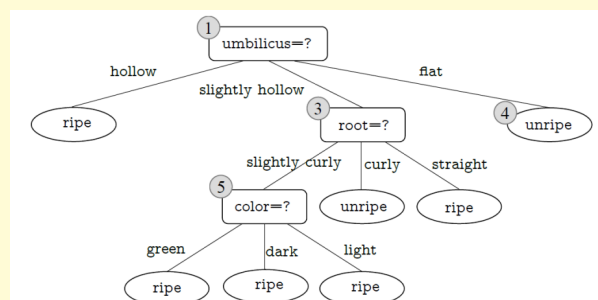
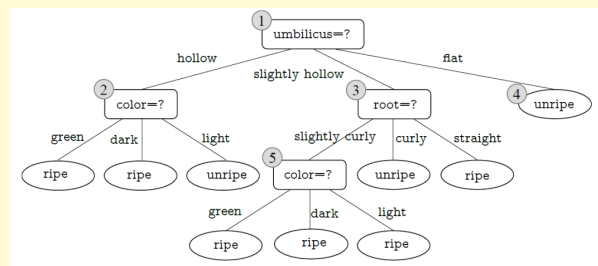
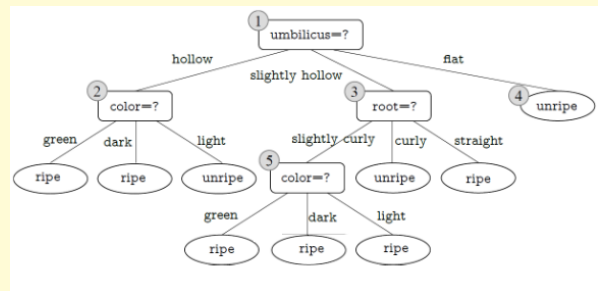
Quindi, la post-potatura esamina il nodo 2.



Poi, la post-potatura esamina il nodo 3 e dopo il nodo 1.



Infine, l'albero decisionale post-potatura è:



Vantaggi del post-pruning:

- La post-potatura mantiene più rami rispetto alla pre-potatura. In generale, la post-potatura è **meno soggetta a underfitting** e porta a una migliore capacità di generalizzazione rispetto alla pre-potatura.

Svantaggi del post-pruning:

- Il **tempo di addestramento del post-pruning è molto più lungo** poiché richiede una strategia bottom-up per esaminare ogni nodo non foglia in un albero decisionale completamente sviluppato.

4.9 Valori continui

Strategia di discretizzazione (bipartizione) Dato un insieme di dati D e una feature continua a , supponiamo che si osservino n valori di a in D e che questi valori vengano ordinati in ordine crescente, indicati con $a_1, a_2, \dots, a_n$. Con un punto di divisione t , D viene partizionato nei sottoinsiemi D_t^- e D_t^+ , dove D_t^- include i campioni con valore di a non maggiore di t , e D_t^+ include i campioni con valore di a maggiore di t . Ci sono $n - 1$ elementi nel seguente insieme di punti di divisione candidati:

$$T_a = \left\{ \frac{a^i + a^{i+1}}{2} \mid 1 \leq i \leq n - 1 \right\}$$

dove il punto medio $\frac{a^i + a^{i+1}}{2}$ viene utilizzato come punto di divisione candidato per l'intervallo $[a^i, a^{i+1}]$.

4.9.1 Densità

ID	color	root	sound	texture	umbilicus	surface	density	sugar	ripe
1	green	curly	muffled	clear	hollow	hard	0.697	0.460	true
2	dark	curly	dull	clear	hollow	hard	0.774	0.376	true
3	dark	curly	muffled	clear	hollow	hard	0.634	0.264	true
4	green	curly	dull	clear	hollow	hard	0.608	0.318	true
5	light	curly	muffled	clear	hollow	hard	0.556	0.215	true
6	green	slightly curly	muffled	clear	slightly hollow	soft	0.403	0.237	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	0.481	0.149	true
8	dark	slightly curly	muffled	clear	slightly hollow	hard	0.437	0.211	true
9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	0.666	0.091	false
10	green	straight	crisp	clear	flat	soft	0.243	0.267	false
11	light	straight	crisp	blurry	flat	hard	0.245	0.057	false
12	light	curly	muffled	blurry	flat	soft	0.343	0.099	false
13	green	slightly curly	muffled	slightly blurry	hollow	hard	0.639	0.161	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	0.657	0.198	false
15	dark	slightly curly	muffled	clear	slightly hollow	soft	0.360	0.370	false
16	light	curly	muffled	blurry	flat	hard	0.593	0.042	false
17	green	curly	dull	slightly blurry	slightly hollow	hard	0.719	0.103	false

Figura 91: Dataset Watermelon 3.0

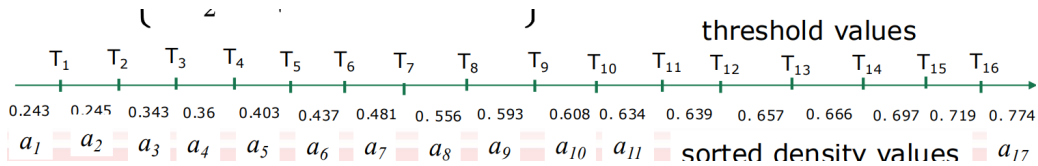


Figura 92: Valori di soglia candidati per la feature continua

I punti di divisione vengono esaminati allo stesso modo delle feature discrete e vengono selezionati i punti di divisione ottimali per i nodi di divisione.

$$\begin{aligned}
 \text{Gain}(D, a) &= \max_{t \in T_a} \text{Gain}(D, a, t) \\
 &= \max_{t \in T_a} \text{Ent}(D) - \sum_{\lambda \in \{l, r\}} \frac{|D_t^\lambda|}{|D|} \text{Ent}(D_t^\lambda)
 \end{aligned}$$

dove $Gain(D, a, t)$ è il guadagno di informazione del bi-partizionamento D per t , e il punto di divisione con il valore maggiore è $Gain(D, a, t)$ selezionato e D_t^l e D_t^r sono rispettivamente gli intervalli sinistro e destro definiti dalla soglia t .

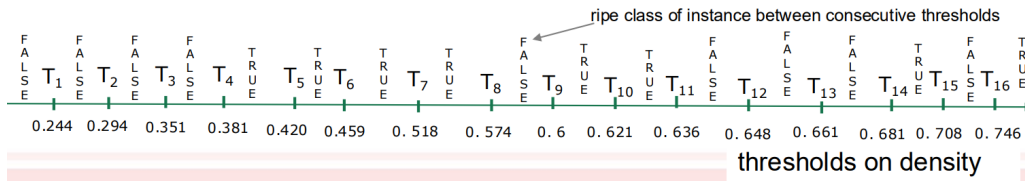


Figura 93: Valori di soglia candidati per la feature continua "densità"

Esempio concreto

ID	color	root	sound	texture	umbilicus	surface	density	sugar	ripe
1	green	curly	muffled	clear	hollow	hard	0.697	0.460	true
2	dark	curly	dull	clear	hollow	hard	0.774	0.376	true
3	dark	curly	muffled	clear	hollow	hard	0.634	0.264	true
4	green	curly	dull	clear	hollow	hard	0.608	0.318	true
5	light	curly	muffled	clear	hollow	hard	0.556	0.215	true
6	green	slightly curly	muffled	clear	slightly hollow	soft	0.403	0.237	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	0.481	0.149	true
8	dark	slightly curly	muffled	clear	slightly hollow	hard	0.437	0.211	true
9	dark	slightly curly	dull	slightly blurry	slightly hollow	hard	0.666	0.091	false
10	green	straight	crisp	clear	flat	soft	0.243	0.267	false
11	light	straight	crisp	blurry	flat	hard	0.245	0.057	false
12	light	curly	muffled	blurry	flat	soft	0.343	0.099	false
13	green	slightly curly	muffled	slightly blurry	hollow	hard	0.639	0.161	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	0.657	0.198	false
15	dark	slightly curly	muffled	clear	slightly hollow	soft	0.360	0.370	false
16	light	curly	muffled	blurry	flat	hard	0.593	0.042	false
17	green	curly	dull	slightly blurry	slightly hollow	hard	0.719	0.103	false

Per la feature densità, il suo set di punti di divisione candidati include 16 valori:

$$T_{density} = \{0.244, 0.294, 0.351, 0.381, 0.420, 0.459, 0.518, 0.574, 0.600, 0.621, 0.636, 0.648, 0.661, 0.681, 0.708, 0.746\}$$

Il guadagno di informazione della densità è 0.262 e il corrispondente punto di divisione è 0.381. A differenza delle feature discrete, una **feature continua** può essere **utilizzata come feature di suddivisione più di una volta** in una sequenza decisionale.

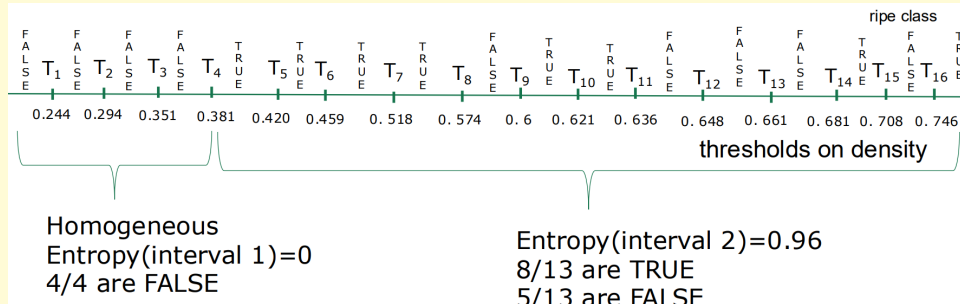


Figura 94: Calcolo dell'entropia negli intervalli con feature continue

$$Information\ Gain = 0.998 - \left(\frac{4}{17} \cdot 0 + \frac{13}{17} \cdot 0.96 \right) = 0.262$$

4.10 Valori mancanti

Nella pratica, i dati sono spesso incompleti, ovvero alcuni valori delle feature risultano mancanti in alcuni campioni.

Possiamo semplicemente scartare i campioni incompleti? Si tratta di un enorme spreco di dati.

L'apprendimento da campioni incompleti solleva due problemi:

1. Come scegliere le funzioni di suddivisione quando ci sono valori mancanti
2. Come dividere un campione con il valore della funzione di divisione mancante?

Dato un training set D e una feature a , sia $\tilde{D}$ il sottoinsieme dei campioni in D che hanno valori di a , $\tilde{D}_v$ denota il sottoinsieme dei campioni in D che assumono il valore a^v , $\tilde{D}_k$ denota il sottoinsieme dei campioni in $\tilde{D}$ appartenenti alla k -esima classe.

ID	color	root	sound	texture	umbilicus	surface	ripe
1	-	curly	muffled	clear	hollow	hard	true
2	dark	curly	dull	clear	hollow	-	true
3	dark	curly	-	clear	hollow	hard	true
4	green	curly	dull	clear	hollow	hard	true
5	-	curly	muffled	clear	hollow	hard	true
6	green	slightly curly	muffled	clear	-	soft	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
8	dark	slightly curly	muffled	-	slightly hollow	hard	true
9	dark	-	dull	slightly blurry	slightly hollow	hard	false
10	green	straight	crisp	-	flat	soft	false
11	light	straight	crisp	blurry	flat	-	false
12	light	curly	-	blurry	flat	soft	false
13	-	slightly curly	muffled	slightly blurry	hollow	hard	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	false
15	dark	slightly curly	muffled	clear	-	soft	false
16	light	curly	muffled	blurry	flat	hard	false
17	green	-	dull	slightly blurry	slightly hollow	hard	false

Figura 95: Dataset Watermelon 2.0 con valori mancanti

Assegniamo un peso w_x ad ogni campion x , e definiamo:

- la proporzione di campioni senza valori mancanti

$$\rho = \frac{\sum_{x \in \tilde{D}} w_x}{\sum_{x \in D} w_x}$$

- la proporzione della classe k -esima in tutti i campioni senza valori mancanti

$$\tilde{p}_k = \frac{\sum_{x \in \tilde{D}_k} w_x}{\sum_{x \in \tilde{D}} w_x} \quad (1 \leq k \leq |\mathcal{Y}|)$$

- la proporzione di campioni che assumono il valore della feature a^v in tutti i campioni senza valori mancanti

$$\tilde{r}_v = \frac{\sum_{x \in \tilde{D}_v} w_x}{\sum_{x \in \tilde{D}} w_x} \quad (1 \leq v \leq V)$$

Esempio

La proporzione di campioni senza valori mancanti:

$$\begin{aligned}\rho_{color} &= \frac{14}{17} & \rho_{sound} &= \frac{15}{17} & \rho_{umbilicus} &= \frac{15}{17} \\ \rho_{root} &= \frac{15}{17} & \rho_{texture} &= \frac{15}{17} & \rho_{surface} &= \frac{15}{17}\end{aligned}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per il **colore**:

$$\tilde{p}_{ripe} = \frac{6}{14} \quad \tilde{p}_{unripe} = \frac{8}{14}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per la **radice**:

$$\tilde{p}_{ripe} = \frac{8}{15} \quad \tilde{p}_{unripe} = \frac{7}{15}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per il **suono**:

$$\tilde{p}_{ripe} = \frac{7}{15} \quad \tilde{p}_{unripe} = \frac{8}{15}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per il **texture**:

$$\tilde{p}_{ripe} = \frac{7}{15} \quad \tilde{p}_{unripe} = \frac{8}{15}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per il **umbilicus**:

$$\tilde{p}_{ripe} = \frac{7}{15} \quad \tilde{p}_{unripe} = \frac{8}{15}$$

la proporzione della classe k-esima in tutti i campioni senza valori mancanti per il **superficie**:

$$\tilde{p}_{ripe} = \frac{7}{15} \quad \tilde{p}_{unripe} = \frac{8}{15}$$

la proporzione di campioni che assumono il valore caratteristico a^v in tutti i campioni senza valori mancanti

$$\begin{aligned}\tilde{r}_{dark} &= \frac{6}{14} & \tilde{r}_{curly} &= \frac{7}{15} & \tilde{r}_{muffled} &= \frac{8}{15} & \tilde{r}_{clear} &= \frac{7}{15} & \tilde{r}_{hollow} &= \frac{7}{15} \\ \tilde{r}_{green} &= \frac{4}{14} & \tilde{r}_{sl.curly} &= \frac{6}{15} & \tilde{r}_{dull} &= \frac{5}{15} & \tilde{r}_{sl.blurry} &= \frac{5}{15} & \tilde{r}_{rl.hollow} &= \frac{4}{15} \\ \tilde{r}_{light} &= \frac{4}{14} & \tilde{r}_{straight} &= \frac{2}{15} & \tilde{r}_{crisp} &= \frac{2}{15} & \tilde{r}_{blurry} &= \frac{3}{15} & \tilde{r}_{flat} &= \frac{4}{15} \\ & & & & & & \tilde{r}_{hard} &= \frac{10}{15} & \tilde{r}_{soft} &= \frac{5}{15}\end{aligned}$$

Per la domanda 1 (come scegliere le caratteristiche di suddivisione), con le definizioni di cui sopra, estendiamo il guadagno informativo a:

$$\begin{aligned}Gain(D, a) &= \rho \times \left(Ent(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \right) \\ &= \rho \times \left(Ent(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \right)\end{aligned}$$

dove:

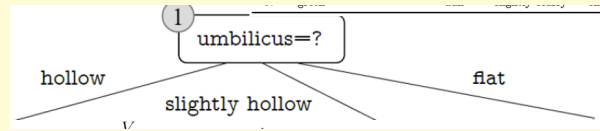
$$Ent(\tilde{D}) = - \sum_{k=1}^{|\mathcal{Y}|} \tilde{p}_k \log_2 \tilde{p}_k$$

ρ è la proporzione di campioni senza valori mancanti, $\tilde{r}_v$ è la proporzione di campioni che assumono il valore di feature v nei campioni senza i valori mancanti, e $\tilde{p}_k$ è la proporzione della classe k-esima nei campioni senza valori mancanti.

Esempio

ID	color	root	sound	texture	umbilicus	surface	ripe
1	-	curly	muffled	clear	hollow	hard	true
2	dark	curly	dull	clear	hollow	-	true
3	dark	curly	-	clear	hollow	hard	true
4	green	curly	dull	clear	hollow	hard	true
5	-	curly	muffled	clear	hollow	hard	true
6	green	slightly curly	muffled	clear	-	soft	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
8	dark	slightly curly	muffled	-	slightly hollow	hard	true
9	dark	-	dull	slightly blurry	slightly hollow	hard	false
10	green	straight	crisp	-	flat	soft	false
11	light	straight	crisp	blurry	flat	-	false
12	light	curly	-	blurry	flat	soft	false
13	-	slightly curly	muffled	slightly blurry	hollow	hard	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	false
15	dark	slightly curly	muffled	clear	-	soft	false
16	light	curly	muffled	blurry	flat	hard	false
17	green	-	dull	slightly blurry	slightly hollow	hard	false

Figura 96: Dataset Watermelon 2.0 con valori mancanti



$$Gain(D, a) = \rho \times \left(Ent(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \right)$$

$$Ent(\tilde{D}) = -\tilde{p}_{ripe} \log_2(\tilde{p}_{ripe}) - \tilde{p}_{unripe} \log_2(\tilde{p}_{unripe})$$

$$p_{umbilicus} = \frac{15}{17} \quad \tilde{p}_{ripe} = \frac{7}{15} \quad \tilde{p}_{unripe} = \frac{8}{15}$$

$$Ent(\tilde{D}) = -\frac{7}{15} \cdot \log_2\left(\frac{7}{15}\right) - \frac{8}{15} \cdot \log_2\left(\frac{8}{15}\right) = 0.996$$

ID	color	root	sound	texture	umbilicus	surface	ripe (Class)
1	—	curly	muffled	clear	hollow	hard	TRUE
2	dark	curly	dull	clear	hollow	—	TRUE
3	dark	curly	—	clear	hollow	hard	TRUE
4	green	curly	dull	clear	hollow	hard	TRUE
5	—	curly	muffled	clear	hollow	hard	TRUE
13	—	slightly curly	muffled	slightly blurry	hollow	hard	FALSE
14	light	slightly curly	dull	slightly blurry	hollow	hard	FALSE

$$\tilde{r}_{hollow} = 7/15$$

$$\sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \Rightarrow \tilde{r}_{hollow} \cdot Ent(\tilde{D}^{hollow}) = \frac{7}{15} \cdot \left[-\frac{5}{7} \cdot \log_2\left(\frac{5}{7}\right) - \frac{2}{7} \cdot \log_2\left(\frac{2}{7}\right) \right] = 0.402$$

$$\sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \Rightarrow \tilde{r}_{sl.hollow} \cdot Ent(\tilde{D}^{sl.hollow}) = \frac{6}{15} \cdot \left[-\frac{3}{6} \cdot \log_2\left(\frac{3}{6}\right) - \frac{3}{6} \cdot \log_2\left(\frac{3}{6}\right) \right] = 0.4$$

$$\sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \Rightarrow \tilde{r}_{flat} \cdot Ent(\tilde{D}^{flat}) = \frac{4}{15} \cdot \left[-\frac{4}{4} \cdot \log_2\left(\frac{4}{4}\right) - 0 \cdot \log_2(0) \right] = 0$$

Esempio

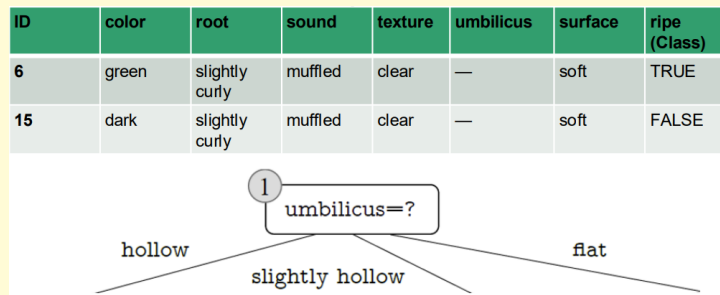
$$\begin{aligned}
 Gain(D, umbilicus) &= \rho_{umbilicus} \times \left(Ent(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v) \right) \\
 &= \frac{15}{17} \times (0.996 - (0.402 + 0.4 + 0)) \\
 &= 0.171
 \end{aligned}$$

Per Q2 (come suddividere un campione con la funzione di suddivisione valore mancante):

- quando il valore di a è noto, inseriamo il campione x nel nodo figlio corrispondente senza modificare il peso w_x .
- quando il valore di a è sconosciuto, inseriamo il campione x in tutti i nodi figli e impostiamo il suo peso nel nodo figlio di valore a^v a $\tilde{r}_v \cdot w_x$.

In altre parole, inseriamo lo stesso campione in diversi nodi figli con probabilità diverse.

Esempio 1



$$\tilde{r}_{hollow} * 1 = \frac{7}{15} \quad \tilde{r}_{sl.hollow} * 1 = \frac{4}{15} \quad \tilde{r}_{flat} * 1 = \frac{4}{15}$$

Esempio 2

ID	color	root	sound	texture	umbilicus	surface	ripe
1	-	curly	muffled	clear	hollow	hard	true
2	dark	curly	dull	clear	hollow	-	true
3	dark	curly	-	clear	hollow	hard	true
4	green	curly	dull	clear	hollow	hard	true
5	-	curly	muffled	clear	hollow	hard	true
6	green	slightly curly	muffled	clear	-	soft	true
7	dark	slightly curly	muffled	slightly blurry	slightly hollow	soft	true
8	dark	slightly curly	muffled	-	slightly hollow	hard	true
9	dark	-	dull	slightly blurry	slightly hollow	hard	false
10	green	straight	crisp	-	flat	soft	false
11	light	straight	crisp	blurry	flat	-	false
12	light	curly	-	blurry	flat	soft	false
13	-	slightly curly	muffled	slightly blurry	hollow	hard	false
14	light	slightly curly	dull	slightly blurry	hollow	hard	false
15	dark	slightly curly	muffled	clear	-	soft	false
16	light	curly	muffled	blurry	flat	hard	false
17	green	-	dull	slightly blurry	slightly hollow	hard	false

All'inizio, il nodo radice include tutti i 17 campioni in D , e tutti i campioni hanno peso pari a 1. Prendendo il **colore** come esempio, l'insieme dei campioni senza valori mancanti di questa caratteristica, indicato dalla tilde $\tilde{D}$, include 14 campioni. L'entropia di D è calcolata come:

$$Ent(\tilde{D}) = - \sum_{k=1}^2 \tilde{p}_k \log_2 \tilde{p}_k = - \left(\frac{6}{14} \log_2 \frac{6}{14} + \frac{8}{14} \log_2 \frac{8}{14} \right) = 0.985$$

Sia $\tilde{D}^1$, $\tilde{D}^2$, e $\tilde{D}^3$ i sottoinsiemi di campioni con $color = green$, $color = dark$ e $color = light$ rispettivamente.

Quindi abbiamo:

$$Ent(\tilde{D}^1) = - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1.000$$

$$Ent(\tilde{D}^2) = - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.918$$

$$Ent(\tilde{D}^3) = - \left(\frac{0}{4} \log_2 \frac{0}{4} + \frac{4}{4} \log_2 \frac{4}{4} \right) = 0.000$$

Il guadagno di informazioni del colore per il sottoinsieme $\tilde{D}$ è:

$$\begin{aligned}
 Gain(\tilde{D}, color) &= Ent(\tilde{D}) - \sum_{v=1}^3 \tilde{r}_v Ent(\tilde{D}^v) \\
 &= 0.985 - \left(\frac{4}{14} \cdot 1.000 + \frac{6}{14} \cdot 0.918 + \frac{4}{14} \cdot 0.000 \right) \\
 &= 0.306
 \end{aligned}$$

Il guadagno di informazioni del colore per il set di dati D è:

$$Gain(D, color) = \rho \times Gain(\tilde{D}, color) = \frac{14}{17} \times 0.306 = 0.252$$

Due valori mancanti: esempio su umbilicus e colore

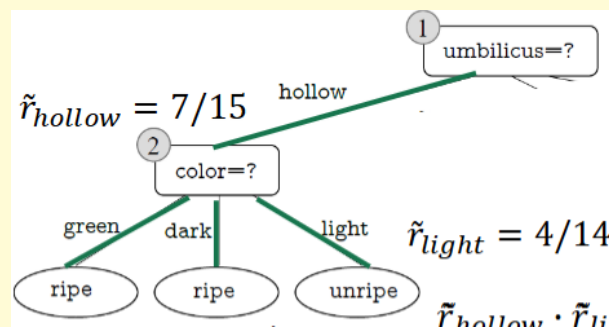
Se avessimo questo albero e questo esempio da predire con valori mancanti per umbilicus e colore:

ID	color	root	sound	texture	umbilicus	surface	ripe (Class)
6	—	slightly curly	muffled	clear	—	soft	TRUE

Seguirebbe anche il percorso con entrambi i valori mancanti ma i suoi pesi sarebbero moltiplicati.

$$\tilde{r}_{hollow} = \frac{7}{15} \quad \tilde{r}_{green} = \frac{4}{14} \quad \tilde{r}_{dark} = \frac{6}{14} \quad \tilde{r}_{light} = \frac{4}{14}$$

$$\tilde{r}_{hollow} \cdot \tilde{r}_{green} \cdot 1 = 0.13 \quad \tilde{r}_{hollow} \cdot \tilde{r}_{dark} \cdot 1 = 0.20 \quad \tilde{r}_{hollow} \cdot \tilde{r}_{light} \cdot 1 = 0.13$$



4.11 Alberi decisionali multivariati

Per gli alberi decisionali, i confini decisionali sono paralleli agli assi.

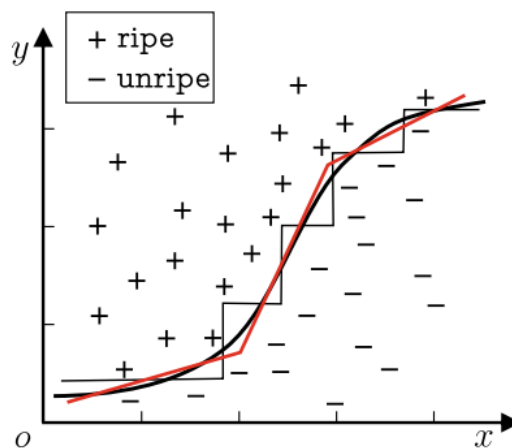


Figura 97: Albero decisionale multivariato

Ogni nodo non foglia non è più un test per una particolare feature, ma una combinazione lineare di feature.

Ogni nodo non foglia è un classificatore lineare nella forma $\sum_{i=1}^d w_i a_i = t$, dove w_i è il peso della feature a_i , e w_i e t vengono appresi dal set di dati e dal set di features del nodo.

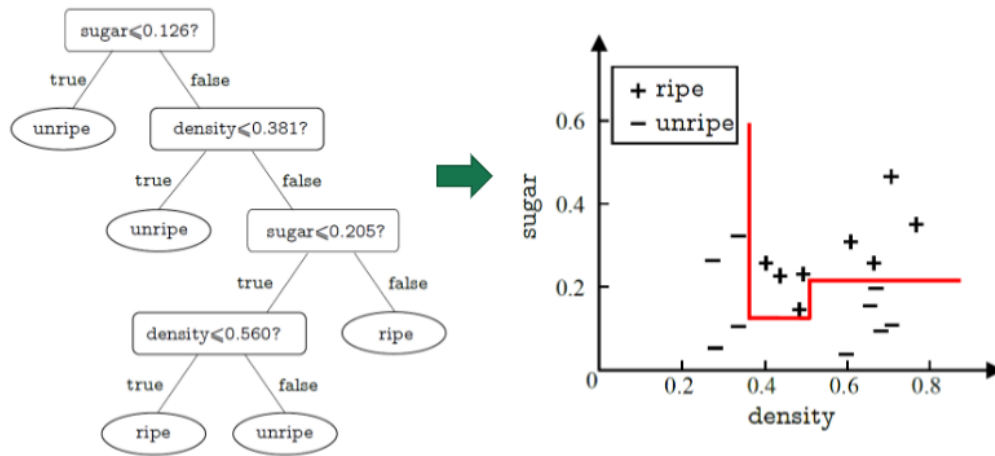


Figura 98: Esempio di albero decisionale multivariato

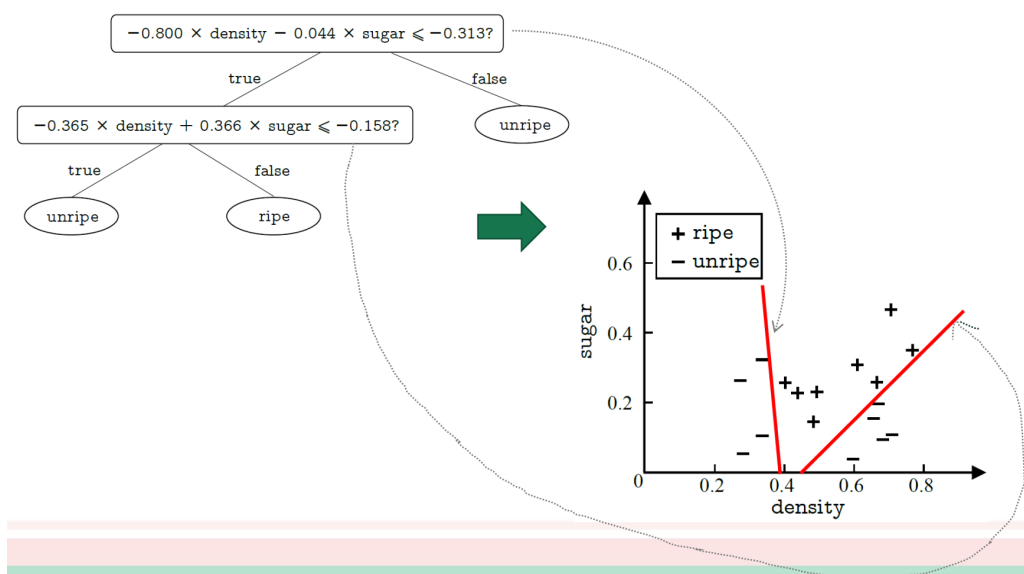


Figura 99: Esempio di albero decisionale multivariato

5 Reti neurali

5.1 Storia delle reti neurali

5.1.1 Prima fase

1943: McCulloch e Pitts proposero il primo modello neurale, ovvero il **modello neuronale M-P**, e dimostrarono in linea di principio che la rete neurale artificiale può calcolare qualsiasi funzione aritmetica e logica.

1949: Hebb pubblicò il libro "The Organization of Behavior" e propose il meccanismo dell'apprendimento biologico dei neuroni, ovvero la **regola di apprendimento di Hebb**.

1958: Rosenblatt ha proposto il **Perceptron** e la sua regola di apprendimento

1960: Widrow e Hoff propongono Adaline e l'**algoritmo dei minimi quadrati medi (LMS)**

1969: Minsky e Papert pubblicarono il libro "Perceptrons", in cui sottolineavano che le **reti neurali a singolo strato non possono risolvere problemi non lineari e non si sa se sia possibile addestrare reti a più strati**. Questa conclusione spinse direttamente la ricerca sulle reti neurali verso un'"era glaciale".

5.1.2 Seconda fase

1982: Il fisico Hopfield propose una rete ricorsiva con memoria associativa e potenza di calcolo ottimizzata, ovvero la **rete di Hopfield**.

1986: Rumelhart et al. hanno pubblicato il libro "Parallel Distributed Processing: Explorations in the Microstructures of Cognition", in cui l'algoritmo BP è stato reinventato.

1987: L'IEEE organizza la prima conferenza internazionale sulle reti neurali a San Diego, California (ICNN).

Primi anni del '90: Sono emerse la teoria dell'apprendimento statistico e la SVM, mentre le reti neurali soffrivano della mancanza di teorie, basandosi pesantemente su tentativi ed errori e piene di trucchi.

Fine anni '90: Le reti neurali entrano in un altro inverno.

5.1.3 Terza fase

2006: Hinton ha proposto la Deep Belief Network (**DBN**), che rende l'ottimizzazione dei modelli profondi relativamente semplice attraverso "pre-training + fine-tuning".

2012: Il team di Hinton ha partecipato alla competizione ImageNet e ha vinto il campionato dell'anno con un punteggio del 10% superiore al secondo classificato utilizzando il modello CNN

Attualmente: Con l'avvento del cloud computing e dell'era dei big data, la potenza di calcolo è notevolmente migliorata, facendo sì che i modelli di deep learning ottenessero un grande successo in vari campi.

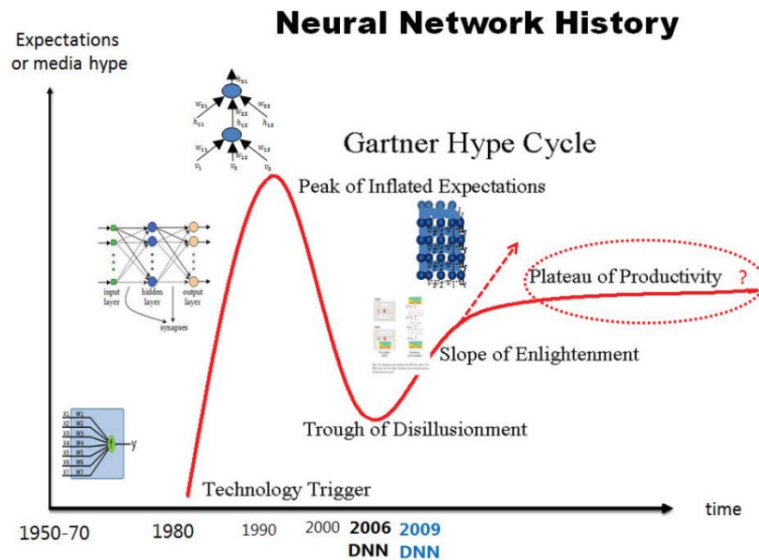


Figura 100: Storia delle reti neurali

5.2 Modello di neurone

5.2.1 Definizione di rete neurale

"Le **reti neurali** sono reti interconnesse massicciamente parallele composte da **elementi semplici** e dalle loro **organizzazioni gerarchiche**, che sono destinate a interagire con gli oggetti del mondo reale allo stesso modo dei **sistemi nervosi biologici**" [Kohonen, 1988]

- Nel contesto dell'apprendimento automatico, le reti neurali si riferiscono all' "**apprendimento tramite reti neurali**", o in altre parole, all'intersezione tra la ricerca sull'apprendimento automatico e la ricerca sulle reti neurali.
- L'elemento base delle reti neurali è il neurone, che è l'"elemento semplice" nella definizione di cui sopra.
- Reti neurali biologiche: i neuroni, quando "**eccitati**", inviano neurotrasmettitori ai neuroni interconnessi per modificarne il potenziale elettrico. Quando il potenziale elettrico supera una soglia, il neurone viene attivato e invierà neurotrasmettitori ad altri neuroni.

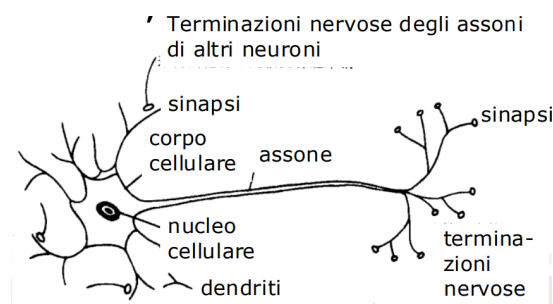


Figura 101: Struttura di un neurone biologico

5.2.2 Modello di neurone M-P (McCulloch-Pitts, 1943)

Input: ricevere segnali di input da n neuroni.

Processo: La somma ponderata dei segnali ricevuti viene confrontata con la soglia

Output: il segnale di uscita è prodotto dalla funzione di attivazione

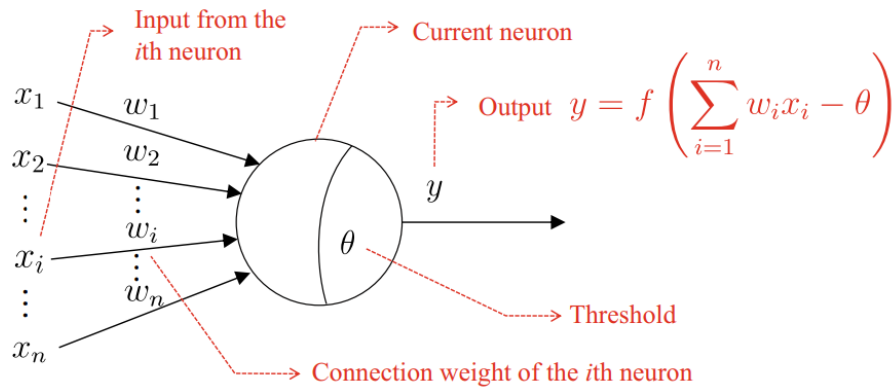


Figura 102: Modello del neurone M-P

5.2.3 Funzione di attivazione

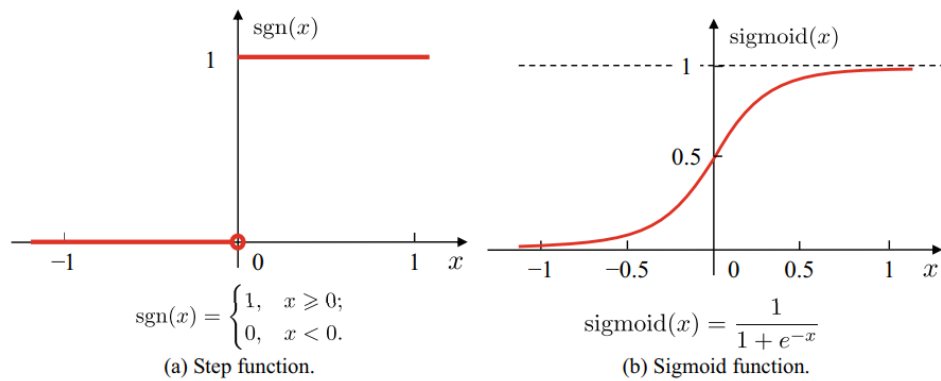


Figura 103: Funzioni tipiche di attivazione dei neuroni

La funzione di attivazione ideale è la **funzione a gradino**, che mappa il valore di input a "0" (non eccitato) e "1" (eccitato).

Le funzioni a gradino sono discontinue e non uniformi, spesso usiamo invece la funzione **sigmoide**.

5.3 Perceptron and Multi-layer Network

5.3.1 Perceptron

Il perceptrone è un classificatore binario costituito da due strati di neuroni. Lo strato di input riceve i segnali e li trasmette al neurone di output M-P (unità logica di soglia).

Perceptron può facilmente implementare "AND", "OR" e "NOT":

- **AND** $x_1 \wedge x_2$: ponendo $w_1 = w_2 = 1, \theta = 2$, allora $y = f(1 \cdot x_1 + 1 \cdot x_2 - 2)$, e $y = 1 \iff x_1 = x_2 = 1$.
- **OR** $x_1 \vee x_2$: ponendo $w_1 = w_2 = 1, \theta = 0.5$, allora $y = f(1 \cdot x_1 + 1 \cdot x_2 - 0.5)$, e $y = 1 \iff x_1 = 1 \vee x_2 = 1$.
- **NOT** $\neg x_1$: ponendo $w_1 = -0.6, w_2 = 0, \theta = -0.5$, allora $y = 0$ dove $x_1 = 1$ e $y = 1$ dove $x_1 = 0$.

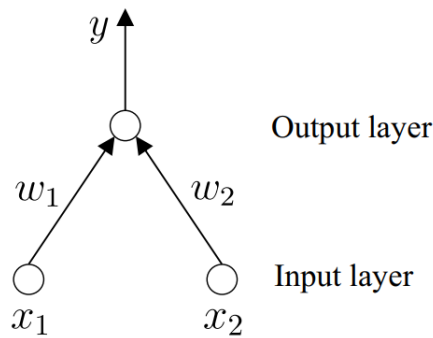


Figura 104: Un perceptron con due neuroni di input

5.3.2 Perceptron Learning

I pesi $w_i (i = 1, 2, \dots, n)$ e la soglia θ possono essere appresi dai dati di addestramento.

La regola di apprendimento, per un campione di addestramento $(\mathbf{x}, y)$, se l'output è $\hat{y}$, allora il peso viene aggiornato da:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(y - \hat{y})x_i$$

dove $\eta \in (0, 1)$ è il tasso di apprendimento.

Il percettrone rimane invariato se predice correttamente il campione $(\mathbf{x}, y)$; altrimenti, il peso viene aggiornato basandosi sul grado dell'errore.

5.3.3 Perceptron che risolve i problemi "AND", "OR" e "Not"

AND

$$x_1 + x_2 - 2 > 0 \Rightarrow x_2 > -x_1 + 2$$

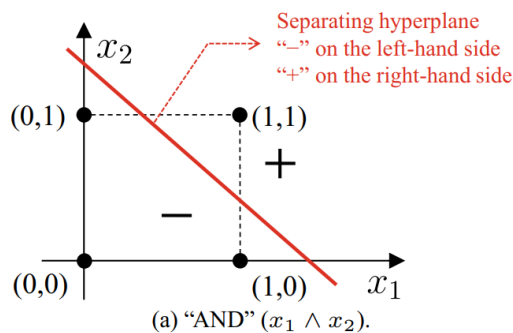


Figura 105: "AND", problema separabile linearmente

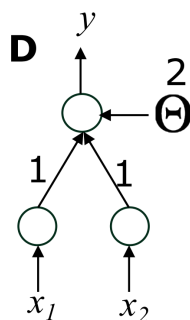


Figura 106: Rete neurale per "AND"

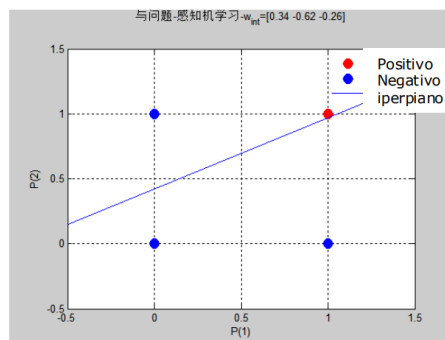


Figura 107: Rappresentazione del perceptron che risolve "AND"

OR

$$x_1 + x_2 - 0.5 > 0 \Rightarrow x_2 > -x_1 + 0.5$$

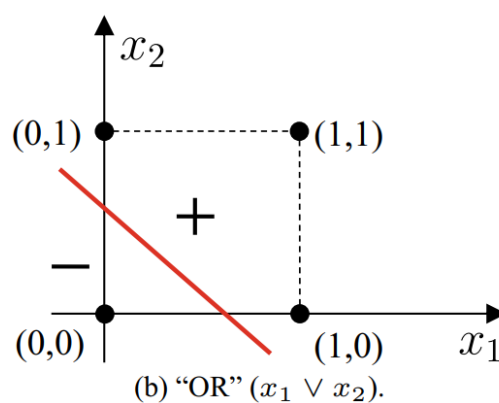


Figura 108: "OR", problema separabile linearmente

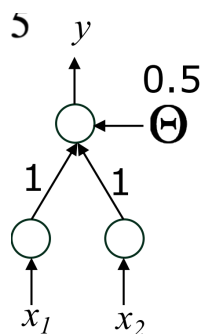


Figura 109: Rete neurale per "OR"

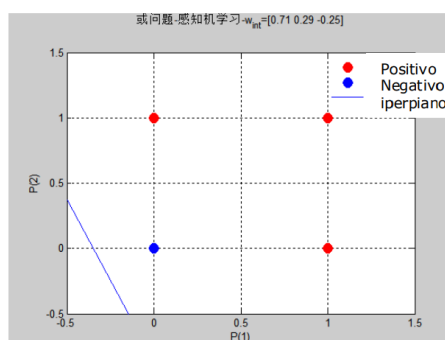


Figura 110: Rappresentazione del perceptron che risolve "OR"

NOT

$$-0.6x_1 - 0.5 > 0 \Rightarrow x_1 > \frac{0.5}{0.6}$$

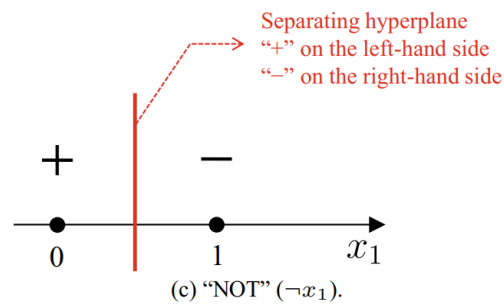


Figura 111: "NOT", problema separabile linearmente

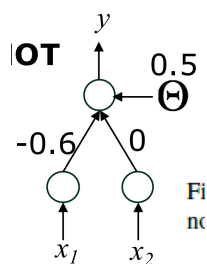


Figura 112: Rete neurale per "NOT"

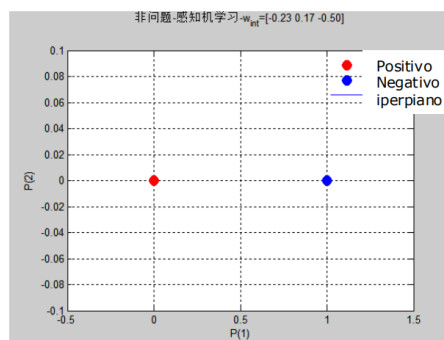


Figura 113: Rappresentazione del perceptron che risolve "NOT"

XOR

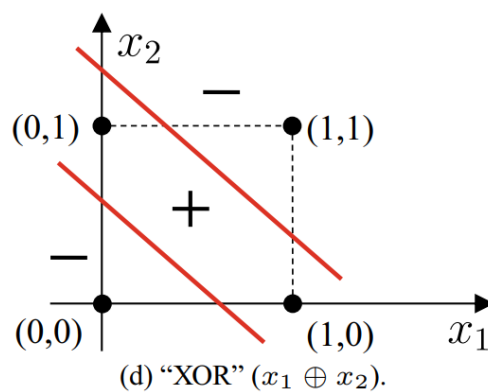


Figura 114: "XOR", problema non separabile linearmente

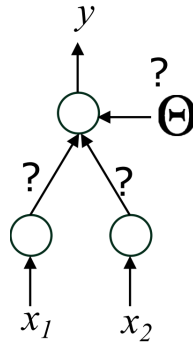


Figura 115: Rete neurale per "XOR"

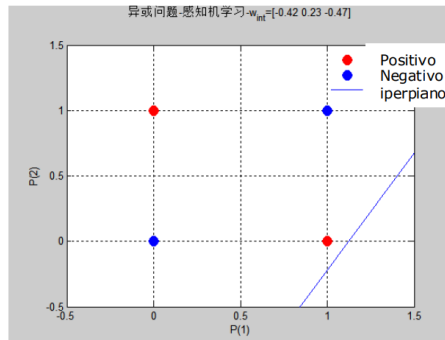


Figura 116: Rappresentazione del perceptron che risolve "XOR"

5.3.4 La capacità di apprendimento dei perceptron

Il processo di apprendimento è garantito per convergere per problemi separabili linearmente; in caso contrario, si verificheranno fluttuazioni. [Minsky e Papert, 1969]

La capacità di apprendimento dei perceptron a un solo strato è piuttosto debole, e può risolvere solo problemi separabili linearmente.

Infatti, i problemi "AND", "OR" e "NOT" sono tutti problemi separabili linearmente e il processo di apprendimento è garantito per convergere a un vettore di pesi appropriato. Tuttavia, i perceptron non possono risolvere problemi separabili non linearmente come "XOR".

Come risolvere problemi separabili non linearmente? **Perceptron multistrato**

5.3.5 Perceptron multistrato

Un perceptrone a due strati che risolve il problema "XOR":

$$b_1 + b_2 - 1.5 > 0 \Rightarrow b_1 + b_2 > 1.5 \Rightarrow x_1 \vee x_2 \text{ AND } \neg(x_1 \wedge x_2) + x_2 + x_1 - 0.5 > 0 \Rightarrow x_2 > -x_1 + 0.5$$

$$-x_1 - x_2 + 1.5 > 0 \Rightarrow 0 \Rightarrow x_2 < -x_1 + 1.5$$

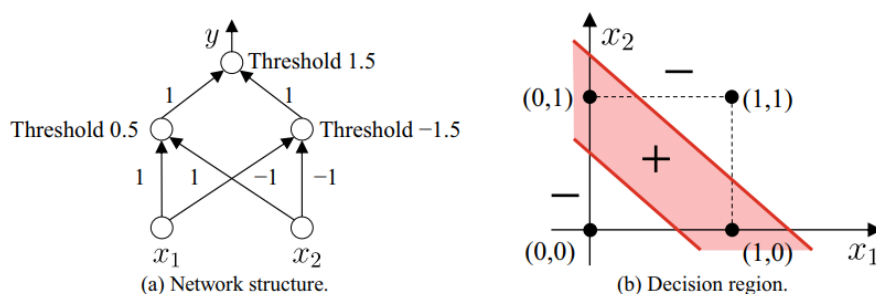


Figura 117: Un perceptron a due strati che risolve il problema "XOR"

Lo strato di neuroni tra lo strato di input e quello di output è noto come **hidden layer**, che ha funzioni di attivazione come lo strato di output.

5.3.6 Multi-layer feed-forward neural networks

Definizione: i neuroni di ogni strato sono completamente connessi con i neuroni dello strato successivo.

Forward: Lo strato di input riceve i segnali esterni, lo strato nascosto e quello di output elaborano (emettono) i segnali.

Learning: imparare dai dati per regolare i "pesi di connessione" e le "soglie"

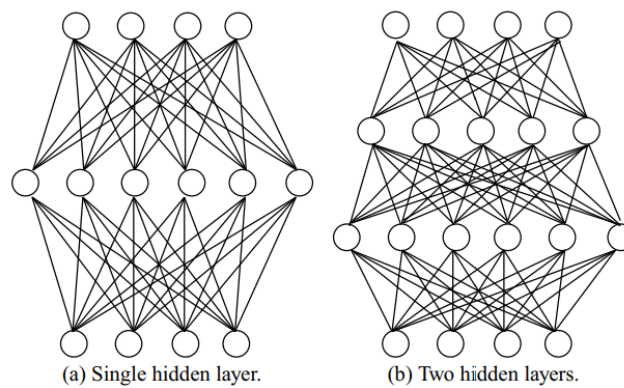


Figura 118: Reti neurali feedforward multistrato

5.4 Algoritmo di backpropagation degli errori

L'algoritmo **Error BackPropagation (BP)** è di gran lunga l'algoritmo di apprendimento delle reti neurali di maggior successo.

Dato un dataset $D = \{(x_i, y_i)\}$, $x_i \in R^d$, $y_i \in R^l$, ($i = 1, 2, \dots, m$) Per semplicità di discussione, data una rete neurale feedforward multistrato con d neuroni di input, l neuroni di output e q strati nascosti.

Notazione:

- θ_j : soglia del neurone j nello strato di output;
- γ_h : soglia del neurone h nello strato nascosto;
- v_{ih} : peso di connessione tra il neurone di input i e il neurone nascosto h ;
- w_{hj} : peso di connessione tra il neurone nascosto h e il neurone di output j ;

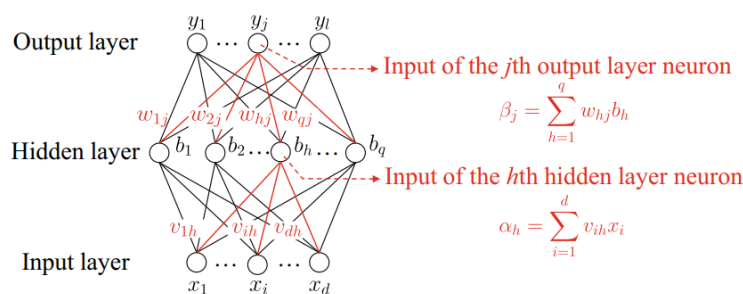


Figura 119: Notazioni delle reti neurali BP

Per il campione (x_k, y_k) , supponiamo che la rete produca $\hat{y}_k$

Forward

- $step_1: b_h = f(a_h - \gamma_h), a_h = \sum_{i=1}^d v_{ih} x_i$
- $step_2: \hat{y}_j^k = f(\beta_j - \theta_j); \beta_j = \sum_{h=1}^q w_{hj} b_h$ (5.3)
- $step_3: E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$

Numero di parametri

- Pesi: v_{ih}, w_{hj}
- Soglia: $\theta_j, \gamma_h (i = 1, \dots, h = 1, \dots, q, j = 1, \dots, l)$
- Totale: $(d + l + 1)q + l$ parametri da ottenere

Tuning dei parametri BP è un algoritmo di apprendimento iterativo e ogni iterazione impiega la regola di apprendimento del perceptron in forma generale e la regola di aggiornamento per qualsiasi parametro v è:

$$v \leftarrow v + \Delta v$$

5.4.1 Algoritmo BP

L'algoritmo BP utilizza la discesa del gradiente, che regola i parametri verso la direzione del gradiente negativo dell'obiettivo. Per l'errore E_k e il tasso di apprendimento η , abbiamo:

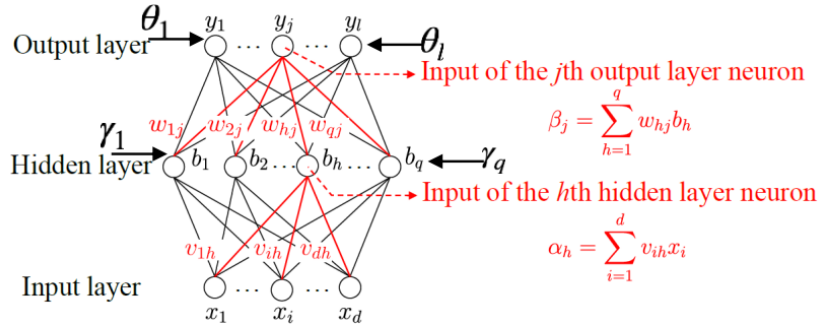


Figura 120: Notazioni delle reti neurali BP con aggiunte

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}} \quad E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$$

$$-\frac{\partial E_k}{\partial w_{hj}} = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial w_{hj}} \quad \hat{y}_j^k = f(\beta_j - \theta_j) \quad \beta_j = \sum_{h=1}^q w_{hj} b_h$$

$$g_j = -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j}$$

$$-\frac{\partial E_k}{\partial \hat{y}_j^k} = -\frac{2}{2} (\hat{y}_j^k - y_j^k) \quad \frac{\partial \hat{y}_j^k}{\partial \beta_j} = f'(\beta_j - \theta_j)$$

f è sigmoide $f'(x) = f(x)[1 - f(x)] = \hat{y}_j^k(1 - \hat{y}_j^k)$

Calcolo finale:

$$g_j = (y_j^k - \hat{y}_j^k) \cdot \hat{y}_j^k \cdot (1 - \hat{y}_j^k) \quad (5.10)$$

$$\Delta w_{hj} = \eta \cdot g_j \cdot b_h \quad (5.11)$$

Similarità che possiamo derivare:

$$\Delta\theta_j = -\eta g_j \quad (5.12)$$

$$\Delta v_{ih} = \eta e_h x_i \quad (5.13)$$

$$\Delta\gamma_h = -\eta e_h \quad (5.14)$$

dove

$$e_h = -\frac{\partial E_k}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} = -\sum_{j=1}^l \frac{\partial E_k}{\partial \beta_j} \frac{\partial \beta_j}{\partial b_h} f'(\alpha_h - \gamma_h) = b_h(1 - b_h) \sum_{j=1}^h w_{hj} g_j \quad (5.15)$$

Il tasso di apprendimento $\eta \in (0, 1)$ controlla la dimensione del passo in ogni round. Un tasso di apprendimento eccessivamente elevato può causare fluttuazioni, mentre un valore troppo basso porta a una convergenza lenta.

Algorithm 2 Error BackPropagation

Require: Training set $D = \{(x_k, y_k)\}_{k=1}^m$; Learning rate η .

- 1: **Process:**
- 2: Randomly initialize connection weights and thresholds from $(0, 1)$
- 3: **repeat**
- 4: **for** $x_k, y_k \in D$ **do**
- 5: Compute the output $\hat{y}_k$ for the current sample according to the current parameters and (5.3);
- 6: Compute the gradient term g_j of the output layer neuron according to (5.10);
- 7: Compute the gradient term e_h of the hidden layer neurons according to (5.15);
- 8: Update connection weights w_{hj} , v_{ih} and thresholds θ_j , γ_h according to (5.11)-(5.14).
- 9: **end for**
- 10: **until** The termination condition is met

Ensure: A feedforward neural network with determined connection weights and thresholds.

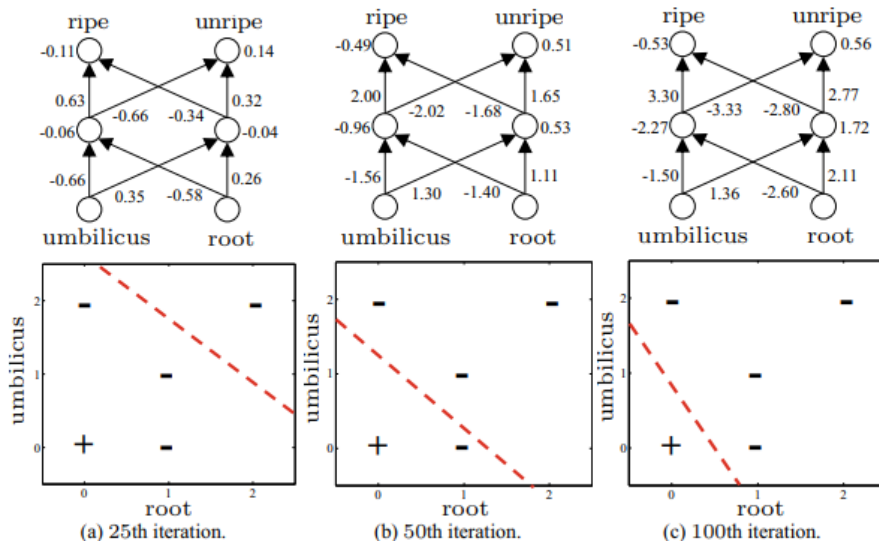


Figura 121: Le variazioni dei parametri e dei limiti decisionali di una rete neurale BP in esecuzione su un set di dati di anguria con 5 campioni e 2 features

Algoritmo BP standard Utilizza un campione alla volta per aggiornare pesi e soglie.

Aggiornamenti frequenti, gli aggiornamenti di campioni diversi potrebbero "compensarsi" a vicenda, richiedendo più iterazioni.

Algoritmo BP accumulato Riduce al minimo l'errore accumulato sull'intero set di addestramento.

$$E = \frac{1}{m} \sum_{k=1}^m E_k$$

Regola i parametri meno frequentemente poiché esegue la regolazione una volta dopo una scansione completa del set di addestramento.

Applicazioni pratiche In alcuni compiti, soprattutto quando il set di addestramento è ampio, l'algoritmo BP accumulato può diventare lento. Al contrario, l'algoritmo BP standard può raggiungere una buona soluzione più rapidamente.

Capacità di rappresentazione delle reti multistrato Una rete neurale feedforward costituita da un singolo strato nascosto con neuroni sufficienti potrebbe approssimare funzioni continue di qualsiasi complessità fino a un'accuratezza arbitraria [Hornik et al., 1989]

Limitazioni delle reti multistrato Le reti neurali BP soffrono di overfitting: l'errore di addestramento diminuisce mentre l'errore di test aumenta.

Non esiste un metodo basato su principi per impostare il numero di neuroni dello strato nascosto, e nella pratica si usa solitamente il metodo per tentativi.

Strategie per alleviare il problema dell'overfitting

Early stopping: Una volta che l'errore di addestramento diminuisce e l'errore di convalida aumenta, il processo di addestramento si interrompe.

Regolarizzazione: Aggiungere un termine di regolarizzazione alla funzione obiettivo, descrivendo la complessità della rete neurale.

6 Support Vector Machines

Il nostro primo obiettivo sarà quello di ridurre le possibili soluzioni solo a quelle che non commettono errori. Questo porterà alla definizione del **marginale funzionale**.

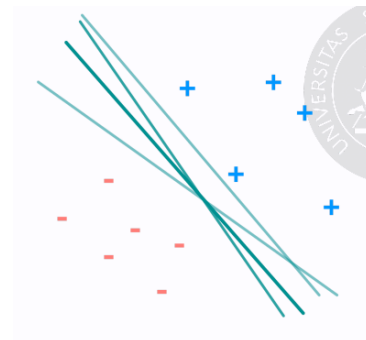


Figura 122: Margine funzionale

Poi potremo concentrarci sul problema di forzare la soluzione a massimizzare la distanza dagli esempi positivi/negativi più vicini. Questo porterà alla definizione del **marginale geometrico**.

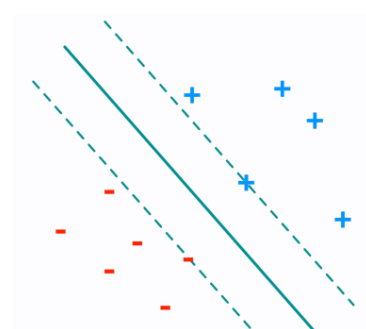


Figura 123: Margine geometrico

6.1 Margine funzionale

Assumiamo che:

- Il problema di classificazione è binario con etichette in $\{+1, -1\}$.
- Il modello è descritto da un iperparametro definito da una funzione $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} - t$
- Classificheremo come positive tutte le $\mathbf{x}$ tali che $f(\mathbf{x}) > 0$

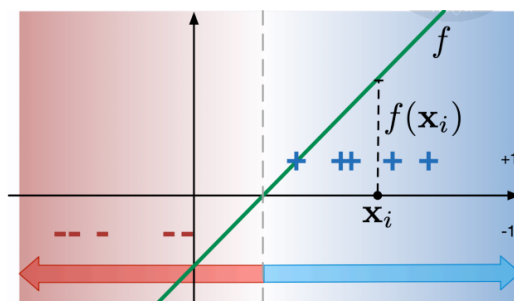


Figura 124: Grafico del margine funzionale

Definiamo il margine funzionale di un esempio $\mathbf{x}_i$ w.r.t il parametro determinato da $\mathbf{w}$ e t come la quantità:

$$\mu(\mathbf{x}_i) = y_i(\mathbf{w} \cdot \mathbf{x}_i - t) = y_i f(\mathbf{x}_i)$$

Notiamo che $\mu(\mathbf{x}_i) > 0 \iff \mathbf{x}_i$ è classificato correttamente.

Nella nostra soluzione vorremo un margine funzionale maggiore di 0 per ogni esempio, ovvero:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i - t) > 0$$

Nota importante

Se riscaldiamo $\mathbf{w}$ e t **cambiamo semplicemente la pendenza dell'iperpiano** senza cambiare la superficie di decisione.

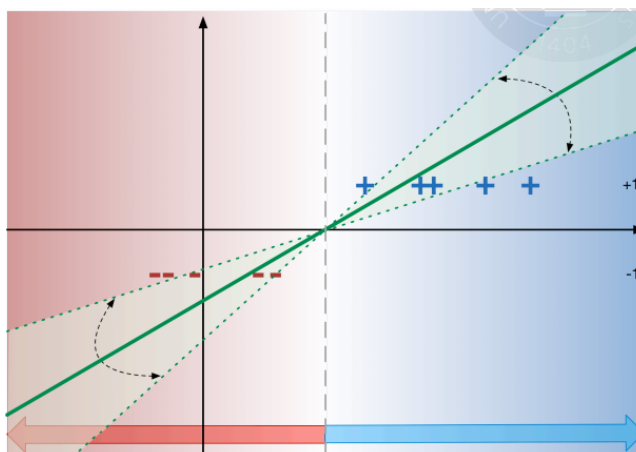


Figura 125: Riscaldamento di $\mathbf{w}$ e t

Data la libertà mostrata nell'immagine e fissata una costante positiva c , è corretto affermare che, ai fini della corretta classificazione degli esempi:

$$y_i(\mathbf{w} \cdot \mathbf{x} - t) > 0$$

è equivalente a:

$$y_i(\mathbf{w} \cdot \mathbf{x} - t) \geq c$$

6.1.1 Support vectors

Siamo liberi di richiedere che ogni esempio nel dataset in esso contenuto:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1$$

e anche che la stretta uguaglianza vale per gli esempi sul confine:

Definizione

Gli esempi che doffisfano:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i - t) = 1$$

risiedono sul limiti di margine e sono chiamati **support vectors**.

6.1.2 Relazione tra l'equazione e il vettore $\mathbf{W}$

Il vettore $\mathbf{w}$ è ortogonale (perpendicolare) all'iperpiano definito dall'equazione $\mathbf{w} \cdot \mathbf{x} - t = 0$.

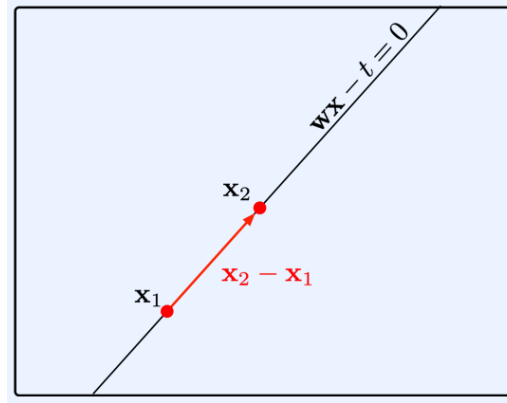


Figura 126: Vettore ortogonale all'iperpiano

Per vedere il perché, consideriamo due punti x_1 e x_2 che risiedono sull'iperpiano. Il vettore differenza $x_2 - x_1$ si trova all'interno dell'iperpiano, il che significa che è parallelo ad esso.

Per confermare che w è ortogonale all'iperpiano, controlliamo che:

$$w \cdot (x_2 - x_1) = 0$$

Poiché entrambi x_1 e x_2 soddisfano l'equazione dell'iperpiano, abbiamo:

$$w \cdot x_1 = t, \quad w \cdot x_2 = t$$

Sostituendo questi valori nell'equazione del prodotto scalare otteniamo:

$$w \cdot (x_2 - x_1) = w \cdot x_2 - w \cdot x_1 = t - t = 0$$

6.2 Margine geometrico

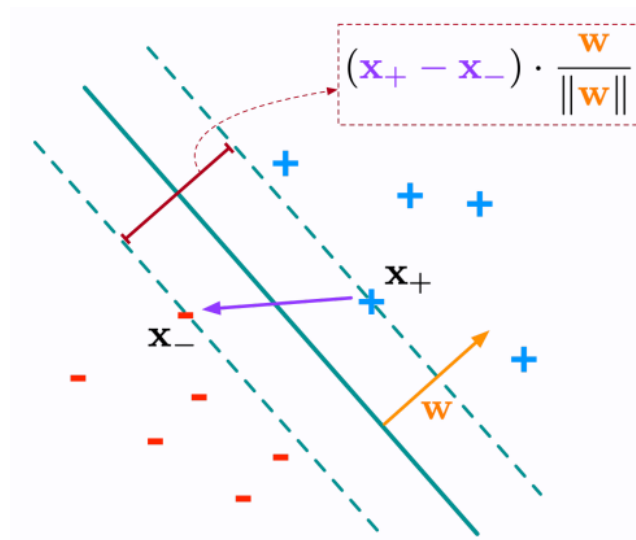


Figura 127: Grafico del margine geometrico

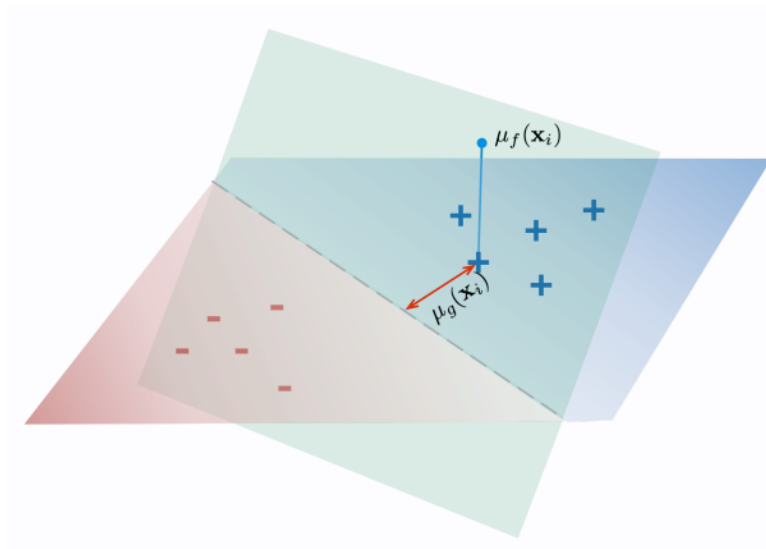


Figura 128: Margine funzionale e geometrico

6.3 SVM: massimizzazione del margine

La dimensione del margine può essere valutata come:

$$\gamma = (\mathbf{x}_+ - \mathbf{x}_-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} = \frac{\mathbf{x}_+ \cdot \mathbf{w}}{\|\mathbf{w}\|} - \frac{\mathbf{x}_- \cdot \mathbf{w}}{\|\mathbf{w}\|}$$

Per $\mathbf{x}_+$ e $\mathbf{x}_-$, essendo gli esempi sul confine, vale che:

$$\mathbf{x}_+ \cdot \mathbf{w} - t = 1 \Rightarrow \mathbf{x}_+ \cdot \mathbf{w} = 1 + t$$

$$-(\mathbf{x}_- \cdot \mathbf{w} - t) = 1 \Rightarrow \mathbf{x}_- \cdot \mathbf{w} = t - 1$$

implicando

$$\mu = \frac{1+t}{\|\mathbf{w}\|} - \frac{t-1}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$

6.4 SVM problema di ottimizzazione

Massimizzando $\frac{1}{\|\mathbf{w}\|}$ è equivalente a minimizzare $\|\mathbf{w}\|$ che, a sua volta equivale a minimizzare $\|\mathbf{w}\|^2$. Il problema di ottimizzazione SVM può essere quindi formulato come:

$$\begin{aligned} \text{minimize}_{\mathbf{w}, t} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{subject to} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1; \quad \forall : 1 \leq i \leq n \end{aligned}$$

6.5 Dualità

Consideriamo il problema di ottimizzazione:

$$\begin{aligned} \text{minimize}_{\mathbf{x}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0; \quad \forall_i : 1 \leq i \leq m \\ & g_i(\mathbf{x}) = 0; \quad \forall_i : 1 \leq i \leq p \end{aligned}$$

La funzione duale di Lagrange è, per definizione:

$$g(\alpha, \beta) = \inf_{\mathbf{x}} \Lambda(\mathbf{x}, \alpha, \beta) = \inf_{\mathbf{x}} \left(f_0(\mathbf{x}) + \sum_{i=1}^m \alpha_i f_i(\mathbf{x}) + \sum_{i=1}^p \beta_i g_i(\mathbf{x}) \right)$$

La **Dualità** può essere pensata come: ... *an organized way to form highly non trivial bounds on optimization problems*

Il problema duale è definito come:

$$\begin{aligned} & \text{maximize}_{\alpha, \beta} \quad g(\alpha, \beta) \\ & \text{subject to} \quad \alpha_i \geq 0; \quad \forall_i : 1 \leq i \leq m \end{aligned}$$

Nei **problemi convessi** il limite è (solitamente) stretto e la massimizzazione del limite (che è la formulazione duale del problema) porta alla stessa soluzione della minimizzazione della funzione originale (la formulazione primale). In tal caso abbiamo una **dualità forte**.

La **dualità forte** richiede che siano soddisfatte un insieme di condizioni, note come condizioni di **Karush-Kuhn-Tucker (KKT)**.

6.5.1 Karush-Kuhn-Tucker conditions

Stazionarietà:

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \alpha, \beta) = 0$$

Fattibilità primordiale:

$$f_i(\mathbf{x}^*) \leq 0, \quad g_j(\mathbf{x}^*) = 0 \quad 1 \leq i \leq m; \quad 1 \leq j \leq p$$

Fattibilità duale:

$$\alpha_i \geq 0 \quad 1 \leq i \leq m$$

Rilassamento complementare:

$$\alpha_i f_i(\mathbf{x}^*) = 0 \quad 1 \leq i \leq m$$

Nota

$$\nabla_{\mathbf{x}} \Lambda(\mathbf{x}, \alpha, \beta) = \nabla f_0(\mathbf{x}^*) + \sum_{i=1}^m \alpha_i \nabla f_i(\mathbf{x}^*) + \sum_{j=1}^p \beta_j \nabla g_j(\mathbf{x}^*)$$

6.5.2 SVM problema duale

Nel caso SVM, iniziamo aggiungendo moltiplicatori per ogni vincolo di disuguaglianza per ottenere la funzione di Lagrange (con il vincolo $\alpha_i \geq 0$):

$$\begin{aligned} \Lambda(\mathbf{w}, t, \alpha) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\mathbf{w} \cdot \mathbf{x}_i - t) - 1] \\ &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i y_i \mathbf{w} \cdot \mathbf{x}_i + t \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\ &= \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w} \cdot \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i + t \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \end{aligned}$$

Per scrivere la formulazione duale dobbiamo ancora calcolare l'estremo inferiore (rispetto a $\mathbf{w}$ e t) della Lagrangiana. Per trovare l'estremo inferiore, iniziamo calcolando le derivate parziali della Lagrangiana.

$$\frac{\partial \Lambda}{\partial t} = \sum_i \alpha_i y_i; \quad \frac{\partial \Lambda}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i$$

ottenendo (impostandoli uguali a 0):

$$\sum_i \alpha_i y_i = 0; \quad \mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$$

Sostituendo nuovamente nella lagrangiana, otteniamo la doppia funzione:

$$g(\alpha) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i$$

che, come accennato, è un limite inferiore generale per la soluzione parametrizzata da α . Massimizzandolo, otteniamo la formulazione duale del problema SVM:

$$\begin{aligned} \text{maximize}_{\alpha} \quad & -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j + \sum_{i=1}^n \alpha_i \\ \text{subject to} \quad & \alpha_i \geq 0; \quad \forall_i : 1 \leq i \leq n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Il problema duale SVM può essere scritto anche in forma matriciale come:

$$\begin{aligned} \text{maximize}_{\alpha} \quad & \mathbf{1}^T \alpha - \frac{1}{2} \alpha^T Y X X^T Y^T \alpha \\ \text{subject to} \quad & \alpha \succeq 0 \\ & y^T \alpha = 0 \end{aligned}$$

$$\text{dove } Y = \text{diag}(y) = \begin{bmatrix} y_1 & 0 & \cdots & 0 \\ 0 & y_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & y_n \end{bmatrix}$$

6.6 Trovare soluzioni al problema SVM

Q: Come si può trovare una soluzione al problema primordiale?

R: Ottimizzazione convessa (algoritmi numerici). **Q:** Come si può trovare una soluzione al problema duale?

R: Ottimizzazione convessa (algoritmi numerici). Perché allora preoccuparsi di formulare il problema duale?

6.6.1 Vantaggi della doppia formulazione

Abbiamo imparato qualcosa sulla soluzione:

- il **vettore soluzione** w è una combinazione lineare dei **support vectors**;
- i moltiplicatori di Lagrange positivi sono associati ai vettori di supporto (il che implica che i **vettori non di supporto hanno moltiplicatori di Lagrange nulli**);
- sia l'apprendimento che la classificazione possono essere effettuati in termini di **prodotti scalari di support vectors**;

Nella formulazione duale, SVM può essere utilizzato per apprendere concetti non lineari utilizzando il **trucco del kernel**;

6.6.2 I moltiplicatori di Lagrange positivi sono associati ai support vectors

Tra le osservazioni precedenti, l'unica che necessita ancora di una giustificazione è l'affermazione che **i moltiplicatori di Lagrange positivi sono associati ai support vectors**.

Questa proprietà deriva dalla condizione di scarto complementare di KKT, che implica:

$$\alpha_i (y_i (w \cdot x_i - t) - 1) = 0$$

6.6.3 Consentire errori di margine

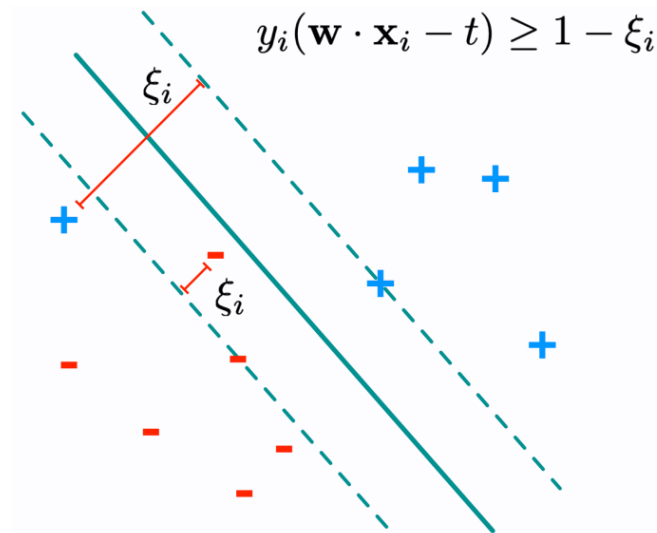


Figura 129: Errori di margine

Introduci una **variabile Slack** per ogni esempio di formazione:

$$\begin{aligned} \underset{\mathbf{w}, t, \xi}{\text{minimize}} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i; \quad \forall : 1 \leq i \leq n \\ & \xi_i \geq 0; \quad \forall : 1 \leq i \leq n \end{aligned}$$

Le variabili Slack allentano i vincoli, ma sono penalizzate nell'obiettivo.

C è un parametro definito dall'utente che bilancia la massimizzazione del margine con la minimizzazione delle variabili Slack:

- Un valore elevato di C indica che gli errori di margine comportano una penalità elevata;
- un valore basso consente più errori di margine (incluse possibili classificazioni errate) al fine di raggiungere un margine ampio.

Nota

Torniamo alla formulazione originale dell'SVM quando:

- $C = 0$
- $C \rightarrow \infty$
- Tutte le risposte precedenti sono sbagliate

Nota

Qual è il $\mathbf{w}$ ottimale se poniamo $C = 0$?

6.6.4 Doppia formulazione

$$\begin{aligned} \underset{\mathbf{w}, t, \xi}{\text{minimize}} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i - t) \geq 1 - \xi_i; \quad \forall : 1 \leq i \leq n \\ & \xi_i \geq 0; \quad \forall : 1 \leq i \leq n \end{aligned}$$

La funzione di Lagrange è:

$$\begin{aligned}
 \Lambda(\mathbf{w}, t, \xi, \alpha, \beta) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - 1 + \xi_i] - \sum_{i=1}^n \beta_i \xi_i \\
 &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n C \xi_i - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i - t) - 1] - \sum_{i=1}^n \alpha_i \xi_i - \sum_{i=1}^n \beta_i \xi_i \\
 &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n C \xi_i - \sum_{i=1}^n \alpha_i \xi_i - \sum_{i=1}^n \beta_i \xi_i \\
 &= \Lambda(\mathbf{w}, t, \alpha_i) + \sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i
 \end{aligned}$$

Impostando la derivata ξ_i dei termini a zero, otteniamo $C = \alpha_i + \beta_i$, il che implica:

$$\Lambda(\mathbf{w}, t, \xi_i, \alpha_i, \beta_i) = \Lambda(\mathbf{w}, t, \alpha_i)$$

che ci permette di scrivere la formulazione duale del problema SVM del **soft margin** come:

$$\begin{aligned}
 \text{maximize}_{\alpha} \quad & -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \mathbf{x}_j + \sum_{i=1}^n \alpha_i \\
 \text{subject to} \quad & 0 \leq \alpha_i \leq C; \quad \forall_i : 1 \leq i \leq n \\
 & \sum_{i=1}^n \alpha_i y_i = 0
 \end{aligned}$$

in forma matriciale:

$$\begin{aligned}
 \text{maximize}_{\alpha} \quad & \mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top Y X X^\top Y^\top \alpha \\
 \text{subject to} \quad & 0 \preceq \alpha \preceq C \mathbf{1} \\
 & y^\top \alpha = 0
 \end{aligned}$$

La condizione di scarto complementare implica che $\beta_i \xi_i = 0$. Se un esempio è un errore marginale ($\xi_i > 0$), allora $\beta_i = 0$ e quindi $\alpha_i = C$, come abbiamo precedentemente stabilito che $C = \alpha_i + \beta_i$.

- $\alpha_i = 0$ sono esempi che sono **al di fuori** (o sul) margine;
- $0 < \alpha_i < C$ sono esempi che sono **sul** margine (cioè, support vectors);
- $\alpha_i = C$ sono esempi che sono **dentro** al margine (cioè, errori di margine).

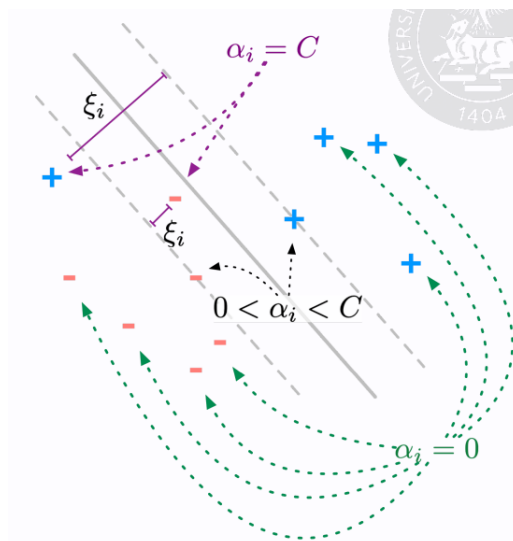


Figura 130: Esempi di slack variable

Soft margin è sufficiente?

Non proprio! La formulazione SVM soft margin non è sempre soddisfacente a causa della limitata capacità di separazione di un iperpiano.

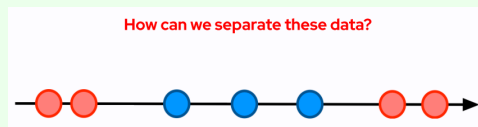


Figura 131: Esempio di soft margin non sufficiente

6.6.5 Espansione delle feature

Proiettiamo i dati in uno spazio dimensionale superiore!

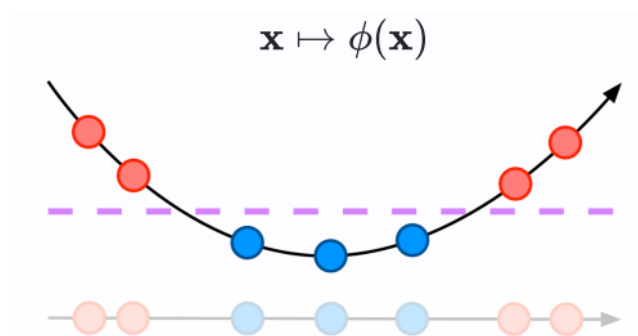


Figura 132: Espansione delle feature

La funzione di "espansione" è:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$$

è chiamata **mappatura delle feature** ed è tipicamente non lineare e $d \ll D$

6.6.6 Non-separable case

Questa intuizione ci porta a una ricetta per utilizzare le SVM anche quando gli esempi non sono linearmente separabili. È possibile utilizzare un approccio basato sui due passaggi seguenti:

1. i vettori di input (spazio di input) vengono proiettati su uno spazio più ampio (**spazio delle feature**);
2. viene calcolato l'iperpiano ottimale nello spazio delle feature (ad esempio, soft-margin SVM).

Teorema di Cover sulla separabilità (informale)

Un problema di classificazione di pattern in uno spazio non lineare ad alta dimensionalità ha maggiori probabilità di essere linearmente separabile rispetto a uno spazio a bassa dimensionalità.

Esempio

Date due feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^6$ definite come:

$$\phi \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = [x_1^2, \quad x_2^2, \quad x_1 x_2, \quad x_1, \quad x_2, \quad 1]^\top$$

Un iperpiano omogeneo nello spazio delle feature determina una superficie nello spazio originale data dall'insieme di punti:

$$\{\mathbf{x} \in \mathbb{R}^2 : \mathbf{w} \cdot \phi(\mathbf{x}) = 0\} \equiv \{\mathbf{x} \in \mathbb{R}^2 : w_1 + w_2 x_1^2 + w_3 x_2^2 + w_4 x_1 x_2 + w_5 x_1 + w_6 x_2 = 0\}$$

che descrive curve di secondo grado sul piano $\mathbb{R}^2$ (ad esempio, ellissi, parabole, iperbole).

Applichiamo SVM nello spazio delle feature definito da ϕ . Poi, la definizione di una generica $\mathbf{w}$ sarà:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \phi(\mathbf{x}_i)$$

e quindi la funzione decisionale sarà:

$$f(\mathbf{x}) = \mathbf{w} \cdot \phi(\mathbf{x}) + t = \sum_{i=1}^n \alpha_i y_i \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}) + t = \sum_{i=1}^n \alpha_i y_i \mathcal{K}(\mathbf{x}_i, \mathbf{x}) + t$$

dove $\mathcal{K}(\mathbf{x}_i, \mathbf{x}) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x})$ è chiamata la **funzione kernel**.

6.6.7 Soft-margin SVM con kernel

$$\begin{aligned} & \text{maximize}_{\alpha} \quad \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i=1}^n \alpha_i \\ & \text{subject to} \quad 0 \leq \alpha_i \leq C; \quad \forall_i : 1 \leq i \leq n \\ & \quad \quad \quad \sum_i y_i \alpha_i = 0 \end{aligned}$$

In forma matriciale possiamo riscrivere il problema come:

$$\begin{aligned} & \text{maximize}_{\alpha} \quad \mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top Y K Y^\top \alpha \\ & \text{subject to} \quad 0 \preceq \alpha \preceq C \mathbf{1} \\ & \quad \quad \quad y^\top \alpha = 0 \end{aligned}$$

Dove $K \in \mathbb{R}^{n \times n}$ nota come **kernel** o **matrice di Gram** contiene tutti i prodotti $\phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$

Nota

Per evitare confusione con la funzione kernel $\mathcal{K}$, nel resto della lezione useremo M per indicare la matrice kernel, anche se K è la notazione più comune.

Matrice Kernel (Gram)

La matrice Gram (o kernel) associata alla funzione kernel $\mathcal{K}(\cdot, \cdot)$, valutata su un subset finito di esempi $X = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, $\mathbf{x}_i \in \mathcal{X}$, è la matrice $M \in \mathbb{R}^{n \times n}$ tale che:

$$M_{i,j} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$$

La matrice M è simmetrica per definizione.

6.7 Kernels

6.7.1 Funzione kernel

Una funzione kernel è una funzione $\mathcal{K} : \mathbb{V} \times \mathbb{V} \rightarrow \mathbb{R}$ per cui si sostiene che esiste una mappatura $\phi : \mathbb{V} \rightarrow \mathbb{F}$ (con $\mathbb{F}$ essendo uno spazio di Hilbert, cioè uno spazio vettoriale completo dotato di un prodotto interno), tale che:

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

ovvero, una funzione kernel calcola il prodotto interno di $\mathbf{x}$ e $\mathbf{x}'$ dopo averli mappati in uno spazio di Hilbert diverso (possibilmente di dimensione molto elevata).

6.7.2 Prodotto interno

Un prodotto interno è una generalizzazione del prodotto scalare a spazi vettoriali arbitrari. È definito assiomaticamente come una funzione che associa a ciascuna coppia di vettori uno scalare (diciamo un numero reale, anche se può essere definito su campi arbitrari). Per essere un prodotto scalare, la funzione deve soddisfare:

- **simmetria:** $\langle \mathbf{x}, \mathbf{x}' \rangle = \langle \mathbf{x}', \mathbf{x} \rangle$
- **linearità nel primo argomento:** $\langle \alpha \mathbf{x} + \beta \mathbf{x}', y \rangle = \alpha \langle \mathbf{x}, y \rangle + \beta \langle \mathbf{x}', y \rangle$
- **determinatezza positiva:** $\langle \mathbf{x}, \mathbf{x} \rangle \geq 0$ e $\langle \mathbf{x}, \mathbf{x} \rangle = 0$ se e solo se $\mathbf{x} = 0$

6.7.3 Motivazioni del kernel

Rappresentativo Estendere i classificatori lineari per far fronte a problemi non lineari;

Computazionale Calcolare una funzione di similarità in uno spazio di feature ad alta dimensionalità senza calcolare effettivamente i vettori delle feature;

Teorica Esistono condizioni per stabilire se una data funzione è un kernel (dimostrare effettivamente che valgono, può essere difficile);

Le proprietà composizionali consentono di costruire nuovi kernel a partire da quelli noti.

6.7.4 Kernel polinomiale

Kernel polinomiale di grado d :

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + 1)^d \text{ o } (\mathbf{x} \cdot \mathbf{x}')^d$$

- $d = 1$ è il kernel lineare;
- per $d = 2$ abbiamo il kernel quadratico;
- per un d generico lo spazio delle feature considerato ha una dimensionalità che è esponenziale in d .

Esempio

Esempio per $d = 2$ e $\mathbf{x} \in \mathbb{R}^2$:

$$\begin{aligned}\mathcal{K}(\mathbf{x}, \mathbf{z}) &= (\mathbf{x} \cdot \mathbf{z} + 1)^2 = (x_1 z_1 + x_2 z_2 + 1)^2 \\&= (x_1 z_1 + x_2 z_2 + 1)(x_1 z_1 + x_2 z_2 + 1) \\&= x_1^2 z_1^2 + x_1 z_1 x_2 z_2 + x_1 z_1 + x_2 z_2 x_1 z_1 + x_2^2 z_2^2 + x_2 z_2 + x_1 z_1 + x_2 z_2 + 1 \\&= x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 + 2x_1 z_1 + 2x_2 z_2 + 1 \\&= \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2} x_1 x_2 \\ \sqrt{2} x_1 \\ \sqrt{2} x_2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} z_1^2 \\ z_2^2 \\ \sqrt{2} z_1 z_2 \\ \sqrt{2} z_1 \\ \sqrt{2} z_2 \\ 1 \end{bmatrix} = \phi(\mathbf{x}) \cdot \phi(\mathbf{z}).\end{aligned}$$

6.7.5 Radial Basis Function kernel

Gaussian (RBF) Kernel:

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

Per derivare la mappatura ϕ associata con K notiamo che:

$$\begin{aligned}\exp\left(-\frac{1}{2\sigma^2}\|\mathbf{x} - \mathbf{x}'\|^2\right) &= \exp\left(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|\mathbf{x}'\|^2}{2\sigma^2}\right) \exp\left(\frac{1}{\sigma^2}(\mathbf{x}^\top \mathbf{x}')\right) \\&= \exp\left(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|\mathbf{x}'\|^2}{2\sigma^2}\right) \sum_{n=0}^{\infty} \frac{1}{n!} \frac{(\mathbf{x}^\top \mathbf{x}')^n}{\sigma^{2n}}\end{aligned}$$

dove abbiamo usato l'espansione in serie di Taylor di $e^x = 1 + x + \frac{x^2}{2!} + \dots$, il kernel gaussiano è una combinazione lineare di infiniti kernel polinomiali di grado crescente.

Nota

- Lo spazio delle feature ha una **dimensionalità infinita**;
- σ è un **iperparametro** che controlla la larghezza della gaussiana, solitamente viene impostata utilizzando la cross-validation o un set di indipendent validation;
- Di solito è meglio **normalizzare i dati prima di calcolare la matrice kernel** (ad esempio, sottraendo la media e dividendo per la deviazione standard). In caso contrario, le features con valori elevati prevarranno nel calcolo del kernel.

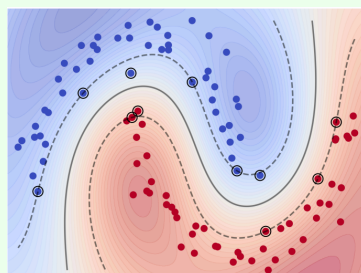


Figura 133: Kernel RBF

6.7.6 Il trucco del kernel

Con il termine "trucco del kernel" ci riferiamo al fatto che, in molti casi, possiamo **calcolare il prodotto interno nello spazio delle feature senza effettivamente mappare gli esempi nello spazio delle feature**.

Ad esempio: sarebbe impossibile calcolare i vettori delle feature per il kernel RBF, poiché lo spazio delle feature ha dimensionalità infinita.

6.7.7 Applicazioni del kernel

Qualsiasi algoritmo per dati vettoriali che possa essere espresso in termini di prodotti scalari può essere implicitamente eseguito nello spazio delle feature associato a un kernel. Ciò porta alle seguenti applicazioni:

- **Kernelizzazione** di metodi lineari o basati sulla distanza più diffusi (ad esempio Perceptron e kNN)
- **Applicazione** di algoritmi per dati vettoriali (ovvero numerici) (SVM, Perceptron, ecc.) **a dati non vettoriali utilizzando kernel ad hoc** (ad esempio kernel per strutture)

Creazione di kernel ad hoc

Esistono molti kernel e il concetto può essere applicato anche a dati strutturati come stringhe o alberi.

In biologia, ad esempio, $\phi(\mathbf{x})$ può essere un vettore che conta il numero di volte in cui specifiche sottostringhe (ad esempio, 'AAA', 'ATA', ...) appaiono in una sequenza di DNA. Il kernel viene quindi calcolato come $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$, ma utilizzando un algoritmo di programmazione dinamica per calcolare il prodotto scalare.

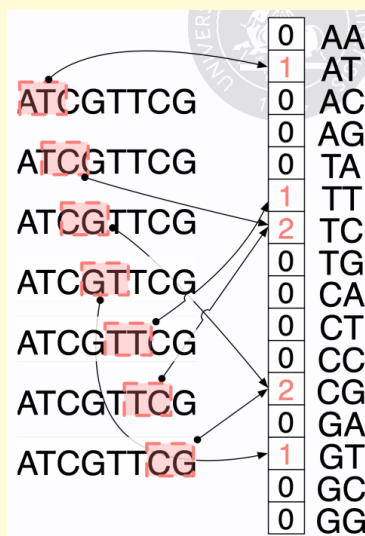


Figura 134: Esempio di kernel per stringhe di DNA

Intuizione

I kernel possono essere considerati funzioni di similarità nello spazio di "output" (kernel). Infatti, se i vettori sono "simili", allora probabilmente puntano in una direzione simile e il prodotto scalare è probabile che sia grande. Se i vettori sono "dissimili", allora probabilmente puntano in direzioni diverse e il prodotto scalare è piccolo.

Reminder: $\mathbf{x} \cdot \mathbf{x}' = |\mathbf{x}| |\mathbf{x}'| \cos(\theta)$

6.7.8 Progettazione di nuovi kernel

Supponiamo di aver creato una funzione $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ che implementa una similarità tra esempi $\mathbf{x}$ e $\mathbf{x}'$. Come possiamo verificare se $\mathcal{K}$ è un kernel valido?

Le condizioni di Mercer Consideriamo una funzione $\mathcal{K}$. Per una sequenza finita di vettori $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ in $\mathbb{V}$, consideriamo la matrice Gram $\mathbf{M}$, $M_{ij} := \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$.

Le condizioni di Mercer affermano che $\mathcal{K}$ è un kernel valido se e solo se per ogni possibile sequenza finita di vettori $\mathbf{M} = \mathbf{M}^T$ e $\mathbf{M} \succeq 0$, cioè, $\mathbf{M}$ è **simmetrica** e **positivamente semidefinita**.

Dimostrazione

Se $\mathcal{K}$ è un kernel, allora $\mathbf{M} \succeq 0$

Dimostriamo la parte "if" dell'affermazione sulle condizioni di Mercer (l'altra direzione non verrà dimostrata qui).

Vogliamo provare che se esiste ϕ tale che $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$, allora per ogni set di punti $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$, $\mathbf{M}$ è simmetrica e positivamente semidefinita.

La **parte simmetrica** è banale (per definizione di prodotto interno).

Dimostriamo ora che $\mathbf{M} \succeq 0$, cioè, che per ogni $\mathbf{z} : \mathbf{z}^T \mathbf{M} \mathbf{z} \geq 0$.

$$\begin{aligned} \mathbf{z}^T \mathbf{M} \mathbf{z} &= \sum_i \sum_j z_i z_j M_{i,j} = \sum_i \sum_j z_i z_j \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) \\ &= \sum_i \sum_j z_i z_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \\ &= \sum_i \sum_j \langle z_i \phi(\mathbf{x}_i), z_j \phi(\mathbf{x}_j) \rangle \\ &= \sum_i \left\langle z_i \phi(\mathbf{x}_i), \sum_j z_j \phi(\mathbf{x}_j) \right\rangle \\ &= \left\langle \sum_i z_i \phi(\mathbf{x}_i), \sum_j z_j \phi(\mathbf{x}_j) \right\rangle \\ &= \left\langle \sum_i z_i \phi(\mathbf{x}_i), \sum_i z_i \phi(\mathbf{x}_i) \right\rangle \geq 0 \end{aligned}$$

6.7.9 Proprietà di chiusura dei kernel

$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') \times \mathcal{K}_2(\mathbf{x}, \mathbf{x}')$	<i>Roughly equivalent to AND ops.</i>
$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') + \mathcal{K}_2(\mathbf{x}, \mathbf{x}')$	<i>Roughly equivalent to OR ops.</i>
$\mathcal{K}(\mathbf{x}, \mathbf{x}') = a\mathcal{K}_1(\mathbf{x}, \mathbf{x}')$	$\forall a > 0$.
$\mathcal{K}(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})f(\mathbf{x}')$	<i>for any function f.</i>
...	...

Figura 135: Proprietà di chiusura dei kernel

Un kernel $\mathcal{K}$ può essere normalizzato (cioè, $\|\phi(\mathbf{x})\| = 1$):

$$\tilde{\mathcal{K}}(\mathbf{x}, \mathbf{x}') = \frac{\mathcal{K}(\mathbf{x}, \mathbf{x}')}{\sqrt{\mathcal{K}(\mathbf{x}, \mathbf{x})\mathcal{K}(\mathbf{x}', \mathbf{x}')}}.$$

7 Classificatori Bayesiani

7.1 Teoria delle decisioni di Bayes

La teoria bayesiana delle decisioni è un approccio decisionale fondamentale nell'ambito della teoria delle probabilità.

- Come prendere decisioni in presenza di incertezza
 - La teoria delle decisioni bayesiana è stata motivata da problemi sorti durante la Seconda Guerra Mondiale: radar per il rilevamento aereo, decifrazione e decifratura di codici. Il compito è stimare lo stato, ma abbiamo solo un'osservazione rumorosa, o corrotta.
 - Oggi trova applicazioni nella finanza, nelle assicurazioni e nel gioco d'azzardo
 - * Sia R un set di ricompense base $\{r_1, r_2, \dots, r_N\}$ per gli N risultati di un evento casuale avente probabilità $\{p_1, p_2, \dots, p_N\}$ con $1 = \sum_i p_i$.
 - * La ricompensa prevista per il gioco d'azzardo è:

$$\sum_{i=1}^N r_i \cdot p_i$$

- Quando tutte le probabilità rilevanti sono note, la teoria decisionale bayesiana prende decisioni di classificazione ottimali basate sulle probabilità e sui costi (invece che sulle ricompense in questo dominio di attività)

7.1.1 Teoria delle decisioni di Bayes nella classificazione

Assumiamo che ci siano N classi di etichette distinte, cioè, $y = \{c_1, c_2, \dots, c_N\}$. Sia λ_{ij} il costo di una classificazione errata di un campione di classe c_j come classe c_i . Poi, con la probabilità a posteriori $P(c_i | x)$ possiamo calcolare la **perdita attesa** di classificare un campione x nella classe c_i , ovvero il rischio condizionale del campione x :

$$R(c_i | x) = \sum_{j=1}^N \lambda_{ij} P(c_j | x)$$

È il costo previsto per classificare gli esempi nella classe c_i , avendo considerato tutte le possibili classi come la classe vera.

$$\lambda_{ij} = \begin{cases} 0, & \text{se } i = j; \\ \lambda_{ij} & \text{altrimenti} \end{cases}$$

Il nostro compito è quello di trovare una regola di decisione $h : X \mapsto Y$ che minimizzi il rischio di classificazione totale:

$$R(h) = \mathbf{E}_x [R(h(x) | x)]$$

Il rischio di classificazione totale è minimizzato quando la condizione di rischio $R(h(x) | x)$ di ogni campione x è minimizzata.

Ciò porta alla regola decisionale di Bayes: per minimizzare il rischio complessivo, classificare ogni campione come la classe che minimizza il rischio condizionale $R(c | x)$

$$h^*(x) = \arg \min_{c \in y} R(c | x)$$

- dove h^* è chiamato il **classificatore di Bayes ottimale**, e il suo rischio totale associato $R(h^*)$ è chiamato **rischio di Bayes**.
- $1 - R(h^*)$ è la migliore prestazione che può essere raggiunta da qualsiasi classificatore, ovvero il limite superiore di accuratezza teoricamente raggiungibile per qualsiasi modello di apprendimento automatico.

Per essere più precisi, se l'obiettivo è minimizzare il tasso di errata classificazione, allora la perdita di errata classificazione λ_{ij} può essere scritta come:

$$\lambda_{i,j} \begin{cases} 0, & \text{se } i = j; \\ 1, & \text{altrimenti} \end{cases}$$

e il rischio condizionale è:

$$R(c | \mathbf{x}) = 1 - P(c | \mathbf{x})$$

dove c è la classe corretta per il campione $\mathbf{x}$.

Quindi, il classificatore ottimale di Bayes che minimizza il tasso di classificazione errata è:

$$h^*(x) = \arg \max_{c \in \mathcal{Y}} P(c | x)$$

che classifica ogni campione $\mathbf{x}$ come la classe che massimizza la sua probabilità a posteriori $P(c | \mathbf{x})$.

Possiamo vedere che la regola decisionale di Bayes si basa sulla probabilità a posteriori $P(c | \mathbf{x})$.

Tuttavia, è spesso difficile ottenerlo nella pratica. Il compito dell'apprendimento automatico è quindi stimare accuratamente la probabilità a posteriori $P(c | \mathbf{x})$ a partire da campioni di addestramento finiti.

In generale, ci sono due strategie:

- **modelli discriminativi:**

- Dato $\mathbf{x}$, predice c stimando $P(c | \mathbf{x})$ direttamente.
- Alberi decisionali, reti neurali BP e macchine a vettori di supporto.

- **modelli generativi:**

- stimare prima la probabilità congiunta $P(\mathbf{x}, c)$ e poi stimare $P(c | \mathbf{x})$.
- Per i modelli generativi, dobbiamo valutare:

$$P(c | \mathbf{x}) = \frac{P(\mathbf{x}, c)}{P(\mathbf{x})}$$

Secondo il teorema di Bayes, $P(c | \mathbf{x})$ può essere riscritto come:

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})}$$

- * $P(c)$: la probabilità a priori rappresenta la proporzione di ciascuna classe nel campione, che può essere stimata dalla frequenza di ciascuna classe nel set di addestramento;
- * $P(\mathbf{x})$: il fattore di prova, che è indipendente dalla classe;
- * $P(\mathbf{x} | c)$: la probabilità condizionata alla classe, nota anche come **verosimiglianza** (likelihood), del campione $\mathbf{x}$ rispetto alla classe c

Esempio

Dato:

- Un medico sa che la meningite provoca rigidità del collo nel 50% dei casi;
- La probabilità a priori che un paziente abbia la meningite è $\frac{1}{50.000}$;
- La probabilità a priori che un paziente abbia il collo rigido è $\frac{1}{20}$.

Se un paziente ha il torcicollo, qual è la probabilità che abbia la meningite?

$$P(M | S) = \frac{P(S | M)P(M)}{P(S)} = \frac{0.5 \times 1/50000}{1/20} = 0.0002$$

7.2 Stima della massima verosimiglianza

Una strategia generale per stimare la probabilità condizionata alla classe consiste nell'ipotizzare una forma fissa di distribuzione di probabilità e quindi stimare i parametri di distribuzione utilizzando i campioni di addestramento.

Sia $P(\mathbf{x} | c)$ la probabilità condizionata alla classe della classe c ,

- Supponiamo che P abbia una forma fissa determinata da un vettore di parametri θ_c . Quindi, il compito è stimare θ_c da un set di addestramento D .

θ_c : Se la distribuzione di probabilità assume la forma gaussiana, il vettore dei parametri $\theta_c = \langle \mu_c, \sigma_c^2 \rangle$

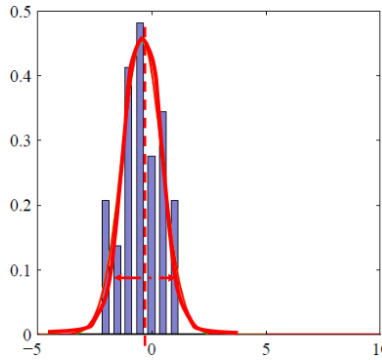


Figura 136: Distribuzione di verosimiglianza con $\mu_c = 0, \sigma_c^2 = 1.5$

Informazioni sul compito di stimare da un set di addestramento:

Il processo di addestramento dei modelli probabilistici è il processo di stima dei parametri. Esistono due modi diversi di pensare ai parametri:

- **(Scuola frequentista)** I parametri hanno valori sconosciuti ma fissi, e quindi possono essere determinati da alcuni approcci come l'ottimizzazione della funzione di verosimiglianza.
- **(Scuola bayesiana)** I parametri sono variabili casuali non osservate che seguono una certa distribuzione, e quindi possiamo assumere distribuzioni a priori per i parametri e stimare la distribuzione a posteriori dai dati osservati.

Sia D_c l'insieme dei campioni di classe c nel training set D , e supponiamo inoltre che i campioni siano campioni i.i.d. Quindi, la verosimiglianza di D_c per un dato parametro θ_c è:

$$P(D_c | \theta_c) = \prod_{\mathbf{x} \in D_c} P(\mathbf{x} | \theta_c)$$

Applicare l'MLE a θ_c significa trovare un valore $\hat{\theta}_c$ del parametro che massimizzi la verosimiglianza $P(D_c | \theta_c)$. Intuitivamente, l'MLE mira a trovare un valore di θ_c che massimizzi la "probabilità" che i dati presenteranno.

Per la distribuzione gaussiana della probabilità, MLE tratta θ_c come variabili e $\mathbf{x}$ come una costante:

$$f_c(x; \theta_c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}}$$

Poiché il prodotto di una sequenza in (7.9) può facilmente portare a un underflow, spesso utilizziamo la logaritmo-verosimiglianza:

$$\begin{aligned} LL(\theta_c) &= \log P(D_c | \theta_c) \\ &= \sum_{\mathbf{x} \in D_c} \log P(\mathbf{x} | \theta_c) \\ &= -\frac{n_c}{2} \log(2\pi\sigma_c^2) - \sum_i \frac{(x_i - \mu_c)^2}{2\sigma_c^2} \end{aligned}$$

e l'MLE di θ_c è $\hat{\theta}_c$:

$$\hat{\theta}_c = \arg \max_{\theta_c} LL(\theta_c)$$

Ad esempio, supponiamo che le features siano continue e che la funzione di densità di probabilità segua la distribuzione gaussiana $p(\mathbf{x} | c) \sim N(\mu_c, \sigma_c^2)$, allora le MLE dei parametri μ_c e σ_c^2 sono:

$$\hat{\mu}_c = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} \mathbf{x}$$

$$\hat{\sigma}_c^2 = \frac{1}{|D_c|} \sum_{\mathbf{x} \in D_c} (\mathbf{x} - \hat{\mu}_c)(\mathbf{x} - \hat{\mu}_c)^T$$

In altre parole, la media stimata della distribuzione gaussiana ottenuta dall'MLE è la media campionaria, e la varianza stimata è la media di $(\mathbf{x} - \hat{\mu}_c)(\mathbf{x} - \hat{\mu}_c)^T$; questo è conforme alla nostra intuizione.

Questo tipo di metodi parametrici semplifica la stima delle probabilità a posteriori, ma l'accuratezza della stima dipende fortemente dalla corrispondenza tra la distribuzione di probabilità ipotetica e la distribuzione dei dati di base sconosciuti. In pratica, una distribuzione di probabilità "ipotizzata" potrebbe portare a risultati fuorvianti.

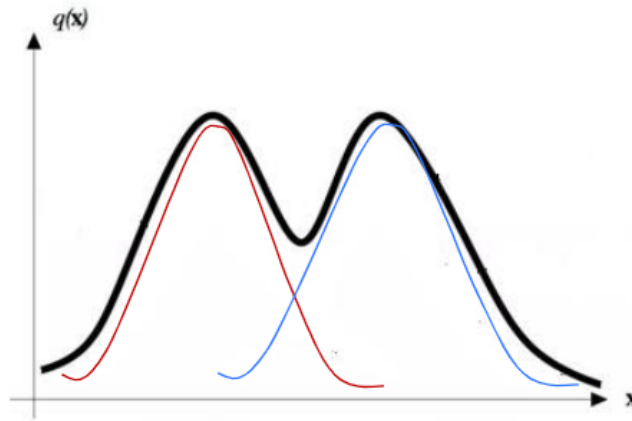


Figura 137: Esempio di adattamento di una distribuzione gaussiana a dati non gaussiani

7.3 Naïve Bayes Classifier

La difficoltà di stimare la probabilità a posteriori $P(c | \mathbf{x})$: non è facile calcolare la probabilità condizionata alla classe $P(\mathbf{x} | c)$ dai campioni di addestramento finiti poiché $P(\mathbf{x} | c)$ è la probabilità congiunta su tutti gli attributi.

- Ad esempio, d proprietà binarie $\rightarrow 2^d$ valori possibili, $2^d \gg$ (il numero di campioni) m .

Nel dataset Watermelon abbiamo $3 * 3 * 3 * 3 * 3 * 2 = 486$ combinazioni di valori e 17 esempi.

Per evitare ciò, il classificatore Bayesiano ingenuo fa l'ipotesi di indipendenza condizionale degli attributi: data una qualsiasi classe nota, si assume che tutti gli attributi siano indipendenti l'uno dall'altro.

Con l'ipotesi di indipendenza condizionale degli attributi, riscriviamo (7.8) come:

$$P(c | \mathbf{x}) = \frac{P(c)P(\mathbf{x} | c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^d P(x_i | c)$$

dove d è il numero di attributi e x_i è il valore assunto sull' i -esimo attributo di $\mathbf{x}$.

Poiché $P(\mathbf{x})$ è lo stesso per tutte le classi, dalla regola decisionale di Bayes (7.6), abbiamo:

$$h_{nb}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} P(c) \prod_{i=1}^d P(x_i | c)$$

che è la formulazione del classificatore Bayes ingenuo.

Per addestrare un classificatore Bayesiano ingenuo, calcoliamo la probabilità a priori $P(c)$ dal set di addestramento D e poi calcoliamo la probabilità condizionata $P(x_i | c)$ per ciascun attributo.

- Sia D_c un sottoinsieme di D contenente tutti i campioni della classe c . Quindi, dati sufficienti campioni i.i.d., la probabilità a priori può essere facilmente stimata da:

$$P(c) = \frac{|D_c|}{|D|}$$

- Per attributi discreti, sia D_{c_i, x_i} un sottoinsieme di D_c contenente tutti i campioni che assumono il valore x_i sull' i -esimo attributo. Quindi, la probabilità condizionata $P(x_i | c)$ può essere stimata da:

$$P(x_i | c) = \frac{|D_{c_i, x_i}|}{|D_c|}$$

- Per le features continue, supponiamo $p(x_i | c) \sim N(\mu_{c,i}, \sigma_{c,i}^2)$, dove $\mu_{c,i}$ e $\sigma_{c,i}^2$ sono, rispettivamente, la media e la varianza della i -esima feature dei campioni di classe c . Quindi, abbiamo:

$$P(x_i | c) = \frac{1}{\sqrt{2\pi\sigma_{c,i}^2}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2\sigma_{c,i}^2}\right)$$

7.3.1 Correzione laplaciana

Se il valore di una feature non è mai apparso insieme a una classe particolare, diventa problematico utilizzare la probabilità. Ad esempio, dato un campione di prova con suono = nitido, il classificatore Bayesiano addestrato sul dataset Watermelon 3.0 predirà 0.

Il risultato della classificazione sarà sempre maturo = falso indipendentemente dai valori delle altre features.

Un simile comportamento è irragionevole.

Per evitare di "rimuovere" le informazioni contenute in altre features, una scelta comune è la correzione laplaciana.

- Sia N il numero di classi distinte nell'insieme di addestramento D , e N_i il numero di valori distinti che la feature i -esima può assumere. Quindi, (7.16) e (7.17) possono essere corrette, rispettivamente, come:

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N} \quad \hat{P}(x_i | c) = \frac{|D_{c_i, x_i}| + 1}{|D_c| + N_i}$$

Esistono diversi modi per utilizzare un classificatore Bayesiano. Se la velocità di previsione è importante, un classificatore Bayesiano può precalcolare tutte le probabilità rilevanti e salvarle per un uso successivo; se i dati di training cambiano frequentemente, possiamo adottare un approccio di apprendimento lento.

7.4 Semi-Naïve Bayes Classifier

Per superare la difficoltà di calcolare $P(c|x)$ nel teorema di Bayes, il classificatore Bayesiano ingenuo fa l'ipotesi di indipendenza condizionale degli attributi, che spesso non è valida nella pratica. Pertanto, i classificatori Bayesiani semi-ingenui sono sviluppati per allentare in una certa misura questa ipotesi.

Una strategia generale utilizzata dai classificatori Bayesiani semi-naïve è lo stimatore unidipendente (ODE). "un dipendente" significa che ogni feature può dipendere al massimo da una feature diversa dalle informazioni sulla classe, ovvero:

$$P(c | x) \propto P(c) \prod_{i=1}^d P(x_i | c, pa_i)$$

dove x_i dipende da pa_i , e pa_i è chiamato feature madre di x_i .

Per ogni feature x_i , se la feature madre pa_i è nota, allora $P(x_i | c, pa_i)$ può essere stimata. Quindi, il problema chiave diventa determinare le features genitrici.

7.4.1 SPODE

L'approccio più semplice è chiamato Super-Parent ODE (SPODE), che presuppone che tutte le feature dipendano da una sola feature, chiamata super-parent. SPODE seleziona la feature super-parent utilizzando metodi di selezione del modello come la convalida incrociata.

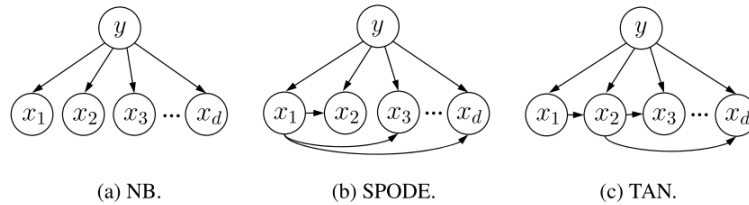


Figura 138: Dipendenze delle features dei classificatori Bayes ingenui e semi-naive

7.4.2 TAN

TAN (Tree augmented Naïve Bayes) [Friedman et al., 1997], basato su un albero di copertura massimo ponderato [Chow e Liu, 1968], semplifica le dipendenze delle features in una struttura ad albero, come mostrato nella Figura 138(c):

- Calcola l'informazione reciproca condizionale per ogni coppia di features:

$$I(x_i, x_j | y) = \sum_{x_i, x_j; c \in y} P(x_i, x_j | c) \log \frac{P(x_i, x_j | c)}{P(x_i | c)P(x_j | c)}$$

- Costruisci un grafo non orientato completo in cui i nodi sono feature. Imposta $I(x_i, x_j | y)$ come peso;
- Costruisci uno spanning tree con peso massimo e seleziona una feature radice. Imposta la direzione di ogni spigolo verso l'esterno della feature radice;
- Aggiungi un nodo di classe y e aggiungi archi diretti da y ad altri nodi di feature.

Sugli alberi di peso minimo Sono utili nella **scoperta causale** (per semplificare grafici complessi ed evidenziare le cause profonde dei disturbi di processo) o nella **progettazione di reti** economicamente vantaggiose.

Un **albero di copertura minimo** (MST) o albero di copertura di peso minimo è:

1. un sottoinsieme degli archi di un grafo non orientato, connesso e ponderato
2. Collega tutti i vertici tra loro, senza cicli
3. Con il minimo peso totale possibile degli archi.

Si tratta di un albero di copertura in cui la somma dei pesi degli spigoli è la più piccola possibile.

Per convertire MST in un albero di copertura a peso massimo, moltiplicare i pesi per -1 e quindi utilizzare un algoritmo per MST:

- **Algoritmo di Kruskal:** ordina gli archi in base al loro peso (negato) in ordine crescente. Aggiungi archi all'albero in crescita purché non formino un ciclo.
- **Algoritmo di Prim:** si parte da un vertice arbitrario e si aggiunge iterativamente l'arco con il peso più alto (negato) che collega un vertice nell'albero a un vertice esterno all'albero, assicurandosi che non si formino cicli.

7.4.3 AODE

L'Averaged One-Dependent Estimator (AODE) [Webb et al., 2005] è un classificatore uni-dipendente più potente, che sfrutta l'apprendimento d'insieme.

- Prova a usare ogni funzionalità come super-genitore per costruire più modelli SPODE
- Integrare quelli supportati da dati di addestramento sufficienti:

$$P(c | \mathbf{x}) \propto \sum_{i=1; |D_{x_i}| \geq m'}^d P(c, x_i) \prod_{j=1}^d P(x_j | c, x_i)$$

dove D_{x_i} è il sottoinsieme di campioni che assumono il valore x_i sulla i -esima feature, e m' è una costante di soglia.

dove N è il numero di classi distinte in D , N_i è il numero di valori distinti che la features i -esima può assumere, D_{c, x_i} è il sottoinsieme di campioni di classe c che assumono il valore x_i sulla features i -esima.

7.5 Reti Bayesiane

Le reti bayesiane, note anche come reti di credenze, utilizzano grafi aciclici diretti (DAG) per rappresentare le dipendenze tra le features e costruire tabelle di probabilità condizionale (CPT) per descrivere le probabilità congiunte delle features.

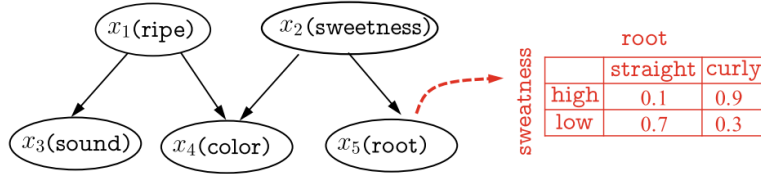


Figura 139: La struttura della rete bayesiana del problema dell'anguria insieme alla tabella di probabilità condizionata della radice

Dalla struttura della rete, vediamo che il colore dipende direttamente dalla maturazione e dalla dolcezza.

Dal CPT, possiamo inoltre ottenere una dipendenza quantificata della radice dalla dolcezza, ad esempio:

$$P(\text{root} = \text{straight} \mid \text{sweetness} = \text{high}) = 0.1$$

7.5.1 Struttura di rete

La struttura della rete bayesiana esprime efficacemente l'indipendenza condizionale tra le features. Dati gli insiemi di nodi padre, la **rete bayesiana presuppone che ogni feature sia indipendente da tutte le sue features non discendenti**.

$B = \langle G, \Theta \rangle$ definisce la probabilità congiunta delle features $x_1, x_2, \dots, x_d$:

$$P(x_1, x_2, x_3, x_4, x_5) = P(x_1)P(x_2)P(x_3 \mid x_1)P(x_4 \mid x_1, x_2)P(x_5 \mid x_2)$$

Prendendo come esempio la Figura 139, la probabilità congiunta è definita come:

$$P_B(x_1, x_2, \dots, x_d) = \prod_{i=1}^d P_B(x_i \mid \pi_i) = \prod_{i=1}^d \theta_{x_i \mid \pi_i}$$

Possiamo vedere che x_3 e x_4 sono indipendenti dato il valore di x_1 (ovvero, $x_3 \perp x_4 \mid x_1$). Inoltre, x_4 e x_5 sono indipendenti dato il valore di x_2 (ovvero, $x_4 \perp x_5 \mid x_2$).

Di seguito sono riportati tre casi tipici di dipendenza tra tre variabili:

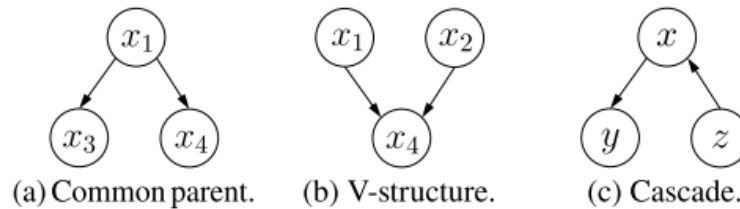


Figura 140: Dipendenze tipiche delle funzionalità nelle reti bayesiane

Possiamo usare la d-separazione per analizzare l'indipendenza condizionale tra variabili in un grafo orientato.

- Trova tutte le strutture a V e quindi aggiungi uno spigolo non orientato
- Trasforma tutti gli spigoli orientati in spigoli non orientati.

Il grafo non orientato prodotto da questo processo è chiamato **grafo morale**. Il nome deriva dal fatto che, in un grafo morale, due nodi che hanno un figlio comune devono essere sposati condividendo un arco.

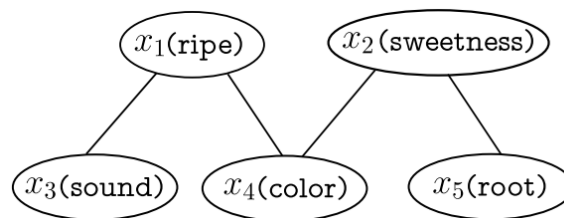


Figura 141: Esempio di grafo morale per 139

7.5.2 Inferenza

Il processo di inferenza del valore di una feature interrogata dai valori della feature osservata è chiamato inferenza, dove i valori della feature osservata sono chiamati evidenza.

Idealmente, vorremmo utilizzare la distribuzione di probabilità congiunta definita dalla rete bayesiana per calcolare le esatte probabilità a posteriori della feature interrogata, data l'evidenza.

7.5.3 Alcuni esempi noti di BN

Alarm

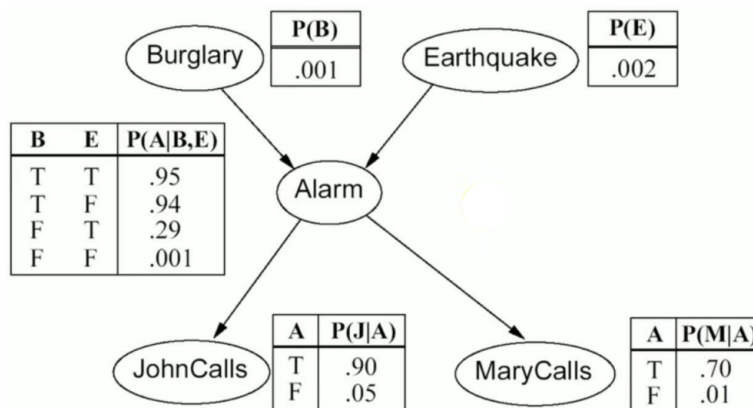


Figura 142: Rete bayesiana Alarm

Diabete di tipo 2 ad esordio

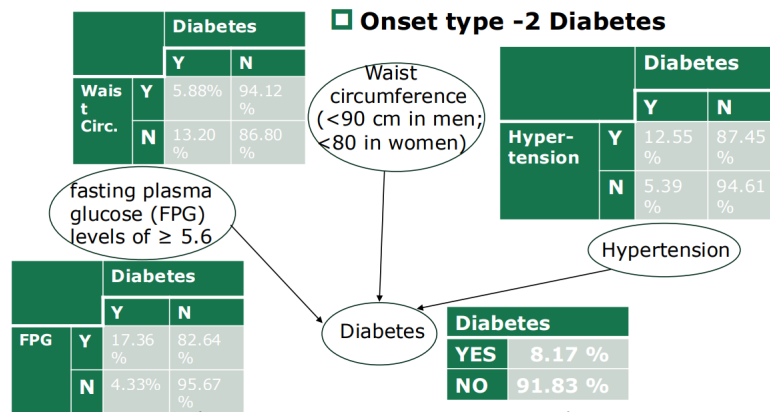


Figura 143: Rete bayesiana per il diabete di tipo 2 ad esordio

Erba bagnata

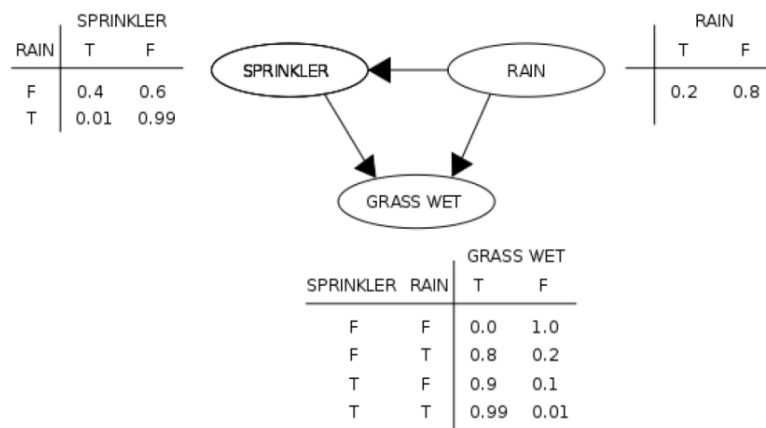
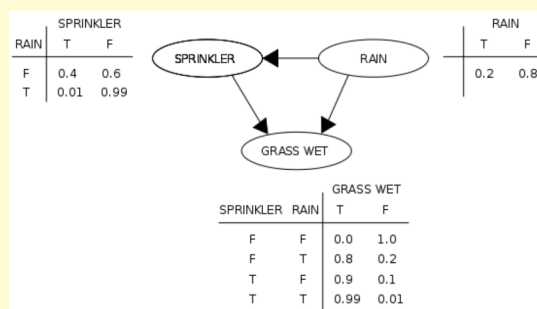


Figura 144: Rete bayesiana per l'erba bagnata

Esercizio sull'inferenza utilizzando un BN



Domanda: Dato che osservi l'erba bagnata (prova), qual è la probabilità che abbia piovuto (feature interrogata)?

Richiamo: Regole fondamentali delle probabilità

- Somma unitaria delle probabilità di una variabile casuale sul dominio della variabile casuale:

$$1 = \sum_{y_i \in \text{Dom}(Y)} P(Y = y_i)$$

- Regola della somma (marginalizzazione):

$$p(X = x_i) = \sum_{y_i \in \text{Dom}(Y)} P(X = x_i, Y = y_i)$$

- Regola del prodotto:

$$P(Y | X) = \frac{P(X, Y)}{P(X)} \Rightarrow P(X, Y) = P(Y | X)P(X)$$

- Variabili indipendenti:

$$P(X, Y) = P(X)P(Y)$$

Marginalizzazione su Watermelon 2.0; su colore e radice

ID	color	root
1	green	curly
2	dark	curly
3	dark	curly
4	green	curly
5	light	curly
6	green	slightly curly
7	dark	slightly curly
8	dark	slightly curly
9	dark	slightly curly
10	green	straight
11	light	straight
12	light	curly
13	green	slightly curly
14	light	slightly curly
15	dark	slightly curly
16	light	curly
17	green	curly

⇒

Contingency table					
		ROOT			
		curly	slightly curly	straight	SUM
C O L O R	green	3	2	1	6
	dark	2	4	0	6
	light	3	1	1	5
	SUM	8	7	2	17

⇒

Probability Table:					
		ROOT			
		curly	slightly curly	straight	SUM
C O L O R	green	3/17	2/17	1/17	6/17
	dark	2/17	4/17	0	6/17
	light	3/17	1/17	1/17	5/17
	SUM	8/17	7/17	2/17	17/17

$P(\text{Color} = \text{green})$

$P(\text{Color} = \text{green}, \text{Root} = \text{curly})$

$P(\text{Root} = \text{curly})$

Figura 145: Marginalizzazione su Watermelon 2.0, su colore e radice

Esercizio

Dato che hai osservato l'erba bagnata, qual è la probabilità che abbia piovuto?

$$P(\text{Rain} = T | \text{Grass_wet} = T) = \frac{P(\text{Rain} = T, \text{Grass_wet} = T)}{P(\text{Grass_wet} = T)}$$

$$\begin{aligned}
 P(\text{Rain} = T, \text{Grass_wet} = T) &= \sum_{\text{Sprinkle}} P(\text{Rain} = T, \text{Grass_wet} = T, \text{Sprinkle}) \\
 &= \sum_{\text{Sprinkle}} P(\text{Grass_wet} = T | \text{Rain} = T, \text{Sprinkle})P(\text{Rain} = T)P(\text{Sprinkle})
 \end{aligned}$$

Esercizio

$$P(\text{Sprinkle}, \text{Rain} = T)$$

RAIN	SPRINKLER	
	T	F
F	0.4	0.6
T	0.01	0.99

	RAIN	
	T	F
	0.2	0.8

$$P(\text{Sprinkle}, \text{Rain}) = P(\text{Sprinkle} \mid \text{Rain})P(\text{Rain})$$

$$\begin{aligned} P(\text{Sprinkle}, \text{Rain} = T) &= P(\text{Sprinkle} \mid \text{Rain} = T)P(\text{Rain} = T) \\ &= P(\text{Sprinkle} = T \mid \text{Rain} = T)P(\text{Rain} = T) + P(\text{Sprinkle} = F \mid \text{Rain} = T)P(\text{Rain} = T) \\ &= 0.01 \cdot 0.2 + 0.99 \cdot 0.2 = 0.2 = P(\text{Rain} = T) \end{aligned}$$

$$\begin{aligned} P(\text{Sprinkle}, \text{Rain} = F) &= P(\text{Sprinkle} = T \mid \text{Rain} = F)P(\text{Rain} = F) + \\ &\quad P(\text{Sprinkle} = F \mid \text{Rain} = F)P(\text{Rain} = F) = \\ &= 0.4 \cdot 0.8 + 0.6 \cdot 0.8 = 0.8 = P(\text{Rain} = F) \end{aligned}$$

Abbiamo scoperto che Sprinkle e Rain sono variabili indipendenti.

$$P(\text{Rain} = T, \text{Grass_wet} = T) \quad \text{Numeratore dell'importo che vogliamo calcolare}$$

SPRINKLER RAIN		GRASS WET	
		T	F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

$$\begin{aligned} P(\text{Rain} = T, \text{Grass_wet} = T) &= \sum_{\text{Sprinkle}} P(\text{Rain} = T, \text{Grass_wet} = T, \text{Sprinkle}) \\ &= \sum_{\text{Sprinkle}} P(\text{Grass_wet} = T \mid \text{Rain} = T, \text{Sprinkle})P(\text{Rain} = T, \text{Sprinkle}) \\ &= 0.8 \cdot 0.2 + 0.99 \cdot 0.2 = 0.358 \end{aligned}$$

$$P(\text{Grass_wet}) \quad \text{Denominatore dell'importo che vogliamo calcolare}$$

$$\begin{aligned} P(\text{Grass_wet} = T) &= \\ &P(\text{Grass_wet} = T \mid \text{Sprinkle} = F, \text{Rain} = T)P(\text{Sprinkle} = F, \text{Rain} = T) \\ &+ P(\text{Grass_wet} = T \mid \text{Sprinkle} = T, \text{Rain} = F)P(\text{Sprinkle} = T, \text{Rain} = F) \\ &+ P(\text{Grass_wet} = T \mid \text{Sprinkle} = T, \text{Rain} = T)P(\text{Sprinkle} = T, \text{Rain} = T) \\ &= 0.8 \cdot P(\text{Sprinkle} = F, \text{Rain} = T) + 0.9 \cdot P(\text{Sprinkle} = T, \text{Rain} = F) \\ &+ 0.99 \cdot P(\text{Sprinkle} = T, \text{Rain} = T) \end{aligned}$$

Esercizio

$P(\text{Sprinkle}, \text{Rain})$ Fattori per ottenere $P(\text{Grass_wet})$

$$P(\text{Sprinkle} = F, \text{Rain} = T) = P(\text{Sprinkle} = F \mid \text{Rain} = T)P(\text{Rain} = T)$$

$$P(\text{Sprinkle} = F, \text{Rain} = T) = 0.99 \cdot 0.2 = 0.198$$

$$P(\text{Sprinkle} = T, \text{Rain} = F) = P(\text{Sprinkle} = T \mid \text{Rain} = F)P(\text{Rain} = F)$$

$$P(\text{Sprinkle} = T, \text{Rain} = F) = 0.4 \cdot 0.8 = 0.32$$

$$P(\text{Sprinkle} = T, \text{Rain} = T) = P(\text{Sprinkle} = T \mid \text{Rain} = T)P(\text{Rain} = T)$$

$$P(\text{Sprinkle} = T, \text{Rain} = T) = 0.01 \cdot 0.2 = 0.002$$

$P(\text{Grass_wet})$ Denominatore dell'importo che vogliamo

$$P(\text{Grass_wet} = T) =$$

$$0.8 \cdot P(\text{Sprinkle} = F, \text{Rain} = T) + 0.9 \cdot P(\text{Sprinkle} = T, \text{Rain} = F)$$

$$+ 0.99 \cdot P(\text{Sprinkle} = T, \text{Rain} = T)$$

Sostituiamo i termini ottenuti:

$$P(\text{Sprinkle} = F, \text{Rain} = T) = 0.198$$

$$P(\text{Sprinkle} = T, \text{Rain} = F) = 0.32$$

$$P(\text{Sprinkle} = T, \text{Rain} = T) = 0.002$$

$$P(\text{Grass_wet} = T) = 0.8 \cdot 0.198 + 0.9 \cdot 0.32 + 0.99 \cdot 0.002 = 0.448$$

Calcolo finale

$$P(\text{Rain} = T \mid \text{Grass_wet} = T) = \frac{P(\text{Rain} = T, \text{Grass_wet} = T)}{P(\text{Grass_wet} = T)} = \frac{0.358}{0.448} = 0.799$$

7.5.4 Learning

Il compito principale dell'apprendimento delle reti bayesiane è trovare la rete bayesiana più "appropriata".

Innanzitutto definiamo una funzione di punteggio per valutare l'idoneità tra una rete bayesiana e un set di addestramento.

- Secondo il principio della lunghezza minima della descrizione (MDL), cerchiamo la rete bayesiana con la lunghezza di codice combinata più breve.

7.5.5 Introduzione alla lunghezza minima della descrizione (MDL)

L'MDL ha le sue origini nella Teoria dell'Informazione ed è stato sviluppato nei campi della statistica, dell'informatica teorica e dell'apprendimento automatico, e più specificamente della Teoria dell'Apprendimento Computazionale.

Storicamente, esistono significati del "principio della lunghezza minima di descrizione" che variano a seconda di cosa si intenda per descrizione.

Nella teoria dell'apprendimento di Rissanen, i modelli sono ipotesi statistiche e le descrizioni sono definite come codici universali.

I modelli di un dato insieme di osservazioni, codificati come bit di dati, possono essere creati sotto forma di programmi per computer che producono quei dati.

Rissanen è stato il primo a derivare automaticamente descrizioni brevi (Criterio Bayesiano dell'Informazione - BIC).

Nella Teoria Algoritmica dell'Informazione, la lunghezza della descrizione di una sequenza di dati è la **lunghezza del programma più piccolo che produce quel set di dati**. È strettamente correlato alla teoria dell'inferenza induttiva di Solomonoff: il modello migliore di un set di dati è rappresentato dal suo archivio autoestraente più breve.

7.5.6 Modelli basati sulla compressione

Consideriamo di nuovo la regola della decisione massima a posteriori:

$$y_{MAP} = \arg \max_y P(X = x | Y = y)P(Y = y)$$

Prendendo i logaritmi negativi, possiamo trasformarlo in una minimizzazione equivalente:

$$y_{MAP} = \arg \min_y -\log P(X = x | Y = y) - \log P(Y = y)$$

- Se un evento ha probabilità p di accadere, il logaritmo negativo di p quantifica il contenuto informativo del messaggio che l'evento è effettivamente accaduto.
- Scriviamo $IC(X | Y)$ e $IC(Y) = -\log_2 P(Y)$.

$$y_{MAP} = \arg \min_y \{IC(X = x | Y = y) + IC(Y = y)\}$$

- L'unità di informazione dipende dalla base del logaritmo: di solito si usano i logaritmi in base 2, nel qual caso l'informazione si misura in bit.
- Ad esempio, se lanci una moneta equilibrata una volta e mi dici che è uscito testa, questo contiene $-\log_2 \frac{1}{2} = 1$ bit di informazione; se lanci un dado equilibrato una volta e mi dici che è uscito sei, il contenuto informativo del tuo messaggio è $-\log_2 \frac{1}{6} = 2.6$ bit.

Per messaggi più brevi, che contengono un contenuto informativo inferiore, sono necessari meno bit.

Sia $L(m)$ la lunghezza in bit di una descrizione del modello m , e sia $L(D | m)$ la lunghezza in bit di una descrizione dei dati D dato il modello m (entrambi misurati da un codice L quasi ottimale). Secondo il principio della lunghezza minima della descrizione, il modello preferito è quello che minimizza la lunghezza della descrizione del modello e dei dati dato il modello:

$$m_{MDL} = \arg \min_{m \in M} (L(m) + L(D | m))$$

$L(m)$ quantifica la complessità del modello, mentre $L(D | m)$ quantifica l'errore del modello.

Quale codifica utilizzare per determinare la complessità del modello $L(m)$ spesso non è semplice e in una certa misura soggettivo. Questo è simile alla prospettiva bayesiana, in cui dobbiamo definire una distribuzione a priori sui modelli. Il punto di vista MDL offre un modo concreto per definire le distribuzioni a priori del modello mediante codici.

MDL bilancia la complessità e l'accuratezza del modello.

7.5.7 Il principio della lunghezza minima della descrizione

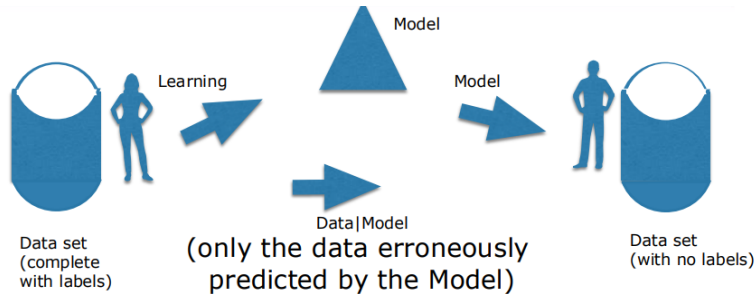


Figura 146: Il principio della lunghezza minima della descrizione

La lunghezza dei messaggi inviati tra due persone misura il costo della consegna delle informazioni scambiate (quantifica la quantità di informazioni del Modello e dei Dati).

$$Cost(Model, Data) = Cost(Model) + Cost(Data | Model)$$

Il modello migliore sarebbe quel modello la cui quantità di informazioni è minima (minimizza $Cost(Model)$), il che è in accordo con il principio del rasoio di Occam sulla preferenza verso modelli semplici.

Inoltre, il modello migliore dovrebbe anche minimizzare l'errore del modello sui dati, che è rappresentato da:

$$Cost(Data \mid Model)$$

È la quantità di informazioni sui dati che dobbiamo rappresentare esplicitamente, separatamente dal modello.

7.5.8 Learning

Dato un set di addestramento $D = \{x_1, x_2, \dots, x_m\}$ il punteggio della funzione di una rete bayesiana $B = \langle G, \Theta \rangle$ su D , può essere scritta come:

$$S(B \mid D) = f(\theta)|B| - LL(B \mid D)$$

- Dove $|B|$ è il numero di parametri della rete bayesiana, $f(\theta)$ è la lunghezza di codifica richiesta per descrivere il parametro θ , e:

$$LL(B \mid D) = \sum_{i=1}^m \log P_B(x_i)$$

la verosimiglianza logaritmica della rete bayesiana.

7.5.9 Inferenza approssimativa

Abbiamo detto che le reti bayesiane sono utili perché consentono l'inferenza su una feature interessante (interrogata).

Idealmente, vorremmo utilizzare la distribuzione di probabilità congiunta definita dalla rete bayesiana per calcolare le esatte probabilità a posteriori.

Un tipico metodo approssimativo per le reti bayesiane è il campionamento di Gibbs.

L'algoritmo di campionamento di Gibbs inizia con un campione q^0 generato casualmente e genera un nuovo campione modificando le variabili non probatorie del campione corrente. Supponiamo di aver campionato T volte e ottenuto n_q campioni coerenti con la query q , quindi la probabilità a posteriori può essere approssimata come:

$$P(Q = q \mid E = e) \simeq \frac{n_q}{T}$$

Poiché ogni passaggio dipende solo dallo stato del passaggio precedente, il campionamento genera una catena di Markov.

Algorithm 3 Gibbs Sampling

Require: Bayesian network $B = \langle G, \Theta \rangle$; Number of samplings T ; Evidence variables E and their values e ; Query variable Q and their values q .

```

1: Process:
2:  $n_q = 0$ ;
3:  $q^0 =$  assign random initial values to  $Q$ ;
4: for  $t = 1, 2, \dots, T$  do
5:   for  $Q_i \in Q$  do
6:      $Z = E \cup Q \setminus \{Q_i\}$ ;
7:      $z = e \cup q^{t-1} \setminus \{q_i^{t-1}\}$ 
8:     Compute distribution  $P_B(Q_i \mid Z = z)$  according to  $B$ ;
9:      $q_i^t = Q_i$  sampled from  $P_B(Q_i \mid Z = z)$ 
10:     $q^t = q^{t-1}$  with  $q_i^{t-1}$  replaced by  $q_i^t$ ;
11:   end for
12:   if  $q^t = q$  then
13:      $n_q = n_q + 1$ 
14:   end if
15: end for
```

Ensure: $P(Q = q \mid E = e) \simeq \frac{n_q}{T}$

7.6 Algoritmo EM

Spesso abbiamo campioni di addestramento incompleti: la radice di un'anguria potrebbe essere stata rimossa, ovvero la radice della feature è sconosciuta. Quindi, in presenza di features non osservate, come possiamo stimare i parametri del modello?

Sia X l'insieme delle variabili osservate, Z l'insieme delle variabili latenti e Θ i parametri del modello. Quindi, la stima di massima verosimiglianza di Θ massimizza la logaritmo-verosimiglianza:

$$LL(\Theta | X, Z) = \ln P(X, Z | \Theta)$$

Non possiamo risolvere direttamente questo problema di massimizzazione a causa delle variabili latenti Z . Tuttavia, possiamo usare l'aspettativa di Z per massimizzare la verosimiglianza marginale logaritmica dei dati osservati:

$$LL(\Theta | X) = \ln P(X | \Theta) = \ln \sum_Z P(X, Z | \Theta)$$

L'algoritmo di Massimizzazione dell'Aspettativa (EM) [Dempster et al., 1977] è un potente metodo iterativo per la stima delle variabili latenti.

Dato il valore di Θ , dai dati di training, possiamo dedurre il valore atteso per ciascuna variabile latente in Z (il passo E).

Dati i valori delle variabili latenti in Z , possiamo stimare Θ con la stima di massima verosimiglianza (il passo M). Inizializziamo (7.35) con Θ^0 e applichiamo iterativamente i seguenti due passi fino alla convergenza:

Deduciamo il valore atteso di Z con Θ^t , indicato con Z^t ;

Stima Θ con la stima di massima verosimiglianza basata su Z^t e le variabili osservate X , indicate con Θ^{t+1} ;

Il processo iterativo sopra descritto è il prototipo dell'algoritmo EM.

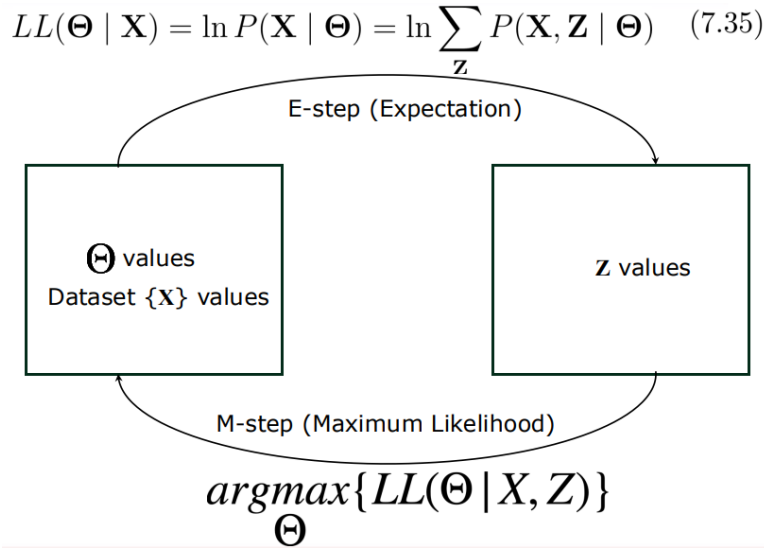


Figura 147: Diagramma di flusso dell'algoritmo EM

Invece di usare l'aspettativa di Z , possiamo calcolare la distribuzione di probabilità $P(Z | X, \Theta^t)$ in base a Θ^t , e i due passaggi precedenti diventano:

- Passo E (Aspettativa): inferire la distribuzione della variabile latente $P(Z | X, \Theta^t)$ in base a Θ^t , e quindi calcolare l'aspettativa della logaritmo-verosimiglianza $LL(\Theta | X, Z)$ rispetto a Z come:

$$Q(\Theta | \Theta^t) = \mathbb{E}_{Z|X, \Theta^t} [LL(\Theta | X, Z)]$$

- Passo M (Massimizzazione): trovare i parametri che massimizzano la logaritmo-verosimiglianza attesa, ovvero

$$\Theta^{t+1} = \arg \max_{\Theta} Q(\Theta | \Theta^t)$$

- l'algoritmo EM trova i parametri che massimizzano la logaritmo-verosimiglianza attesa nel passo E. Iterando i due passaggi precedenti fino a convergere a un ottimo locale.

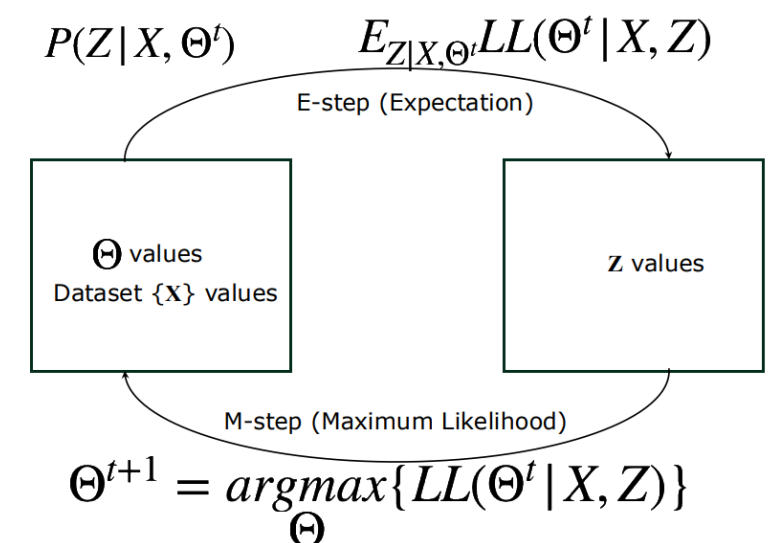


Figura 148: Diagramma di flusso dell'algoritmo EM (versione 2)

8 Ensemble Learning

8.1 Cos'è Ensemble Learning

Ensemble Learning si riferisce a un insieme di metodi che apprendono una funzione target addestrando un certo numero di learners individuali e combinando le loro previsioni.

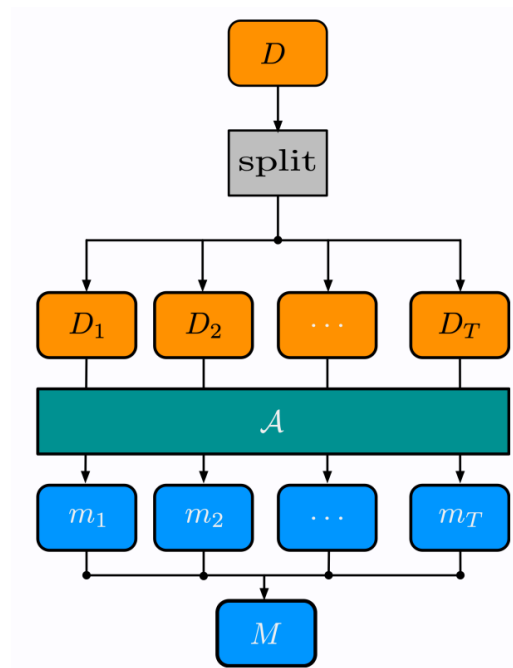


Figura 149: Esempio di Ensemble Learning

8.1.1 Come creare un ensemble efficace?

Quando si progettano gli ensemble bisogna prendere due decisioni:

- Come si dividono i dati/si generano i classificatori di base?
- Come si combinano i classificatori di base in un classificatore finale?

8.1.2 Come si integrano/combinano i classificatori di base

Approccio comune: votare a maggioranza:

$$M(x) = f\left(\sum_{t=1}^T w_t m_t(x)\right)$$

dove:

- w_t è il "peso" associato al modello m_t ;
- $w_t > w_u$ è da intendere come " m_t è più affidabile di m_u ";
- tipicamente $w_t > 0$ (sebbene si possa dare il significato $w_t < 0$ che " m_t è più spesso sbagliato che giusto")

Sono possibili progetti più fantasiosi, ma poco comuni.

8.1.3 Come generare classificatori di base?

Molte scelte:

- Bagging
- Boosting
- ECOC
- Stacking
- ...

8.2 Bagging (Bootstrap Aggregating)

```
1 def bagging(D: Dataset, A: WeakAlgo, T: int) -> List[Classifier]:
2     classifiers = []
3     for t in range(T):
4         Dt = D.bootstrap_replicate()
5         mt = A(Dt)
6         classifiers.append(mt)
7
8     return classifiers
```

Il classificatore ensemble viene quindi calcolato come il voto di maggioranza dei classificatori nell'insieme:

$$M(x) = \text{sign} \left(\sum_{t=1}^T m_t(x) \right)$$

8.2.1 Bootstrap replicates

In qualsiasi algoritmo di apprendimento ensemble è importante promuovere la diversità tra i classificatori: ovviamente, **se tutti i classificatori sono identici, l'insieme non migliorerà rispetto al classificatore debole.**

Nel Bagging, questa diversità viene promossa costruendo **repliche bootstrap** del set di dati originale D .

Bootstrap replica

Una **replica bootstrap** di un dataset D è un nuovo dataset costruito estraendo **con rimpiazzamento** $|D|$ esempi da D .

Il campionamento bootstrap ha la proprietà interessante di creare un insieme di esempi che seguono la stessa distribuzione del set di dati originale.

In media una **replica bootstrap conterrà** $\sim 63\%$ di esempi unici dal dataset originale.

Dimostrazione

La probabilità che un esempio specifico **non** sia selezionato è $(1 - \frac{1}{n})^n$ che, al limite, equivale a $\frac{1}{e} \simeq 0.3678$.

Ne consegue che, in media, solo $(1 - \frac{1}{e}) \times n \simeq 0.63 \times n$ esempi, (cioè, $\sim 63\%$ degli esempi) saranno inclusi nella replica bootstrap.

8.2.2 Stima dell'errore Out-of-bag

Possiamo **sfruttare il fatto che non tutti gli esempi vengono utilizzati da ciascun weak learner** per calcolare una **stima dell'errore di generalizzazione** commesso dall'ensemble **senza** bisogno di un set di test separato.

Innanzitutto, abbiamo bisogno di un modo per calcolare la **previsione dell'ensemble** utilizzando solo classificatori addestrati su repliche bootstrap che **non contengono** l'esempio esaminato:

$$H^{oob}(\mathbf{x}) = \arg \max_y \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y) \cdot \mathbb{I}(\mathbf{x} \notin D_t)$$

Possiamo quindi **stimare l'errore di generalizzazione** come al solito:

$$\epsilon^{oob} = \frac{1}{|D|} \sum_{i=1}^{|D|} \mathbb{I}(H^{oob}(\mathbf{x}_i) \neq y_i)$$

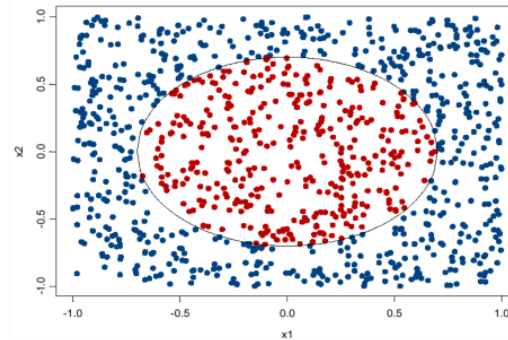


Figura 150: Esempio di Bagging

Confine di decisione CART

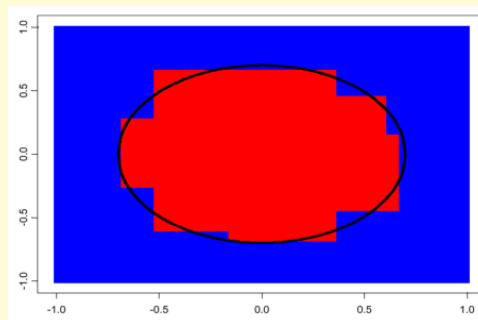


Figura 151: Confine di decisione di un albero CART

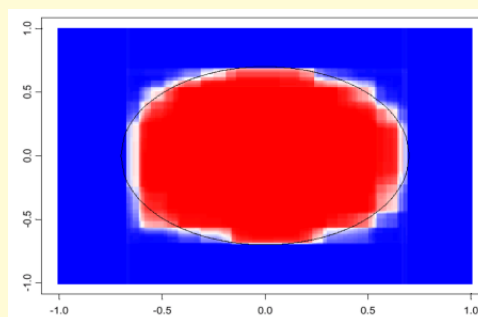


Figura 152: Confine di decisione di un ensemble di 100 alberi CART tramite Bagging

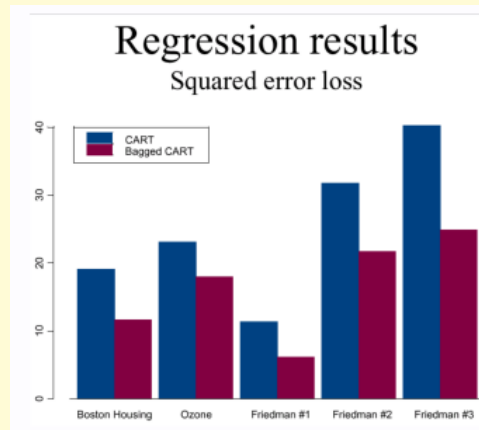


Figura 153: Bagging vs CART in un problema di regressione

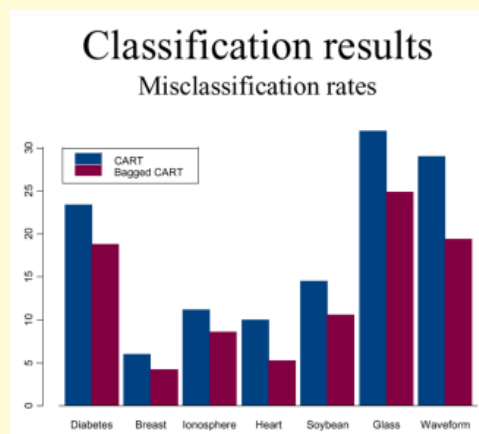


Figura 154: Bagging vs CART in un problema di classificazione

8.2.3 Random Forests (RFs)

Una **random forest** è un'istanza del **Bagging** che utilizza **alberi decisionali non potati** come learner debole.

L'unica differenza rispetto allo schema precedente è che l'algoritmo di apprendimento dell'**albero decisionale** è **modificato** in modo da considerare un sottoinsieme casuale F' dell'insieme di feature F in ogni punto di divisione.

Breiman (2001), che propose RFs, affermò che $|F'|$ è mantenuto fisso e tale che $|F'| \ll |F|$.

Un approccio comune è di scegliere $|F'| = \sqrt{|F|}$.

8.3 AdaBoost (Adaptive Boosting)

Definizione

Una classe di concetti è apprendibile (o **fortemente apprendibile**) se esiste un algoritmo in tempo polinomiale che raggiunge un basso errore con un'elevata confidenza per tutti i concetti della classe. Un modello più debole di apprendibilità, chiamato **apprendibilità debole**, abbandona il requisito che l'apprendista sia in grado di raggiungere un'accuratezza arbitrariamente elevata; un algoritmo di apprendimento debole deve solo produrre un'ipotesi che ha prestazioni leggermente migliori [...] rispetto a un'ipotesi casuale.

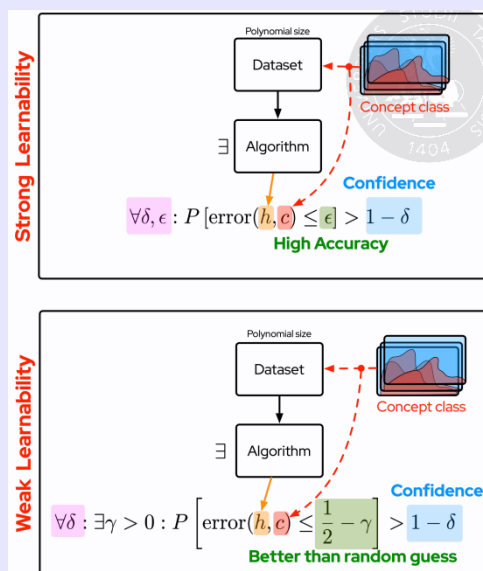


Figura 155: Strong vs Weak Learnability

Nota

In questo contesto, "tempo polinomiale" significa che l'algoritmo applica i risultati dopo aver visto un numero polinomiale di esempi.

8.3.1 Il problema dell'aumento delle ipotesi

Il concetto di apprendibilità debole è stato introdotto da Kearns e Valiant [...] che hanno lasciato **aperta la questione se le nozioni di apprendibilità forte e debole siano equivalenti**. Questa questione è stata definita problema di **potenziamento dell'ipotesi**, poiché dimostrare che le nozioni sono equivalenti richiede un metodo per potenziare la bassa accuratezza delle ipotesi di un algoritmo di apprendimento debole. [...]

[...] **Kearns (1988)**, considerando il problema del potenziamento delle ipotesi, fornisce un argomento convincente che **scredita l'approccio naturale** che consiste nel cercare di aumentare l'accuratezza di un algoritmo di apprendimento debole eseguendo la procedura più volte e prendendo il "voto di maggioranza" delle ipotesi di output. Inoltre, **Kearns e Valiant [...]** mostrano che, **sotto una distribuzione uniforme sullo spazio delle istanze, le funzioni booleane monotone sono debolmente, ma non fortemente, apprendibili**.

Questo dimostra che **l'apprendibilità forte e debole non sono equivalenti quando vengono imposte determinate restrizioni alla distribuzione dello spazio delle istanze**. Pertanto, non sembrava improbabile che i modelli di apprendimento forte e debole si dimostrassero non equivalenti anche per distribuzioni non vincolate.

8.3.2 Boosting

L'algoritmo di boosting (e la sua variante migliorata AdaBoost) ha permesso di rispondere in modo positivo (e costruttivo) al problema del boosting dell'ipotesi.

Teorema

Una classe di concetti $\mathcal{C}$ è debolmente apprendibile se e solo se è fortemente apprendibile.

In altri termini: sia A un algoritmo di apprendimento in grado di fornire una soluzione leggermente migliore di un'ipotesi casuale su ogni istanza del problema, **se esiste** una procedura di boosting in grado di fornire una soluzione accurata quanto necessario che utilizza A come apprendista di base (e il miglioramento dell'accuratezza cresce esponenzialmente con il numero di volte in cui A viene utilizzato).

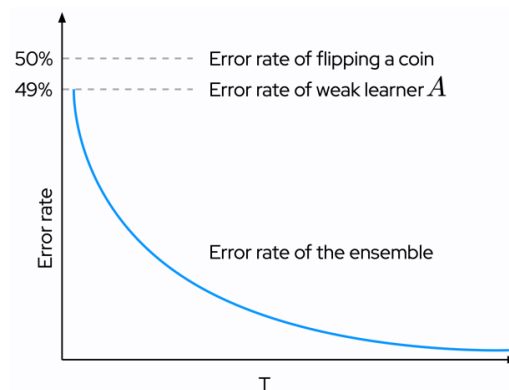


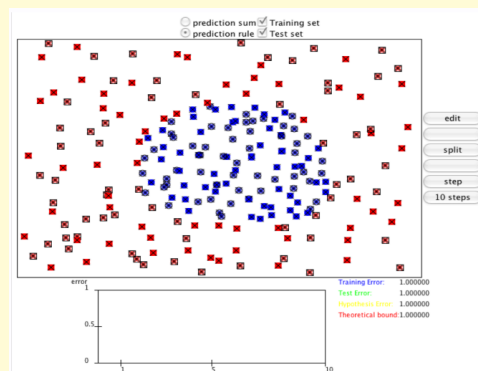
Figura 156: Esempio di Boosting

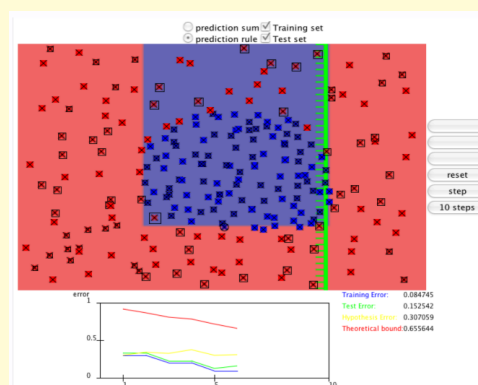
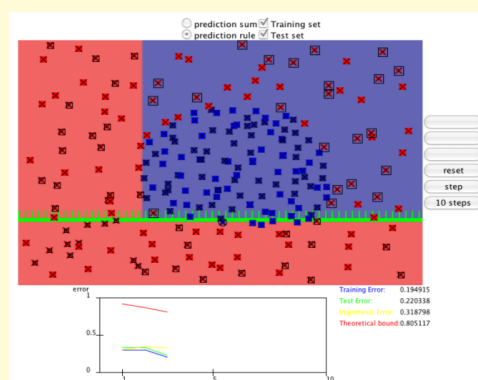
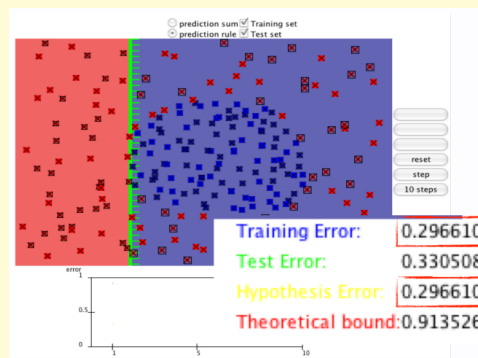
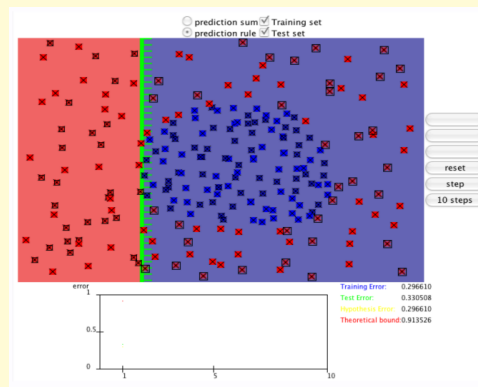
Idea Assegna un peso ad ogni esempio in D

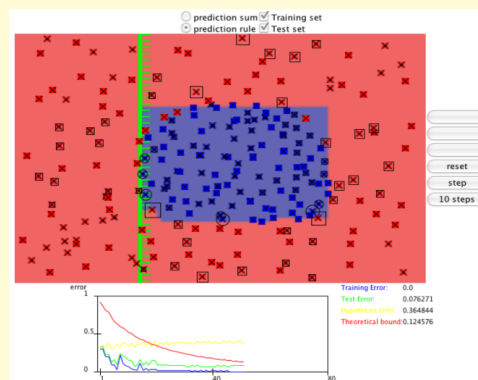
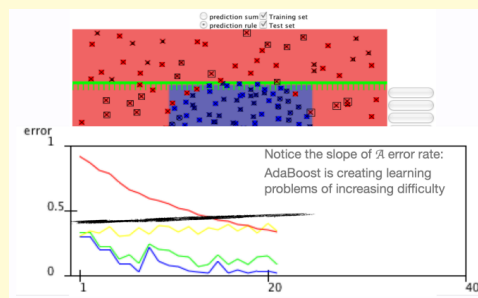
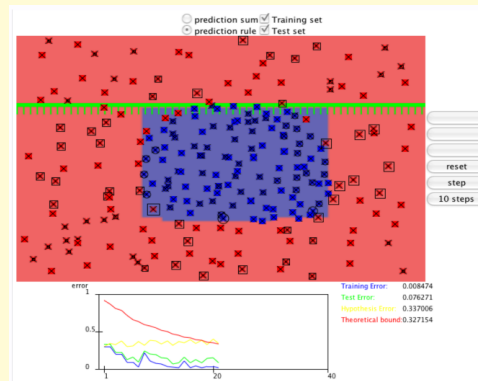
- inizialmente, tutti gli esempi hanno lo stesso peso
- man mano che l'incremento procede, i pesi vengono adeguati in base ai risultati della previsione:
 - i pesi degli esempi correttamente previsti vengono abbassati
 - vengono sollevati i pesi degli esempi mal previsti

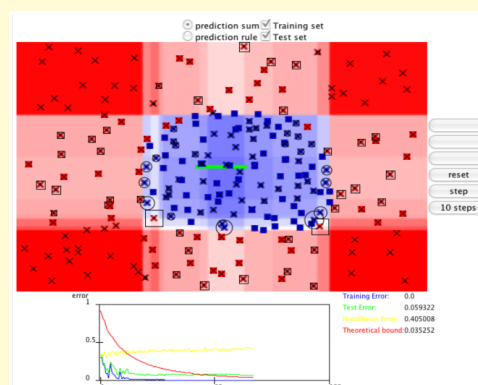
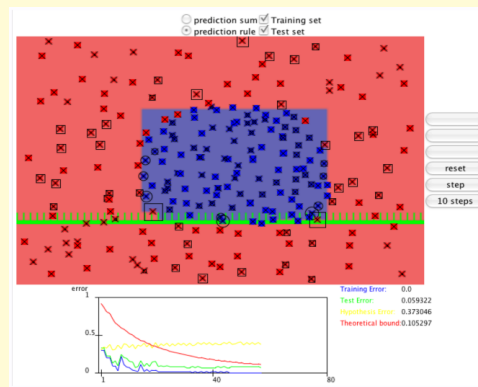
Risultati Man mano che l'apprendimento procede, il learner è costretto a concentrarsi su porzioni dello spazio di esempio che non sono state adeguatamente affrontate nelle iterazioni precedenti.

Esempio









Esperimento: Boosting vs CART

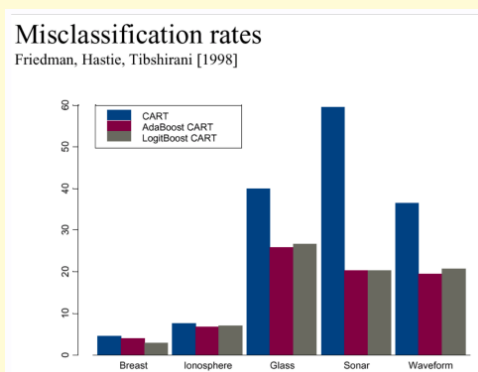


Figura 157: Boosting vs CART in un problema di classificazione

Algorithm 4 AdaBoost pseudo-code

```
1:  $\mathbf{w}^1 \leftarrow \left[ \frac{1}{|D|}, \frac{1}{|D|}, \dots, \frac{1}{|D|} \right]$ 
2: for  $t \in \{1, \dots, T\}$  do
3:    $m_t \leftarrow A(D, \mathbf{w}^t)$ 
4:    $\epsilon_t \leftarrow \sum_{i=1}^{|D|} w_i^t \cdot \mathbf{1}_{y_i \neq m_t(x_i)}$ 
5:    $\alpha_t \leftarrow \frac{1}{2} \ln \left( \frac{1-\epsilon_t}{\epsilon_t} \right)$ 
6:   for  $i \in \{1, \dots, |D|\}$  do
7:      $w_i^{t+1} \leftarrow w_i^t \cdot \exp(-\alpha_t \cdot y_i \cdot m_t(x_i))$ 
8:   end for
9:    $\mathbf{w}^{t+1} \leftarrow \text{normalize}(\mathbf{w}^{t+1})$ 
10: end for
11: return  $M(x) = \text{sign} \left( \sum_{t=1}^T \alpha_t \cdot m_t(x) \right)$ 
```

8.3.3 Derivazione dell'algoritmo AdaBoost

L'algoritmo AdaBoost può essere derivato come una **minimizzazione greedy** della **perdita esponenziale** subita dall'ensemble. A ogni passo, AdaBoost seleziona in modo greedy l'ipotesi debole e il peso α_t che minimizza la perdita esponenziale di M_t .

Supponiamo di aver già costruito parte dell'ensemble:

$$M_{T-1}(x) = \sum_{t=1}^{T-1} \alpha_t \cdot m_t(x)$$

e concentriamoci su come scegliere in modo ottimale α_T e m_T sotto la perdita esponenziale.

Nota

La derivazione che segue utilizza pesi non normalizzati per semplicità. Il fattore di normalizzazione $Z_t = \sum_i w_i^t$ è omissso nelle equazioni, poiché è una costante a ogni passo che si annulla quando si risolve per l'ottimale α_t .

La perdita esponenziale di:

$$M_T(x) = M_{T-1}(x) + \alpha_T m_T(x)$$

è:

$$\begin{aligned} E &= \sum_i \exp(-y_i M_T(x_i)) \\ &= \sum_i \exp(-y_i (M_{T-1}(x_i) + \alpha_T m_T(x_i))) \\ &= \sum_i \exp(-y_i M_{T-1}(x_i)) \cdot \exp(-y_i \alpha_T m_T(x_i)) \\ &= \sum_i w_i^T \cdot \exp(-y_i \alpha_T m_T(x_i)) \\ &= \sum_{\{i|y_i=m_T(x_i)\}} w_i^T \cdot e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T \cdot e^{\alpha_T} \end{aligned}$$

Nota

w_i^T è il peso dell'esempio i al passo T :

$$w_i^T = \begin{cases} \frac{1}{|D|}, & \text{se } T = 1 \\ \exp(-y_i M_{T-1}(x_i)), & \text{altrimenti} \end{cases}$$

Ricordando che $M_T(x) = M_{T-1}(x) + \alpha_T m_T(x)$, possiamo riscrivere il peso w_i^T come:

$$\begin{aligned} w_i^T &= e^{-y_i M_{T-1}(x_i)} \\ &= e^{-y_i (M_{T-2}(x_i) + \alpha_{T-1} m_{T-1}(x_i))} \\ &= e^{-y_i M_{T-2}(x_i)} \cdot e^{-y_i \alpha_{T-1} m_{T-1}(x_i)} \\ &= w_i^{T-1} \cdot e^{-\alpha_{T-1} y_i m_{T-1}(x_i)} \end{aligned}$$

che è la formula utilizzata nell'algoritmo per aggiornare i pesi.

$$\begin{aligned} E &= \sum_{\{i|y_i=m_T(x_i)\}} w_i^T e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{\alpha_T} \\ &= \sum_{\{i|y_i=m_T(x_i)\}} w_i^T e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{-\alpha_T} - \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{-\alpha_T} \\ &= \sum_{\{i|y_i=m_T(x_i)\}} w_i^T e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{\alpha_T} - \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{-\alpha_T} \\ &= \sum_i w_i^T e^{-\alpha_T} + \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T (e^{\alpha_T} - e^{-\alpha_T}) \end{aligned}$$

Quindi, per minimizzare l'errore w.r.t. m_T è sufficiente minimizzare

$$\sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T (e^{\alpha_T} - e^{-\alpha_T})$$

Prima osservazione: dovremmo scegliere m_T in questo modo per ridurre al minimo il numero di classificazioni errate.

Seconda osservazione: Per scegliere il peso ottimale α_t , possiamo trovare α_t che annulla la derivata di E .

$$\begin{aligned} \frac{\partial E}{\partial \alpha_T} &= \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T e^{\alpha_T} - \sum_{\{i|y_i = m_T(x_i)\}} w_i^T e^{-\alpha_T} \\ e^{\alpha_T} \sum_{\{i|y_i \neq m_T(x_i)\}} w_i^T - e^{-\alpha_T} \sum_{\{i|y_i = m_T(x_i)\}} w_i^T &= 0 \\ \Downarrow \\ e^{\alpha_T} \sum_i w_i^T \mathbb{I}(y_i \neq m_T(x_i)) - e^{-\alpha_T} \sum_i w_i^T \mathbb{I}(y_i = m_T(x_i)) &= 0 \\ \Downarrow \\ e^{\alpha_T} \epsilon_T - e^{-\alpha_T} (1 - \epsilon_T) &= 0 \\ \Downarrow \\ \alpha_T &= \frac{1}{2} \ln \left(\frac{1 - \epsilon_T}{\epsilon_T} \right) \end{aligned}$$

8.3.4 Imparare da set di dati ponderati

Come soddisfare la necessità di avere un algoritmo di apprendimento che apprenda da un set di dati pesato?

- è molto facile modificare la maggior parte degli algoritmi di apprendimento per gestire istanze ponderate:
 - ID3: Facile modificare l'entropia, è sufficiente creare equazioni di guadagno di informazione che tengano conto dei pesi associati agli esempi, piuttosto che considerare solo il conteggio (che presuppone semplicemente che tutti i pesi siano pari a 1)

- Bayes ingenuo: idem
- k-NN: usa il peso per regolare gli esempi "voti"
- ...
- Ricampionamento del set di dati

8.3.5 Ricampionamento del set di dati

Ricampionamento: in alternativa alla modifica degli algoritmi di apprendimento per supportare set di dati ponderati, possiamo costruire un nuovo set di dati campionato in base alla distribuzione indotta dai pesi.

```

1  def resample(D: Dataset, w: List[float]) -> Dataset:
2      Dt = []
3      for i in range(len(D)):
4          v = random.uniform(0, 1)
5          for j in range(len(D)):
6              v -= w[j]
7              if v <= 0:
8                  Dt.append(D[j])
9                  break
10     return Dt

```

8.3.6 Apprendimento da istanze ponderate

Quanti esempi sono "sufficienti"?

- Più alto è il numero, meglio D' approssima un set di dati che segue la distribuzione indotta da w .
- come regola generale: $|D'| = |D|$ di solito funziona abbastanza bene.
- Perché non utilizziamo sempre il ricampionamento invece della riponderazione?
 - Il ricampionamento può essere sempre applicato, ma purtroppo richiede più risorse e produce risultati meno accurati. Si dovrebbe utilizzare questa tecnica solo quando è troppo costoso (o impraticabile) ricorrere alla riponderazione.

8.4 Perché l'ensemble learning funziona?

- Decomposizione bias-varianza
- Benefici dell'ensemble learning
 - Statistico
 - Rappresentativa
 - Computazionale

8.4.1 Scomposizione Bias/Varianza

Sia $E[\epsilon(\mathbf{x})]$ l'errore medio del modello indotto da un algoritmo di apprendimento A su un esempio $\mathbf{x}$ (la media è calcolata ripetendo l'esecuzione dell'algoritmo su diversi set di dati).

Come abbiamo visto:

$$E[\epsilon(\mathbf{x})] = \text{Bias}(\mathbf{x}) + \text{Var}(\mathbf{x}) + \text{Noise}(\mathbf{x})$$

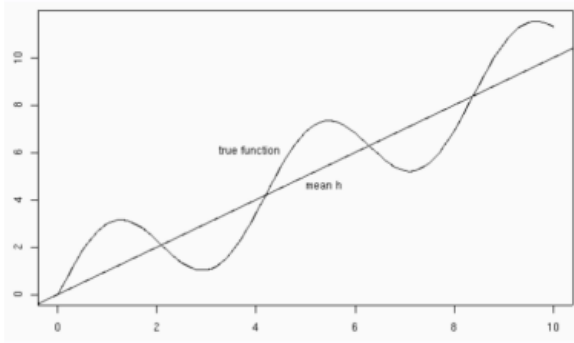


Figura 158: Esempio di Bias

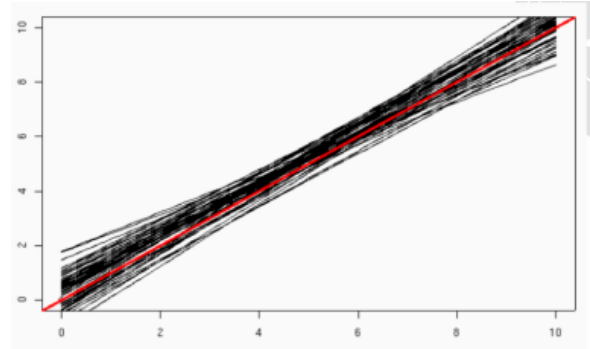


Figura 159: Esempio di Varianza

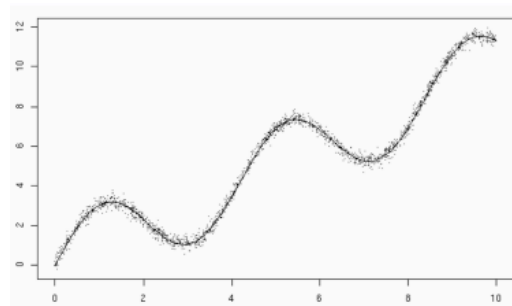


Figura 160: Esempio di Noise

8.4.2 Bagging è una macchina per la riduzione della varianza

Tutti i metodi di ensemble riducono sia la componente di distorsione che quella di varianza dell'errore. Tuttavia, argomentazioni empiriche e teoriche dimostrano che il **Bagging agisce principalmente come una macchina per la riduzione della varianza**.

Il bagging è più efficace sui classificatori ad **alta varianza e basso bias** (ad esempio, alberi decisionali).

Può essere interessante riflettere su cosa fa il bagging in termini di distribuzione di probabilità della variabile casuale che conta il numero di classificazioni errate effettuate dall'insieme.

8.4.3 Bagging e distribuzione binomiale

In un problema di classificazione binaria, abbiamo un errore se più della metà dei membri dell'insieme sono sbagliati, ovvero possiamo calcolare la probabilità di errore come segue.

Supponiamo che la probabilità di un errore da parte di un apprendista debole sia $p < 0.5$ (ovvero, sia fissa e almeno migliore di una stima casuale) e sia X la variabile stocastica che conta il numero di errori nelle T prove, quindi $X \sim \text{Bin}(p, T)$ e la probabilità di errore dell'insieme è:

$$P\left(X > \left\lfloor \frac{T}{2} \right\rfloor\right) = \sum_{x=\left\lfloor \frac{T}{2} \right\rfloor+1}^T P(X = x)$$

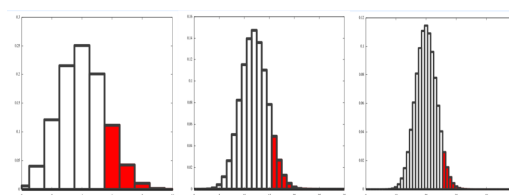


Figura 161: Esempio di Bagging e distribuzione binomiale

È semplice verificare che, secondo le ipotesi attuali, la probabilità che X sia maggiore di $\frac{T}{2}$ si avvicina a zero al crescere di T .

L'evidenza empirica dimostra che AdaBoost è molto efficace nel ridurre la componente di distorsione dell'errore, riducendo al contempo la componente di varianza.

8.4.4 Benefit di ensemble learning

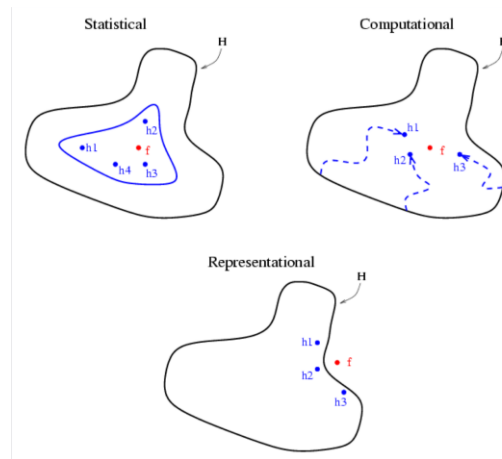


Figura 162: Benefici di Ensemble Learning

Statistico Data una quantità finita di dati, molte ipotesi sono in genere ugualmente valide. Come può l'algoritmo di apprendimento selezionarle?

Ricetta del **classificatore Bayesiano Ottimale**: prendere una maggioranza ponderata di tutte le ipotesi ponderate in base alla loro probabilità a posteriori. In altre parole, dare il massimo peso alle ipotesi coerenti con i dati.

Quindi, **l'ensemble learning può essere visto come un'approssimazione della regola di Bayesiano Ottimale** (che è probabilmente il miglior classificatore possibile).

Rappresentativa La funzione target desiderata potrebbe non essere implementabile con classificatori individuali, ma può essere approssimata mediante la media d'insieme.

Supponiamo di voler costruire un confine decisionale con alberi decisionali. I confini decisionali degli alberi decisionali sono iperpiani paralleli agli assi cartesiani.

Mediando un gran numero di tali "scale", il confine decisionale diagonale può essere approssimato con un'accuratezza arbitrariamente buona.

Computazionale Tutti gli algoritmi di apprendimento effettuano una sorta di ricerca in uno spazio di ipotesi per trovarne una che sia "sufficientemente buona" per i dati di training forniti.

Poiché gli spazi di ipotesi interessanti sono enormi/infiniti, la ricerca euristica è essenziale (ad esempio, ID3 esegue una ricerca greedy nello spazio dei possibili alberi decisionali).

Quindi l'apprendimento potrebbe rimanere bloccato in un minimo locale. Una strategia per evitare i minimi locali: **ripetere la ricerca più volte con riavvii casuali** → Bagging.

9 Clustering

9.1 Problema del Clustering

È ampiamente studiato e utilizzato nell'apprendimento non supervisionato.

L'obiettivo del Clustering è partizionare il dataset in diversi sottoinsiemi disgiunti (cluster).

Il clustering può essere utilizzato da solo per identificare la struttura intrinseca dei dati, ma può anche fungere da tecnica di pre-elaborazione per altre attività di apprendimento come la classificazione.

Si può vedere come un task di pre processamento dei dati.

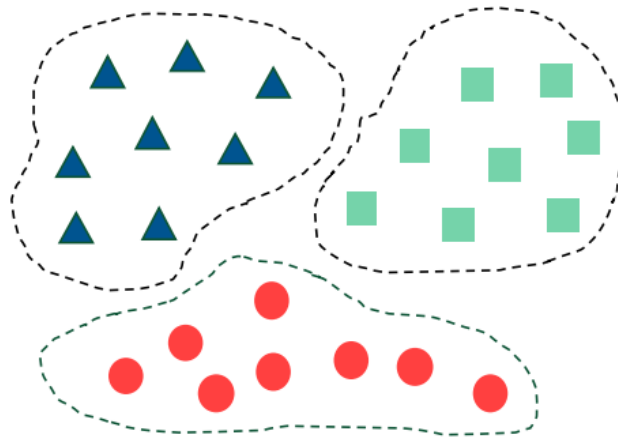


Figura 163: Esempio di clustering

9.1.1 Formalizzazione

Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, contenente m esempi non etichettati, dove ogni esempio $x_i = (x_{i1}; x_{i2}; \dots; x_{in})$ è un vettore n -dimensionale. Poi, un algoritmo di clustering partiziona il dataset D in k cluster disgiunti $\{C_l \mid l = 1, 2, \dots, k\}$, dove $C_{l'} \cap_{l' \neq l} C_l = \phi$ e $D = \bigcup_{l=1}^k C_l$.

Di conseguenza, denotiamo $\lambda_j \in \{1, 2, \dots, k\}$ come l'etichetta degli esempi x_j , (cioè, $x_j \in C_{\lambda_j}$). Quindi il risultato del clustering può essere rappresentato come un vettore di etichette di cluster $\lambda = (\lambda_1, +; \lambda_2; \dots; \lambda_m)$ con m elementi.

9.2 Misura delle prestazioni

Le misure di performance per il clustering sono anche chiamate **indici di validità**.

Intuitivamente, vogliamo che le cose di un certo tipo si uniscano; ovvero, **i campioni nello stesso cluster dovrebbero essere il più simili possibile**, mentre i campioni di cluster diversi dovrebbero essere il più diversi possibile. In altre parole, cerchiamo cluster con un'**elevata similarità e bassa distanza intracluster** e una **bassa similarità e alta distanza intercluster**.

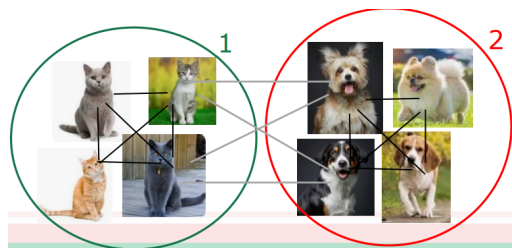


Figura 164: Esempio di clustering con esempi reali

9.2.1 Misura delle prestazioni per il clustering

External index Confronto del risultato del clustering con un modello di riferimento (etichette fornite dagli esperti).

Internal index Valutare il risultato del clustering senza utilizzare alcun modello di riferimento (ad esempio, varianza, coesione, ecc.)

9.2.2 Misura della performance - Indice esterno

Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, supponiamo che un algoritmo di clustering produca i cluster $C = \{C_1, C_2, \dots, C_k\}$, e un modello di riferimento fornisce i cluster $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$. Di conseguenza, indichiamo con λ e λ^* i vettori delle etichette di clustering di C e C^* , rispettivamente.

Per ogni coppia di campioni possiamo definire i seguenti quattro termini:

$$\begin{aligned} a &= |SS|, SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\} \\ b &= |SD|, SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\} \\ c &= |DS|, DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\} \\ d &= |DD|, DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\} \end{aligned} \quad (1)$$

Dove D = diverso cluster stimato, S = stesso cluster stimato.

Esempio

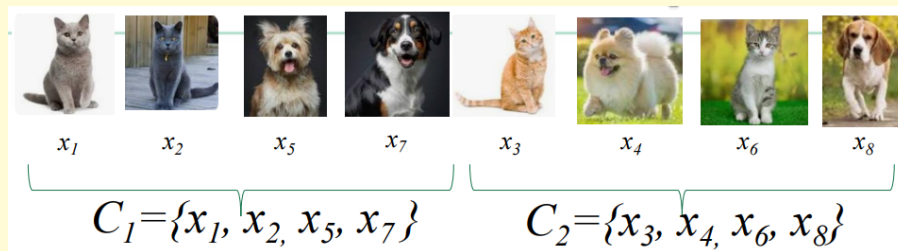


Figura 165: Esempio di indice esterno

$$C^* = \{C_1^*, C_2^*\} \quad C_1^* = \{x_1, x_2, x_3, x_6\} \quad C_2^* = \{x_4, x_5, x_7, x_8\}$$

$$\lambda = \{1, 1, 2, 2, 1, 2, 1, 2\} \quad \lambda^* = \{1, 1, 1, 2, 2, 1, 2, 2\}$$

$$a = |SS|, SS = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$(x_1, x_2), (x_4, x_8), (x_3, x_6), (x_5, x_7)$$

G G C C G G C C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4		
True D			
Total col			28
$\frac{m \cdot m - 1}{2}$			

Esempio

$$b = |SD|, SD = \{(x_i, x_j) \mid \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$(x_1, x_5), (x_1, x_7), (x_2, x_5), (x_2, x_7),$
 G C G C G C G C
 $(x_3, x_4), (x_3, x_8), (x_4, x_6), (x_6, x_8)$
 G C G C G C G C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4		
True D	8		
Total col	12	16	28

$$c = |DS|, DS = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$(x_1, x_3), (x_1, x_6), (x_2, x_3), (x_2, x_6),$
 G G G G G G G G
 $(x_4, x_5), (x_4, x_7), (x_5, x_8), (x_7, x_8)$
 C C C C C C C C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8		16
Total col	12	16	28

$$d = |DD|, DD = \{(x_i, x_j) \mid \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$(x_1, x_4), (x_1, x_8), (x_2, x_4), (x_2, x_8),$
 C G G C G C G C
 $(x_3, x_5), (x_3, x_7), (x_5, x_6), (x_6, x_7)$
 G C G C C G G C

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8	8	16
Total col	12	16	28

Indicatori con valori compresi tra 0 e 1, più alto è il valore, migliore è la qualità del clustering:

- **Jaccard Coefficient, JC:**

$$JC = \frac{a}{a + b + c} = \frac{Set_1 \cap Set_2}{Set_1 \cup Set_2}$$

- **Fowlkes and Mallows Index, FMI:**

$$(\text{geometric mean}) FMI = \sqrt{\frac{a}{a + b} \cdot \frac{a}{a + c}}$$

- **Rand Index, RI:**

$$RI = \frac{2(a + d)}{m(m - 1)}$$

Esempio

- **Jaccard Coefficient, JC:**

$$JC = \frac{a}{a + b + c} = \frac{4}{20} = 0.2$$

- **Fowlkes and Mallows Index, FMI:**

$$(\text{geometric mean}) FMI = \sqrt{\frac{a}{a + b} \cdot \frac{a}{a + c}} = \sqrt{\frac{4}{12} \cdot \frac{4}{12}} = \frac{4}{12} = 0.33$$

- **Rand Index, RI:**

$$RI = \frac{2(a + d)}{m(m - 1)} = \frac{12}{28} = 0.428$$

Pairs	$\wedge$ S	$\wedge$ D	Total row
True S	4	8	12
True D	8	8	16
Total col	12	16	28

9.2.3 Misura della performance - Indice interno

Cluster Cohesion: Misura quanto strettamente correlati sono gli oggetti in un cluster

- Esempio: Sum Squared Error (SSE)
- Cohesion è misurato da **Within cluster Sum of Squares (WSS):**

$$WSS = \sum_i \sum_{x \in C_i} (x - m_i)^2$$

Più piccolo è, meglio è.

Cluster Separation: Misura quanto un cluster è distinto o ben separato dagli altri cluster

- Separation è misurata da **Between cluster Sum of Squares (BSS):**

$$BSS = \sum_i |C_i| (m - m_i)^2$$

- dove $|C_i|$ è la dimensione del cluster i , m_i e m_j sono le medie dei cluster i e la media globale, rispettivamente.

Più grande è, meglio è.

Esempio - SSE

$$BSS + WSS = constant$$

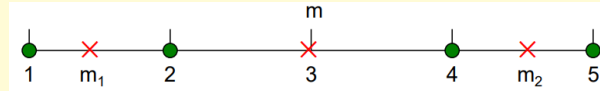


Figura 166: Esempio per il calcolo di WSS e BSS

K=1 cluster:

- $WSS = (1 - 3)^2 + (2 - 3)^2 + (4 - 3)^2 + (5 - 3)^2 = 10$
- $BSS = 4 \times (3 - 3)^2 = 0$
- $Total = 10 + 0 = 10$

K=2 clusters:

- $WSS = (1 - 1.5)^2 + (2 - 1.5)^2 + (4 - 4.5)^2 + (5 - 4.5)^2 = 1$
- $BSS = 2 \times (3 - 1.5)^2 + 2 \times (4.5 - 3)^2 = 9$
- $Total = 1 + 9 = 10$

R^2 : un modo per combinarli

$$R^2 = 1 - \frac{WSS}{SSE}$$

Più grande è, meglio è.

L'indice R-quadrato misura la **proporzione della varianza totale (SSE) nei dati che è spiegata dalla soluzione di clustering (WSS)**, dove la varianza totale (SSE) è la varianza dell'intero set di dati, e la varianza all'interno dei cluster (WSS) è la somma delle varianze di ciascun cluster.

Dati i cluster generati $C = \{C_1, C_2, \dots, C_k\}$, definiamo i seguenti quattro termini:

- La distanza media tra i campioni nel cluster C :

$$avg(C) = \frac{2}{|C|(|C| - 1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$



Figura 167: Distanza media tra i campioni nel cluster C

- La distanza più grande tra i campioni nel cluster C :

$$diam(C) = \max_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$



Figura 168: Distanza più grande tra i campioni nel cluster C

- La distanza tra due campioni più vicini nei cluster C_i e C_j :

$$d_{min}(C_i, C_j) = \min_{x_i \in C_i, x_j \in C_j} dist(x_i, x_j)$$



Figura 169: Distanza tra due campioni più vicini nei cluster C_i e C_j

- La distanza tra i centroidi dei cluster C_i e C_j :

$$d_{cen}(C_i, C_j) = dist(\mu_i, \mu_j)$$



Figura 170: Distanza tra i centroidi dei cluster C_i e C_j

- **Davies-Bouldin Index, DBI:**

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{cen}(C_i, C_j)} \right)$$



Figura 171: Davies-Bouldin Index

Più piccolo è, meglio è.

- **Dunn Index, DI:**

$$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{min}(C_i, C_j)}{\max_{1 \leq l \leq k} diam(C_l)} \right) \right\}$$

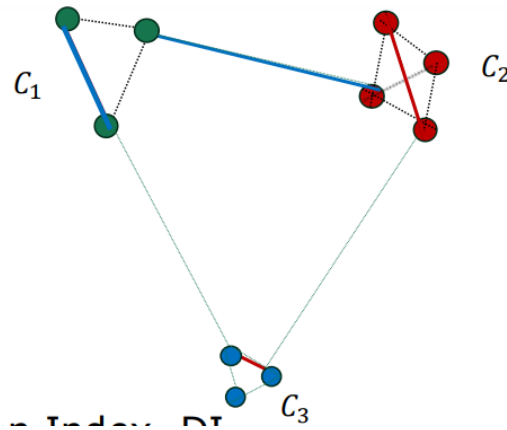


Figura 172: Dunn Index

Più grande è, meglio è.

- **Hausdorff distance:**

$$Dist_h(X, Z) = \max\{dist_h(X, Z), dist_h(Z, X)\} \quad \text{dove} \quad dist_h(X, Z) = \max_{x \in X} \min_{z \in Z} \|x - z\|_2$$

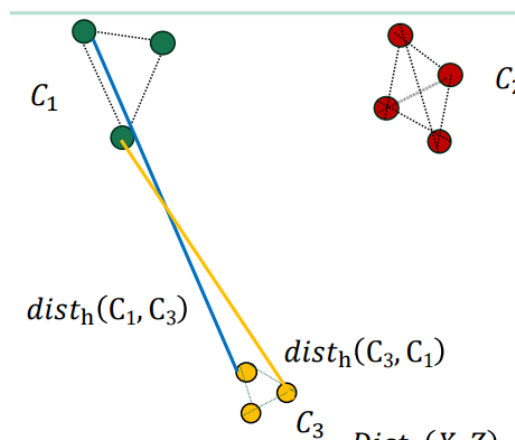


Figura 173: Hausdorff distance

Più grande è, meglio è.

Un'equivalenza utile

$$avg(C) = \frac{2}{|C|(|C| - 1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

Consideriamo ogni coppia di oggetti solo una volta.

$$Dis(C) = \frac{1}{|C|^2} \sum_{x_i \in C} \sum_{x_j \in C} [x_i - x_j]^2 = 2 \cdot Var(C)$$

Consideriamo ogni coppia di oggetti due volte (con gli oggetti in ordine inverso) con:

$$Var(C) = \frac{1}{|C|} \sum_{x \in C} (x - \bar{x})^2 \quad \bar{x} = \frac{1}{|C|} \sum_{x \in C} x$$

È stato dimostrato che $\bar{x}$ è la posizione che minimizza la somma del quadrato della distanza euclidea con i punti di un cluster.

9.3 Calcolo della distanza

Gli assiomi della misura della distanza:

- Non negatività: $dist(x_i, x_j) \geq 0$
- Identità degli indiscernibili: $dist(x_i, x_j) = 0 \Leftrightarrow x_i = x_j$
- Simmetria: $dist(x_i, x_j) = dist(x_j, x_i)$
- Subadditività (Disuguaglianza triangolare): $dist(x_i, x_j) \leq dist(x_i, x_k) + dist(x_k, x_j)$

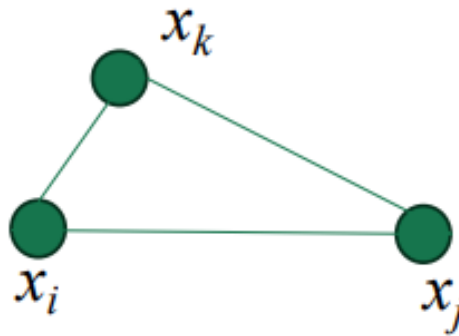


Figura 174: Calcolo della distanza tra due punti in uno spazio n-dimensionale

Una misurazione che viene comunemente utilizzata è **Minkowski distance**:

$$dist(x_i, x_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{1/p}$$

- Per $p = 1$, otteniamo la **distanza di Manhattan** (o distanza L1)
 - Per $p = 2$, otteniamo la **distanza euclidea** (o distanza L2)
- La distanza euclidea è una generalizzazione del teorema di Pitagora.

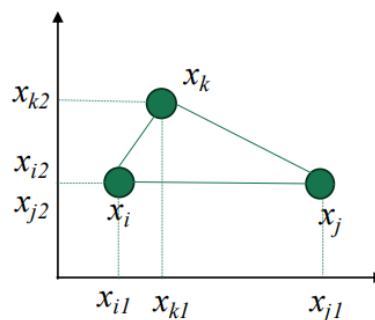


Figura 175: Teorema di Pitagora

9.3.1 Attributi

Attributo continuo avente domini infiniti (può essere manipolato da qualsiasi matematica)

Attributo ordinale (può essere ordinato o differenziato)

Attributo categoriale (possono essere enumerati o distinti) aventi domini finiti

Ad esempio, un attributo categoriale con dominio $\{1, 2, 3\}$ è più simile a un attributo ordinale o continuo poiché la distanza può essere calcolata con i valori dell'attributo, ovvero 1 è più vicino a 2 che a 3.

Attributo non ordinale (o nominale) Se il dominio di un attributo è discreto, come $\{aereo, treno, nave\}$, allora la distanza non può essere calcolata direttamente con i valori dell'attributo, e tali attributi sono attributi non ordinali.

9.3.2 Value Difference Metric, VDM

Sia $m_{u,a}$ il numero di campioni che assumono valore a sull'attributo u , e $m_{u,a,i}$ il numero di campioni all'interno dell' i -esimo cluster che assumono valore a sull'attributo u , e k è il numero di cluster. Quindi la distanza VDM tra due valori categoriali a e b dell'attributo u è:

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

9.3.3 Misura della distanza con tipi di attributi misti

MinkovDM_p

$$MinkovDM_p(\vec{x}_i, \vec{x}_j) = \left(\sum_{u=1}^{n_c} |x_{iu} - x_{ju}|^p + \sum_{u=n_c+1}^n VDM_p(x_{iu}, x_{ju}) \right)^{\frac{1}{p}}$$

$n_c \rightarrow$ numero di attributi continui.

$n \rightarrow$ numero totale di attributi categoriali.

Distanza pesata Quando attributi diversi hanno importanza diversa:

$$dist(\vec{x}_i, \vec{x}_j) = (\omega_1 \cdot |x_{i1} - x_{j1}|^p + \omega_2 \cdot |x_{i2} - x_{j2}|^p + \dots + \omega_n \cdot |x_{in} - x_{jn}|^p)^{\frac{1}{p}}$$

$$\omega_i \geq 0, \sum_{i=1}^n \omega_i = 1$$

9.4 Prototype Clustering

Prototype Clustering, noto anche come **clustering basato su prototipi**, presuppone che la struttura del clustering possa essere rappresentata con un **insieme di prototipi**.

Algoritmi In genere, tali algoritmi partono da alcuni prototipi iniziali, per poi aggiornarli e ottimizzarli iterativamente.

9.4.1 K-means Clustering

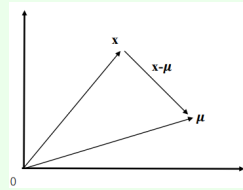
Dato un dataset $D = \{x_1, x_2, \dots, x_m\}$, l'obiettivo del K-means è quello di **minimizzare l'errore quadratico dei cluster** $C = \{C_1, C_2, \dots, C_k\}$:

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

dove μ_i è il vettore medio del cluster C_i .

Intuitivamente, E rappresenta la vicinanza tra il vettore medio di un cluster e i campioni all'interno di quel cluster, dove un valore E più piccolo indica una maggiore similarità intra-cluster.

The norm-2 of a vector



$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix} \quad \mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix} \quad \mathbf{X} - \mu = \begin{bmatrix} X_1 - \mu_1 \\ X_2 - \mu_2 \\ \vdots \\ X_n - \mu_n \end{bmatrix}$$

$$\|x - \mu\|_2 = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots + (x_n - \mu_n)^2}$$

$$\|x - \mu\|_2^2 = (x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots + (x_n - \mu_n)^2 = \sum_{i=1}^n (x_i - \mu_i)^2$$

$$\|x - \mu\|_2^2 = (X - \mu)^T (X - \mu) \quad (\text{in notazione vettoriale})$$

Algoritmo (ottimizzazione iterativa)

- **inizializza i vettori medi dei cluster ripeti**
 1. (aggiorna) i cluster
 2. calcola i nuovi vettori medi dei cluster
- **finché i cluster non cambiano**

Algorithm 5 k-means Clustering

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Number of clusters k .

- 1: **Process:**
- 2: Randomly select k samples as the initial mean vectors $\{\mu_1, \mu_2, \dots, \mu_k\}$.
- 3: **repeat**
- 4: $C_i = \emptyset$ ($1 \leq i \leq k$)
- 5: **for** $j = 1, 2, \dots, m$ **do**
- 6: Compute the distance between sample x_j and each mean vector μ_i ($1 \leq i \leq k$):
 $d_{ji} = \|x_j - \mu_i\|_2$
- 7: According to the nearest mean vector, decide the cluster label of x_j : $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$
- 8: Move x_j to the corresponding cluster: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$
- 9: **end for**
- 10: **for** $i = 1, 2, \dots, k$ **do**
- 11: Compute the updated mean vectors: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$
- 12: **if** $\mu'_i \neq \mu_i$ **then**
- 13: Update the current mean vector μ_i to μ'_i
- 14: **else**
- 15: Leave the current mean vector unchanged
- 16: **end if**
- 17: **end for**
- 18: **until** All mean vectors remain unchanged

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

Un esempio per l'algoritmo k-means

Prendiamo il set di dati 4.0 dell'anguria nella tabella seguente come esempio per dimostrare l'algoritmo k-means. Per semplicità di discussione, rappresentiamo il campione con l'ID i come x_i .

ID	density	sugar	ID	density	sugar	ID	density	sugar
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

Figura 176: Set di dati 4.0 dell'anguria

Supponendo di impostare $k = 3$, l'algoritmo seleziona causalmente tre campioni x_6, x_{12}, x_{24} come vettori medi iniziali dei cluster, cioè, $\mu_1 = (0.403; 0.237)$, $\mu_2 = (0.343; 0.099)$, $\mu_3 = (0.478; 0.437)$

Poi, per il campione $x_1 = (0.697; 0.460)$, la sua distanza dai tre vettori medi μ_1, μ_2, μ_3 è 0.369, 0.506, e 0.220, rispettivamente. Quindi, x_1 viene assegnato al cluster C_3 . Similmente, valutiamo tutti i campioni del dataset e troviamo i seguenti cluster:

$$C_1 = \{x_3, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_4, x_{15}, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

$$\mu'_1 = (0.493; 0.207), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.602; 0.396)$$

Risultati dell'algoritmo k-means

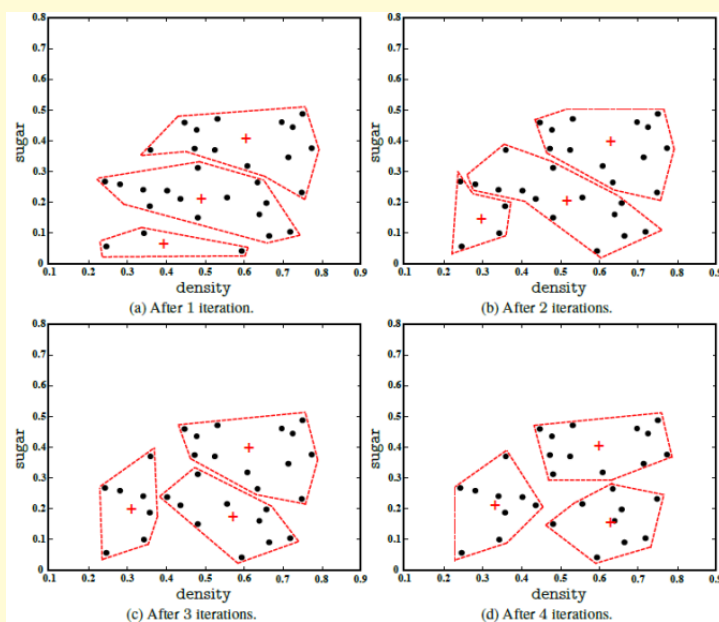


Figura 177: Risultati dell'algoritmo k-means sul set di dati 4.0 dell'anguria con $k = 3$. I campioni e i vettori delle medie sono rappresentati rispettivamente da "●" e "+". Le linee tratteggiate rosse indicano i confini dei cluster

9.4.2 Learning Vector Quantization (LVQ)

A differenza del tipico algoritmo di clustering, LVQ presuppone che i **campioni di dati siano etichettati** e che il **processo di clustering sia assistito da informazioni supervisionate**.

Dato un dataset $D = \{(x_1, y_2), (x_2, y_2), \dots, (x_m, y_m)\}$, LVQ mira a imparare un set n-dimensionale di vettori di prototipi $\{p_1, p_2, \dots, p_q\}$ dove ogni vettore di prototipi rappresenta un cluster.

La quantizzazione è il processo di riduzione della precisione di un segnale o di un insieme di valori mappando un insieme più ampio (continuo) di input su un insieme più piccolo e discreto di output. Questa tecnica è utilizzata in vari campi, come l'elaborazione digitale dei segnali e l'apprendimento automatico, per rendere i dati più efficienti riducendo i requisiti di memoria e di calcolo, anche se a costo di una certa perdita di accuratezza.

Algorithm 6 Learning Vector Quantization (LVQ)

Require: Training set $D = \{(x_1, y_2), (x_2, y_2), \dots, (x_m, y_m)\}$; Number of clusters q . Initial labels of prototype vectors $\{t_1, t_2, \dots, t_q\}$. Learning rate η .

- 1: **Process:**
- 2: Initialize a set of prototype vectors $\{p_1, p_2, \dots, p_q\}$;
- 3: **repeat**
- 4: Randomly pickup a sample (x_j, y_j) from the data set D ;
- 5: Compute the distance between x_j and $p_i (1 \leq i \leq q)$: $d_{ji} = \|x_j - p_i\|_2$;
- 6: Find the nearest prototype vector p_{i^*} for x_j : $i^* = \arg \min_{i \in \{1, 2, \dots, q\}} d_{ji}$;
- 7: **if** $y_j = t_{i^*}$ **then**
- 8: $p' = p_{i^*} + \eta \cdot (x_j - p_{i^*})$; // p is moved toward x_j (their distance is decreased by η)
- 9: **else**
- 10: $p' = p_{i^*} - \eta \cdot (x_j - p_{i^*})$; // p is moved away from x_j (their distance is increased by η)
- 11: **end if**
- 12: Update the prototype vector p_{i^*} to p' ;
- 13: **until** The termination condition is met

Ensure: Prototype vectors $\{p_1, p_2, \dots, p_q\}$.

$$\|p' - x_j\|_2 = \|p_{i^*} + \eta \cdot (x_j - p_{i^*}) - x_j\|_2 = (1 - \eta) \cdot \|p_{i^*} - x_j\|_2$$

Risultati del Clustering

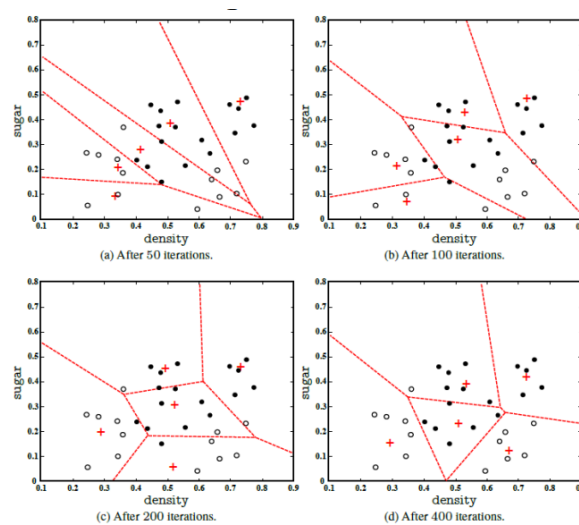


Figura 178: Risultati dell'algoritmo LVQ dopo diverse iterazioni sul set di dati 4.0 dell'anguria con $q = 5$. I simboli "•", "o", "+" rappresentano, rispettivamente, i campioni di classe c_1 , i campioni di classe c_2 e i vettori prototipo. Le linee tratteggiate rosse mostrano la tassellatura di Voronoi

Un vettore prototipo p_i definisce una regione R_i , dove la distanza da qualsiasi campione al suo interno a p_i non è maggiore della distanza da qualsiasi altro vettore prototipo. Le regioni formano una **tassellatura di Voronoi**.

Se tutti i campioni sono rappresentati dai vettori prototipo, allora abbiamo effettuato una **compressione con perdita di dati**: ecco perché si chiama **quantizzazione vettoriale**.

9.5 Density Clustering

Applicando il clustering k-means ai dati seguenti, risulta difficile per il clustering basato su prototipi trovare le informazioni sulla densità, il che porta a risultati ulteriori rispetto a quelli attesi:

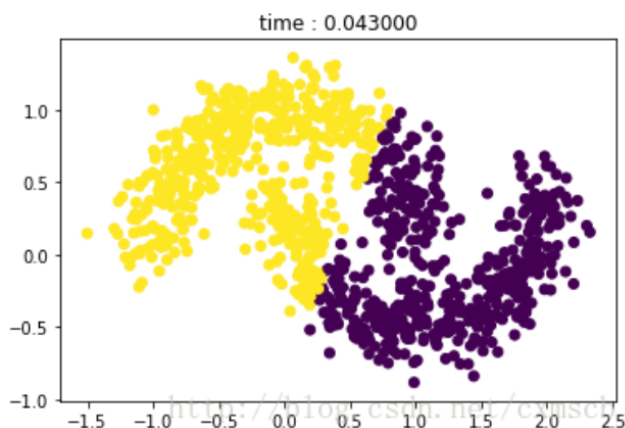


Figura 179: Esempio di clustering basato su densità, partenza

Clustering basato sulla densità: valutare la connettività tra i campioni dal punto di vista della densità ed espandere i cluster aggiungendo campioni collegabili.

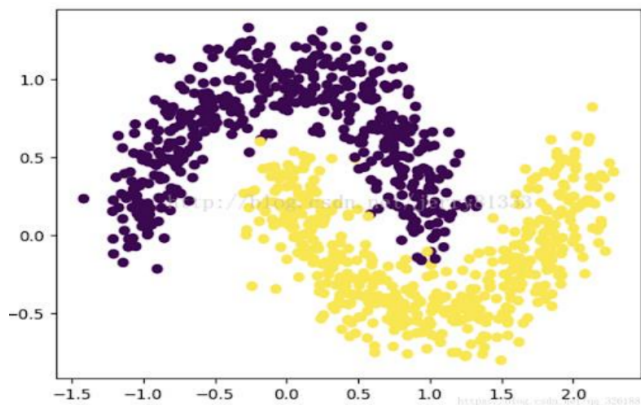


Figura 180: Esempio di clustering basato su densità, risultato

9.5.1 Definizione di clustering di densità

Il **clustering di densità** è anche noto come clustering basato sulla densità. Supponendo che la struttura del clustering possa essere determinata dalle densità delle distribuzioni campionarie.

In genere, gli algoritmi di clustering di densità **valutano la connettività tra i campioni dal punto di vista della densità**. Espandono i cluster aggiungendo campioni collegabili.

Presentiamo **DBSCAN**, un algoritmo rappresentativo di clustering di densità.

9.5.2 DBSCAN

L'algoritmo DBSCAN caratterizza la densità delle distribuzioni campionarie mediante una **coppia di parametri di vicinato** (ϵ , $MinPts$).

ϵ – **neighborhood**: per $x_j \in D$, il suo ϵ – *neighborhood* include tutti i campioni in D la cui distanza da x_j non è maggiore di ϵ ;

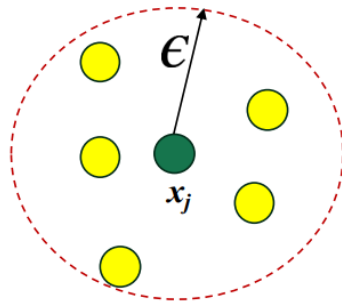


Figura 181: ϵ – *neighborhood* di un campione x_j

Core object: se l' ϵ – *neighborhood* di un campione x_j contiene almeno *MinPts* campioni, allora x_j è un core object;

Directly density-reachable: x_j è detto directly density-reachable da x_i se x_i è un core object e x_j è all'interno dell' ϵ – *neighborhood* di x_i ;

Density-reachable: x_j è detto density-reachable da x_i se esiste una catena di campioni $p_1, p_2, \dots, p_n$ dove $p_1 = x_i, p_n = x_j$, e p_{i+1} è directly density-reachable da p_i ;

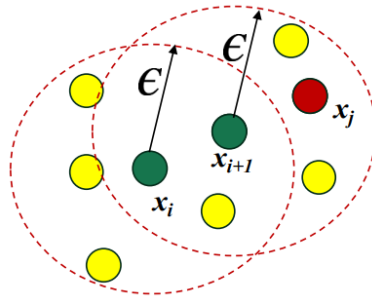


Figura 182: Esempio di density-reachable: x_j è density-reachable da x_i tramite la catena di campioni

x_j è density-reachable da x_i dalla media di x_{i+1}

Density-connected: x_i e x_j sono detti density-connected se esiste un campione x_k tale che entrambi sono density-reachable da x_k .

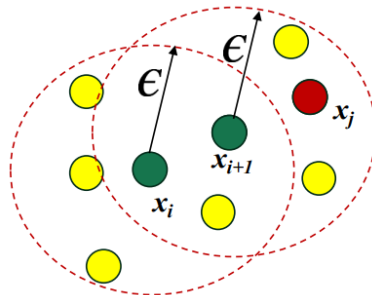


Figura 183: Esempio di density-connected: x_i e x_j sono density-connected tramite x_k

x_j e x_i sono density-connected perché esiste x_{i+1}

Esempio

Sia $MinPts = 3$.

I cerchi tratteggiati mostrano l' ϵ - neighborhood di ciascun campione.

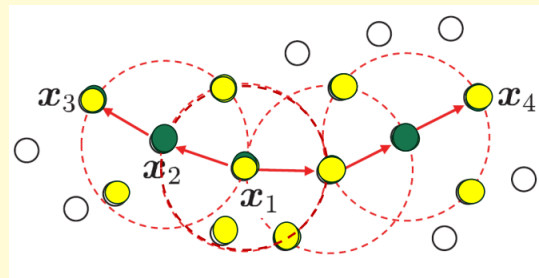


Figura 184: I concetti di base di DBSCAN ($MinPts = 3$): i cerchi tratteggiati mostrano il ϵ - neighborhood; x_1 è un oggetto centrale; x_2 è direttamente raggiungibile in base alla densità da x_1 ; x_3 è raggiungibile in base alla densità da x_1 ; x_3 e x_4 sono connessi in base alla densità

- x_1 è un core object
- x_2 è directly density-reachable da x_1
- x_3 è density-reachable da x_1 tramite x_2
- x_3 e x_4 sono density-connected

Definizione di un cluster Il più grande insieme di campioni connessi in base alla densità, derivati da relazioni raggiungibili in base alla densità.

Formalizzazione Dati i parametri di vicinato, un cluster è un sottoinsieme non vuoto con le seguenti proprietà:

- **Connectivity:** $x_i \in C, x_j \in C \Rightarrow x_i$ e x_j sono density-connected
- **Maximality:** $x_i \in C, x_j$ è density-reachable da $x_i \Rightarrow x_j \in C$

In realtà:

1. Se x è un core object e
2. sia $X = \{x' \in D \mid x' \text{ è density-reachable da } x\}$ denota l'insieme dei campioni density-reachable da x ,

allora si può dimostrare che X è un cluster che soddisfa sia la connettività che la massimalità.

Algorithm 7 DBSCAN

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Neighborhood parameters $(\epsilon, MinPts)$.

```
1: Process:
2: Initialize the set of core objects:  $\Omega = \emptyset$ ;
3: for  $j = 1, 2, \dots, m$  do
4:   Determine the  $\epsilon$  - neighborhood  $N_\epsilon(x_j)$  of sample  $x_j$ ;
5:   if  $|N_\epsilon(x_j)| \geq MinPts$  then
6:     Add  $x_j$  to the set of core objects:  $\Omega = \Omega \cup \{x_j\}$ ;
7:   end if
8: end for
9: Initialize the number of clusters:  $k = 0$ ;
10: Initialize the set of unprocessed samples:  $\Gamma = D$ ;
11: while  $\Gamma \neq \emptyset$  do
12:   Keep a copy of the current unprocessed data set:  $\Gamma_{old} = \Gamma$ ;
13:   Randomly select a core object  $o \in \Omega$ , and initialize the queue  $Q = \langle o \rangle$ ;
14:    $\Gamma = \Gamma \setminus \{o\}$ ;
15:   while  $Q \neq \emptyset$  do
16:     Dequeue the first sample  $q$  from  $Q$ ;
17:     if  $|N_\epsilon(q)| \geq MinPts$  then
18:       Letting  $\Delta = N_\epsilon(q) \cap \Gamma$ ;
19:       Enqueue the samples in  $\Delta$  into  $Q$ ;
20:        $\Gamma = \Gamma \setminus \Delta$ ; //  $\Gamma$  contains examples not in the next cluster
21:     end if
22:   end while
23:    $k = k + 1$  generate cluster  $C_k = \Gamma_{old} \setminus \Gamma$ ; //  $\Gamma_{old}$  by a double subtraction satisfies the maximality
24:    $\Omega = \Omega \setminus C_k$ ;
25: end while
```

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

Risultati del Clustering

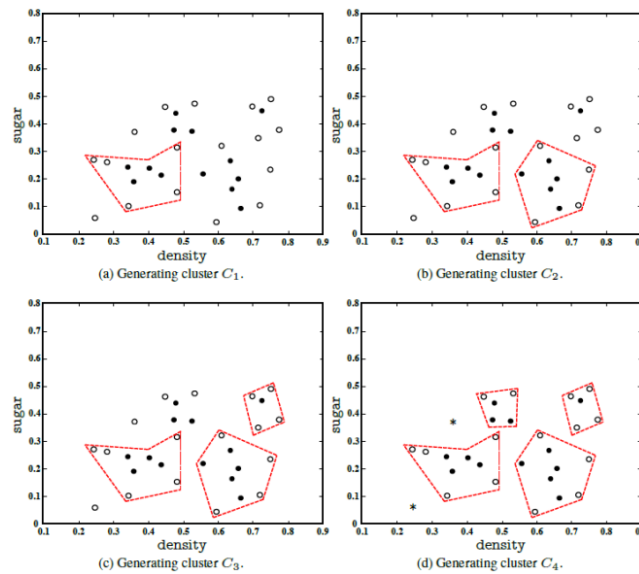


Figura 185: Risultati dell'algoritmo DBSCAN con $\epsilon = 0,11$ e $MinPts = 5$. I simboli "•", "○", "*" rappresentano rispettivamente gli oggetti core, gli oggetti non core e i campioni rumorosi. Le linee tratteggiate rosse mostrano i cluster.

9.5.3 Come calcolare i parametri DBSCAN

I cluster sono aree dello spazio delle istanze con la stessa elevata densità.

La densità di esempi nell'intorno di ciascun esempio viene verificata calcolando la distanza con il suo k-esimo vicino più prossimo. Chiamiamola "distanza di raggiungibilità". Impostiamo k uguale a un valore costante che viene impostato come parametro MinPts.

- Se il punto è un punto centrale, la densità nel suo intorno è alta (il punto è circondato da altri punti vicini) e la distanza di raggiungibilità è bassa
- Se il punto è un punto di rumore, è un outlier (il punto è isolato) e la sua raggiungibilità è elevata.

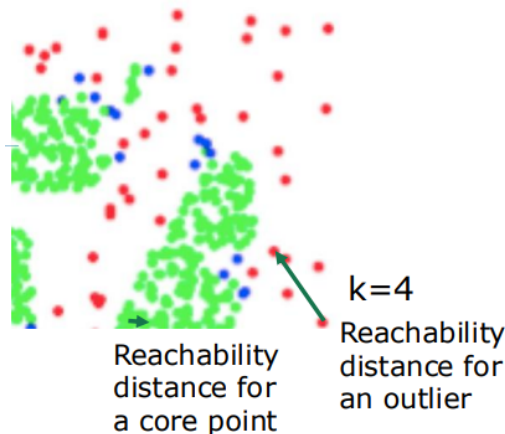


Figura 186: Esempio di parametri DBSCAN calcolati con il metodo della distanza di raggiungibilità

La distanza di raggiungibilità viene calcolata per ogni punto

Il set di dati è ordinato in ordine crescente per distanza di raggiungibilità

Nel grafico seguente ogni punto è rappresentato con il suo ordine in ascissa e la sua distanza di raggiungibilità sull'asse y.

Quando il grafico mostra un "ginocchio", questo è il valore corretto della distanza a cui dovremmo impostare il valore del parametro Eps e distinguere tra punti centrali e punti di rumore (e di confine).

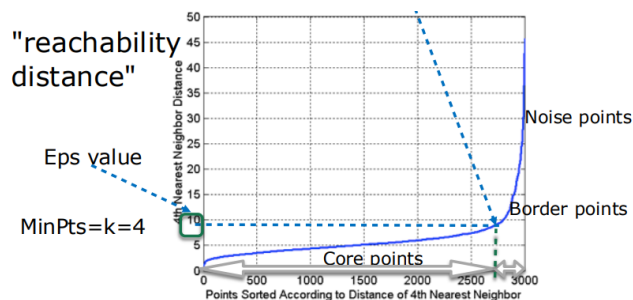


Figura 187: Esempio di selezione del parametro Eps in DBSCAN

9.6 Hierarchical Clustering

Il clustering gerarchico mira a creare una **struttura di clustering ad albero** dividendo un set di dati in diversi livelli. La gerarchia dei cluster può essere formata adottando una strategia bottom-up o top-down.

9.6.1 Punti di forza del clustering gerarchico

Non è necessario presupporre un numero specifico di cluster. È possibile ottenere qualsiasi numero desiderato di cluster "tagliando" il dendrogramma al livello appropriato.

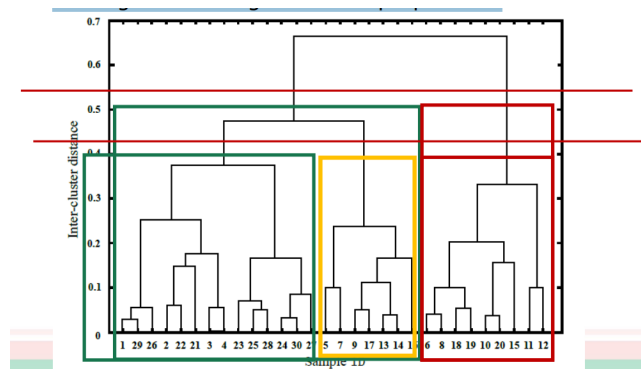


Figura 188: Esempio di dendrogramma

Possono corrispondere a tassonomie significative.

9.6.2 AGNES

Algoritmo AGNES (clustering gerarchico **bottom-up**)

- inizia considerando ogni campione nel set di dati come un cluster iniziale.
- Quindi, a ogni round, due cluster più vicini vengono uniti per formare un nuovo cluster.
- Questo processo si ripete finché il numero di cluster non raggiunge il valore predefinito.

Definiamo le distanze di cluster dati C_i e C_j in forme diverse.

Distanza minima:

$$d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$$

Distanza massima:

$$d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$$

Distanza media:

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i| \cdot |C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$

Dendrogramma

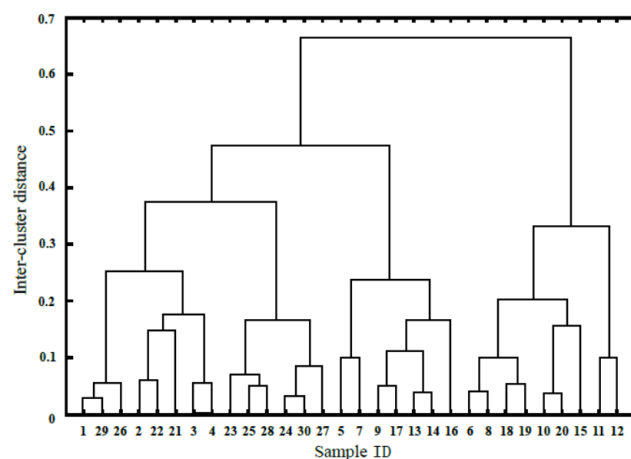


Figura 189: Il dendrogramma (utilizzando d_{max}) generato dall'algoritmo AGNES sul set di dati 4.0 dell'anguria. L'asse x mostra l'ID dei campioni e l'asse y mostra le distanze tra i cluster

Algorithm 8 AGNES

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Cluster distance metric function d ; Number of clusters k .

```
1: Process:
2: for  $j = 1, 2, \dots, m$  do
3:    $C_j = \{x_j\}$ ; //Starts with singleton clusters
4: end for
5: for  $i = 1, 2, \dots, m$  do
6:   for  $j = i + 1, \dots, m$  do
7:      $M(i, j) = d(C_i, C_j)$  //Matrix of point to point distances (symmetric)
8:      $M(j, i) = M(i, j)$ 
9:   end for
10: end for
11: Set the current number of clusters:  $q = m$ ; //Actual number of tree levels
12: while  $q > k$  do //Until the number of levels is reached
13:   Find two clusters  $C_{i^*}$  e  $C_{j^*}$  that have shortest distance;
14:   Merge  $C_{i^*}$  e  $C_{j^*}$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
15:   for  $j = j^* + 1, j^* + 2, \dots, q$  do
16:     Change the index of  $C_j$  to  $C_{j-1}$ ; //Shifts clusters indexes so that cluster  $j$  will be forgotten
17:   end for
18:   Delete the  $j^*$ th row and  $j^*$ th column of the distance matrix  $M$ ;
19:   for  $j = 1, 2, \dots, q - 1$  do
20:      $M(i^*, j) = d(C_{i^*}, C_j)$  //Updates the distance matrix considering the presence of the new
    cluster  $i^*$  and all other clusters  $j$ 
21:      $M(j, i^*) = M(i^*, j)$ 
22:   end for
23:    $q = q - 1$ ;
24: end while
```

Ensure: Clusters $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

Esempio: aggiornamento della matrice delle distanze

Quando due cluster più vicini (C_2 e C_5) vengono uniti, dobbiamo aggiornare la matrice di prossimità:

- Mantenere solo una riga/colonna ($_2$)
- Eliminare l'altra (C_5)
- Aggiornare le distanze tra gli altri cluster e quello nuovo (C_2^*)

	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix

Figura 190: Matrice di prossimità

Esempio: aggiornamento della matrice delle distanze

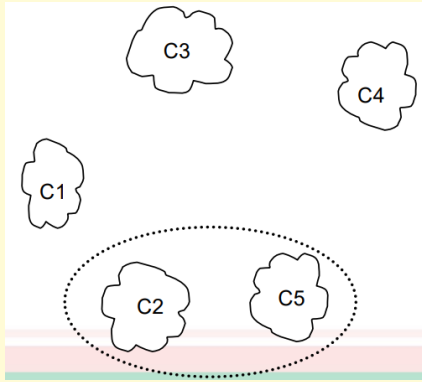


Figura 191: Aggiornamento della matrice delle distanze dopo la fusione di C_2 e C_5

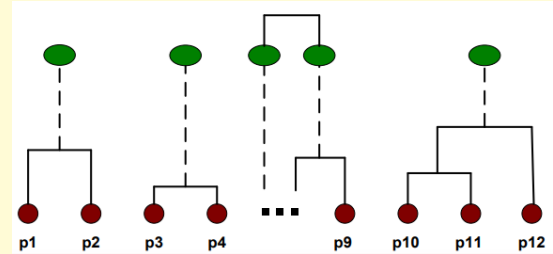


Figura 192: Rappresentazione della matrice delle distanze aggiornata

9.6.3 Risultati del Clustering

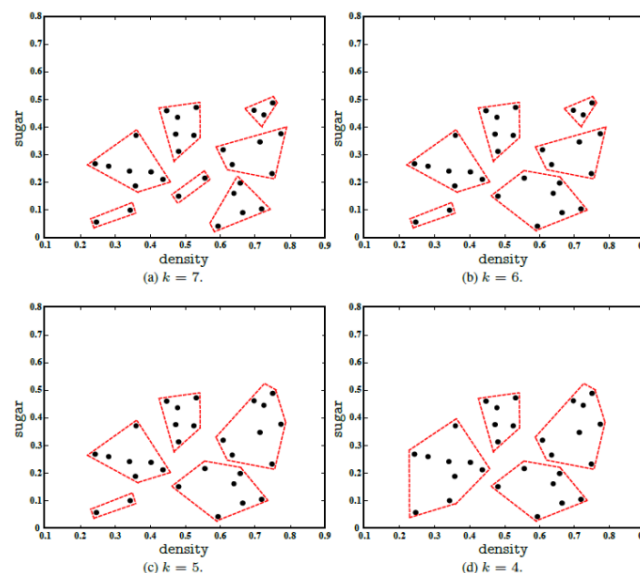


Figura 193: Risultati dell'algoritmo AGNES (utilizzando $d_{\max}$) sul set di dati 4.0 dell'anguria con diverso numero di cluster ($k = 7, 6, 5, 4$). Il simbolo "•" rappresenta i campioni e le linee tratteggiate rosse mostrano i cluster.

9.7 Probabilistic Clustering

9.7.1 Prototype Clustering - Mixture of-Gaussian Clustering

A differenza di k-means e LVQ, il clustering misto di gaussiane **non utilizza vettori prototipo**, ma **modelli probabilistici** per rappresentare le strutture di clustering.

Definizione di distribuzione gaussiana multivariata Per un vettore casuale x in uno spazio campione n -dimensionale χ ,

$$p(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

dove μ è il vettore n -dimensionale della media e Σ è la matrice di covarianza $n \times n$. Scriviamo la funzione di densità di probabilità come $p(x | \mu, \Sigma)$.

9.7.2 Calcolo della matrice di covarianza

Matrice di dati X (m oggetti, n features)

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix}$$

Matrice di covarianza $\Sigma(n \times n)$ L'elemento σ_{lj} è la covarianza tra le features l e j su tutti gli oggetti: misura quanto fortemente le features l e j variano insieme negli m oggetti.

Se $l = j$ è la varianza della feature l .

$$\Sigma = \begin{bmatrix} \sigma_{11} & \cdots & \sigma_{1n} \\ \vdots & \ddots & \vdots \\ \sigma_{1n} & \cdots & \sigma_{nn} \end{bmatrix} \quad \sigma_{lj} = \frac{1}{m} \sum_{k=1}^m (x_{kl} - \bar{x}_l)(x_{kj} - \bar{x}_j)$$

9.7.3 Mostrare la correlazione tra due features

Feature positivamente correlate (valori dei punti nelle linee rosse e verdi)

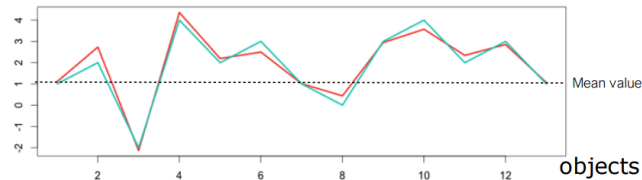


Figura 194: Feature positivamente correlate

Scatter plots:

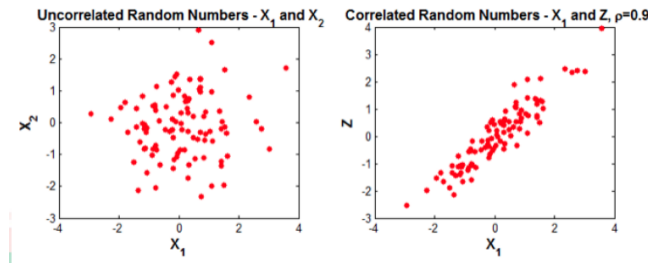


Figura 195: Scatter plots di due feature correlate positivamente

$$\rho = \frac{\sigma_{12}}{\sqrt{\sigma_{11} \cdot \sigma_{22}}}$$

9.7.4 Definizione della distribuzione mista di Gaussiane

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

Consiste di k mixture components, ogni componente corrisponde a una distribuzione gaussiana. μ_i e Σ_i sono i parametri della i -esimo mixture component. $\alpha_i > 0$ sono i coefficienti di mixture corrispondenti, dove $\sum_{i=1}^k \alpha_i = 1$ (la mixture è finita).

Un modello di mixture a due cluster

data											
A	51	B	62	B	64	A	48	A	39	A	51
A	43	A	47	A	51	B	64	B	62	A	48
B	62	A	52	A	52	A	51	B	64	B	64
B	64	B	64	B	62	B	63	A	52	A	42
A	45	A	51	A	49	A	43	B	63	A	48
A	42	B	65	A	48	B	65	B	64	A	41
A	46	A	48	B	62	B	66	A	48		
A	45	A	49	A	43	B	65	B	64		
A	45	A	46	A	40	A	46	A	48		
model											

Figura 196: Esempio di mixture a due cluster

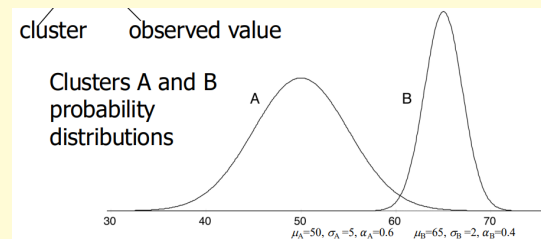


Figura 197: Funzione di densità di probabilità della mixture a due cluster

Supponiamo che i campioni siano generati da una distribuzione mista di Gaussiane:

1. Innanzitutto, seleziona i componenti della mixture gaussiana utilizzando la distribuzione a priori definita da $\alpha_1, \alpha_2, \dots, \alpha_k$, dove α_i è la probabilità di selezionare il i -esimo componente della mixture.
2. Quindi, genera campioni campionando dalle funzioni di densità di probabilità dei componenti della mixture selezionati.

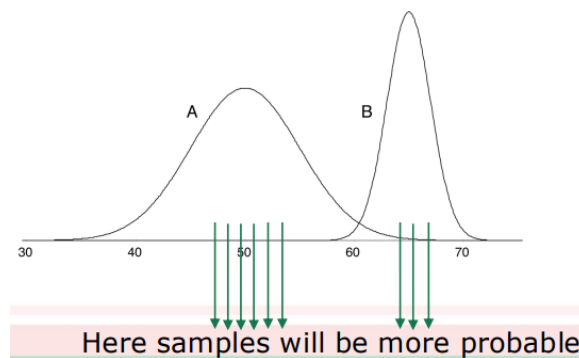


Figura 198: Posizione in cui i campioni sono più probabili

9.7.5 Ottimizzazione dei parametri del modello

Massima likelihood

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

$$LL(D) = \ln \left(\prod_{j=1}^m p_M(x_j) \right) = \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(x_j | \mu_i, \Sigma_i) \right)$$

$$\frac{\partial LL(D)}{\partial \mu_i} = 0 \quad \Rightarrow \quad \mu_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

$$\gamma_{ji} = p_M(z_j = i | x_j) (1 \leq i \leq k); \quad \gamma_{ji} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(x_j | \mu_l, \Sigma_l)}$$

Esempio

Dato il cluster A , viene selezionato in base alla probabilità α_i :

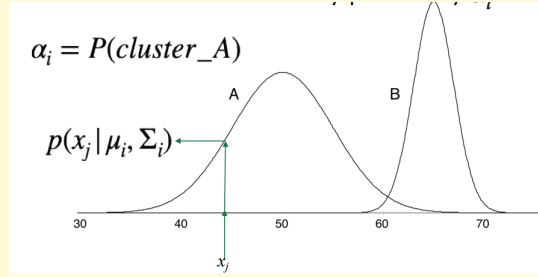


Figura 199: Esempio di ottimizzazione dei parametri del modello

$$\gamma_{ji} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l \cdot p(x_j | \mu_l, \Sigma_l)}$$

$$\mu_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

Sia:

$$\frac{\partial LL(D)}{\partial \Sigma_i} = 0 \quad \Rightarrow \quad \Sigma_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu_i)(x_j - \mu_i)^T}{\sum_{j=1}^m \gamma_{ji}}$$

Moltiplicatore di Lagrange:

$$\alpha_i = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$$

Algorithm 9 Mixture of-Gaussian Clustering

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Number of Gaussian mixture components k .

```

1: Process:
2: repeat
3:   for  $j = 1, 2, \dots, m$  do
4:     According to (9.30), compute the posterior probabilities that  $x_j$  is generated by each
     Gaussian mixture component, i.e.,  $\gamma_{ji} = p_M(z_j = i | x_j) (1 \leq i \leq k)$ 
5:   end for
6:   for  $i = 1, 2, \dots, k$  do
7:     Compute the updated mean vector:  $\mu'_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$ 
8:     Compute the updated covariance matrix:  $\Sigma'_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu'_i)(x_j - \mu'_i)^T}{\sum_{j=1}^m \gamma_{ji}}$ 
9:     Compute the updated mixture coefficient:  $\alpha'_i = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$ 
10:  end for
11:  Update the model parameters:  $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$  to  $\{(\alpha'_i, \mu'_i, \Sigma'_i) \mid 1 \leq i \leq k\}$ 
12: until The termination condition is met
13:  $C_i = \emptyset (1 \leq i \leq k)$ ;
14: for  $j = 1, 2, \dots, m$  do
15:   Determine the cluster label  $\lambda_j$  of  $x_j$  according to (9.31);  $\lambda_j = \arg \max_{i \in \{1, 2, \dots, k\}} \gamma_{ji}$ 
16:   Move  $x_j$  to the corresponding cluster:  $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$ 
17: end for
Ensure: Clusters  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ .

```

Risultati del Clustering

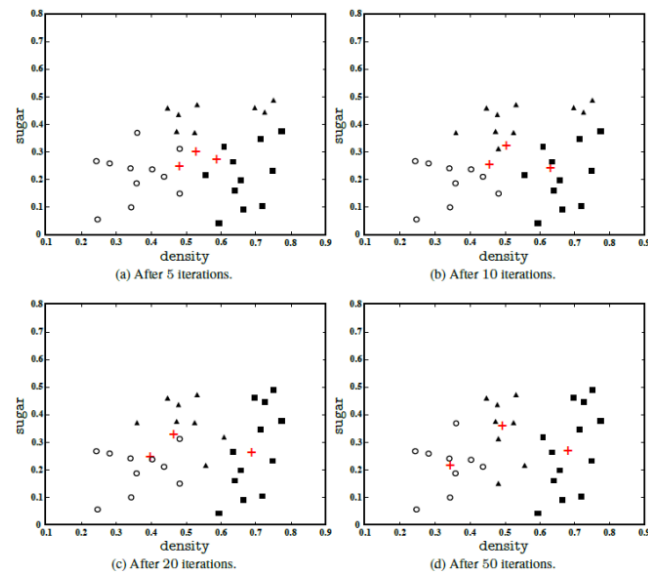


Figura 200: Risultato del clustering misto Gaussiano dopo 5, 10, 20 e 50 iterazioni

10 Riduzione della dimensionalità e Apprendimento delle metriche

10.1 k-Nearest Neighbor Learning

10.1.1 Introduzione a k-nearest neighbor learning

L'apprendimento k-Nearest Neighbor (kNN) è un metodo di apprendimento supervisionato comune:

- Ottenere campioni di training e una metrica di distanza specifica;
- Dato un campione di test, trovare i k campioni più vicini nel set di addestramento, prendere il voto di questi campioni per la classificazione o la media per la regressione. È possibile utilizzare anche il voting ponderato o la media: minore è la distanza, maggiore è il peso.
 - **Voting:** predire il campione di test come l'etichetta di classe più frequente nei k vicini
 - **Averaging:** prevedere il campione di prova come media dei k output a valori reali

10.1.2 Lazy Learning e Eager Learning

L'apprendimento k-Nearest Neighbor non sembra avere un processo di formazione esplicito e appartiene all'apprendimento pigro.

Lazy learning: memorizza semplicemente i campioni nella fase di addestramento e non fa nulla finché non vengono ricevuti i campioni di prova.

Eager learning: imparare dai campioni nella fase di formazione

10.1.3 Illustrazione di k-Nearest Neighbor Learning

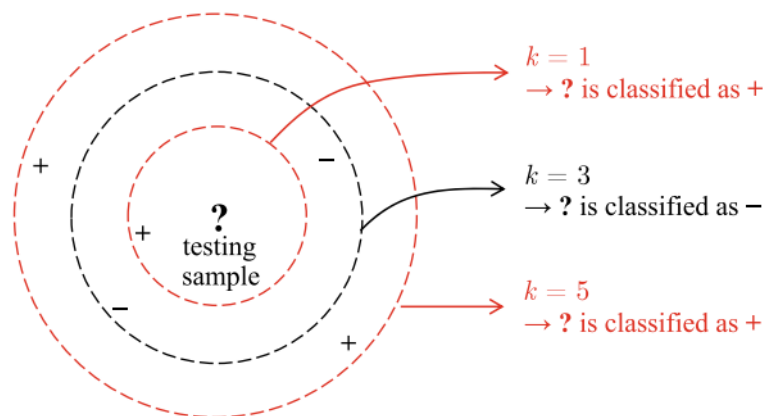


Figura 201: Classificatore kNN. Le linee tratteggiate sono linee equidistanti; il campione di prova è classificato come positivo quando $k = 1$ o $k = 5$ ed è classificato come negativo quando $k = 3$

Possiamo osservare che il parametro k gioca un ruolo importante in questo caso, poiché valori di k diversi possono portare a risultati di classificazione molto diversi. Inoltre, calcoli di distanza diversi possono anche portare a quartieri significativamente diversi, quindi a risultati di classificazione diversi.

10.1.4 Errore di classificazione binaria di 1NN

Supponiamo di aver selezionato una distanza appropriata, ovvero di poter identificare k vicini appropriati. Quindi, diamo un'occhiata all'uso del classificatore One-Nearest Neighbor (1NN con $k = 1$) per problemi di classificazione binaria.

Dato un campione di test x , supponiamo che il suo vicino più prossimo sia z , il tasso di classificazione errata di 1NN è la probabilità che x e z abbiano etichette di classe diverse, ovvero:

$$P(err) = 1 - \sum_{x \in \mathcal{Y}} P(x | x) P(x | z)$$

Supponiamo che i campioni siano i.i.d. e che possiamo sempre trovare un campione all'interno di un intervallo positivo δ arbitrariamente piccolo per qualsiasi x . In altre parole, per qualsiasi campione di test, il campione di training può sempre essere trovato all'interno di un intervallo arbitrariamente piccolo.

Sia $c^* = \arg \max_{c \in \mathcal{Y}} P(c | x)$ denotiamo il risultato della classificazione Bayes ottimale per il campione di test x :

$$\begin{aligned} P(err) &= 1 - \sum_{c \in \mathcal{Y}} P(x | x) P(c | z) \simeq 1 - \sum_{c \in \mathcal{Y}} P^2(x | x) \leq 1 - P^2(c^* | x) \\ &= (1 + P(c^* | x))(1 - P(c^* | x)) \leq 2(1 - P(c^* | x)). \end{aligned}$$

Sebbene il classificatore 1NN sia semplice, il suo errore di generalizzazione è al massimo il doppio dell'errore del classificatore ottimale di Bayes!

10.2 Low-dimensional Embedding

10.2.1 Maledizione della dimensionalità

Le nostre discussioni nella sezione precedente si basano su un importante presupposto: possiamo sempre trovare un campione entro un intervallo positivo arbitrariamente piccolo δ per qualsiasi x , il che significa che il campionamento deve essere sufficientemente denso, ovvero un campionamento denso:

- Supponiamo $\delta = 0,001$ e che ci sia una sola feature, abbiamo bisogno di 1000 campioni distribuiti uniformemente all'interno dell'intervallo di valori normalizzati della feature data. Tuttavia, questo è il caso solo quando la dimensione è 1. Se la dimensione aumenta, la situazione cambierà significativamente. Ad esempio, date 20 features, avremmo bisogno di $(10^3)^{20} = 10^{60}$ campioni per soddisfare il requisito di campionamento denso (*).
- Poiché il numero di feature può facilmente superare le migliaia nella pratica, è generalmente impraticabile soddisfare il requisito del campionamento denso. Inoltre, i calcoli delle distanze, richiesti in molti metodi di apprendimento, diventano difficili nello spazio ad alta dimensionalità. Ad esempio, anche il prodotto interno non è più semplice nei casi ad alta dimensionalità.
- L'elevata dimensionalità crea problemi, come la scarsità di dati e la difficoltà di calcolo delle distanze, che si presentano con tutti i metodi di apprendimento automatico. Questo fenomeno è noto come la maledizione della dimensionalità.

(*) Nota: il numero di particelle elementari nell'universo osservabile è di circa 10^{80} .

Un approccio generale per alleviare la maledizione della dimensionalità è la **riduzione della dimensionalità**.

- La conversione dello spazio delle feature originale ad alta dimensionalità in un **sottospazio a bassa dimensionalità** tramite **trasformazioni matematiche (embedding)**.

Perché è possibile eseguire la riduzione della dimensionalità?

- In molti casi, sebbene i campioni osservati o raccolti dalle persone siano ad alta dimensionalità, **ciò che è strettamente correlato al compito di apprendimento potrebbe essere solo una distribuzione a bassa dimensionalità**, ovvero vi è un'immersione a bassa dimensionalità nello spazio ad alta dimensionalità.

10.2.2 Multiple Dimensional Scaling

Il Multiple Dimensional Scaling (MDS) proietta un set di dati dallo spazio originale in uno spazio a dimensione inferiore, preservando le distanze tra i campioni. È una classe di tecniche e risale a un articolo del 1952 [Togerson]

Gli emedding in LLM sono una nuova versione.

È disponibile in scikit learn. Si basa su tre fasi:

1. Calcola la matrice GRAM $= X^T X$
2. Decomposizione degli autovettori: mantiene gli autovalori d^* più grandi
3. Projection in the space of the eigenvectors to obtain the final data matrix $\tilde{X}$

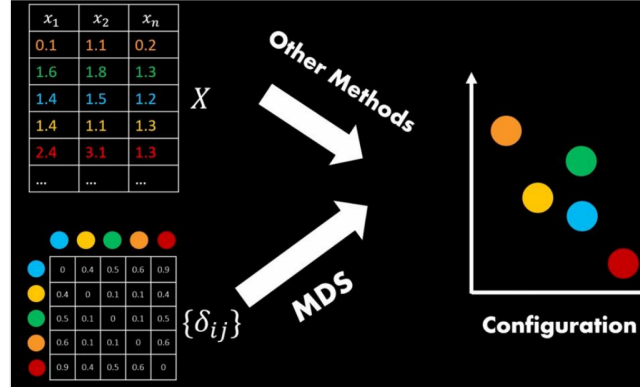


Figura 202: Illustrazione di Multiple Dimensional Scaling

Sia $D \in \mathbb{R}^{m \times m}$ la matrice delle distanze di m campioni nello spazio originale, dove la distanza tra i campioni x_i e x_j è $dist_{ij}$.

$$D = \begin{bmatrix} dist_{11} & dist_{12} & \cdots & dist_{1m} \\ dist_{21} & dist_{22} & \cdots & dist_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ dist_{m1} & dist_{m2} & \cdots & dist_{mm} \end{bmatrix}$$

$$dist_{ij} = \|x_i - x_j\| = \langle x_{i1} \mid \dots \mid x_{id} \rangle - \langle x_{j1} \mid \dots \mid x_{jd} \rangle = \langle x_{i1} - x_{j1} \mid \dots \mid x_{id} - x_{jd} \rangle$$

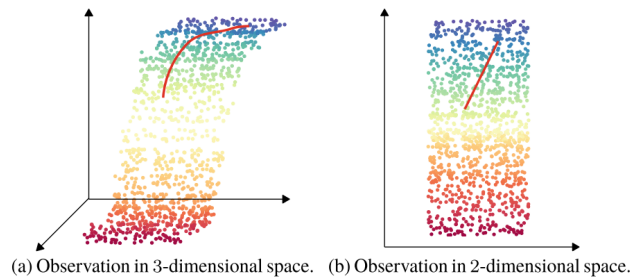


Figura 203: Un'illustrazione dell'incorporamento a bassa dimensionalità, X a sinistra, Z a destra

Il compito è ottenere la rappresentazione del campione nello spazio d' -dimensionale, e la distanza tra due campioni qualsiasi è la stessa in entrambi gli spazi, ovvero, $\|z_i - z_j\| = dist_{ij}$

Sia $B = Z^T Z \in \mathbb{R}^{m \times m}$ la matrice del prodotto interno dei campioni proiettati

$b_{ij} = z_i^T z_j$ indica il prodotto interno del campione proiettato i -esimo e j -esimo

$$B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mm} \end{bmatrix}$$

$$z_i = \langle z_{i1} \mid \dots z_{id'} \rangle \quad z_j = \langle z_{j1} \mid \dots z_{jd'} \rangle$$

$$\begin{bmatrix} z_{i1} \\ z_{i2} \\ \dots \\ z_{id'} \end{bmatrix} \cdot \begin{bmatrix} z_{j1} & z_{j2} & \dots & z_{jd'} \end{bmatrix} = \sum_{k=1}^{d'} z_{ik} z_{jk} = b_{ij}$$

Abbiamo:

$$dist_{ij}^2 = \|z_i\|^2 + \|z_j\|^2 - 2z_i^\top z_j = b_{ii} + b_{jj} - 2b_{ij}$$

Perché:

$$dist_{ij} = z_i - z_j = \langle z_{i1}, \dots, z_{id'} \rangle - \langle z_{j1}, \dots, z_{jd'} \rangle$$

$$= \langle z_{i1} - z_{j1}, \dots, z_{id'} - z_{jd'} \rangle,$$

$$dist_{ij}^2 = \|\langle z_{i1} - z_{j1}, \dots, z_{id'} - z_{jd'} \rangle\|^2$$

$$= \sum_{k=1}^{d'} (z_{ik} - z_{jk})^2 = \sum_{k=1}^{d'} (z_{ik}^2 + z_{jk}^2 - 2z_{ik}z_{jk}).$$

Supponiamo che i campioni proiettati z siano **centrati sullo zero**, ovvero $\sum_{i=0}^m z_i = 0$. Quindi, possiamo sapere che le sommatorie delle colonne e delle righe della matrice del prodotto scalare B sono:

$$\sum_{i=1}^m b_{ij} = \sum_{j=1}^m b_{ij} = 0$$

Perché?:

$$b_{ij} = \sum_{k=1}^{d'} z_{ik} \cdot z_{jk} = z_{i1} \cdot z_{j1} + \dots + z_{id'} \cdot z_{jd'}$$

$$\sum_{i=1}^m b_{ij} = z_{11} \cdot z_{j1} + \dots + z_{1d'} \cdot z_{jd'} +$$

$$z_{21} \cdot z_{j1} + \dots + z_{2d'} \cdot z_{jd'} +$$

$$\vdots$$

$$z_{m1} \cdot z_{j1} + \dots + z_{md'} \cdot z_{jd'}$$

$$= (z_{11} + z_{21} + \dots + z_{m1}) \cdot z_{j1} +$$

$$(z_{11} + z_{21} + \dots + z_{m1}) \cdot z_{j2} +$$

$$\vdots$$

$$(z_{11} + z_{21} + \dots + z_{m1}) \cdot z_{jd'} = 0$$

Indichiamo con $tr(\cdot)$ la traccia di una matrice.

$$tr(B) = b_{11} + b_{22} + \dots + b_{mm}$$

Possiamo ottenere la seguente equazione:

$$\sum_{i=1}^m dist_{ij}^2 = tr(B) + mb_{ij}, \quad \sum_{j=1}^m dist_{ij}^2 = tr(B) + mb_{ij}, \quad \sum_{i=1}^m \sum_{j=1}^m dist_{ij}^2 = 2mtr(B)$$

Quindi potremmo ottenere la matrice del prodotto scalare dalla matrice B delle distanze D tramite la proprietà di distanza preservata:

$$dist_{i\cdot}^2 = \frac{1}{m} \sum_{j=1}^m dist_{ij}^2 \quad dist_{\cdot j}^2 = \frac{1}{m} \sum_{i=1}^m dist_{ij}^2 \quad dist_{\cdot\cdot}^2 = \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m dist_{ij}^2$$

$$b_{ij} = -\frac{1}{2}(dist_{ij}^2 - dist_{i\cdot}^2 - dist_{\cdot j}^2 + dist_{\cdot\cdot}^2)$$

Perché?:

$$dist_{ij}^2 = \|z_i\|^2 + \|z_j\|^2 - 2z_i^\top z_j = b_{ii} + b_{jj} - 2b_{ij} \Rightarrow b_{ij} = -\frac{1}{2}(dist_{ij}^2 - b_{ii} - b_{jj})$$

dove sostituiamo b_{ii} , b_{jj} e $tr(B)$ da:

$$b_{ii} = \frac{1}{m}(\sum_j dist_{ij}^2 - tr(B)) \quad b_{jj} = \frac{1}{m}(\sum_i dist_{ij}^2 - tr(B))$$

$$tr(B) = \frac{1}{2m} \sum_i \sum_j dist_{ij}^2$$

- Calcoliamo la decomposizione degli autovalori della matrice , ovvero, $B = V\Lambda V^\top$
- $\Lambda = diag(\lambda_1, \dots, \lambda_m)$ è la matrice diagonale degli autovalori.
- V è la matrice degli autovettori corrispondenti.
- Assumiamo che ci siano d^* non-zero eigenvalues.
- La matrice diagonale corrispondente e la matrice degli autovettori sono: $\Lambda_* = diag(\lambda_1, \lambda_2, \dots, \lambda_{d^*})$ e V_*
- Quindi (proiezione), Z potrebbe essere scomposto come: $Z = \Lambda_*^{\frac{1}{2}} V_*^\top \in \mathbb{R}^{d^* \times m}$
- In pratica, richiediamo solo che le distanze ridotte in termini dimensionali siano vicine alle distanze originali, anziché uguali.
- In questi casi possiamo prendere $d' \ll d$ più grandi autovalori.
- Realizziamo una matrice diagonale $\tilde{\Lambda} = diag(\lambda_1, \lambda_2, \dots, \lambda_{d'})$
- Denotiamo $\tilde{V}$ come la matrice degli autovettori corrispondenti.
- Quindi, abbiamo:

$$Z = \tilde{\Lambda}^{\frac{1}{2}} \tilde{V}^\top \in \mathbb{R}^{d' \times m}$$

10.2.3 Decomposizione degli autovalori di B

$$B = V\Lambda V^\top$$

$$B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mm} \end{bmatrix} \quad V = \begin{bmatrix} \nu_{11} & \nu_{12} & \cdots & \nu_{1d} \\ \nu_{21} & \nu_{22} & \cdots & \nu_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ \nu_{m1} & \nu_{m2} & \cdots & \nu_{md} \end{bmatrix} \quad \Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_m \end{bmatrix}$$

$$V\Lambda = \begin{bmatrix} \nu_{11}\lambda_1 & \nu_{12}\lambda_2 & \cdots & \nu_{1d}\lambda_d^* \\ \nu_{21}\lambda_1 & \nu_{22}\lambda_2 & \cdots & \nu_{2d}\lambda_d^* \\ \vdots & \vdots & \ddots & \vdots \\ \nu_{m1}\lambda_1 & \nu_{m2}\lambda_2 & \cdots & \nu_{md}\lambda_d^* \end{bmatrix} \quad V^\top = \begin{bmatrix} \nu_{11} & \nu_{21} & \cdots & \nu_{m1} \\ \nu_{12} & \nu_{22} & \cdots & \nu_{m2} \\ \vdots & \vdots & \ddots & \vdots \\ \nu_{1d^*} & \nu_{2d^*} & \cdots & \nu_{md^*} \end{bmatrix} \quad V\Lambda V^\top = \begin{bmatrix} \sum_j \nu_{1j}^2 \lambda_j & \sum_j \nu_{1j} \nu_{2j} \lambda_j \\ \sum_j \nu_{2j} \nu_{1j} \lambda_j & \sum_j \nu_{2j}^2 \lambda_j \\ \vdots & \vdots \\ \sum_j \nu_{mj} \nu_{1j} \lambda_j & \sum_j \nu_{mj} \nu_{2j} \lambda_j \end{bmatrix}$$

Algorithm 10 Multiple Dimensional Scaling

Require: Distance matrix $D \in \mathbb{R}^{m \times m}$, where $dist_{ij}$ is the distance from x_i to x_j ; Dimension d of the low-dimensional space.

1: **Process:**

2: Compute $dist_{i.}^2$, $dist_{.j}^2$, and $dist_{..}^2$ from D .

3: Compute inner product matrix B according to (10.10);

4: Perform eigenvalue decomposition on matrix B ;

5: Take the d largest eigenvalues to make a diagonal matrix $\tilde{\Lambda}$, and $\tilde{V}$ is the corresponding eigenvector matrix

Ensure: The matrix $\tilde{V}\tilde{\Lambda}^{\frac{1}{2}} \in \mathbb{R}^{m \times d'}$, where each row gives the coordinates of the sample in the low-dimensional space.

10.2.4 Riduzione delle dimensioni lineari

In generale, il metodo più semplice per ottenere un sottospazio a dimensione inferiore è applicare trasformazioni lineari allo spazio ad alta dimensione originale.

Dato un campione $X = (x_1, x_2, \dots, x_m) \in \mathbb{R}^{d \times m}$ in uno spazio a d dimensioni, i campioni trasformati nello spazio d' -dimensionale sono:

$$Z = W^T X$$

dove $W \in \mathbb{R}^{d \times d'}$ è la matrice di trasformazione, $Z \in \mathbb{R}^{d' \times m}$ è la rappresentazione nello spazio a bassa dimensionalità.

La matrice di trasformazione W può essere vista come d' vettori di base con dimensione d .

$\{w_1, w_2, \dots, w_{d'}\}$ può essere visto come un nuovo sistema di coordinate nello spazio originale.

Se w_i e w_j ($i \neq j$) sono ortogonali ($w_i^T w_j = 0$), il nuovo sistema di coordinate è un sistema di coordinate ortogonali, e W è una matrice ortogonale.

I metodi che eseguono la riduzione della dimensionalità mediante trasformazioni lineari sono chiamati riduzione lineare della dimensionalità.

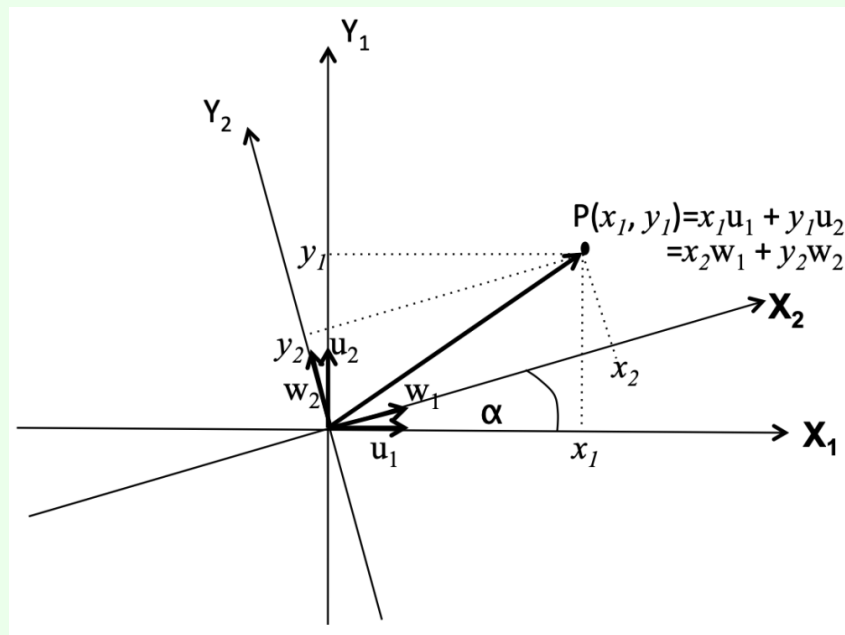


Figura 204: Illustrazione della moltiplicazione, rotazione e proiezione matrice-vettore

$$\begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix} \cdot \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} m_{11}v_1 + m_{12}v_2 \\ m_{21}v_1 + m_{22}v_2 \end{pmatrix}$$

$$\mathbf{W} = \begin{pmatrix} \mathbf{w}_1 & \mathbf{w}_2 \\ \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$$

$$\mathbf{W} \mathbf{u} = \mathbf{w}$$

$$\begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} \mathbf{u}_1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \mathbf{w}_1 \\ \cos \alpha \\ \sin \alpha \end{pmatrix}$$

$$\begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \cdot \begin{pmatrix} \mathbf{u}_2 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{w}_2 \\ -\sin \alpha \\ \cos \alpha \end{pmatrix}$$

Figura 205: Illustrazione della rotazione matrice-vettore

Per trovare x_2 e y_2 dobbiamo proiettare il vettore che definisce P su w_1 e w_2 :

$$\begin{cases} x_2 = \mathbf{W}_1 \cdot \mathbf{p} = \langle \cos(\alpha)u_1, \sin(\alpha)u_2 \rangle \cdot \langle x_1u_1, y_1u_2 \rangle = x_1 \cos(\alpha) + y_1 \sin(\alpha) \\ y_2 = \mathbf{W}_2 \cdot \mathbf{p} = \langle -\sin(\alpha)u_1, \cos(\alpha)u_2 \rangle \cdot \langle x_1u_1, y_1u_2 \rangle = -x_1 \sin(\alpha) + y_1 \cos(\alpha) \end{cases}$$

10.3 Analisi delle componenti principali (PCA)

Per i campioni in uno spazio delle feature ortogonale, come possiamo usare un iperpiano per rappresentare i campioni?

Intuitivamente, se un tale iperpiano esistesse, allora dovrebbe avere le seguenti proprietà:

1. **Errore di ricostruzione minimo:** i campioni dovrebbero avere brevi distanze da questo iperpiano;
2. **Massima varianza:** le proiezioni dei campioni sull' iperpiano dovrebbero rimanere distanti l'una dall'altra.

Le due proprietà di cui sopra portano a due derivazioni equivalenti della PCA.

10.3.1 Errore di ricostruzione minimo

Centrare la matrice campione, $\sum_i x_i = 0$, e il nuovo sistema di coordinate ottenuto è $\{w_1, w_2, \dots, w_d\}$. w_i è un vettore di base ortonormale, che soddisfa:

$$\|w_i\|_2 = 1, w_i^T w_j = 0 (i \neq j)$$

Eliminiamo alcune dimensioni nel nuovo sistema di coordinate, riducendo la dimensione a $d' < d$, poi abbiamo la proiezione $z_i = (z_{i1}; z_{i2}; \dots; z_{id'})$.

$z_{ij} = w_j^T x_i$ è la j-esima coordinata dimensionale tramite la proiezione di x_i su w_j . Ricostruiamo x_i da z_i come:

$$\hat{x}_i = \sum_{j=1}^{d'} z_{ij} w_j$$

Considera tutti i campioni, la distanza dei campioni grezzi x_i e dei campioni ricostruiti $\hat{x}_i$ è:

$$\begin{aligned} \sum_{i=1}^m \left\| \sum_{j=1}^{d'} z_{ij} w_j - x_i \right\|_2^2 &= \sum_{i=1}^m z_i^T z_i - 2 \sum_{i=1}^m z_i^T W^T x_i + \text{const} \\ &= \alpha - \text{tr} \left(W^T \left(\sum_{i=1}^m x_i x_i^T \right) W \right) \end{aligned}$$

Dovremmo minimizzare l'equazione di cui sopra. Poiché w_j è un vettore di base ortonormale, ed $\sum_i x_i x_i^T$ è la matrice di covarianza, abbiamo:

$$\begin{aligned} \min_W & -\text{tr}(W^T X X^T W) \\ \text{s.t.} & W^T W = I \end{aligned}$$

Quanto sopra è l'obiettivo di ottimizzazione della PCA.

10.3.2 Massima varianza

Il campione x_i corrisponde a un punto proiettato $W^T x_i$. Se desideriamo che le proiezioni dei campioni rimangano distanti l'una dall'altra, allora la **varianza dei campioni** proiettati dovrebbe essere **massimizzata**. La matrice di covarianza dei campioni proiettati è $\sum_i W^T x_i x_i^T W$, allora abbiamo:

$$\begin{aligned} \max_W & \text{tr}(W^T X X^T W) \\ \text{s.t.} & W^T W = I \end{aligned}$$

Questo problema è equivalente a:

$$\begin{aligned} \min_W & -\text{tr}(W^T X X^T W) \\ \text{s.t.} & W^T W = I \end{aligned}$$

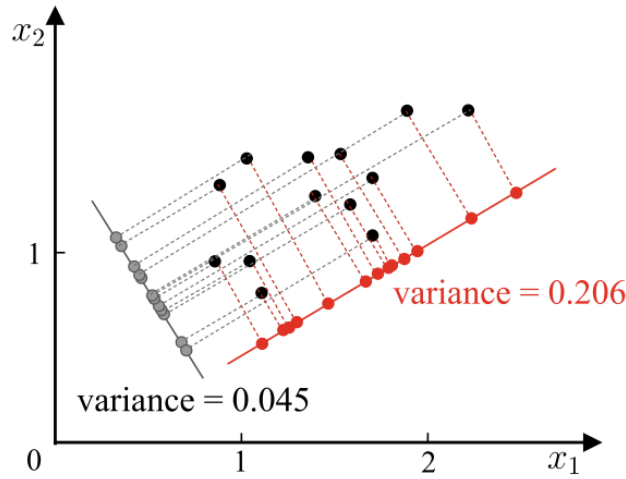


Figura 206: Per allontanare il più possibile le proiezioni dei campioni l'una dall'altra (le linee rosse), dobbiamo massimizzare la varianza dei campioni proiettati.

10.3.3 Risolvere PCA

$$\begin{aligned} \max_W \quad & \text{tr}(W^T X X^T W) \\ \text{s.t.} \quad & W^T W = I \end{aligned}$$

$$\begin{aligned} \min_W \quad & -\text{tr}(W^T X X^T W) \\ \text{s.t.} \quad & W^T W = I \end{aligned}$$

Applicando i moltiplicatori di Lagrange alle equazioni precedenti:

$$X X^T w = \lambda_i w$$

Eseguiamo la decomposizione degli autovalori sulla matrice di covarianza $X X^T$ e otteniamo gli autovalori ordinati: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. Quindi, costruiamo la soluzione utilizzando gli autovettori dei primi d' autovalori, ovvero, $W^* = (w_1, w_2, \dots, w_{d'})$

Algorithm 11 Principal Component Analysis

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Dimension d' of the lower-dimensional space.

1: **Process:**

2: Center all samples: $x_1 \leftarrow x_i - \frac{1}{m} \sum_{i=1}^m x_i$;

3: Compute the covariance matrix $X X^T$ of samples;

4: Perform eigvalue decomposition on the covariance matrix $X X^T$;

5: Take the eigenvectors $w_1, w_2, \dots, w_{d'}$ corresponding to the d' largest eigenvalues;

Ensure: The projection matrix $W^* = (w_1, w_2, \dots, w_{d'})$

In genere, la dimensione dello spazio a dimensione inferiore viene specificata dall'utente o selezionata eseguendo una convalida incrociata con valori d' diversi, ovvero confrontando le prestazioni di un classificatore kNN negli spazi a dimensione ridotta prodotti da d' valori diversi. Per la PCA, è anche possibile impostare una soglia t della varianza mantenuta (ad esempio, $t = 95\%$) dalla prospettiva di ricostruzione per trovare il minimo d' soggetto a:

$$\frac{\sum_{i=1}^{d'} \lambda_i}{\sum_{i=1}^d \lambda_i} \geq t$$

Con solo W^* e il vettore medio dei campioni, la PCA può proiettare nuovi campioni nello spazio a dimensione inferiore applicando semplici operazioni di sottrazione vettoriale e moltiplicazione matrice-vettore.

Ridurre la dimensionalità:

- produce una perdita di informazioni;

- può portare a un campionamento relativamente più denso;
- può ridurre il rumore: le direzioni degli autovalori più piccoli spesso sono correlate ai rumori: scartare queste dimensioni aiuta a ridurre il rumore.

10.4 Kernelized Linear Dimension Reduction

I metodi di riduzione lineare della dimensionalità trasformano uno spazio ad alta dimensionalità in uno spazio a bassa dimensionalità tramite una mappatura lineare. In pratica, tuttavia, sono spesso necessarie mappe non lineari per trovare la corretta immersione a bassa dimensionalità:

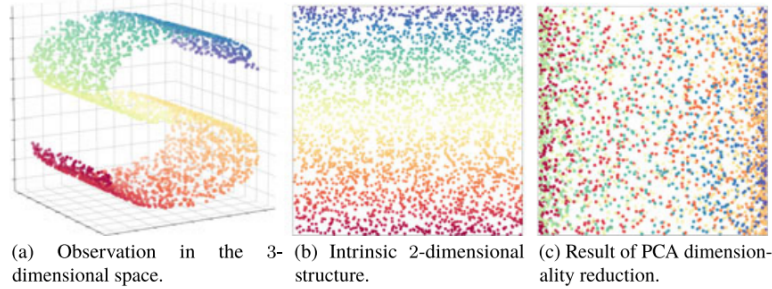


Figura 207: I 3000 campioni nello spazio tridimensionale vengono campionati da una regione quadrata dello spazio bidimensionale intrinseco e poi incorporati in una superficie a forma di S nello spazio tridimensionale. In questi casi, l'esecuzione di una riduzione lineare della dimensionalità comporta una perdita di informazione della struttura a bassa dimensionalità. I colori dei punti campione mostrano la struttura a bassa dimensionalità.

La Figura (a) mostra l'incorporamento di punti dati campionati da una regione quadrata mostrata nella Figura (b) su una superficie a forma di S in uno spazio tridimensionale. I risultati dell'analisi PCA (nella Figura (c)) perdono la struttura originale a bassa dimensionalità.

10.4.1 Kernelized PCA (KPCA)

Un approccio generale alla riduzione della dimensionalità non lineare consiste nel kernelizzare i metodi di riduzione della dimensionalità lineare tramite kernel tricks.

Supponiamo di proiettare i dati dallo spazio delle feature ad alta dimensionalità a un iperpiano generato da $W = (w_1, \dots, w_d)$. Allora per w_j , abbiamo:

$$\left(\sum_{i=1}^m z_i z_i^T \right) w_j = \lambda_j w_j$$

z_i è l'immagine di x_i nello spazio delle feature ad alta dimensionalità. Quindi abbiamo:

$$w_j = \frac{1}{\lambda_j} \left(\sum_{i=1}^m z_i z_i^T \right) w_j = \sum_{i=1}^m z_i \frac{z_i^T w_j}{\lambda_j} = \sum_{i=1}^m z_i \alpha_i^j$$

Supponendo che z_i è ottenuto mappando il campione originale x_i tramite ϕ :

$$z_i = \phi(x_i), i = 1, 2, \dots, m$$

Sappiamo che $W = \sum_{i=1}^m z_i \alpha_i$.

Se la forma esplicita di ϕ è nota, allora possiamo usarla per mappare i campioni nello spazio delle feature ad alta dimensionalità e quindi applicare la PCA, ovvero:

$$\left(\sum_{i=1}^m \phi(x_i) \phi(x_i)^T \right) w_j = \lambda_j w_j$$

con

$$w_j = \sum_{i=1}^m \phi(x_i) \alpha_i^j$$

La forma esatta di ϕ è generalmente sconosciuta. Quindi, introduciamo la funzione kernel:

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Sostituendo $w_j = \sum_{i=1}^m \phi(x_i) \alpha_i^j$ dentro $(\sum_{i=1}^m \phi(x_i) \phi(x_i)^T) w_j = \lambda_j w_j$ otteniamo:

$$K \alpha^j = \lambda_j \alpha^j$$

K è la matrice kernel corrispondente a k con:

$$(K)_{ij} = k(x_i, x_j), \alpha^j = (\alpha_1^j, \dots, \alpha_m^j)$$

Quello sopra è un problema di decomposizione degli autovalori: quindi, prendiamo semplicemente gli autovettori corrispondenti ai d' massimi autovalori in K .

Per un nuovo campione x , la sua coordinata proiettata nella j -esima dimensione è:

Sostituendo $w_j = \sum_{i=1}^m \phi(x_i) \alpha_i^j$ in z_j , dove z_j è la coordinata proiettata di x nella j -esima dimensione, otteniamo:

$$z_j = w_j^T \phi(x) = \sum_{i=1}^m \alpha_i^j \phi(x_i)^T \phi(x) = \sum_{i=1}^m \alpha_i^j k(x_i, x)$$

dove α_i è stata normalizzata.

Abbiamo potuto osservare che il KPCA è computazionalmente costoso poiché somma tutti i campioni per calcolare le coordinate proiettate.

10.5 Manifolds learning

L'apprendimento di varietà è un approccio di riduzione della dimensionalità che utilizza alcuni concetti delle varietà topologiche.

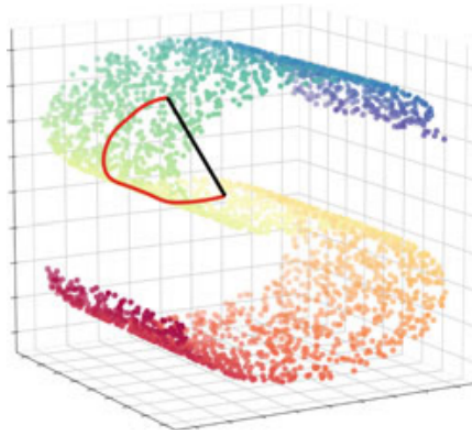
Una varietà è uno spazio topologico localmente omeomorfo allo spazio euclideo. Soddisfa localmente le proprietà dello spazio euclideo, il che significa che possiamo utilizzare la distanza euclidea.

Se una varietà a bassa dimensionalità è immersa nello spazio ad alta dimensionalità, allora i campioni, che sembrano essere molto complessi nello spazio ad alta dimensionalità, hanno localmente le stesse proprietà dei campioni nello spazio euclideo. Pertanto, possiamo stabilire localmente mappe di riduzione della dimensionalità e poi estenderle all'intero spazio.

Quando la dimensionalità viene ridotta a 2 o 3, possiamo visualizzare naturalmente i dati, e quindi l'apprendimento di varietà è utile anche per scopi di visualizzazione.

10.5.1 Isometric Mapping (Isomap)

Le distanze euclidee in linea retta misurate in uno spazio ad alta dimensionalità possono essere fuorvianti poiché le linee rette nello spazio ad alta dimensionalità potrebbero non esistere nell'immersione della varietà a bassa dimensionalità. La distanza geodetica è la distanza intrinseca tra i due punti.



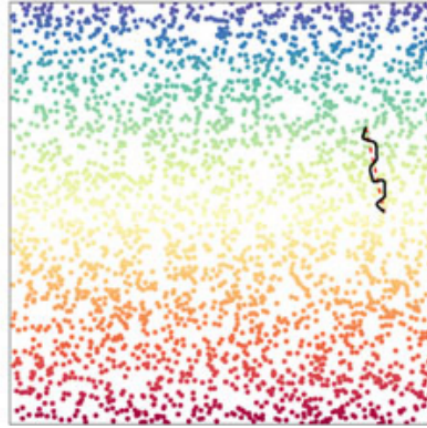
(a) The geodesic distance and the straight-line distance.

Figura 208: Non possiamo usare la distanza in linea retta nello spazio ad alta dimensionalità per calcolare la distanza geodetica (rossa) nell'immersione della varietà a bassa dimensionalità

Una varietà è localmente omeomorfa allo spazio euclideo e possiamo trovare i punti adiacenti di ciascun punto in base alla distanza euclidea.

Quindi, si può costruire un grafo di vicinato. Nel grafo di vicinato, solo i punti adiacenti sono connessi; la **distanza geodetica viene misurata trovando il cammino più breve** tra due punti nel grafo di vicinato.

Per trovare il cammino più breve tra due punti in un grafo di vicinato, possiamo utilizzare il noto algoritmo di Dijkstra o l'algoritmo di Floyd.



(b) The geodesic distance and the neighborhood distance.

Figura 209: è possibile un'approssimazione con la distanza del vicinato

Algorithm 12 Isometric Mapping

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Nearest neighbor parameter k ; Dimension d' of the low-dimensional space.

1: **Process:**

2: **for** $i = 1, 2, \dots, m$ **do**

3: Find the k nearest neighbors of x_i ;

4: Set The distance between x_i and the k nearest neighbors to the Euclidean distance, and set the distances from x_i to other sample points as positive infinity;

5: **end for**

6: Compute the distance $\text{dist}(x_i, x_j)$ between every pair sample points using a shortest path algorithm;

7: Use $\text{dist}(x_i, x_j)$ as input for the MDS algorithm;

8: **return** The output of the MDS algorithm;

Ensure: The low-dimensional projections $Z = \{z_1, z_2, \dots, z_m\}$ of the data set D .

10.5.2 Locally Linear Embedding (LLE)

Locally Linear Embedding (LLE) mira a mantenere le relazioni lineari tra campioni adiacenti.

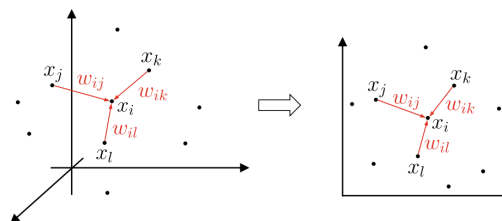


Figura 210: La relazione di ricostruzione dei campioni nello spazio ad alta dimensionalità viene preservata nello spazio a bassa dimensionalità

$$x_i = w_{ij}x_j + w_{ik}x_k + w_{il}x_l$$

- LLE inizia identificando gli indici di neighborhood set Q_i per i campioni x_i
- Quindi trova i pesi di ricostruzione lineare w_{ij} dei campioni in Q_i :

$$\begin{aligned} \min_{w_1, w_2, \dots, w_m} \sum_{i=1}^m \left\| x_i - \sum_{j \in Q_i} w_{ij} x_j \right\|_2^2 \\ \text{s.t.} \quad \sum_{j \in Q_i} w_{ij} = 1 \end{aligned}$$

x_i e x_j sono noti.

Ponendo $C_{jk} = (x_i - x_j)^T (x_i - x_k)$, allora w_{ij} ha una soluzione in forma chiusa:

$$w_{ij} = \frac{\sum_{k \in Q_i} C_{jk}^{-1}}{\sum_{l, s \in Q_i} C_{ls}^{-1}}$$

Poiché LLE si conserva w_i nello spazio a bassa dimensionalità, le coordinate a bassa dimensionalità z_i di x_i possono essere ottenute da:

$$\min_{z_1, z_2, \dots, z_m} \sum_{i=1}^m \left\| z_i - \sum_{j \in Q_i} w_{ij} z_j \right\|_2^2$$

Sia $Z = (z_1; z_2; \dots; z_m) \in \mathbb{R}^{d' \times m}$, $(W)_{ij} = w_{ij}$. In forma matriciale la formula soprastante diventa:

$$M = (I - W)^T (I - W)$$

E l'ottimizzazione può essere riscritta come:

$$\begin{aligned} \min_z \quad & \text{tr}(Z M Z^T) \\ \text{s.t.} \quad & Z Z^T = I \end{aligned}$$

Algorithm 13 Locally Linear Embedding

Require: Data set $D = \{x_1, x_2, \dots, x_m\}$; Nearest neighbor parameter k ; Dimension d' of the low-dimensional space.

- 1: **Process:**
- 2: **for** $i = 1, 2, \dots, m$ **do**
- 3: Find the k nearest neighbors of x_i ;
- 4: Compute the reconstruction weights w_{ij} for $j \in Q_i$;
- 5: Set $w_{ij} = 0$ for $j \notin Q_i$;
- 6: **end for**
- 7: Obtain M using $M = (I - W)^T (I - W)$
- 8: Perform eigenvalue decomposition on matrix M ;
- 9: **return** The eigenvectors corresponding to the d' smallest non-zero eigenvalues of M ;

Ensure: The low-dimensional projections $Z = \{z_1, z_2, \dots, z_m\}$ of the data set D .

10.6 Apprendimento delle metriche

Nell'apprendimento automatico, lo scopo principale della riduzione della dimensionalità è trovare uno spazio a dimensione inferiore, in cui le prestazioni di apprendimento siano migliori rispetto a quelle dello spazio originale ad alta dimensionalità.

Poiché ogni spazio corrisponde a una metrica di distanza definita sulle feature del campione, la ricerca di uno spazio appropriato è in effetti la ricerca di una metrica di distanza appropriata.

Allora, perché non apprendere direttamente la metrica di distanza appropriata?

Per due campioni a d dimensioni x_i e x_j , la loro distanza euclidea al quadrato può essere scritto come:

$$dist_{ed}^2(x_i, x_j) = \|x_i - x_j\|_2^2 = dist_{ij,1}^2 + dist_{ij,2}^2 + \dots + dist_{ij,d}^2$$

$dist_{ij,k}$ è la distanza tra x_i e x_j nella k -esima dimensione. Supponiamo che diverse feature abbiano importanze diverse, quindi possiamo introdurre i pesi delle feature w :

$$\begin{aligned} dist_{wed}^2(x_i, x_j) &= \|x_i - x_j\|_2^2 = w_1 \cdot dist_{ij,1}^2 + w_2 \cdot dist_{ij,2}^2 + \dots + w_d \cdot dist_{ij,d}^2 \\ &= (x_i - x_j)^T W (x_i - x_j) \end{aligned}$$

$w_i \geq 0$, $W = diag(w)$ è una matrice diagonale, $(W)_{ii} = w_i$, e i pesi regolabili sono i parametri da apprendere dai dati.

Notiamo che gli elementi non diagonali di W sono nulli, il che significa che gli assi sono ortogonali. In pratica, tuttavia, questo spesso non è vero. Quindi, sostituendo W con una matrice semidefinita positiva simmetrica M , otteniamo la **distanza di Mahalanobis**:

$$dist_{mah}^2(x_i, x_j) = (x_i - x_j)^T M (x_i - x_j) = \|x_i - x_j\|_M^2$$

dove M è anche chiamata **matrice metrica**, e l'apprendimento metrico riguarda l'apprendimento di M . Per garantire che le distanze siano non negative e simmetriche, M deve essere una matrice semidefinita positiva, ovvero deve esistere una matrice P tale che $M = PP^T$.

- Per imparare M , dobbiamo porci un obiettivo.
- Supponiamo di voler migliorare le prestazioni del classificatore più vicino, in tal caso M può essere incorporato direttamente nella misura delle prestazioni del classificatore e l' M ottimale si ottiene ottimizzando la misura delle prestazioni.

Semidefinito positivo

Le matrici semidefinite positive M sono definite in modo tale che gli scalari $x^T M x$ e $z^T M z$ siano non negativi per ogni vettore x con valori positivi.

10.6.1 Neighbourhood Component Analysis (NCA)

I classificatori neighbor più prossimo di solito utilizzano il metodo di voto per la classificazione, ovvero ogni campione vicino ha 1 voto e ogni campione non vicino ha 0 voti.

Invece di utilizzare il voto binario, possiamo anche utilizzare il metodo di voto probabilistico, in cui la probabilità di un campione x_j influenza la classificazione di un campione x_i , che è data da:

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|_M^2)}{\sum_l \exp(-\|x_i - x_l\|_M^2)}$$

p_{ij} è massimizzato quando $i = j$. L'impatto di x_j su x_i diminuisce con l'aumentare della distanza tra x_i e x_j . Se l'obiettivo è massimizzare l'accuratezza del LOO, allora l'accuratezza di x_i è la probabilità che altri campioni possano classificarlo correttamente:

$$p_i = \sum_{j \in \Omega_i} p_{ij}$$

Ω_i è l'insieme di indici di campioni con la stessa etichetta di classe di x_i .

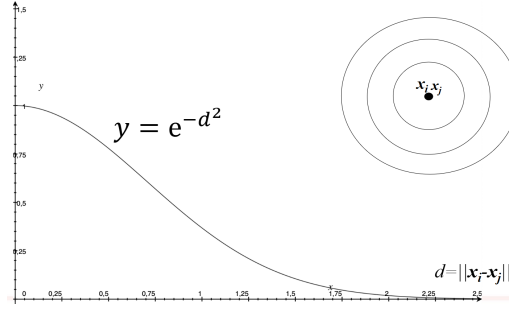


Figura 211: Grafico della funzione di base radiale

Quindi, la precisione LOO per l'intero set di dati è:

$$\sum_{i=1}^m p_i = \sum_{i=1}^m \sum_{j \in \Omega_i} p_{ij}$$

Con $p_{ij} = \frac{\exp(-\|x_i - x_j\|_M^2)}{\sum_l \exp(-\|x_i - x_l\|_M^2)}$ e $M = PP^T$, il target di NCA è:

$$\min_P 1 - \sum_{i=1}^m \sum_{j \in \Omega_i} \frac{\exp(-\|P^T x_i - P^T x_j\|_2^2)}{\sum_l \exp(-\|P^T x_i - P^T x_l\|_2^2)}$$

Risolvendo questo problema si ottiene la matrice metrica della distanza M che massimizza l'accuratezza LOO del classificatore del vicino più prossimo.

Oltre agli obiettivi di apprendimento supervisionato, come la minimizzazione del tasso di errore, possiamo anche introdurre la conoscenza del dominio nell'obiettivo di ottimizzazione dell'apprendimento metrico.

Se sappiamo già che alcuni campioni sono simili o dissimili, possiamo definire un insieme di vincoli di collegamento obbligatorio $\mathcal{M}$ e un insieme di vincoli di collegamento impossibile $\mathcal{C}$, dove $(x_i, x_j) \in \mathcal{M}$ indica che x_i e x_j sono simili, e $(x_i, x_j) \in \mathcal{C}$ indica che x_i e x_j sono dissimili:

$$\begin{aligned} \min_M \quad & \sum_{(x_i, x_j) \in \mathcal{M}} \|x_i - x_j\|_M^2 \\ \text{s.t.} \quad & \sum_{(x_i, x_k) \in \mathcal{C}} \|x_i - x_k\|_M \geq 1 \\ & M \succeq 0 \end{aligned}$$

Il vincolo $M \succeq 0$ garantisce che M sia semidefinita positiva. Il problema di cui sopra minimizza la distanza totale tra campioni simili mantenendo le distanze individuali tra campioni dissimili non inferiori a 1.

Diversi metodi di apprendimento metrico ottengono buone matrici metriche di distanza simmetrica semidefinita positiva M basate su obiettivi diversi.

Quando M è una matrice di rango basso, mediante la decomposizione agli autovalori di M , possiamo sempre ottenere un insieme di vettori di base ortogonali, dove d è minore di $\text{rank}(M)$ il numero originale di feature.

Quindi, il risultato dell'apprendimento metrico può derivare una matrice $P \in \mathbb{R}^{d \times \text{rank}(M)}$ a dimensione ridotta che può essere utilizzata per la riduzione della dimensionalità.

11 Selezione degli Attributi e Apprendimento Sparso

11.1 Feature

Feature: Nell'apprendimento automatico, gli attributi sono chiamati features.

Diversi tipi di feature:

- **Feature rilevanti:** utili per il compito di apprendimento;
- **Feature irrilevanti:** inutili per il compito di apprendimento;
- **Feature ridondanti:** le informazioni che contengono possono essere dedotte da altre features.

Esempio: le features dell'anguria

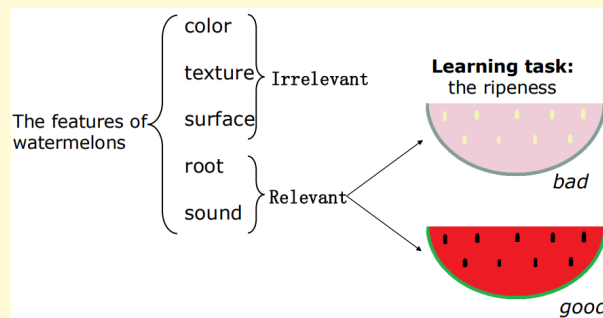


Figura 212: Esempio di features rilevanti, irrilevanti e ridondanti per il compito di classificazione dell'anguria.

11.2 Selezione delle Feature

Selezione delle feature

- Seleziona le feature irrilevanti dal set di feature fornito;
- Assicuratevi che le feature importanti non vengano ignorate.

Motivi

- Alleviare la maledizione della dimensionalità: costruire il modello basandosi su un numero inferiore di attributi.
- Ridurre la difficoltà di apprendimento: conservare le informazioni chiave.

11.2.1 Metodo generale per la selezione delle features

Valutare tutti i possibili sottoinsiemi di features: **poco pratico** poiché soffre dell'esplosione combinatoria.

Un metodo più pratico:



Figura 213: Due passaggi chiave: ricerca del sottoinsieme e valutazione del sottoinsieme.

11.2.2 Ricerca di sottoinsieme

Seleziona il sottoinsieme di feature che contiene informazioni importanti con la ricerca greedy.

Forward search: aggiunge le feature rilevanti.

Backward search: rimuove le feature irrilevanti.

Bidirectional search: aggiunge features rilevanti e rimuove feature irrilevanti simultaneamente.

11.2.3 Forward search

Il sottoinsieme ottimale viene inizializzato con un insieme vuoto e l'insieme delle feature viene inizializzato con tutte le feature fornite.

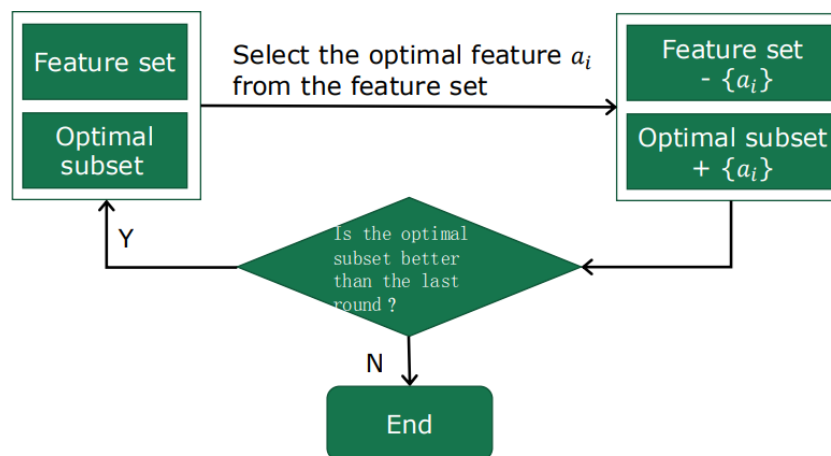


Figura 214: Procedura di forward search.

11.2.4 Valutazione del sottoinsieme

Ogni sottoinsieme di feature corrisponde a una partizione del set di dati. Ogni partizione corrisponde a un'assegnazione di valore del sottoinsieme di feature.

L'etichetta fornisce la partizione della verità di base.

Possiamo valutare il sottoinsieme di features stimando la differenza tra queste due partizioni. Minore è la differenza, migliore è il sottoinsieme di features.

11.2.5 Valutare il sottoinsieme con entropia informativa

Il sottoinsieme delle features A determina una partizione del set di dati D :

- Il dataset D viene suddiviso in V sottoinsiemi in base a quei valori delle feature su A , e ogni sottoinsieme è indicato come D^v .
- $Ent(D^v)$ è l'entropia informativa del sottoinsieme D^v .

Definizione dell'entropia informativa

$$Ent(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

p_i denota la porzione dell' i -esima classe.

L'etichetta Y fornisce la partizione di verità di base del set di dati D .

- $Ent(D)$ è l'entropia informativa del set di dati D .

Il guadagno informativo del sottoinsieme di features A è:

$$Gian(A) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$$

11.2.6 Metodi di selezione delle features comunemente utilizzati

Il metodo di selezione delle feature può essere ottenuto combinando il metodo di ricerca di sottoinsiemi di feature e il metodo di valutazione dei sottoinsiemi.

Esistono tre tipi di metodi di selezione delle feature comunemente utilizzati:

- Metodi di filtro
- Metodi wrapper
- Metodi di embedding

Metodi di filtro I metodi di filtro selezionano le features senza considerare gli studenti conseguenti. Relief (features Rilevanti) [Kira e Rendell, 1992]

- Incorporare una statistica di rilevanza per misurare l'importanza della feature.
- L'importanza di un sottoinsieme di features è determinata dalla somma dei componenti della statistica di rilevanza corrispondenti.
- Selezionare le features con una componente maggiore di una soglia specificata dall'utente.
- In alternativa, selezionare le features corrispondenti alle componenti più grandi.

Determina la statistica di rilevanza in Relief

- **Near-hit:** $x_{i,nh}$ è il campione più vicino a x_i che ha la stessa etichetta;
- **Near-miss:** $x_{i,nm}$ è il campione più vicino a x_i che ha un'etichetta diversa.
- La componente statistica di rilevanza della feature j è:

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i,nh}^j)^2 + \text{diff}(x_i^j, x_{i,nm}^j)^2$$

Nota

Quando j è discreta, $\text{diff}(x_a^j, x_b^j) = 0$ se $x_a^j = x_b^j$; altrimenti, $\text{diff}(x_a^j, x_b^j) = 1$.
Quando j è continua, $\text{diff}(x_a^j, x_b^j) = |x_a^j - x_b^j|$, quando x_a^j e x_b^j sono normalizzati in $[0, 1]$.

Una componente statistica di rilevanza maggiore significa che $x_{i,nh}$ è più vicino di $x_{i,nm}$ alla feature j . Pertanto, la feature j è più utile per distinguere classi diverse.

La complessità computazionale di Relief è lineare rispetto al numero di campionamenti e al numero di features originali, e quindi è un metodo efficiente.

Estendi Relief alle classi multiple Relief è progettato per la classificazione binaria e la sua variante Relief-F [Kononenko, 1994] può gestire la classificazione multi-classe.

- Dato un set di dati con $|\mathcal{Y}|$ classi. Per un campione x_i con etichetta k .
 - Near-hit: $x_{i,nh}$ è il campione più vicino a x_i che ha la stessa etichetta k ;
 - Near-miss: Trova un quasi incidente per ogni classe diversa da k , indicato da $x_{i,l,nm}$ ($l = 1, 2, \dots, |\mathcal{Y}|; l \neq k$)
 - La componente statistica di rilevanza della feature k è:

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i,nh}^j)^2 + \sum_{l \neq k} \text{diff}(p_l \times \text{diff}(x_{i,l,nm}^j)^2)$$

p_l è la porzione di campioni di classe l nel set di dati D .

Metodi wrapper I metodi wrapper utilizzano direttamente le prestazioni degli studenti successivi come metrica di valutazione per il sottoinsieme di features.

- I metodi wrapper mirano a trovare il sottoinsieme di features più utile, su misura per il learner in questione.
- I metodi wrapper sono solitamente migliori dei metodi filtro in termini di prestazioni finali del learner, poiché la selezione delle features è ottimizzata per il learner in questione.
- I metodi wrapper sono spesso molto più costosi dal punto di vista computazionale, poiché addestrano il learner più volte durante la selezione delle features.

Metodi LVW Le ricerche LVW (Las Vegas Wrapper) [Liu e Setiono, 1996] presentano sottoinsiemi che utilizzano una **strategia randomizzata** nell'ambito del metodo Las Vegas, e i sottoinsiemi vengono valutati in base all'errore di classificazione finale.

- Genera un sottoinsieme di feature casuale
- Stima l'errore del learner utilizzando la convalida incrociata sul sottoinsieme di feature
- Esegui più volte e seleziona il sottoinsieme di feature con l'errore più piccolo tra più sottoinsiemi di feature generati casualmente come soluzione finale*.

*LVW potrebbe non restituire nulla se il tempo di esecuzione è limitato.

Metodi embedding I metodi embedding unificano il processo di selezione delle feature e il processo di addestramento dell'allievo in un processo di ottimizzazione congiunto, ovvero le feature vengono selezionate automaticamente durante l'addestramento.

- Per un modello di regressione lineare con errore quadratico e L_2 , la regolarizzazione viene utilizzata per prevenire l'overfitting. L'obiettivo di ottimizzazione è (Ridge regression [Tikhonov and Arsenin, 1977]):

$$\min_w \sum_{i=1}^m (y_i - w^\top x_i)^2 + \lambda \|w\|_2^2$$

- **LASSO** [Tibshirani, 1996] può essere ottenuto sostituendo la norma L_2 con la norma L_1 :

$$\min_w \sum_{i=1}^m (y_i - w^\top x_i)^2 + \lambda \|w\|_1$$

Un metodo di inclusione che ha maggiori probabilità di portare a soluzioni sparse.

L'utilizzo della norma L_1 ha maggiori probabilità di portare a una soluzione sparsa. Supponiamo che x abbia solo due features, quindi la soluzione w ha due componenti w_1 e w_2 . Le soluzioni devono fare un **compromesso** tra il termine di errore quadratico e il termine di regolarizzazione. In altre parole, le soluzioni giacciono sulle **intersezioni** tra i contorni del termine di errore quadratico e del termine di regolarizzazione. Dalla figura, quando si usa la norma L_1 , le intersezioni spesso giacciono sugli assi, ovvero w_1 o w_2 è 0.

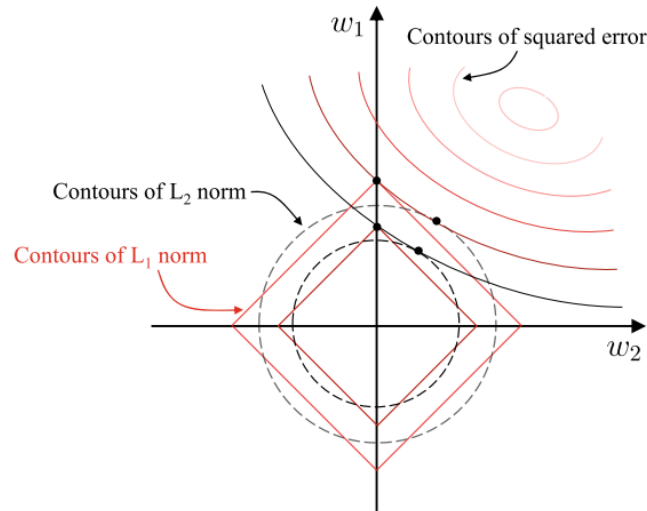


Figura 215: È più facile ottenere soluzioni sparse utilizzando la regolarizzazione L_1 che utilizzando la regolarizzazione L_2

Un contorno è una linea composta da punti con lo stesso valore (errore).

Risolvi la norma L_1 (1) L'obiettivo di ottimizzazione con il termine di regolarizzazione è:

$$\min_x f(x) + \lambda \|x\|_1$$

Per risolverlo, utilizziamo il metodo proposto in: **Proximal Gradient Descend (PGD)** [Boyd and Vandenberghe, 2004]

$f(x)$, nel caso univariato, può essere approssimato in prossimità del valore $x = a$, meglio che con un'estrapolazione lineare, tramite lo sviluppo di Taylor del secondo ordine:

$$P_2(x) = f(a) + f'(a)(x - a) + \frac{1}{2}f''(a)(x - a)^2$$

Proximal Gradient Descend (PGD)

$$\min_x f(x) + \lambda \|x\|_1$$

$f(x)$ (in forma vettoriale) può essere approssimata tramite lo sviluppo di Taylor del secondo ordine in prossimità:

$$f(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2}(x - x_k)^\top \frac{\delta^2 f(x_k)}{\delta x_k^2} (x - x_k)$$

Supponiamo che $f(x)$ sia differenziabile e $\nabla f(x)$ soddisfa la condizione L-Lipschitziana, ovvero esiste una costante $L > 0$ tale che:

$$\|\nabla f(x') - \nabla f(x)\|_2 \leq L\|x' - x\|_2$$

Utilizzando la condizione L-Lipschitziana Una condizione L-Lipschitziana è una proprietà di una funzione $f(x)$ che limita la sua velocità di variazione.

La condizione afferma che per due punti qualsiasi x e x' , la differenza assoluta nell'output della funzione non è maggiore di una costante L moltiplicata per la differenza assoluta nei suoi input, il che significa che la velocità di variazione della funzione è limitata:

$$|f(x) - f(x')| \leq L \cdot |x - x'|$$

Ora, supponiamo che $\nabla f(x)$ soddisfi la condizione L-Lipschitziana; esiste una costante $L > 0$ tale che:

$$\|\nabla f(x') - \nabla f(x)\|_2 \leq L\|x' - x\|_2$$

Risolvi la norma L_1 (2) Sostituisci la condizione L-Lipschitziana:

$$\|\nabla f(x') - \nabla f(x)\|_2 \leq L\|x' - x\|_2$$

nell'espansione di Taylor del secondo ordine:

$$f(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2}(x - x_k)^\top \frac{\delta^2 f(x_k)}{\delta x_k^2} (x - x_k)$$

Otteniamo:

$$\begin{aligned} f(x) &\simeq f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{L}{2}\|x - x_k\|_2^2 \\ &= f(x_k) + \nabla f(x_k) \cdot (x - x_k) + \frac{L}{2}(x - x_k)^2 \\ &= f(x_k) + \frac{L}{2} \left[\frac{2}{L} \nabla f(x_k) + (x - x_k) \right]^2 \\ &= \frac{L}{2} \left\| x - \left(x_k - \frac{1}{L} \nabla f(x_k) \right) \right\|_2^2 + \text{costante} \end{aligned}$$

Sostituisci l'approssimazione di $f(x)$ nell'obiettivo di ottimizzazione:

$$\min_x \sum_{i=1}^m \frac{L}{2} \left\| x - \left(x_k - \frac{1}{L} \nabla f(x_k) \right) \right\|_2^2 + \lambda \|x\|_1$$

Il minimo del termine quadratico è in:

$$x_{k+1} = x_k - \frac{1}{L} \nabla f(x_k)$$

Risolviamo tramite la discesa del gradiente di $\nabla f(x_k)$. Ogni iterazione (i) mira a trovare una soluzione ottimale nelle vicinanze di x_k .

$$x_{k+1} = \min_x \sum_{i=1}^m \frac{L}{2} \left\| x - \left(x_k - \frac{1}{L} \nabla f(x_k) \right) \right\|_2^2 + \lambda \|x\|_1$$

Scriviamo $z^i = x_k - \frac{1}{L} \nabla f(x_k)$ la soluzione trovata all'iterazione i . Esiste una soluzione in forma chiusa:

$$x_{k+1}^i = \begin{cases} z^i - \frac{\lambda}{L}, & \frac{\lambda}{L} < z^i; \\ 0, & |z^i| \leq \frac{\lambda}{L}; \\ z^i + \frac{\lambda}{L}, & z^i < -\frac{\lambda}{L}. \end{cases}$$

11.3 Rappresentazione sparsa

Consideriamo il set di dati come una matrice, in cui ogni riga corrisponde a un campione e ogni colonna corrisponde a una feature.

Ci sono molti elementi nulli nella matrice D , ma gli elementi nulli non giacciono su tutte le righe o colonne.

Vantaggi della rappresentazione sparsa:

- Rendere molti problemi di classificazione del testo linearmente separabili;
- Minori spese generali di archiviazione

È possibile convertire un set di dati denso in una rappresentazione opportunamente sparsa per sfruttarne i vantaggi?

11.4 Apprendimento del dizionario

Il processo di costruzione di un dizionario appropriato per convertire i campioni in una rappresentazione sparsa è chiamato apprendimento del dizionario.

Dato un set di dati $D = [x_1, x_2, \dots, x_m] \in \mathbb{R}^{n \times k}$, gli obiettivi da imparare sono:

- una matrice dizionario $B \in \mathbb{R}^{d \times k}$
- per ogni campione x_i una rappresentazione sparsa $\alpha_i \in \mathbb{R}^k$ dove k è la dimensione del vocabolario (solitamente specificata dall'utente)

La forma più semplice di apprendimento del dizionario è:

$$\min_{B, \alpha_i} \sum_{i=1}^m \|x_i - B\alpha_i\|_2^2 + \lambda \sum_{i=1}^m \|\alpha_i\|_1$$

11.4.1 Risolvi l'apprendimento del dizionario

Ispirandosi a LASSO, l'apprendimento del dizionario può essere risolto mediante l'ottimizzazione alternata:

1. Fissare B e risolvere α_i per ogni campione x_i :

$$\min_{\alpha_i} \|x_i - B\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1$$

2. Gli α_i trovati sono utilizzati per ottimizzare il dizionario B :

$$\min_B \|X - BA\|_F^2$$

Norma di Frobenius

$\|\cdot\|_F$ è la norma di Frobenius:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

11.4.2 Risolvi il dizionario di apprendimento di KSVD

KSVD [Aharon et al., 2006] è una generalizzazione di Kmeans.

Richiede una strategia di aggiornamento colonna per colonna della matrice B (aggiorna solo una colonna di B in ogni iterazione) per risolvere il compito di ottimizzazione:

$$\min_B \|X - BA\|_F^2 = \min_{b_i} \left\| X - \sum_{j=1}^k b_j \alpha^j \right\|_F^2$$

La formula può essere riscritta come:

$$\min_{b_i} \left\| \left(X - \sum_{j \neq i} b_j \alpha^j \right) - b_i \alpha^i \right\|_F^2 = \min_{b_i} \|E_i - b_i \alpha^i\|_F^2$$

Utilizzando la decomposizione in valori singolari del primo termine fissato, trova i vettori ortogonali corrispondenti ai valori singolari più grandi di E_i .

Per preservare la scarsità, vengono preservati solo gli elementi diversi da zero α_i e E_i preserva solo i termini prodotto diversi da zero di b_i e α_i .

Ripetere più volte per generare la soluzione ottimale.

11.5 Rilevamento compresso

Recuperare le informazioni complete da quelle parzialmente disponibili.

Nelle telecomunicazioni, ad esempio, possiamo ricostruire completamente il segnale originale dai segnali digitali convertiti, se il segnale digitale è compresso e subisce perdita di pacchetti?

Una soluzione a questi problemi è il rilevamento compressivo [Cándes et al., 2006, Donoho, 2006].

Supponiamo che esista un segnale discreto x di lunghezza m ;

- campionando a una frequenza molto inferiore a quella richiesta dal teorema di campionamento di Nyquist(*), otteniamo un segnale campionato y di lunghezza n ($n \ll m$):

$$y = \Phi x$$

Φ è una matrice di misurazione di dimensione $n \times m$;

In generale, x non può essere recuperato da y poiché $n \ll m$.

Tuttavia, se esistesse una trasformazione lineare Ψ che soddisfa $x = \Psi s$ e s è un vettore sparso:

$$y = \Phi \Psi s = A s$$

Quando A la proprietà di isometria limitata, s può essere quasi completamente ricostruita.

(*) Il teorema del campionamento di Nyquist stabilisce le condizioni per una ricostruzione completa del segnale a partire dai campioni: la frequenza di campionamento è almeno il doppio della frequenza più alta del segnale analogico.

11.5.1 Restricted Isometry Property

Restricted Isometry Property, RIP [Cándes, 2008]:

Per una matrice A di dimensione $n \times m$ con $n \ll m$, diciamo che A soddisfa la proprietà di isometria k -ristretta ($k - RIP$) se esiste una costante $\delta_k \in (0, 1)$ tale che:

$$(1 - \delta_k) \|s\|_2^2 \leq \|A_k s\|_2^2 \leq (1 + \delta_k) \|s\|_2^2$$

dove A_k è una sottomatrice di A .

11.5.2 L'oggetto di ottimizzazione e la soluzione al rilevamento compresso

Se A soddisfa $k - RIP$, s può essere quasi del tutto ricostruita da y , e successivamente x :

$$\begin{aligned} \min_s & \|s\|_0 \\ \text{s.t.} & y = A s \end{aligned}$$

$\|s\|_0$ il numero di elementi non nulli nel vettore s .

Minimizzare la norma L_0 è un problema NP-hard. Fortunatamente, in determinate condizioni, minimizzare la norma L_0 condivide la stessa soluzione con minimizzare la norma L_1 [Cándes et al., 2006]:

$$\begin{aligned} \min_s & \|s\|_1 \\ \text{s.t.} & y = A s \end{aligned}$$

Questa formula può essere convertita in una forma equivalente a LASSO, che può quindi essere risolta tramite discesa del gradiente prossimale, ovvero utilizzando Basis Pursuit DeNoising [Chen et al., 1998].

11.6 Completamento della matrice

	Gulliver's Travels	Robinson Crusoe	The Story of Mankind	A Study of History	Gitanjali
Emma	5	?	?	3	2
Michael	?	5	3	?	5
John	5	3	?	?	?
Sarah	3	?	5	4	?

Figura 216: Le valutazioni dei libri da parte di alcuni lettori

Considerando le valutazioni come un **segnale parzialmente disponibile**, è possibile **recuperare il segnale originale** tramite il meccanismo di rilevamento compresso? In generale, la preferenza dei lettori ha una **rappresentazione scarsa**. Pertanto, la risposta è "Sì".

Il completamento della matrice può essere utilizzato per risolvere questo problema.

11.6.1 L'oggetto di ottimizzazione e la soluzione per il completamento della matrice

L'oggetto di ottimizzazione del completamento della matrice è:

$$\begin{aligned} \min_X & \text{rank}(X) \\ \text{s.t. } & X_{ij} = A_{ij}, (i, j) \in \Omega \end{aligned}$$

Il vincolo indica che la matrice recuperata non dovrebbe modificare gli elementi non mancanti.

- X : il segnale da recuperare;
- $\text{rank}(X)$: il rango della matrice X .
- A : il segnale osservato;
- Ω : l'insieme degli indici di pedice degli elementi non mancanti in A .

NP-hard. Usa la norma nucleare per approssimarlo:

$$\begin{aligned} \min_X & \|X\|_* \\ \text{s.t. } & X_{ij} = A_{ij}, (i, j) \in \Omega \end{aligned}$$

$\|X\|_*$ indica la norma nucleare, che è la somma dei valori singolari della matrice.

Si tratta di un problema di ottimizzazione convessa che può essere risolto con la Programmazione Semi-Definita (SDP). In determinate condizioni, A può essere ricostruito perfettamente con $O(mr \log^2 m)$ elementi osservati. [Recht, 2011]

12 Computational Learning Theory

Preoccupazioni

- Come descrivere il processo di **apprendimento**?
- Quali tipi di problemi sono **apprendibili**?
- Quali tipi di problemi sono **facili da apprendere**?
- Per un dato algoritmo di apprendimento, è possibile prevederne le prestazioni teoricamente?
- In che modo i risultati teorici possono guidare la progettazione di algoritmi per problemi pratici?

12.1 Alcuni concetti e notazioni

Set di campioni: campioni indipendenti e identicamente distribuiti, considerando solo il problema della classificazione binaria:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}, \quad x_i \in \mathcal{X}, y_i \in \mathcal{Y} = \{-1, +1\}$$

h è una mappatura da $\mathcal{X}$ a $\mathcal{Y}$:

- **Errore di generalizzazione:** errore previsto del classificatore:

$$E(h; \mathbf{D}) = P_{x \sim \mathbf{D}}(h(x) \neq y)$$

- **Errore empirico:** l'errore medio del classificatore su un dato insieme di campioni:

$$\hat{E}(h; \mathcal{D}) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(h(x_i) \neq y_i)$$

Poiché $\mathcal{D}$ è un campionamento indipendente e identicamente distribuito da $\mathbf{D}$, l'aspettativa dell'errore empirico di h è uguale al suo errore di generalizzazione. Quando la scrittura non è fuorviante, $E(h; \mathbf{D})$ e $\hat{E}(h; \mathcal{D})$ sono abbreviati rispettivamente come $E(h)$ e $\hat{E}(h)$.

Errore previsto $E(h; \mathcal{D}) = P_{x \sim \mathcal{D}}(h(x) \neq y)$

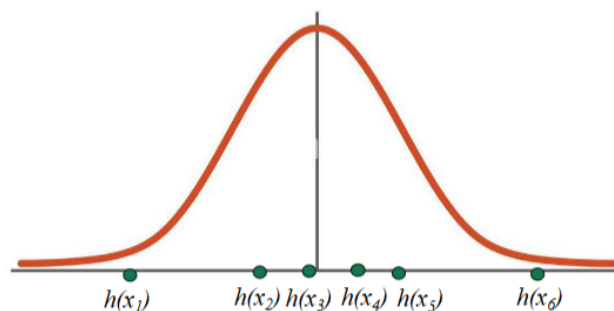


Figura 217: Errore previsto

Parametro di errore ϵ ϵ è un limite superiore di $E(h)$, dove $E(h) \leq \epsilon \Rightarrow$ L'errore massimo che possiamo tollerare per un modello appreso.

Consistenza Una mappatura h si dice coerente con $\mathbf{D}$ se l'errore empirico di h sul set di dati $\mathbf{D}$ è 0.

Disaccordo Per due mappature qualsiasi $h_1, h_2 \in \mathcal{X} \rightarrow \mathcal{Y}$, la loro differenza può essere misurata dal disaccordo:

$$d(h_1, h_2) = P_{x, \mathcal{D}}(h_1(x) \neq h_2(x))$$

12.2 Cos'è l'apprendimento

12.2.1 Concetti

Il concetto fornisce una mappatura dallo spazio campione $\mathcal{X}$ allo spazio etichetta $\mathcal{Y}$, e determina l'etichetta di verità di base y del campione x .

- **Concetto target:** Un concetto c è detto concetto target se $c(x) = y$ per ogni campione (x, y) .
- **Classe concettuale:** L'insieme di tutti i concetti target che desideriamo apprendere è chiamato classe di concetti, indicata da c .

ID	Color	Root	Sound	Ripe (Y)
1	green	curly	muffled	true
2	dark	curly	muffled	true
3	green	straight	crisp	false
4	dark	slightly curly	dull	false

Figura 218: Esempio delle angurie

Il concetto da apprendere (maturazione) potrebbe essere espresso in tre caratteristiche: "maturo è un cocomero con *colore* = A , *radice* = B e *suono* = C ", e dai dati dovremmo sostituire le variabili A , B e C con valori che non contraddicono l'evidenza dei dati.

12.2.2 Spazio delle ipotesi

L'insieme di tutti i possibili concetti per un dato algoritmo di apprendimento $\mathcal{L}$ è chiamato spazio delle ipotesi, indicato con $\mathcal{H}$.

- Poiché la classe di concetti di ground-truth è sconosciuta agli algoritmi di apprendimento, $\mathcal{H}$ e c sono solitamente diversi. Un algoritmo di apprendimento costruisce $\mathcal{H}$ raccogliendo tutti i concetti che si ritiene siano i concetti target.
- Poiché non è noto se i concetti raccolti siano concetti target di ground-truth, $h \in \mathcal{H}$ è definito ipotesi, che fornisce una mappatura dallo spazio campionario $\mathcal{X}$ allo spazio delle etichette $\mathcal{Y}$.

12.2.3 Separabili e non separabili

- **Separabile:** Se $c \in \mathcal{H}$, allora $\mathcal{H}$ contiene un'ipotesi che può classificare correttamente tutte le istanze, e tale problema di apprendimento si dice separabile o coerente rispetto all'algoritmo di apprendimento $\mathcal{L}$.
- **Non separabile:** Se $c \notin \mathcal{H}$, allora $\mathcal{H}$ non contiene alcuna ipotesi che possa classificare correttamente tutte le istanze, e tale problema di apprendimento si dice non separabile o incoerente rispetto all'algoritmo di apprendimento $\mathcal{L}$.

12.2.4 Perché non possiamo apprendere esattamente il concetto target

Dato un set di addestramento $\mathbf{D}$, vorremmo che l'algoritmo di apprendimento $\mathcal{L}$ possa apprendere un'ipotesi h che è vicina al concetto target c .

Perché non apprendere esattamente il concetto target c ? Il processo di apprendimento automatico è limitato da molti fattori:

- Dato che il set di addestramento D , solitamente, contiene un numero di campioni finito, esistono spesso molte ipotesi equivalenti che non possono essere distinte dagli algoritmi di apprendimento su D .
- Inoltre, **esiste una certa casualità nel campionamento di D da $\mathcal{D}$** , e quindi le ipotesi apprese da diversi set di addestramento di uguali dimensioni potrebbero essere diverse.

12.3 Cos'è apprendibile

12.3.1 Probably Approximately Correct, PAC

Speriamo di apprendere un modello migliore con una maggiore comprensione, ovvero un'ipotesi che sia **probabilmente approssimativamente corretta**. Indichiamo con δ la confidenza e la definizione formale come segue:

Definizione di identità PAC

Un algoritmo di apprendimento $\mathcal{L}$ si dice che PAC identifichi la classe di concetto C dallo spazio delle ipotesi $\mathcal{H}$ se, per ogni $x \in C$ e distribuzione $\mathcal{D}$, e $0 < \epsilon, \delta < 1$, l'algoritmo di apprendimento $\mathcal{L}$ restituisce un'ipotesi $h \in \mathcal{H}$ soddisfacendo:

$$P(E(h) \leq \epsilon) \geq 1 - \delta$$

Un tale algoritmo di apprendimento $\mathcal{L}$ ha una probabilità di almeno $1 - \delta$ di apprendere un'approssimazione del concetto target c con un errore al massimo di ϵ .

Ipotesi di probabilità h approssimativamente corretta

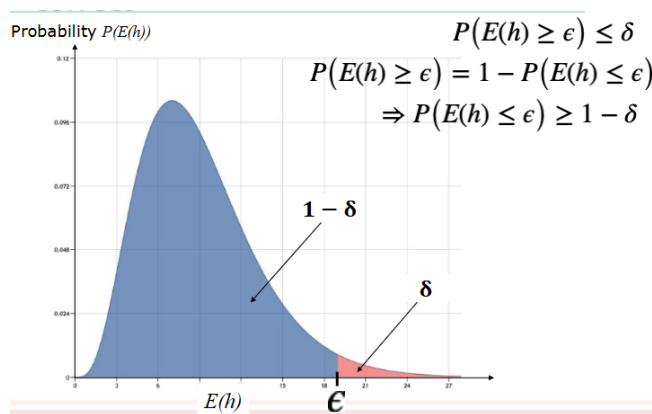


Figura 219: Ipotesi di probabilità h approssimativamente corretta

Relazione tra le principali dimensioni del problema La definizione di PAC apprendibile fornisce una relazione tra le principali dimensioni del problema di apprendimento:

- La dimensione del set di addestramento m .
- L'errore massimo tollerabile ϵ .
- La confidenza $1 - \delta$.
- La complessità dei dati $size(x)$.
- La complessità di concetto target $size(c)$.

Definizione di PAC apprendibile

Una **classe di concetto** target $\mathcal{C}$ si dice che sia apprendibile tramite PAC rispetto allo spazio delle ipotesi $\mathcal{H}$ se esiste un algoritmo di apprendimento $\mathcal{L}$ tale che:
per ogni $0 < \epsilon, \delta < 1$ e distribuzione $\mathcal{D}$, l'algoritmo di apprendimento $\mathcal{L}$ il PAC può identificare la classe di concetto $\mathcal{C}$ dallo spazio delle ipotesi $\mathcal{H}$, per ogni $m \geq \text{poly}(\frac{1}{\epsilon}, \frac{1}{\delta}, \text{size}(x), \text{size}(c))$, dove $\text{poly}(\cdot, \cdot, \cdot, \cdot)$ è una funzione polinomiale e m è il numero di campioni di addestramento i.i.d. estratti da $\mathcal{D}$.

Per gli algoritmi di apprendimento, è necessario considerare la **complessità del tempo di esecuzione**. Pertanto, definiamo ulteriormente:

Definizione di algoritmo di apprendimento PAC

Una classe di concetto $\mathcal{C}$ si dice che sia **efficientemente apprendibile tramite PAC** dal suo algoritmo di apprendimento PAC $\mathcal{L}$ se $\mathcal{C}$ è apprendibile tramite PAC da $\mathcal{L}$ entro un tempo polinomiale $\text{poly}(\frac{1}{\epsilon}, \frac{1}{\delta}, \text{size}(x), \text{size}(c))$.

Supponiamo che l'algoritmo di apprendimento $\mathcal{L}$ elabori ogni campione con un tempo costante, allora la complessità del tempo di esecuzione è equivalente alla complessità del campione e potremmo concentrarci solo sulla complessità del campione:

Definizione di complessità dei campioni

La complessità campionaria di un algoritmo di apprendimento PAC $\mathcal{L}$ è la dimensione campionaria più piccola $m \geq \text{poly}(\frac{1}{\epsilon}, \frac{1}{\delta}, \text{size}(x), \text{size}(c))$ richiesta da $\mathcal{L}$.

12.3.2 L'importanza dell'apprendimento PAC

- L'apprendimento PAC fornisce un quadro formale per descrivere la capacità di apprendimento degli algoritmi di apprendimento e, all'interno di questo quadro, è possibile discutere teoricamente molte questioni importanti.
 - Quali sono i requisiti per apprendere un buon modello per un dato problema di apprendimento?
 - Quali sono le condizioni affinché un algoritmo apprenda in modo efficace?
 - Quanti campioni di addestramento sono necessari per apprendere un buon modello?
- L'analisi della **complessità temporale** degli algoritmi si trasforma nell'analisi della **complessità del campione**.

12.4 Spazio delle ipotesi Finito

La complessità dello spazio delle ipotesi $\mathcal{H}$ è uno dei fattori importanti che influenzano l'apprendibilità.

- In generale, un $\mathcal{H}$ più grande ha maggiori probabilità di contenere il concetto target che stiamo cercando, sebbene uno spazio di ipotesi più ampio renda anche più difficile trovare il concetto target.
- $\mathcal{H}$ è chiamato spazio di ipotesi finito se $|\mathcal{H}|$ è finito, e spazio di ipotesi infinito altrimenti.

La complessità dello spazio delle ipotesi $\mathcal{H}$ è uno dei fattori importanti che influenzano la difficoltà di apprendimento del compito.

Apprendimento PAC appropriato Se lo spazio delle ipotesi è lo stesso della classe concettuale (ovvero $\mathcal{H} = \mathcal{C}$), allora si dice che $\mathcal{C}$ è correttamente apprendibile tramite PAC rispetto a $\mathcal{H}$.

- Intuitivamente, significa che la capacità dell'algoritmo di apprendimento corrisponde correttamente al problema di apprendimento.
- Tuttavia, nell'applicazione pratica, di solito non sappiamo nulla della classe di concetto, quindi è poco pratico progettare un algoritmo di apprendimento con lo stesso spazio di ipotesi della classe di concetto.

Pertanto, è più realistico studiare i casi in cui lo spazio delle ipotesi e la classe dei concetti sono diversi $\mathcal{H} \neq \mathcal{C}$.

12.4.1 Casi separabili

Il concetto target c è nello spazio delle ipotesi $\mathcal{H}$, ovvero $c \in \mathcal{H}$.

Dato un training set D di dimensione m , come possiamo trovare un'ipotesi che soddisfi il vincolo di un dato parametro di errore?

Strategia di apprendimento naturale

- Poiché le etichette dei campioni in D sono assegnate dal concetto target $c \in \mathcal{H}$, qualsiasi ipotesi che classifichi erroneamente qualsiasi campione in D non deve essere il concetto target c .
- Basta eliminare tutte le ipotesi incoerenti con D e mantenere le restanti.
- Quando il training set D è sufficientemente ampio, possiamo continuare a eliminare le ipotesi incoerenti da $\mathcal{H}$ finché non rimane una sola ipotesi, che deve essere il concetto target c .
- In generale, a causa delle dimensioni limitate del training set D , potrebbero esserci più di un'ipotesi equivalente nello spazio delle ipotesi $\mathcal{H}$, ed è impossibile distinguere ulteriormente i vantaggi e gli svantaggi di queste false ipotesi equivalenti.

Di quanti campioni abbiamo bisogno per apprendere una buona approssimazione del concetto target c ?

- Per prima cosa stimiamo la probabilità di avere un'ipotesi che funziona perfettamente sul set di addestramento.
- Supponiamo che l'errore di generalizzazione di un'ipotesi sia maggiore di ϵ .
- Per qualsiasi campione i.i.d. estratto dalla distribuzione D , abbiamo:

$$\begin{aligned} P(h \text{ classifica correttamente } x_i) &= P(h(x_i) = c(x_i)) \\ &= 1 - P(h(x_i) \neq c(x_i)) \\ &= 1 - E(h) \\ &< 1 - \epsilon \end{aligned}$$

- Poiché D contiene campioni m i.i.d, la probabilità che h e D siano coerenti è data da:

$$\begin{aligned} P(h(x_1) = y_1) \wedge P(h(x_2) = y_2) \wedge \dots \wedge P(h(x_m) = y_m) \\ = [1 - P(h(x) \neq y)]^m < (1 - \epsilon)^m \end{aligned}$$

- Dobbiamo assicurarci che la probabilità totale di avere ipotesi coerenti con D e con un errore di generalizzazione maggiore di ϵ non sia maggiore di δ .

$$P(h \in \mathcal{H} : E(h) > \epsilon > \epsilon \wedge \hat{E}(h) = 0) < |\mathcal{H}|(1 - \epsilon)^m < |\mathcal{H}|e^{-\epsilon m}$$

$$|\mathcal{H}|e^{-\epsilon m} \leq \delta \Rightarrow m \geq \frac{1}{\epsilon} (\ln |\mathcal{H}| + \ln \frac{1}{\delta})$$

$$(1 - \epsilon)^m \leq e^{-m\epsilon}$$

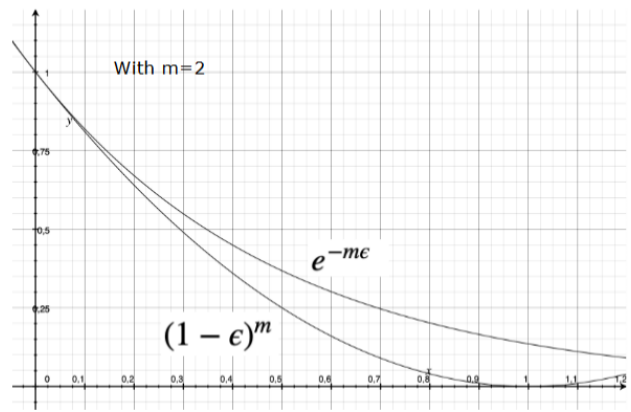


Figura 220: $(1 - \epsilon)^m \leq e^{-\epsilon m}$

$$m \geq \frac{1}{\epsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right)$$

- L'equazione trovata sopra mostra che ogni spazio di ipotesi finito $\mathcal{H}$ è apprendibile tramite PAC
- All'aumentare del numero di campioni, l'errore di generalizzazione dell'ipotesi di output h converge verso 0 a un tasso di convergenza di $O(\frac{1}{m})$.

12.4.2 Casi Non separabili

Per problemi di apprendimento difficili, il concetto target c di solito non si trova nello spazio delle ipotesi $\mathcal{H}$. Supponiamo $h \in \mathcal{H}$, $\hat{E}(h) \neq 0$ perché $\hat{E}(h; \mathcal{D}) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(h(x_i) \neq y_i)$ cioè ogni ipotesi in $\mathcal{H}$ classifica erroneamente almeno un esempio di addestramento.

Disuguaglianza di Hoeffding

Date le variabili m i.i.d. x_i , tali che: $0 \leq x_i \leq 1$, per ogni $\epsilon > 0$,

$$P(E(h) - \hat{E}(h) \geq \epsilon) \leq \exp(-2m\epsilon^2) \quad P(\hat{E}(h) - E(h) \geq \epsilon) \leq \exp(-2m\epsilon^2)$$

$$P(|E(h) - \hat{E}(h)| \geq \epsilon) \leq 2 \exp(-2m\epsilon^2)$$

Applicando la disuguaglianza di Hoeffding a $E(h)$:

1.

$$P(|E(h) - \hat{E}(h)| \geq \epsilon) \leq 2 \exp(-2m\epsilon^2) \Rightarrow P(E(h) \geq \epsilon) \leq 2 \exp(-2m\epsilon^2)$$

2.

$$P(|E(h) - \hat{E}(h)| \geq \epsilon) \leq 2 \exp(-2m\epsilon^2) \Rightarrow P(E(h) \geq \epsilon) = 1 - P(E(h) \leq \epsilon)$$

$$P(E(h) \geq \epsilon) \leq \delta \Rightarrow \delta = 2 \exp(-2m\epsilon^2)$$

$$\Rightarrow \epsilon = \pm \sqrt{\frac{1}{2m} \ln \left(\frac{\delta}{2} \right)}$$

$$P(|E(h) - \hat{E}(h)| \geq \epsilon) \leq 2 \exp(-2m\epsilon^2) \Rightarrow$$

$$\hat{E}(h) - \sqrt{\frac{1}{2m} \ln \left(\frac{\delta}{2} \right)} \leq E(h) \leq \hat{E}(h) + \sqrt{\frac{1}{2m} \ln \left(\frac{\delta}{2} \right)}$$

Dimostra che, per un grande m , l'errore empirico di $\hat{E}(h)$ è una buona approssimazione del suo errore di generalizzazione $E(h)$.

Teorema 12.1

Sia $\mathcal{H}$ uno spazio di ipotesi finito. Allora, per qualsiasi $h \in \mathcal{H}$ e $0 < \delta < 1$, abbiamo

$$P \left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{\ln |\mathcal{H}| + \ln \left(\frac{2}{\delta} \right)}{2m}} \right) \geq 1 - \delta$$

Dimostrazione

Siano $h_1, h_2, \dots, h_{|\mathcal{H}|}$ le ipotesi in $\mathcal{H}$, la probabilità $P \left(\exists h \in \mathcal{H} : |E(h) - \hat{E}(h)| \geq \epsilon \right)$

$$\begin{aligned} P \left((|E(h_1) - \hat{E}(h_1)| \geq \epsilon) \vee (|E(h_2) - \hat{E}(h_2)| \geq \epsilon) \vee \dots \vee (|E(h_{|\mathcal{H}|}) - \hat{E}(h_{|\mathcal{H}|})| \geq \epsilon) \right) \\ \leq \sum_{h \in \mathcal{H}} P \left(|E(h) - \hat{E}(h)| \geq \epsilon \right) \end{aligned}$$

Applicando la disuguaglianza di Hoeffding: $P \left(|E(h) - \hat{E}(h)| \geq \epsilon \right) \leq 2 \exp(-2m\epsilon^2)$

$$\begin{aligned} \sum_{h \in \mathcal{H}} P \left(|E(h) - \hat{E}(h)| \geq \epsilon \right) &\leq 2|\mathcal{H}| \exp(-2m\epsilon^2) \\ \Rightarrow \sum_{h \in \mathcal{H}} P \left(|E(h) - \hat{E}(h)| \leq \epsilon \right) &\geq 1 - 2|\mathcal{H}| \exp(-2m\epsilon^2) \end{aligned}$$

Il teorema è dimostrato ponendo:

$$\begin{aligned} \delta &= 2|\mathcal{H}| \exp(-2m\epsilon^2) \\ \Rightarrow \frac{\delta}{2|\mathcal{H}|} &= \exp(-2m\epsilon^2) \Rightarrow \ln \left(\frac{\delta}{2|\mathcal{H}|} \right) = -2m\epsilon^2 \Rightarrow -\ln \left(\frac{\delta}{2|\mathcal{H}|} \right) = 2m\epsilon^2 \\ \Rightarrow \ln \left(\frac{2|\mathcal{H}|}{\delta} \right) &= 2m\epsilon^2 \Rightarrow \ln \left(\frac{2}{\delta} \right) + \ln(|\mathcal{H}|) = 2m\epsilon^2 \Rightarrow \frac{\ln \left(\frac{2}{\delta} \right) + \ln(|\mathcal{H}|)}{2m} = \epsilon^2 \\ \Rightarrow \epsilon &= \sqrt{\frac{\ln(|\mathcal{H}|) + \ln \left(\frac{2}{\delta} \right)}{2m}} \Rightarrow P \left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{\ln |\mathcal{H}| + \ln \left(\frac{2}{\delta} \right)}{2m}} \right) \geq 1 - \delta \end{aligned}$$

Il Teorema 12.1 mostra che l'errore empirico è una buona approssimazione dell'errore di generalizzazione quando la dimensione del campione m è sufficientemente ampia. In questo caso $c \notin \mathcal{H}$, la scelta migliore è trovare l'ipotesi con il minimo errore di generalizzazione. Il Teorema 12.1 evidenzia in realtà un principio generale di apprendimento:

Empirical Risk Minimization, ERM: Sia h l'ipotesi di output dell'algoritmo di apprendimento $\mathcal{L}$, se:

$$\hat{E}(h) = \min_{h' \in \mathcal{H}} \hat{E}(h')$$

Chiamiamo $\mathcal{L}$ l'algoritmo che soddisfa il **principio di minimizzazione del rischio empirico**.

Questo approccio generalizza l'apprendimento PAC in un apprendimento agnostico in cui $c \notin \mathcal{H}$:

Definizione di apprendimento agnostico PAC

Uno spazio delle ipotesi $\mathcal{H}$ è detto agnosticamente apprendibile tramite PAC se esiste un algoritmo di apprendimento $\mathcal{L}$ tale che, per ogni $0 < \epsilon, \delta < 1$ e distribuzione $\mathcal{D}$, l'algoritmo di apprendimento $\mathcal{L}$ restituisce un'ipotesi $h \in \mathcal{H}$ soddisfacendo:

$$P \left(E(h) - \min_{h' \in \mathcal{H}} E(h') \leq \epsilon \right) \geq 1 - \delta$$

per ogni $m \geq \text{poly} \left(\frac{1}{\epsilon}, \frac{1}{\delta}, \text{size}(x), \text{size}(c) \right)$, dove m è il numero di campioni di addestramento i.i.d. estratti da $\mathcal{D}$.

Il teorema 12.1 mostra che ogni spazio di ipotesi finito è agnosticamente apprendibile tramite PAC. La complessità campionaria dell'algoritmo di apprendimento $\mathcal{L}$ è la dimensione minima m del campione che soddisfa i requisiti di cui sopra.

12.5 Spazio delle ipotesi Infinito

Gli spazi delle ipotesi nelle applicazioni del mondo reale sono solitamente infiniti:

- Tutti gli intervalli nel dominio reale.
- Tutti gli iperpiani nello spazio $\mathbb{R}^d$.

Per studiare l'apprendibilità di tali casi, dobbiamo misurare la complessità degli spazi delle ipotesi $|\mathcal{H}|$:

- Vapnik-Chervonenkis Dimension (VC).
- Rademacher Complexity.

12.6 Dimensione Vapnik-Chervonenkis (VC)

Notazione Dato uno spazio delle ipotesi $\mathcal{H}$ e un insieme di istanze $D = \{x_1, x_2, \dots, x_m\}$, dove ogni ipotesi h può etichettare ogni istanza in D , indicato da:

$$h|_D = \{(h(x_1), h(x_2), \dots, h(x_m))\} \quad \text{con } h \in \mathcal{H}$$

- Ogni elemento di $h|_D$ si chiama **dicotomia**.
- Il numero di dicotomie generate dalle ipotesi in D su $\mathcal{H}$ aumenta all'aumentare di m .

Per esempio, in un problema di classificazione binaria abbiamo:

- ci sono al massimo 4 dicotomie date 2 istanze;
- ci sono al massimo 8 dicotomie date 3 istanze;

Esempio

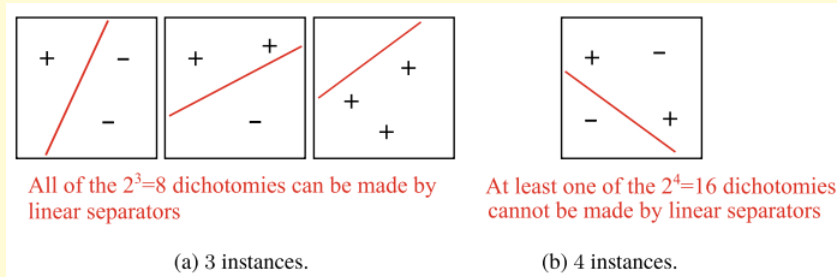


Figura 221: La dimensione VC dello spazio delle ipotesi di tutti i separatori lineari nel piano reale bidimensionale è 3

Ci sono al massimo 8 dicotomie date 3 istanze: $2^3 = 8$.

$$h|_D = (+, +, +) \quad h|_D = (+, +, -) \quad h|_D = (+, -, -)$$

$$h|_D = (-, +, +) \quad h|_D = (-, +, -) \quad h|_D = (-, -, -)$$

Ipotesi diverse nello spazio delle ipotesi $\mathcal{H}$ possono avere risultati uguali o diversi per gli esempi in D . Il numero di dicotomie potrebbe essere finito anche per uno spazio di ipotesi infinito $\mathcal{H}$.

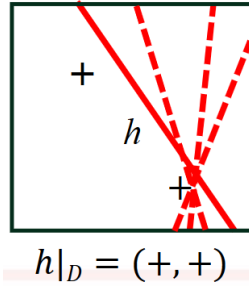


Figura 222: Esempio di dicotomie in uno spazio di ipotesi infinito

Esempio

Ci sono al massimo 2^m dicotomie su m istanze.

Dato $m = 3$:

$$h|_D = (+, +, +) \quad h|_D = (+, +, -) \quad h|_D = (+, -, -)$$

$$h|_D = (-, +, +) \quad h|_D = (-, +, -) \quad h|_D = (-, -, -)$$

$$2^3 = 8$$

Definizione di funzione di crescita

Per $m \in \mathbb{N}$, la funzione di crescita $\Pi_{\mathcal{H}}(\cdot)$ di uno spazio delle ipotesi $\mathcal{H}$ è definita come:

$$\Pi_{\mathcal{H}}(m) = \max_{\{x_1, x_2, \dots, x_m\} \subseteq \mathcal{X}} |\{(h(x_1), h(x_2), \dots, h(x_m)) | h \in \mathcal{H}\}|$$

- La funzione di crescita fornisce il maggior numero di dicotomie che lo spazio delle ipotesi $\mathcal{H}$ può generare su m istanze.
- Più dicotomie ci sono, maggiore è il **potere di rappresentazione**, cioè **migliore è l'adattabilità** ai problemi di apprendimento.
- La funzione di crescita descrive il potere di rappresentazione di uno spazio delle ipotesi $\mathcal{H}$, che riflette anche la **complessità dello spazio delle ipotesi**.

12.6.1 Dicotomia

Per il problema della classificazione binaria, ogni possibile risultato contrassegnato dall'ipotesi h in $\mathcal{H}$ è chiamato **dicotomia**.

12.6.2 Shattering

Diciamo che uno spazio di ipotesi $\mathcal{H}$ può **frantumare** un insieme di dati D se $\mathcal{H}$ può generare tutte le possibili dicotomie di D .

Ricorda la definizione di un Apprendista Probabile Approssimativamente Corretto (PAC) di un concetto utilizzando l'ipotesi h appresa su un set di dati il cui numero di esempi è m :

$$O\left(|E(h) - \hat{E}(h)| \geq \epsilon\right) \leq 2 \exp(-2m\epsilon^2)$$

Utilizzare una funzione di crescita $\Pi_{\mathcal{H}}(m)$ per presentare la relazione tra l'errore empirico $\hat{E}(h)$ e l'errore di generalizzazione $E(h)$:

Teorema 12.2

Per ogni $h \in \mathcal{H}$, $m \in \mathbb{N}$, $0 < \epsilon < 1$, abbiamo:

$$P(|E(h) - \hat{E}(h)| > \epsilon) \leq 4\Pi_{\mathcal{H}}(2m) \exp\left(-\frac{m\epsilon^2}{8}\right)$$

Definizione della dimensione di Vapnik-Chervonenkis

La dimensione VC di uno spazio di ipotesi $\mathcal{H}$ è la dimensione del più grande insieme di istanze D frantumato da $\mathcal{H}$

$$VC(\mathcal{H}) = \max\{m : \Pi_{\mathcal{H}}(m) = 2^m\}$$

La dimensione VC di $\mathcal{H}$ è d se esiste un insieme di istanze di dimensione d frantumato da $\mathcal{H}$ mentre non esiste alcun insieme di istanze di dimensione $d + 1$ frantumato da $\mathcal{H}$.

$$VC(\mathcal{H}) = d$$

Esempio: Intervallo $[a, b]$ sul dominio reale

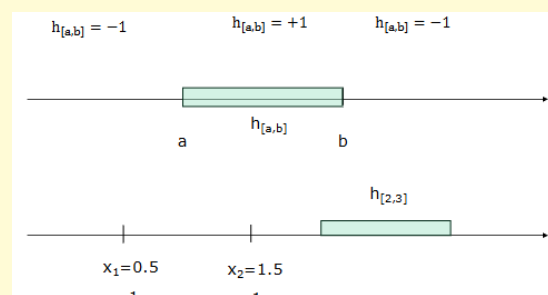
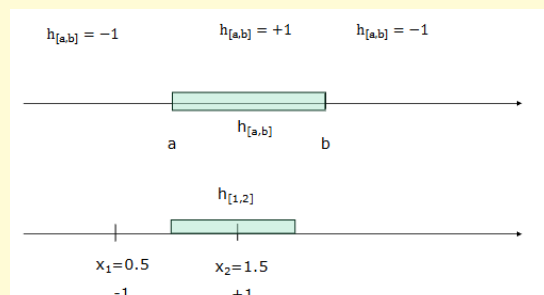
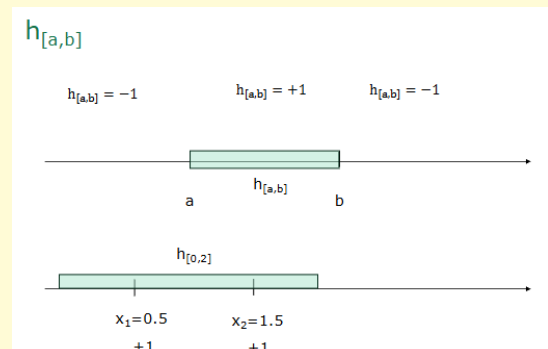
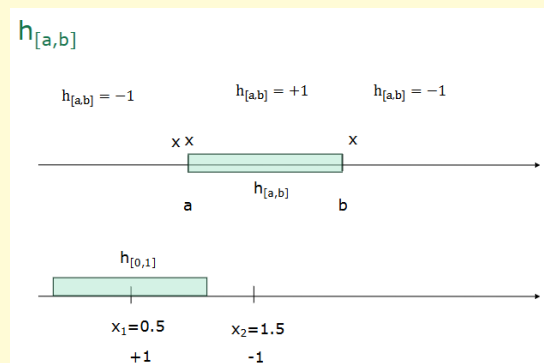
Sia $\mathcal{H} = \{h_{[a,b]} : a, b \in \mathbb{R}, a \leq b\}$ l'insieme di tutti gli intervalli chiusi nel dominio reale $\mathcal{X} = \mathbb{R}$. Per ogni $x \in \mathcal{X}$, abbiamo $h_{[a,b]}(x) = +1$ se $x \in [a, b]$ e -1 altrimenti.

Ponendo $x_1 = 0.5, x_2 = 1.5$ allora $\{x_1, x_2\}$ è frantumato dalle ipotesi $\{h_{[0,1]}, h_{[0,2]}, h_{[1,2]}, h_{[2,3]}\}$ da $\mathcal{H}$.

Da qui la dimensione VC di $\mathcal{H}$ è almeno 2.

Tuttavia, non c'è alcuna ipotesi $h_{[a,b]} \in \mathcal{H}$ che possa generare la dicotomia $\{(x_3, +), (x_4, -), (x_5, +)\}$ per un dataset contenente 3 istanze $\{x_3, x_4, x_5\}$, dove $x_3 < x_4 < x_5$.

Da qui la dimensione VC di $\mathcal{H}$ è 2.



x_1 e x_2 sono frammentate dallo spazio delle ipotesi $\mathcal{H} = \{h_{[0,1]}, h_{[0,2]}, h_{[1,2]}, h_{[2,3]}\}$ perché producono tutte le dicotomie possibili.

Esempio: Separatori lineari nel piano reale 2D

Sia $\mathcal{H}$ l'insieme di tutti i separatori lineari nel piano reale bidimensionale $X = \mathbb{R}^2$. Dalla Figura vediamo che esiste un insieme di dati di dimensione 3 frammentato da $\mathcal{H}$, mentre non esiste alcun insieme di istanze di dimensione 4 frammentato da $\mathcal{H}$. Quindi, la dimensione VC dello spazio di ipotesi $\mathcal{H}$ di tutti i separatori lineari nel piano reale bidimensionale è 3.

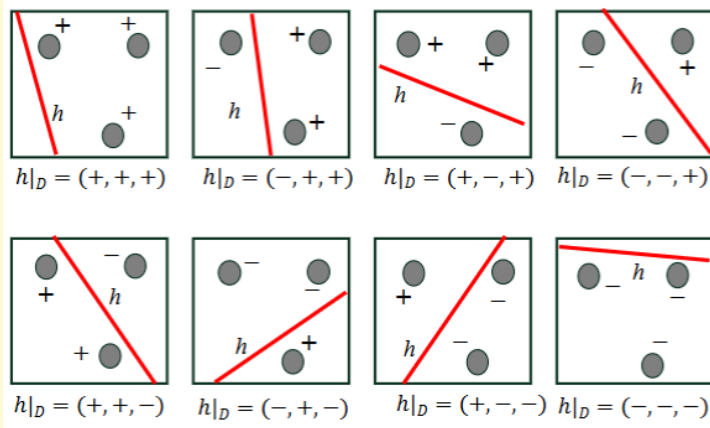
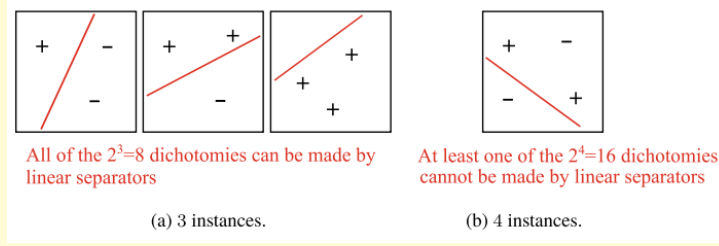


Figura 223: D di dimensione 3 viene frantumato da separatori lineari

La relazione tra la dimensione VC e la funzione di crescita $\Pi_{\mathcal{H}}(m)$ è data dal seguente lemma.

Lemma di Sauer

Se la dimensione VC di uno spazio di ipotesi $\mathcal{H}$ è d , allora per ogni $m \in \mathbb{N}$, abbiamo:

$$\Pi_{\mathcal{H}}(m) \leq \sum_{i=0}^d \binom{m}{i}$$

Esempio

$$\begin{aligned} h|_D = (+, +, +) \quad h|_D = (+, +, -) \quad h|_D = (+, -, -) \\ h|_D = (-, +, +) \quad h|_D = (-, +, -) \quad h|_D = (-, -, -) \end{aligned}$$

$$\begin{aligned} \binom{3}{1} &= \frac{3!}{(3-1)!1!} = \frac{3 \cdot 2}{2} = 3 \\ \binom{3}{2} &= \frac{3!}{(3-2)!2!} = \frac{3 \cdot 2}{2} = 3 \\ \binom{3}{3} &= \frac{3!}{(3-3)!3!} = 1 \\ \binom{3}{0} &= \frac{3!}{(3-0)!0!} = 1 \end{aligned}$$

12.6.3 Limite superiore della funzione di crescita

Possiamo calcolare il limite superiore della funzione di crescita:

Corollario: Se la dimensione VC di uno spazio di ipotesi $\mathcal{H}$ è d , allora per ogni intero $m \geq d$, abbiamo:

$$\Pi_{\mathcal{H}}(m) \leq \left(\frac{e \cdot m}{d}\right)^d$$

dove e è la costante di Eulero ($\lim_{n \rightarrow \infty} (1 + \frac{1}{n})^n$)

Abbiamo il limite VC:

Teorema 12.3

Se la dimensione VC di uno spazio di ipotesi $\mathcal{H}$ è d , allora per ogni $m > d, 0 < \delta < 1$, e $h \in \mathcal{H}$, abbiamo:

$$P \left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{8d \ln \frac{2em}{d} + 8 \ln \frac{4}{\delta}}{m}} \right) \geq 1 - \delta$$

Dimostrazione

Dal teorema 12.2: $P(|E(h) - \hat{E}(h)| > \epsilon) \leq 4\Pi_{\mathcal{H}}(2m) \exp\left(-\frac{m\epsilon^2}{8}\right)$. Dal corollario $\Pi_{\mathcal{H}}(m) \leq \left(\frac{e \cdot m}{d}\right)^d$ e sostituendo quanto sopra, abbiamo:

$$4\Pi_{\mathcal{H}}(2m) \exp\left(-\frac{m\epsilon^2}{8}\right) \leq 4 \left(\frac{2em}{d}\right)^d \exp\left(-\frac{m\epsilon^2}{8}\right) = \delta$$

Risolvendo per ϵ :

$$\epsilon = \sqrt{\frac{8d \ln \frac{2em}{d} + 8 \ln \frac{4}{\delta}}{m}}$$

Sapendo che $P(|E(h) - \hat{E}(h)| > \epsilon) \leq \delta$, otteniamo:

$$P(|E(h) - \hat{E}(h)| \leq \epsilon) \geq 1 - \delta$$

Sostituendo ϵ nell'equazione sopra, otteniamo la conclusione del teorema.

Dal limite VC del Teorema 12.3:

$$P \left(|E(h) - \hat{E}(h)| \leq \sqrt{\frac{8d \ln \frac{2em}{d} + 8 \ln \frac{4}{\delta}}{m}} \right) \geq 1 - \delta$$

Osserviamo che il limite dell'errore di generalizzazione dipende solo dalla dimensione del campione m e converge a una velocità di $O\left(\frac{1}{\sqrt{m}}\right)$.

Il limite VC è indipendente dalla distribuzione dei dati $\mathcal{D}$ e il dataset D .

È libero da distribuzione e indipendente dai dati.

Analogamente al teorema 12.1, dal teorema 12.3 osserviamo che quando la dimensione VC dello spazio è limitata e la dimensione campionaria m è sufficientemente grande, l'errore empirico $\hat{E}(h)$ è una buona approssimazione dell'errore di generalizzazione $E(h)$. Pertanto, per l'algoritmo di apprendimento L che soddisfa il principio di minimizzazione del rischio empirico, vale il seguente teorema:

Teorema 12.4

Ogni spazio di ipotesi $\mathcal{H}$ con dimensione VC finita è (agnostico) apprendibile tramite PAC.

12.7 Rademacher Complexity

- Il limite VC è **indipendente dalla distribuzione e dai dati**, il che rende l'analisi del limite dell'errore di generalizzazione "universale".
- Poiché non tiene conto del set di dati, il limite VC è generalmente lasco.
- È possibile tenere conto della distribuzione dei dati?

La **complessità di Rademacher** è un'altra caratterizzazione della complessità dello spazio delle ipotesi, e la differenza rispetto alla dimensione VC risiede nella **considerazione della distribuzione dei dati in un certo senso**.

Dato un set di dati $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, l'errore empirico di un'ipotesi h è dato da:

$$\begin{aligned}\hat{E}(h) &= \frac{1}{m} \sum_{i=1}^m \mathbb{I}(h(x_i) \neq y_i) \\ &= \frac{1}{m} \sum_{i=1}^m \frac{1 - h(x_i)y_i}{2} \\ &= \frac{1}{2} - \frac{1}{2m} \sum_{i=1}^m h(x_i)y_i\end{aligned}$$

Supponendo che y_i e $h(x_i)$ diano $+1$ o -1 .

Se concordano su x_i il termine $\frac{1-h(x_i)y_i}{2}$ dà $\frac{1-1 \cdot 1}{2} = 0$ o $\frac{1-(-1 \cdot -1)}{2} = 0$.

Se non concordano, il termine dà $\frac{1-h(x_i)y_i}{2}$ rappresenta 1 errore.

- Dove $\frac{1}{m} \sum_{i=1}^m h(x_i)y_i$ rappresenta la coerenza tra i valori previsti $h(x_i)$ e i valori reali y_i .
- Assume il valore massimo 1 se $h(x_i) = y_i$, per ogni $i \in \{1, 2, \dots, m\}$.
- L'ipotesi con il più piccolo errore empirico è:

$$\arg \max_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m h(x_i)y_i$$

Il set di dati potrebbe essere stato corrotto da alcuni rumori. In questi casi, a volte è meglio selezionare un'ipotesi $\mathcal{H}$ che abbia considerato l'influenza dei rumori casuali

$$\sup_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i h(x_i)$$

Nota

È probabile che non possiamo trovare il valore massimo poiché $\mathcal{H}$ è infinito, quindi usiamo l'estremo superiore.

La variabile casuale di Rademacher σ_i : assume il valore $+1$ o -1 con una probabilità equivalente di $\frac{1}{2}$.

Considera tutte le ipotesi in $\mathcal{H}$ e prendi l'aspettativa come:

$$\mathbb{E}_\sigma \left[\sup_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \sigma_i h(x_i) \right]$$

- dove $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$.
- Prende valore in $[0, 1]$, esprime il potere di rappresentazione dell'ipotesi $\mathcal{H}$.
 - Ugua le a 0 quando $|\mathcal{H}| = 1$, cioè, c'è una sola ipotesi in $\mathcal{H}$.

$$* \mathbb{E}_\sigma[h(x_i)] = 1 \cdot 0.5 \cdot h(x_i) - 1 \cdot 0.5 \cdot h(x_i) = 0$$

- Uguale a 1 quando $\mathcal{H} = 2^m$ e $\mathcal{H}$ frantuma D , cioè, per qualsiasi σ , esiste un'ipotesi tale che $h(x_i) = \sigma_i (i = 1, 2, \dots, m)$ (corrisponde perfettamente al rumore casuale osservato).

Definizione di Complessità empirica di Rademacher

La complessità empirica di Rademacher di uno spazio funzionale $\mathcal{F}$ rispetto a Z è definita come:

$$\hat{R}_Z(\mathcal{F}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{F}} \frac{1}{m} \sum_{i=1}^m \sigma_i f(z_i) \right]$$

- Sia $\mathcal{F} : \mathcal{Z} \rightarrow \mathbb{R}$ uno spazio funzionale a valori reali, e $Z = \{z_1, z_2, \dots, z_m\}$ un set di istanze i.i.d., dove $z_i \in \mathcal{Z}$.
- La complessità empirica di Rademacher **misura la correlazione tra la funzione spaziale $\mathcal{F}$ e il rumore casuale** nel set di dati Z .

Definizione di Complessità di Rademacher

La complessità di Rademacher di uno spazio funzionale $\mathcal{F}$ rispetto a una distribuzione $\mathcal{D}$ su Z è definita come:

$$R_m(\mathcal{F}) = \mathbb{E}_{Z \subseteq \mathcal{Z}: |Z|=m} [\hat{R}_Z(\mathcal{F})]$$

Sulla base della complessità di Rademacher, possiamo definire il limite dell'errore di generalizzazione dello spazio delle funzioni $\mathcal{F}$.

Teorema 12.5

Sia $\mathcal{F} : \mathcal{Z} \rightarrow [0, 1]$ uno spazio funzionale a valori reali, e $Z = \{z_1, z_2, \dots, z_m\}$ un set di campioni i.i.d. estratti da $\mathcal{D}$ su $\mathcal{Z}$. Allora, per ogni $\delta \in (0, 1)$ e $f \in \mathcal{F}$, vale quanto segue con una probabilità di almeno $1 - \delta$:

$$\mathbb{E}[f(z)] \leq \frac{1}{m} \sum_{i=1}^m f(z_i) + 2R_m(\mathcal{F}) + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

$$\mathbb{E}[f(z)] \leq \frac{1}{m} \sum_{i=1}^m f(z_i) + 2\hat{R}_Z(\mathcal{F}) + 3\sqrt{\frac{\ln(2/\delta)}{2m}}$$

Poiché $\mathcal{F}$ nel Teorema 12.5 è una funzione a valori reali sull'intervallo $[0, 1]$, il Teorema 12.5 è applicabile solo ai **problemi di regressione**.

Teorema 12.6: Limite di errore di generalizzazione

Sia $\mathcal{H} : \mathcal{X} \rightarrow \{-1, +1\}$ uno spazio delle ipotesi e $D = \{x_1, x_2, \dots, x_m\}$ un set di istanze i.i.d. estratte da $\mathcal{D}$ su $\mathcal{X}$. Allora, per ogni $\delta \in (0, 1)$ e $h \in \mathcal{H}$, quanto segue è valido con una probabilità di almeno $1 - \delta$:

$$E(h) \leq \hat{E}(h) + R_m(\mathcal{H}) + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

$$E(h) \leq \hat{E}(h) + \hat{R}_D(\mathcal{H}) + 3\sqrt{\frac{\ln(2/\delta)}{2m}}$$

Il teorema 12.6 si adatta ai problemi di classificazione binaria.

Il teorema 12.6 (limite di Rademacher) è correlato a $\mathcal{D}$ e D .

Il limite di errore di generalizzazione basato sulla complessità di Rademacher dipende dalla distribuzione dei dati del problema di apprendimento specifico, simile a un limite "su misura" per il problema. Pertanto, è solitamente più stretto del limite di errore di generalizzazione basato sulla dimensione VC.

Per la complessità di Rademacher e la funzione di crescita:

Teorema 12.7

La complessità di Rademacher $R_m(\mathcal{H})$ e la funzione di crescita $\Pi_{\mathcal{H}}(m)$ di uno spazio delle ipotesi $\mathcal{H}$ soddisfano:

$$R_m(\mathcal{H}) \leq \sqrt{\frac{2 \ln \Pi_{\mathcal{H}}(m)}{m}}$$

Dal Teorema 12.6, dal Teorema 12.7, dal Corollario 12.2:

$$E(h) \leq \hat{E}(h) + \sqrt{\frac{2 \ln \frac{em}{d}}{m}} + \sqrt{\frac{\ln(1/\delta)}{2m}}$$

In altre parole, possiamo derivare il limite VC dalla complessità di Rademacher e dalla funzione di crescita.

12.8 Stabilità

Che le prestazioni di generalizzazione vengano analizzate in base alla dimensione VC e alla complessità di Rademacher, i risultati sono indipendenti dall'algoritmo di apprendimento specifico, il che consente di considerare la natura del problema di apprendimento stesso senza l'algoritmo di apprendimento specifico.

Ma d'altra parte, per ottenere i risultati dell'analisi relativi all'algoritmo, dobbiamo trovare un altro modo.

L'analisi di stabilità è una direzione degna di attenzione in questo campo.

- Si studia se l'output dell'algoritmo cambia notevolmente al variare dell'input (set di addestramento).

12.8.1 Due cambi di set di allenamento

Dato $D = \{z_1 = (x_1, y_1), z_2 = (x_2, y_2), \dots, z_m = (x_m, y_m)\}$, $x_i \in \mathcal{X}$ sono istanze i.i.d. estratte da una distribuzione $\mathcal{D}$ e $y_i \in \{-1, +1\}$. Sia $\mathcal{H} : \mathcal{X} \rightarrow \{-1, +1\}$ uno spazio delle ipotesi, e $\mathcal{L}_D \in \mathcal{H}$ l'ipotesi appresa da un algoritmo di apprendimento $\mathcal{L}$ sul training set D .

- Sia $D^{\setminus i}$ il set D con l'iesimo esempio rimosso, cioè:

$$D^{\setminus i} = \{z_1, \dots, z_{i-1}, z_{i+1}, \dots, z_m\}$$

- Sia D^i il set D con l'iesimo esempio sostituito da un nuovo esempio, cioè:

$$D^i = \{z_1, z_2, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_m\}$$

dove $z'_i = (x'_i, y'_i)$, x'_i segue la distribuzione $\mathcal{D}$ ed è dipendente da D .

12.8.2 Loss function

$\ell(\mathcal{L}_D(x), y) : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+$ abbreviato a $\ell(\mathcal{L}_D, z)$, caratterizza la differenza tra l'etichetta predetta $\mathcal{L}_D(x)$ e l'etichetta reale y .

Errore di generalizzazione

$$\ell(\mathcal{L}, D) = \mathbb{E}_{x \in \mathcal{X}, z=(x,y)}[\ell(\mathcal{L}_D, z)]$$

Errore empirico

$$\hat{\ell}(\mathcal{L}, D) = \frac{1}{m} \sum_{i=1}^m \ell(\mathcal{L}_D, z_i)$$

Errore leave-one-out

$$\ell_{loo}(\mathcal{L}, D) = \frac{1}{m} \sum_{i=1}^m \ell(\mathcal{L}_{D \setminus i}, z_i)$$

Definizione di Stabilità uniforme

Per ogni $x \in \mathcal{X}$, $z = (x, y)$, un algoritmo di apprendimento $\mathcal{L}$ soddisfa:

$$|\ell(\mathcal{L}_D, z) - \ell(\mathcal{L}_{D \setminus i}, z)| \leq \beta \quad i = 1, 2, \dots, m$$

$\mathcal{L}$ si dice che soddisfi $\mathcal{L}$ β -stabilità uniforme rispetto alla funzione di perdita ℓ .

- Se un algoritmo di apprendimento L soddisfa la stabilità β -uniforme rispetto a una funzione di perdita ℓ , allora:

$$\begin{aligned} & |\ell(\mathcal{L}_D, z) - \ell(\mathcal{L}_{D^i}, z)| \\ & \leq |\ell(\mathcal{L}_D, z) - \ell(\mathcal{L}_{D \setminus i}, z)| + |\ell(\mathcal{L}_{D^i}, z) - \ell(\mathcal{L}_{D \setminus i}, z)| \\ & \leq 2\beta \end{aligned}$$

il che significa che la stabilità dell'esclusione di un'istanza implica la stabilità della sostituzione di un'istanza.

Se la funzione di perdita ℓ è limitata:

For all D and $z = (x, y)$, $0 \leq \ell(\mathcal{L}_D, z) \leq M$, allora

Teorema 12.8

Dato un set di dati D con m istanze i.i.d. estratte dalla distribuzione $\mathcal{D}$. Se un algoritmo di apprendimento $\mathcal{L}$ soddisfa la stabilità β -uniforme rispetto a una funzione di perdita ℓ limitata superiormente da M , allora, per qualsiasi $m \geq 1$ e $0 < \delta < 1$ quanto segue vale con una probabilità di almeno $1 - \delta$:

$$\begin{aligned} \ell(\mathcal{L}, D) & \leq \hat{\ell}(\mathcal{L}, D) + 2\beta + (4m\beta + M) \sqrt{\frac{\ln(1/\delta)}{2m}} \\ \ell(\mathcal{L}, D) & \leq \ell_{loo}(\mathcal{L}, D) + \beta + (4m\beta + M) \sqrt{\frac{\ln(1/\delta)}{2m}} \end{aligned}$$

Il teorema 12.8 mostra il limite di errore di generalizzazione dell'algoritmo di apprendimento $\mathcal{L}$ derivato dall'analisi di stabilità.

- Il tasso di convergenza tra l'errore empirico e l'errore di generalizzazione è $\beta\sqrt{m}$;
- Quando $\beta = O\left(\frac{1}{m}\right)$, il tasso di convergenza diventa $O\left(\frac{1}{\sqrt{m}}\right)$.
- che è coerente con quelli del limite VC e del limite Rademacher.

L'analisi della stabilità dell'algoritmo di apprendimento si concentra su:

$$|\hat{\ell}(\mathcal{L}, D) - \ell(\mathcal{L}, D)|$$

l'analisi della complessità dello spazio delle ipotesi considera:

$$\sup_{h \in \mathcal{H}} |\hat{E}(h) - E(h)|$$

In altre parole, l'analisi di stabilità non considera necessariamente ogni ipotesi, ma analizza solo il limite di errore di generalizzazione dell'ipotesi di output $\mathcal{L}_D$

Qual è la relazione tra stabilità e apprendibilità? Innanzitutto, si deve supporre che $\beta\sqrt{m} \rightarrow 0$, solo in questo modo l'algoritmo di apprendimento stabile possa avere una certa capacità di generalizzazione, ovvero la perdita di esperienza converge alla perdita di generalizzazione, altrimenti l'apprendibilità è impossibile. Per l'algoritmo di apprendimento che soddisfa il principio ERM, valgono i seguenti teoremi:

Teorema 12.9

Se un algoritmo di apprendimento ERM $\mathcal{L}$ è stabile, allora lo spazio delle ipotesi $\mathcal{H}$ è apprendibile.

- La stabilità dell'algoritmo di apprendimento porta all'apprendimento dello spazio delle ipotesi.
- La stabilità e lo spazio delle ipotesi possono essere collegati dalla funzione di perdita ℓ .